스마트 제조_2일차

류영표 강사
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com

학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

HISTORY OF ARTIFICIAL INTELLIGENCE

Key milestones • Updated October 2026

1950
Turing Test
Can machines exhibit human-like intelligence?

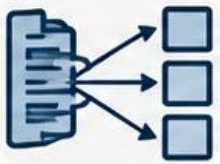
1956
Dartmouth Workshop
Birth of AI as a field of research

1980s
Expert Systems
Knowledge-based systems and rule-based AI

1997
Deep Blue defeats Kasparov
First major victory of AI over a world chess champion

2012
AlexNet / Deep Learning
Breakthrough in image recognition with deep neural networks

2017
Transformer Architecture
Attention mechanism revolutionizes AI models



2020
GPT-3
Large-scale language model with broad capabilities

2022
ChatGPT Launch
Widespread adoption of conversational AI

2023
GPT-4
Multimodal capabilities and stronger reasoning

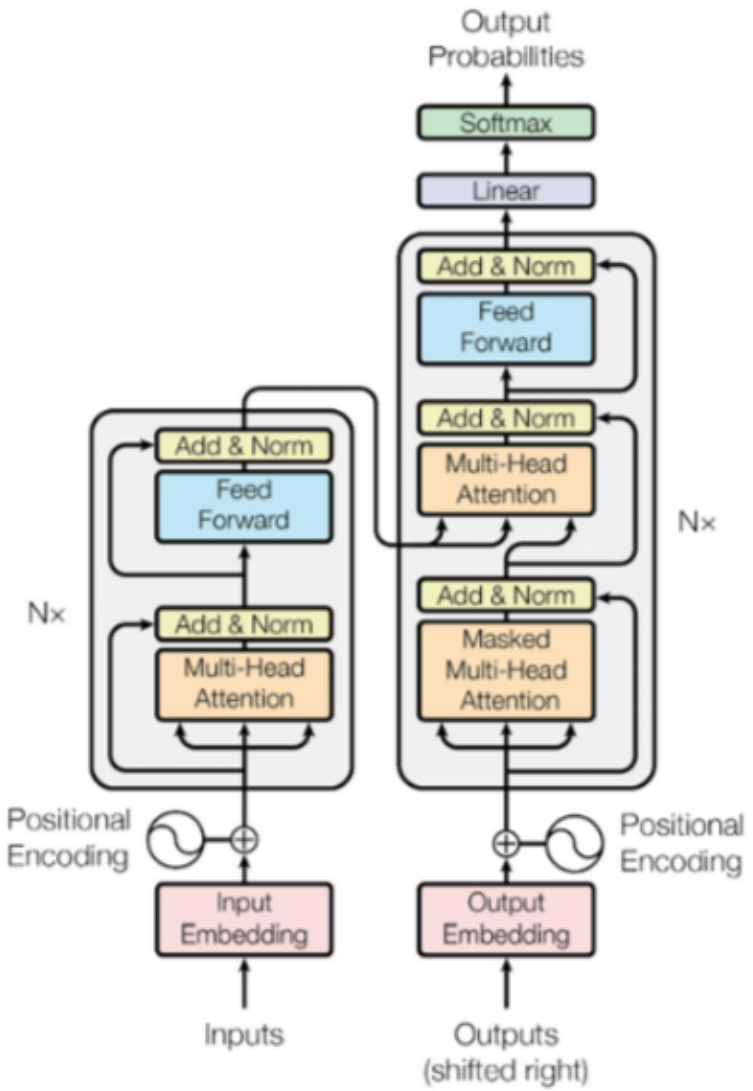
2025
Reasoning & Multimodal AI
Gemini 3 announced (Nov 2025)

2026
Agentic AI & Advanced Models
GPT-5 family, Gemini 3.5/3.8, Claude 5.5

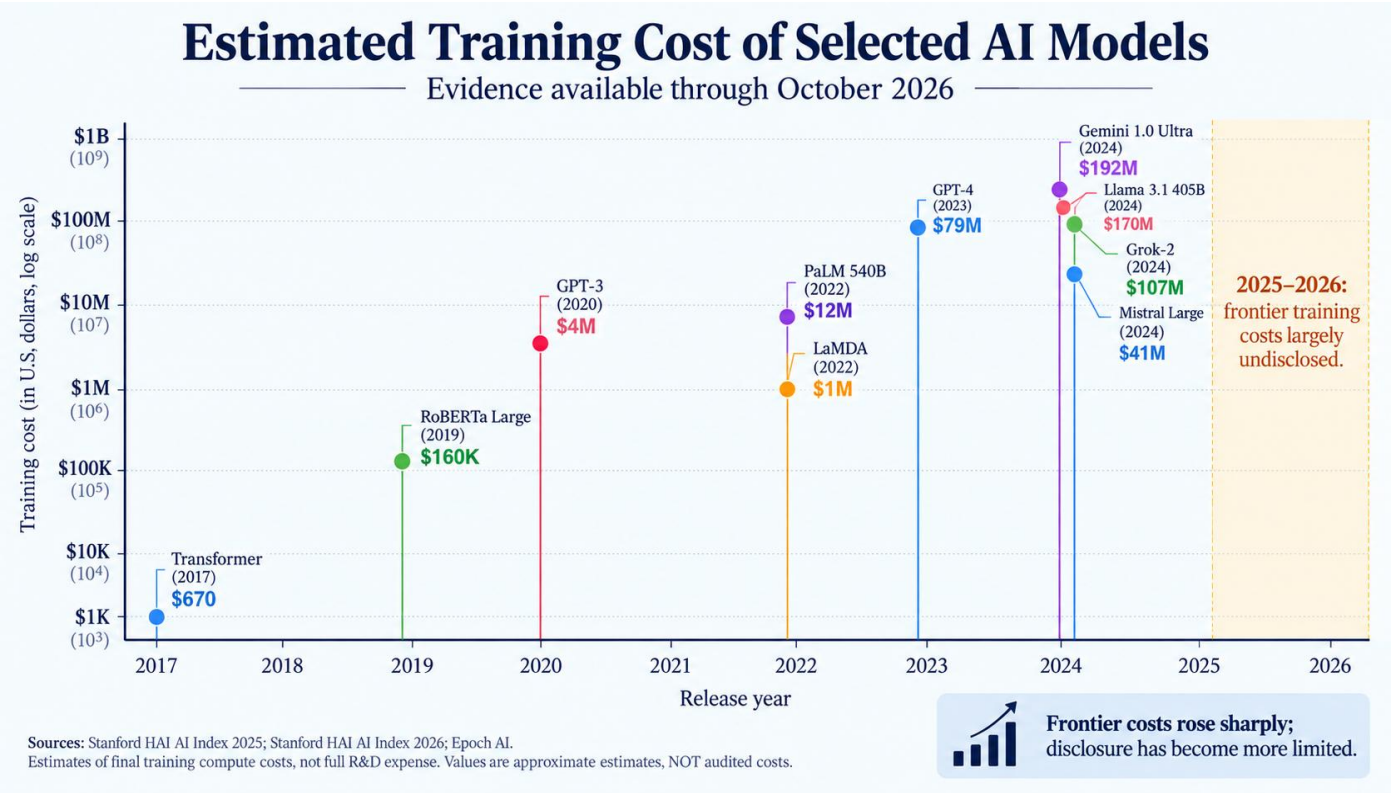
2017, Transformer 등장으로 시작된 AI 패러다임 전환

Transformer

Attention Is All You Need



2018 ~ LLM(Large Language Model)의 폭발적 성장



스케일링 법칙(Scaling Law)

- 모델의 크기(파라미터)와 학습 데이터가 늘어날수록 지능이 향상된다는 법칙이 증명된 시기
- GPT 시리즈 : GPT- 1에서 GPT- 6로 이어지며 인간 수준의 언어 능력을 확보
- ChatGPT 모멘텀 : 2022년, 대화형 인터페이스를 통해 AI가 대중의 일상으로 진입

대규모언어 모델(LLM, Large Language Model)

정의 (Definition)

방대한 텍스트 데이터로 학습되어 인간의 언어를 이해하고 생성하는 초거대 AI 모델입니다.

핵심 기술 (Core Tech)

문맥을 파악하는 트랜스포머(Transformer) 아키텍처와 어텐션 메커니즘을 기반으로 작동합니다.

주요 기능 (Capabilities)

자연어 생성, 문서 요약, 번역, 감정 분석, 코드 작성 등 다양한 지적 작업을 수행합니다.

대표 모델 (Examples)

OpenAI의 GPT-6, Google의 Gemini, Anthropic의 Claude, Meta의 LLaMA 등이 있습니다.

현재: LLM의 3대 핵심 능력

고도화된 추론

단순 검색을 넘어, 복잡한 문제를 단계별로 생각하고 논리적으로 해결하는 추론(Reasoning) 능력을 갖추었습니다.

코드 생성

자연어를 코드로 변환하고, 버그를 수정하며, 전체 시스템 아키텍처를 제안하는 수준에 도달했습니다.

멀티모달

텍스트뿐만 아니라 이미지, 음성, 비디오를 동시에 이해하고 생성하는 통합 지능으로 진화했습니다.

2026: 차세대 LLM과 Agent 시대

GPT-5, Gemini 3 등 거대 모델이 이끄는 업무 자동화 혁신

- **GPT-6** 고급 추론, 코딩, 컴퓨터 조작, 대화형 UI
- **Gemini 3** 멀티모달 이해, 추론, 긴 문맥 처리
- **Claude 5** 복잡한 추론, 코딩, 점문 업무 수행
- **Llama 4** 오픈소스 최강자. 기업 맞춤형 파인튜닝 용이.
- **Mistral Large 4** 개방형 AI 생태계 및 효율적인 모델 활용

AI Agent 시대 확립

단순한 질문-답변을 넘어,
AI가 스스로 계획하고 업무를 완결하는
진정한 자동화

- SW 개발 자동화 (Copilot Workspace)
- PC 조작 자동화 (Operator AI)
- Multi-Agent 협업 시스템 도입

2026 AI 인프라: GPU 수요와 추론 비용

GPU 인프라 수요 확대
NVIDIA 데이터센터 매출 2026년 2분기 **890억 달러,**
전년 대비 **117% 증가.**

AI 클라우드 투자 확대
2026년 AI 최적화 IaaS 지출 **423억 달러** 전망,
전년 대비 **96% 증가**.

추론이 핵심 비용으로
2026년 추론 인프라 지출 **233억 달러,**
학습 190억 달러 전망.

모델 단가는 하락
2024년 1월~2026년 4월 언어모델 품질조정 가격
약 80% 하락.

Agent 사용량 증가
반복적인 도구 호출과 긴 작업으로
총 토큰 사용량 증가 가능.

> ***"토큰당 비용은 낮아져도, 사용량 증가로 총 운영비는 커질 수 있습니다."***

출처: NVIDIA FY2027 Q2 (2026.08); Gartner (2026.08); OECD (2026).

AI 거품론과 인프라의 도전

2026년: 투자 집중과 실질적 가치 창출 사이

1. **투자 집중**
   최첨단 모델 개발은 소수 기업 중심,
   자본·연산 인프라 집중 심화.

2. **인프라 부담**
   GPU·전력·데이터센터 비용과
   공급망 리스크 확대.

3. **소형 모델의 부상**
   목적 특화 SLM과 공개 가중치 모델로
   비용·통제력 확보.

4. **비용과 생산성**
   추론 단가는 하락해도 대규모 사용량과
   검증 비용은 증가할 수 있음.

5. **실무 적용의 과제**
   정확성·보안·운영비를 함께 평가해야
   지속 가능.

핵심 질문: AI 도입 비용을 넘어, 검증 가능한 실질적 성과를 만드는가?

출처: Stanford HAI, AI Index Report 2025·2026

2026: 클라우드 및 AI 인프라의 도전 과제

더 큰 모델, 더 많은 에이전트, 더 높은 인프라 요구

GPU 수요의 폭발적 증가

초거대 모델과 멀티모달, AI Agent 확산으로
글로벌 GPU 수요가 2024-2026년 연평균 **2-3배 증가.**

클라우드 비용 상승 및 제약

AI 워크로드 확산으로 클라우드 인프라 **비용이 급증**하고,
GPU 가용성 부족으로 대기 시간과 구축 지연 발생.

추론 인프라 비용 확대

Agent 기반 서비스와 실시간멀티모달 처리로
추론 비용이 전체 AI 운영비의 **50% 이상**을 차지.

모델 다양화와 복잡한 운영

폐쇄형(고성능)과 개방형(저비용) 모델의 혼재로
모델 **관리, 보안, 거버넌스가 더욱 복잡**해짐.

데이터 보안 및 규제 강화

기업 데이터의 클라우드 활용이 늘어나면서
보안, 개인정보, AI 규제 대응 비용이 증가.

적정 기술 전략

(Appropriate Technology)

무조건적인 초대형 모델이 아니라,
목적과 환경에 맞는 최적의 조합을
선택하는 것이 핵심입니다.

하이브리드 인프라

클라우드 + 온프레미스 + 엣지
환경을 조합해 비용과 성능을
최적화

효율적 모델 활용

작은 전문 모델(SLM)과
멀티모달 모델을 적재적소에
활용하여 비용 효율화

온디바이스 AI 확대

민감한 데이터는 기기 내에서
처리하여 보안과 운영 비용을
동시에 해결

AI Agents

Top 6 Most Popular Use Cases of AI Agents

Voice Agents

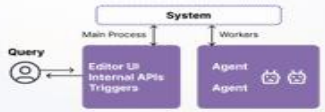
Agents used via voice, often used in customer service

Agentic RAG

Knowledge retrieval based AI Agent workflow

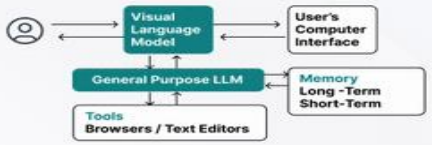
Agentic RAG

These are AI Agent creation/ development platforms



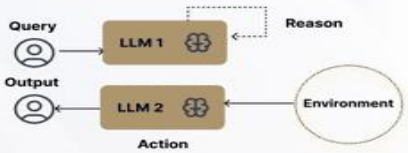
UI Interaction Agents

Versatile agents that can navigate the UI and perform interactions



Coding Agents

AI Agents for building and debugging applications 10x faster



Tool-based Agents

These agents are niche and are based on the tools used per workflow

미래: 멀티 에이전트 협업

전문 역할을 가진 여러 AI 에이전트가 함께 협력하여
복잡한 문제를 해결하는 시대

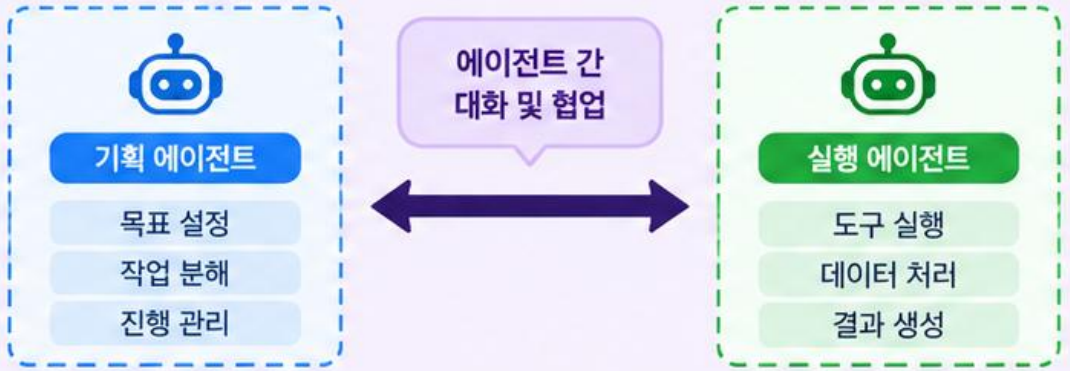
1 Conversable Agent

도구와 역할을 가진 대화형 에이전트

필요에 따라 전문 에이전트를 생성하거나 연결하여
맞춤형 에이전트 시스템을 구성할 수 있습니다.

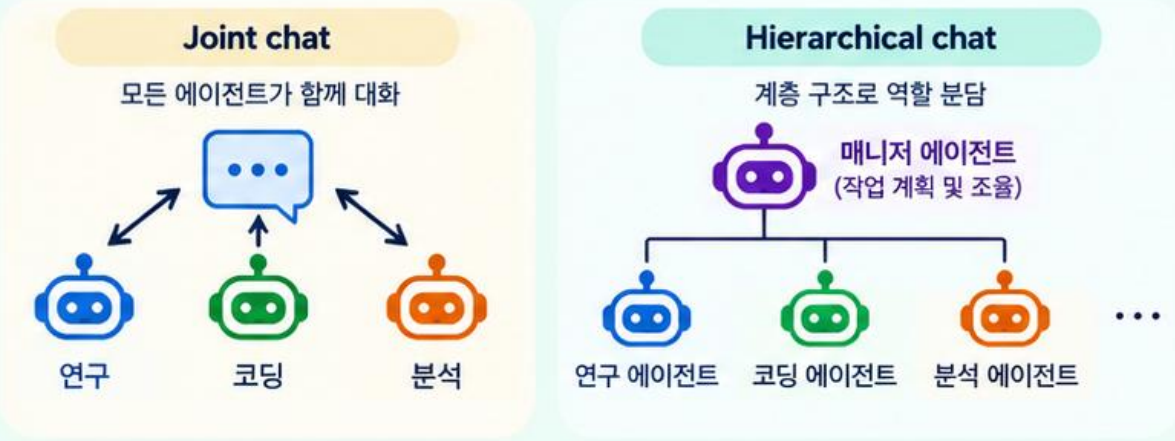
2 Multi-Agent Conversations

여러 에이전트가 서로 대화하며 협업



3 Flexible Conversation Patterns

목적에 따라 다양한 협업 방식 지원

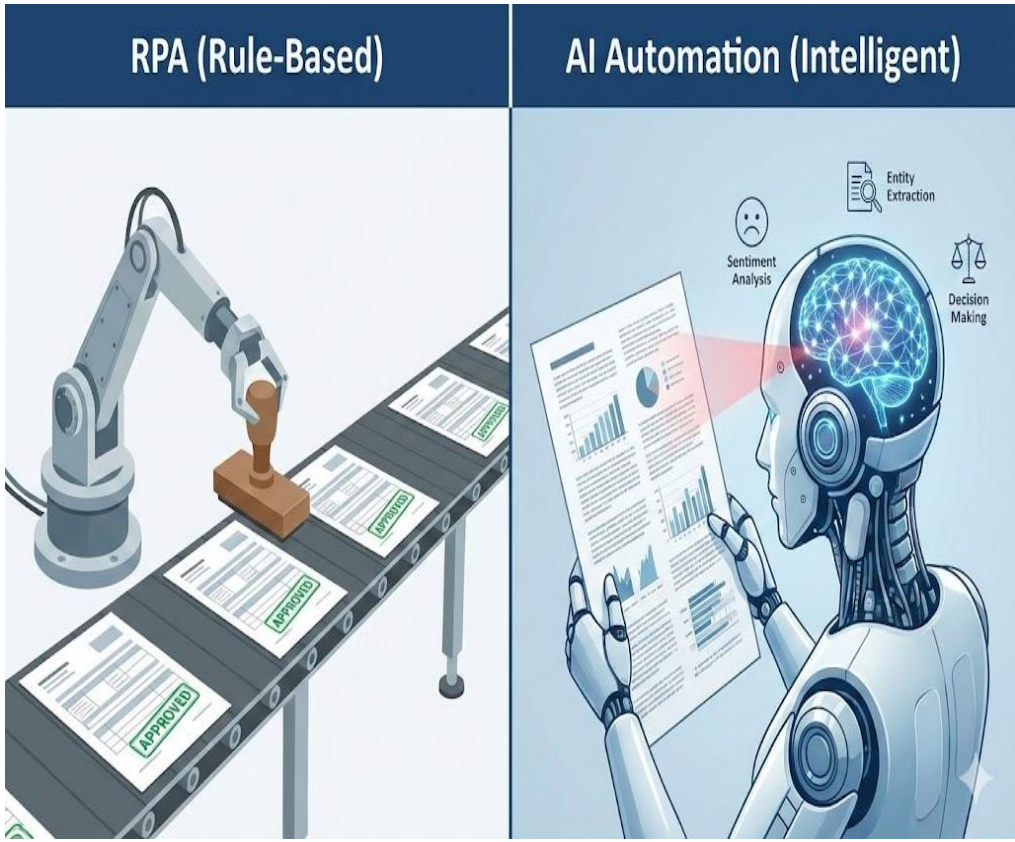


패러다임의 변화(RPA에서 지능형 자동화로)

- AI 자동화(AI Automation) 란?

  : 머신러닝과 인지 기술을 활용하여, 비정형 데이터(이미지, 텍스트, 음성)를 이해하고 스스로 판단하여 업무를 수행하는 프로세스

- RPA(Robotic Process Automation) : 사람이 반복적으로 수행하는 업무를 소프트웨어 로봇이 대신 자동화하는 기술



- 기존 자동화(RPA) : “시키는 대로 수행” (Rule-Based)

  - 정해진 규칙에 따라 반복 업무 처리(ex. 송장 데이터 입력)

  - 예외 상황 발생 시 사람의 개입 필요

- AI 자동화(Intelligent) : “스스로 판단하고 수행”(Data-Driven)

  - 비정형 데이터(문서, 이미지, 자연어)를 이해하고 분석

  - 맥락을 파악하여 최적의 의사결정 지원

자율 에이전트와 인간의 협업

에이전트 AI(Autonomous Agents):

- 사용자의 목표(Goal)를 이해하고 스스로 계획을 수립하여 실행
- 여러 도구와 시스템을 자율적으로 활용하여 복잡한 업무 완수

인간과 AI의 협업(Human-AI Collaboration):

- 인간은 전략 수립, 창의적 결정, 최종 검토와 같은 고부가가치 업무에 집중
- AI는 데이터 수집, 분석, 초안 분석 등 실행 업무를 전달하는 파트너 역할수행

AI로 진화하는 비즈니스 자동화

AI와 RPA의 결합으로 지능형 자동화가 실현됩니다

1 [기술] 뇌와 손의 결합 (The Core)

RPA의 실행력과 AI의 판단력이 결합하여 지능형 자동화의 핵심이 됩니다.

RPA (손)
- 정해진 규칙대로 실행
- 반복 업무 자동화
- 높은 정확성·속도

+

AI/ML (뇌)
- 데이터를 학습하고 판단
- 비정형 데이터 처리
- 상황에 맞는 의사결정

→

지능형 로봇 탄생
스스로 판단하고 실행하는 업무 자동화

2 [실행] 전방위 자동화 (The Action)

AI와 RPA가 협업하여 업무 전 과정을 자동화합니다.

UI 자동화 (좌측)
- 클릭, 입력 등 PC 화면 조작
- 웹 시스템·업무 도구 연동
- 반복 업무의 자동 처리

+

프로세스 자동화 (우측)
- 복잡한 업무 흐름 자동화
- 여러 시스템 간 연계 실행
- 예외 상황 처리 및 모니터링

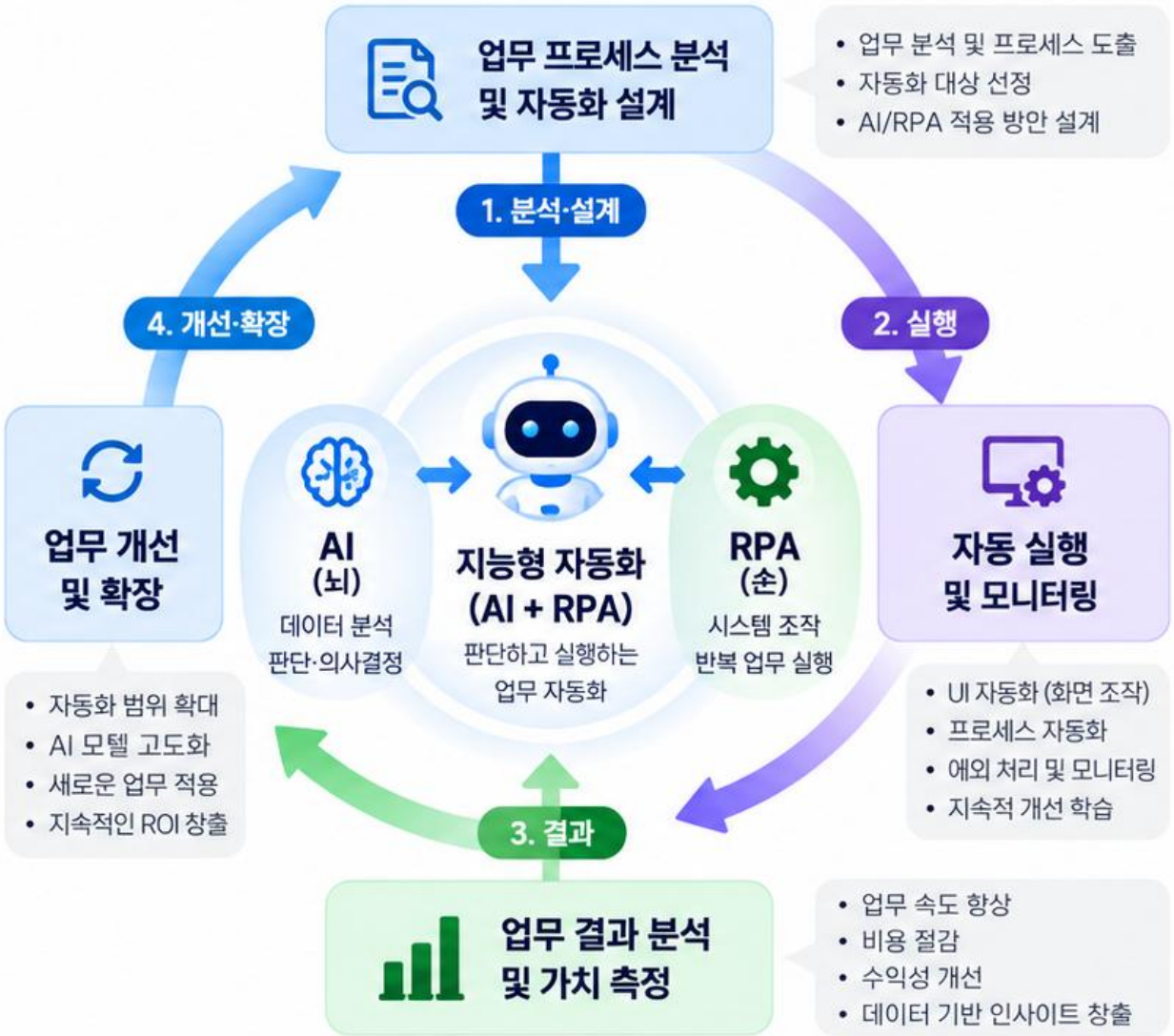
3 [결과] 비즈니스 가치 창출 (The Value)

자동화를 통해 업무 효율을 높이고, 기업의 이익과 성장을 실현합니다.

운영 효율성 향상
- 업무 처리 속도 향상
- 운영 비용 절감
- 사람의 반복 작업 부담 감소

+

기업 가치 창출
- 수익성(Profit) 개선
- 데이터 기반 의사결정 강화
- 지속적인 성장 기반 마련



핵심 메시지

RPA의 실행력과 AI의 판단력이 결합하여,
더 빠르고, 더 똑똑한, 더 큰 가치를 만드는 비즈니스 자동화가 실현됩니다.

AI Agent | 목표를 이해하고, 계획하고, 도구를 사용해 스스로 실행하는 AI 비서

AI 시대, 역할의 경계가 무너졌다: 새로운 인재의 조건

최근 시대가 급변하면서 여러가지 인사이트들이 공유되고 있는건 다들 알고 계실겁니다.
그 중 저희쪽에서도 봐야 할 인사이트가 담긴 글이 있어서 공유드리니 꼭 봐주시길 당부드립니다.

😲<"T자형 인재는 죽었다" - AI 시대, 이제 'π형 풀스택 인간'만 살아남는다>
- "코딩 잘하면 끝? 이제는 제품 감각 + 엔지니어링 + 도메인 지식 없으면 도태된다"

AI가 코드를 짜주고 디자인을 뽑아주는 시대. 그런데 왜 아직도 우리 회사엔 PM, 개발자, 디자이너가 따로 있을까? 답은 간단하다. AI가 아무리 똑똑해져도, '무엇을 만들지' 결정하고 '왜 이게 좋은지' 판단하는 건 여전히 사람 몫이기 때문. 문제는, 그 '사람'의 정의가 바뀌고 있다는 것.

Microsoft 수석 프로덕트 엔지니어가 직접 경고하는 역할의 대재편. 이젠 한 가지만 잘해서는 안 된다. 두 가지 전문성에 폭넓은 기반 지식을 갖춘 'π형 인재'가 되거나, 도태되거나 둘 중 하나다.

1. 역할의 경계가 무너지고 있다 🌊

* 과거엔 명확했다 - PM은 기획, 개발자는 코딩, 디자이너는 디자인
* 이제는 "프로덕트 엔지니어"가 등장 - 기획부터 코딩, 배포, 모니터링까지 혼자 다 함
* 예전엔 웹 프레임워크 모르면 프로토타입 못 만들었지만, 지금은 Claude Code로 몇 시간이면 끝
* 저자는 실제로 명함에 "Principal Product Engineer"라고 적혀있음
* Figma 디자인을 몇 시간 만에 작동하는 웹 프로토타입으로 변환하는 게 일상

2. 스킬 상품화의 시대가 왔다 📦

* 예전엔 "디자인을 코드로 옮기는 능력"이 희소했지만 이젠 누구나 가능
* 하지만 이건 나쁜 소식이 아님 - Excel이 회계사를 없앤 게 아니라 더 고차원 업무를 하게 만든 것처럼
* 3D 프린터가 제조업을 파괴하지 않았고, 로봇 수술기가 외과의사를 대체하지 않았듯이
* AI는 증강(augmentation) 도구일 뿐, 전문성과 판단력을 대체하지 못함
* 문제는 '어떤 전문성'이 중요한지가 바뀌고 있다는 것

11월 14일 금요일

3. 창의성과 취향이 새로운 무기다 🎨

* LLM은 수천 개 아이디어를 뽑아내지만, 어떤 게 '진짜 좋은지'는 못 가림
* AI 결과물이 다 똑같이 보이는 이유 - 기존 데이터를 재탕하기 때문
* 둥근 모서리에 보라-파랑 그라데이션... 다 AI 슬롭(slop)의 전형
* 아이폰 이전에도 스마트폰은 있었지만, Apple의 취향이 시장을 바꿈
* 기술적으로 맞는 것과 진짜 매력적인 것의 차이를 구분하는 눈이 필요

4. 소통 능력이 곧 프롬프팅 능력이다 💬

* 더 명확한 요구사항을 작성할수록 더 나은 결과물이 나옴
* GitHub Spec Kit 프로젝트의 핵심 - "Spec-Driven Development"
* 사전 기획의 비용이 LLM의 이해도를 높이고 더 나은 코드를 만듦
* 바이브 코딩(vibe coding)으로는 좋은 제품 못 만듦
* 복잡함을 명확함으로 번역하는 능력이 더 중요해짐

5. 크로스 도메인 지식이 기본이 됐다 🔗

* 모든 분야의 전문가일 필요는 없지만, 조각들이 어떻게 맞물리는지는 알아야 함
* 프론트엔드 엔지니어가 아니어도 프론트엔드를 '충분히' 알아야 함
* Claude Code로 REST API 설정하는 법을 아는 비개발자가 순수 개발자를 앞지름
* 스택 전반의 제약과 트레이드오프를 이해해야 제대로 된 대화 가능
* 최신 도구들이 뭔지 알고 써먹을 줄 알아야 생산성이 나옴

6. AI 증강 스킬은 필수 역량이다 🤖

* 모든 AI/ML 논문을 읽을 필요는 없음
* 언제, 어떻게 AI 도구를 활용하는지만 알면 됨
* LLM이 잘하는 것과 못하는 것을 구분하고, 이를 체인으로 엮는 능력
* 어떤 작업에 어떤 모델이 적절한지 판단하는 감각
* AI 결과물이 맞는지 검증하는 방법을 아는 것

AI 시대, 역할의 경계가 무너졌다: 새로운 인재의 조건

7. 제품 감각이 핵심 차별화 요소다 🎯

11월 14일 금요일

* 무엇을 만들지, 왜 만드는지 아는 게 땜질쟁이와 제품 전문가의 차이
* 고객 피드백 두 조각을 이어붙이지 못하는 PM들 - 그러지 마라
* 기술적으로 인상적이지만 아무도 원하지 않는 걸 만드는 걸 막아주는 스킬
* 사용자에게 기능을 물어볼게 아니라 패턴을 찾아서 솔루션을 만들어야 함
* 작은 버전으로 진짜 가치를 전달하는 것부터 시작

8. 실행 속도가 복리로 쌓인다 ⚡

* 아이디어는 싸다, 출시가 어렵다 - 바이브 코딩으로도 마찬가지
* 바이브 코딩이 수천 개 SaaS 사업을 만들었나? 아니다, 훨씬 적음
* 왜냐면 엔드투엔드로 하는 게 여전히 어렵기 때문
* 빠르게 움직이면서도 '왜' 하는지를 잊지 않는 것
* 언제 밟아야 할지 감각을 키워야 함

9. 학습 민첩성이 생존을 결정한다 📚

* 기술 지식의 반감기가 계속 짧아지고 있음
* 웹 프레임워크는 "피자 먹고 싶어"라고 말하기도 전에 사라짐
* 프레임워크 암기가 아니라 전이 가능한 이해가 필요
* 근본 원리를 다른 곳에 어떻게 적용할지 아는 능력
* 내가 뭘 모르는지 발견하고, 누구에게 물어볼지 아는 것

10. 주도성이 모든 것을 가른다 🚀

* High Agency - "우리가 해결할 거야, 일 시작하자" 태도
* 핑계 없음, 허락 기다리지 않음, 쓸데없는 것에 시간 안 씀
* 장애물이 나타나면 밀어붙이는 불도저가 되어라
* 남이 가르쳐주길 기다리지 말고 스스로 배워라
* 과거에 연연하지 말고 앞으로 나아가라

🥧π형 인재 시대의 도래 - T자형은 끝났다

과거 T자형 인재는 하나의 깊은 전문성에 얕고 넓은 지식을 더한 형태였다. 역할이 명확하고 핸드오프가 깔끔할 때는 괜찮았다. PM은 PM 일, 개발자는 개발자 일.

하지만 지금 일은 훨씬 더 지저분하다. 루프는 타이트하고, 아이디어에서 임팩트까지 가는 가장 빠른 길은 여러 분야를 넘나든다. 이젠 π(파이)형이 답이다.

넓은 기반 - 프론트엔드, 백엔드, 데이터, 디자인, 배포를 충분히 말할 수 있고 결정하고 출시할 수 있는 수준

첫 번째 축 - 제품 감각. 취향, 판단력, 애매한 고객 문제를 명확하고 가치있는 결과물로 바꾸는 능력, 디테일에 대한 깊은 주의

두 번째 축 - 엔지니어링 장인정신. 프로덕션 제약 속에서 안전하고 단순하게 실제로 만드는 스킬

AI는 올바른 것을 선택하고, 잘 다듬고, 실제 환경에서 운영하는 비용은 낮추지 못한다. 그건 당신 몫이다. 모델은 증폭기지 관리자가 아니다. 당신이 의도, 아키텍처, 책임을 소유하는 동안 실행을 가속화할 뿐이다.

미래는 풀스택이다. 지금 올라타라.

출처: "You Need To Become A Full Stack Person" by Ian Vanagas (November 3, 2025)

에이전트의 핵심 작동 원리 (The Agent Loop)

목표를 이해하고, 스스로 생각하고, 도구를 사용해 행동하며, 결과를 관찰하여 목표를 달성하는 AI

"목표를 주면, 스스로 생각하고(ReAct), 도구를 사용해 행동하고, 결과를 관찰하여 목표를 달성하는 AI"

1 Goal (목표 입력)

"경쟁사 A의 이번 달 뉴스 기사를 요약해서 보고서 써줘."

2 Thought (계획/추론)

검색이 필요하군.
먼저 검색 키워드를 정하고 계획을 세우자.

3 Action (도구 사용)

웹 검색 | 문서 분석 | 데이터 처리

4 Observation (결과 관찰)

검색 결과를 확인해보니 추가 분석이 필요하군.
(필요 시 2단계로 이동)

5 Final Answer (완료)

최종 목표 달성!
뉴스 요약 보고서를 제출합니다.

결과를 바탕으로 필요한 경우 반복 (ReAct)

1 Goal (목표 입력)

- 사용자의 목표와 요구사항 이해
- 작업 범위와 성공 기준 정의
- 필요한 정보와 제약 조건 파악

예시 "경쟁사 A의 이번 달 뉴스 기사를 요약해서 보고서 써줘."

2 Thought (계획/추론)

- 필요한 작업을 분석하고 계획 수립
- 검색 키워드, 활용 도구, 실행 순서 결정
- 중간 목표 및 대안 전략 고려

예시 "검색 도구가 필요하군. 먼저 검색 키워드를 정하고 계획을 세우자."

3 Action (도구 사용)

- 웹 검색, 문서 분석, 데이터 처리 등 적절한 도구 실행
- 여러 도구를 **순차적/병렬**로 활용
- 외부 시스템과 연동하여 실제 작업 수행

예시 [웹 브라우징 도구 실행] → 경쟁사 A 관련 뉴스 검색

4 Observation (결과 관찰)

- 실행 결과를 확인하고 평가
- 목표 달성 여부 판단
- 부족한 부분이 있으면 2단계로 돌아가 계획을 수정하고 재실행

예시 "검색 결과를 확인해보니 추가 분석이 필요하군. (필요 시 2단계로 이동)"

5 Final Answer (완료)

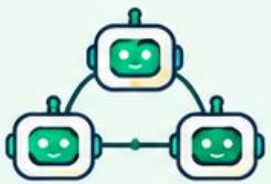
- 최종 목표 달성 및 결과물 생성
- 사용자에게 명확한 형성으로 전달
- 필요 시 추가 제안 및 후속 작업 안내

예시 "경쟁사 A의 이번 달 주요 뉴스 요약 보고서를 작성했습니다."

💡 **단순한 답변 생성이 아닌, 실행(Action)이 가능한 AI**

에이전트 도구, 목적에 따라 어떻게 나뉘는가?

목적과 개발 수준에 따라 에이전트 도구는 크게 세 가지로 분류됩니다.

카테고리	목적	주요 특징	예시 도구
1 **개발 프레임워크** (Agent Framework)	**카스텀 에이전트 설계** 개발자가 직접 에이전트를 설계하고 구축하기 위한 도구 모음	✓ 에이전트 아키텍처 설계 지원 ✓ 다양한 도구와 서비스 연동 ✓ 메모리, 계획, 실행 로직 제공 ✓ 유연한 커스터마이징 가능	LangChain LlamaIndex Crew AI AutoGen (Microsoft)
2 **멀티 에이전트 플랫폼** (Multi-Agent Platform) 	**복잡한 문제 해결** 여러 전문 에이전트가 협업하여 복잡한 워크플로우를 처리	✓ 다중 에이전트 협업 및 역할 분담 ✓ 업무 프로세스 자동화 ✓ 기업용 워크플로우 지원 ✓ 비개발자도 쉽게 사용 가능	MS AutoGen Crew AI LangGraph Vertex AI Agent (Google)
3 **전문화된 서비스/완제품** (Specialized Service)	**특정 직무 대체 및 실행** 톡정 목표(검색, 코딩, 분석 등)를 도구를 사용해 자율적으로 수행하는 완제품 서비스	✓ 목적에 최적화된 에이전트 ✓ 별도 개발 없이 바로 사용 ✓ 도구 연동 및 실제 작업 실행 ✓ 생산성 및 업무 효율성 향상	Perplexity (검색/리서치) Devin (코딩) ChatGPT (범용 업무) Gemini (분석/멀티모달)

참고 : ChatGPT, Claude, Gemini 등의 대화형 AI는 내장된 Tool-Use(함수 호출) 기능을 통해 사용자 목표를 실행하는 에이전트의 역할을 수행합니다.
Perplexity는 자율적인 검색 에이전트에 해당됩니다.

서비스 장애

ZDNET Korea 뉴스 지스타2025 인공지능 스테이블코인 IT's

NordVPN 최대 70% 할인
+ Saily eSIM 데이터 증정

AWS 15시간 먹통 후 복구…업계 "소수 클라우드 집중 구조 한계"

DNS 오류로 전 세계 서비스 마비…복원력·다중 리전 분산 전략 필요성 제기

컴퓨팅 | 입력 :2025/10/21 11:02 수정: 2025/10/21 14:06

한정호 기자 | 기자 페이지 구독 기자의 다른기사 보기

[등록하기] 베리타스와 코헤시티의 통합! 11월 27일 레질리언스 웨비나 등록하고 기념품 받으세요!

세계 최대 클라우드 서비스인 아마존웹서비스(AWS)에 대규모 장애가 발생해 전 세계 주요 인터넷 서비스가 마비됐다. 현재 대부분 복구가 완료됐지만 일부 서비스는 여전히 불안정한 상태로, 인터넷 인프라 집중화의 구조적 취약성이 다시 드러났다는 평가가 나온다.

ACC 2025
효율을 넘어 성과로, AI가 바꾸는 새로운 비즈니스 전략
ACC 2025
2025년 12월 11일(목)
사전등록 >

21일 업계와 주요 외신에 따르면 이번 장애는 지난 20일 미국 버지니아 북부 리전(US-EAST-1)에서 발생했다. AWS의 핵심 데이터베이스(DB) 서비스 '다이나모DB'의 도메인 네임 시스템(DNS) 오류가 근본 원인으로 지목됐다.

DNS는 인터넷의 전화번호부 역할을 하는 핵심 인프라로, 주소(URL)를 실제 서버의 IP로 연결해주는 기능을 한다. 이 시스템이 멈추자 수많은 앱과 웹사이트가 데이터 접근을 잃고 동작을 멈췄다.

Cloudflare 시스템 상태

Cloudflare Global Network에 문제가 발생했습니다.

조사 중 - Cloudflare는 여러 고객에게 영향을 미칠 수 있는 문제를 인지하고 조사 중입니다. 더 자세한 정보는 추가 정보가 확보되는 대로 제공해 드리겠습니다.
2025년 11 월 18일 - 11:48 UTC

지원 포털 가용성 문제

조사 중 - 현재 지원 포털 제공업체에 문제가 발생하여 고객이 지원 사례를 확인하거나 답변하는 데 오류가 발생할 수 있습니다. 고객 문의에 대한 답변은 영향을 받지 않으며, 고객은 Cloudflare 대시보드를 통한 라이브 채팅(비즈니스 및 엔터프라이즈)이나 비상 전화(엔터프라이즈)를 통해 문의하실 수 있습니다.
당사는 문제의 전체적인 영향을 파악하고 완화하기 위해 타사 제공업체와 협력하고 있습니다. 더 자세한 내용은 곧 업데이트될 예정입니다.
2025년 11 월 18일 - 11:17 UTC

LAX(로스앤젤레스) 2025-11-18

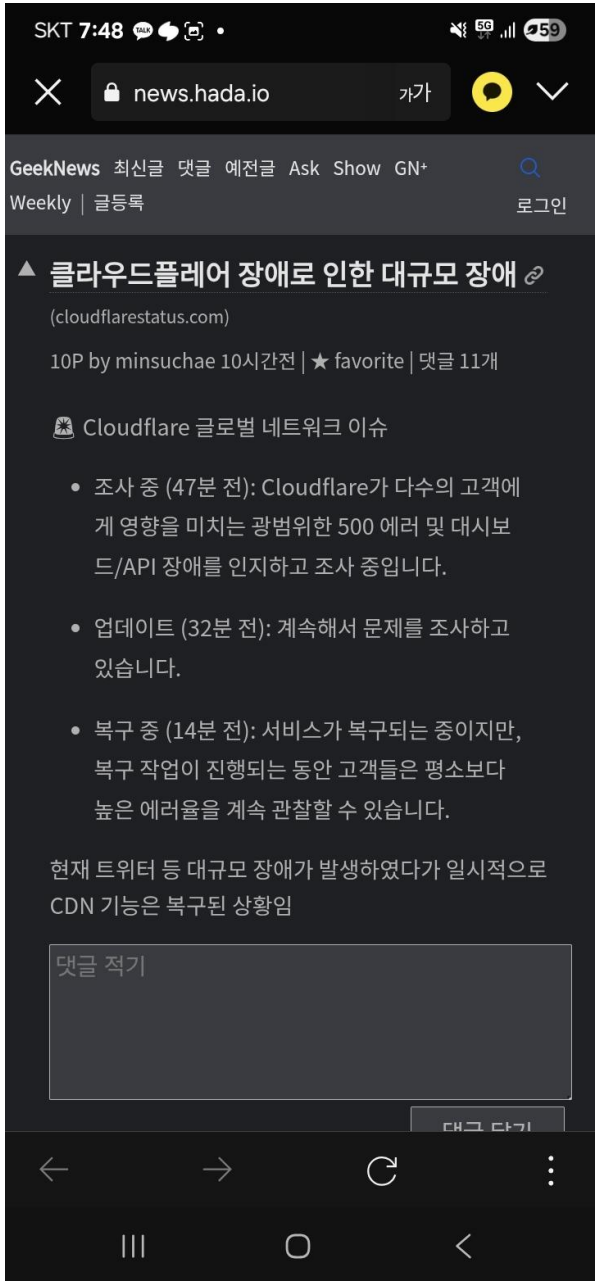
진행 중 - 현재 예정된 유지 관리가 진행 중입니다. 필요한 경우 업데이트를 제공해 드리겠습니다.
2025년 11 월 18일 - 10:00 UTC

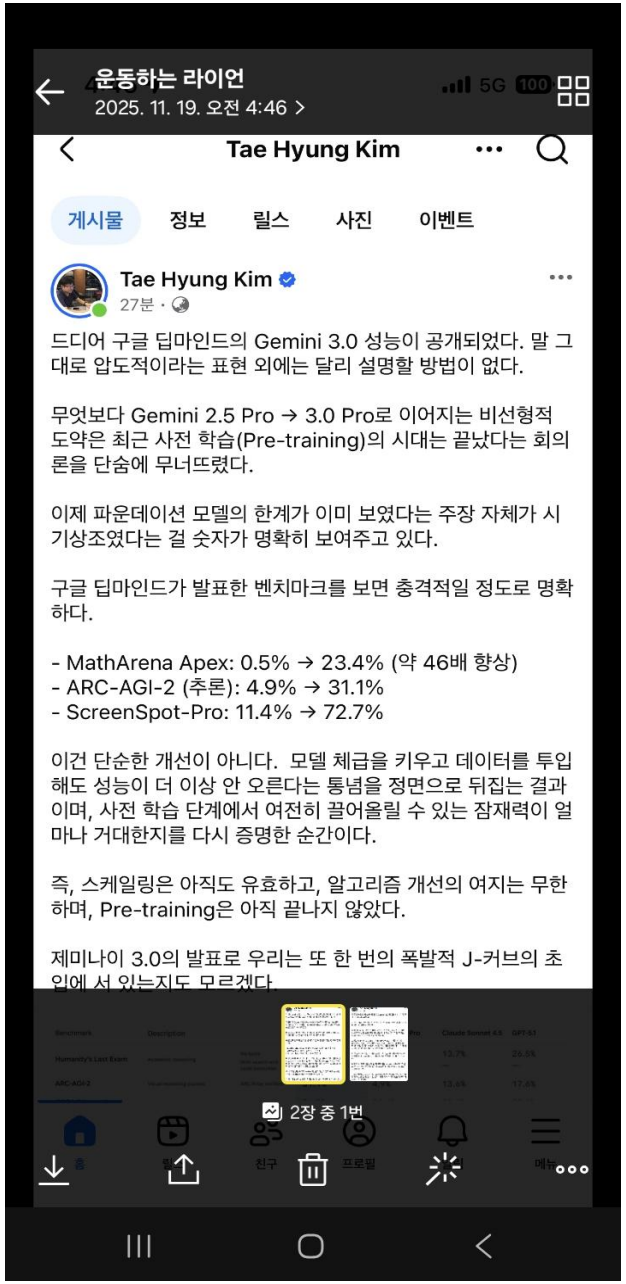
예정 - 2025년 11월 18일 오전 10시에서 오후 2시(UTC) 사이에 LAX(로스앤젤레스) 데이터 센터에서 예정된 유지 관리가 진행됩니다.

이 위치에서 트래픽이 재라우팅될 수 있으므로, 영향을 받는 지역의 최종 사용자는 유지 관리 기간 동안 지연 시간이 약간 증가할 수 있습니다. 이 위치에서 당사에 접속하는 PNI/CNI 고객의 경우, 이 데이터 센터의 네트워크 인터페이스를 일시적으로 사용할 수 없게 될 수 있으므로 유지 관리 기간 동안 트래픽이 다른 곳에서 장애 조치될 것으로 예상해야 합니다.

이제 Cloudflare 대시보드를 통해 이러한 알림을 구독하고 이메일, PagerDuty 및 웹훅(플랜에 따라 다름)을 통해 직접 업데이트를 받을 수 있습니다. https://developers.cloudflare.com/notifications/notification-available/#cloudflare-

서비스 장애


SKT 7:48
news.hada.io
가가
GeekNews 최신글 댓글 예전글 Ask Show GN+
Weekly | 글등록
로그인
클라우드플레어 장애로 인한 대규모 장애
(cloudflarestatus.com)
10P by minsuchae 10시간전 | ★ favorite | 댓글 11개
Cloudflare 글로벌 네트워크 이슈
조사 중 (47분 전): Cloudflare가 다수의 고객에게 영향을 미치는 광범위한 500 에러 및 대시보드/API 장애를 인지하고 조사 중입니다.
업데이트 (32분 전): 계속해서 문제를 조사하고 있습니다.
복구 중 (14분 전): 서비스가 복구되는 중이지만, 복구 작업이 진행되는 동안 고객들은 평소보다 높은 에러율을 계속 관찰할 수 있습니다.
현재 트위터 등 대규모 장애가 발생하였다가 일시적으로 CDN 기능은 복구된 상황임
댓글 적기

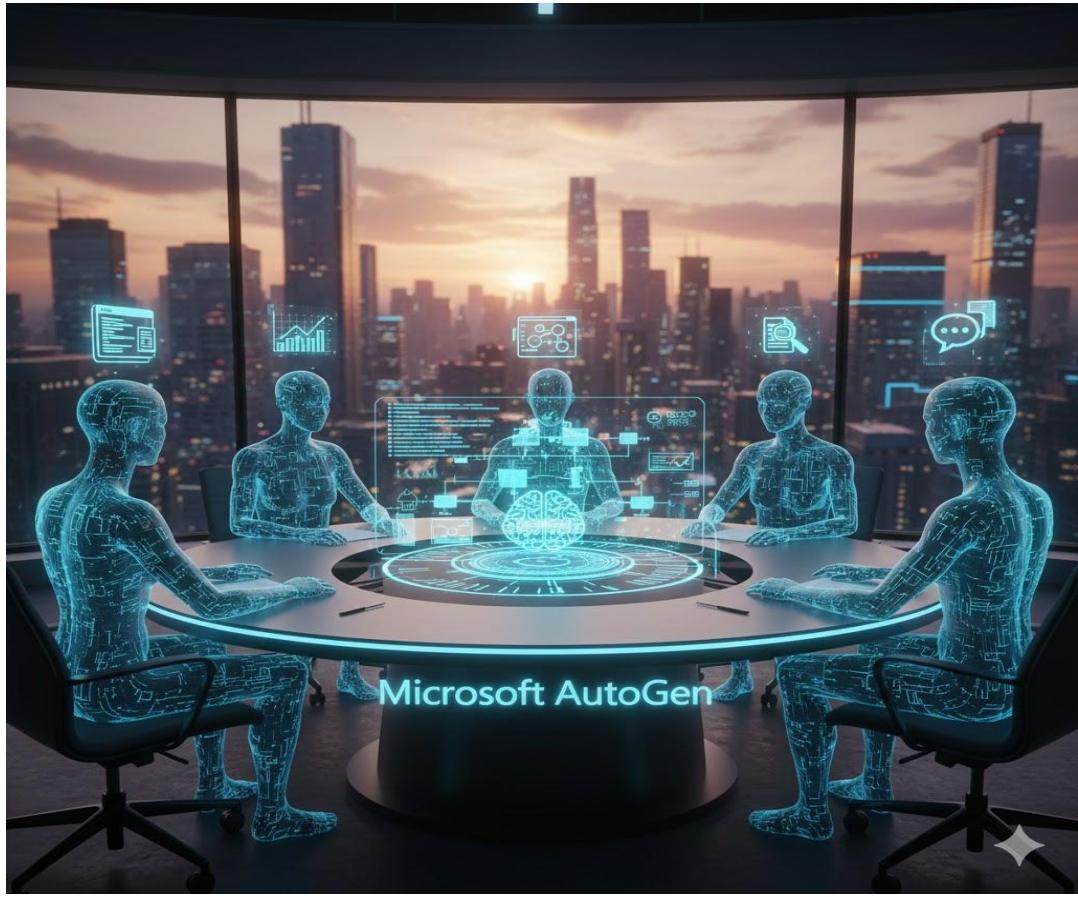

운동하는 라이언
2025. 11. 19. 오전 4:46
Tae Hyung Kim
게시물 정보 릴스 사진 이벤트
Tae Hyung Kim
27분 ·
드디어 구글 딥마인드의 Gemini 3.0 성능이 공개되었다. 말 그대로 압도적이라는 표현 외에는 달리 설명할 방법이 없다.
무엇보다 Gemini 2.5 Pro → 3.0 Pro로 이어지는 비선형적 도약은 최근 사전 학습(Pre-training)의 시대는 끝났다는 회의론을 단숨에 무너뜨렸다.
이제 파운데이션 모델의 한계가 이미 보였다는 주장 자체가 시기상조였다는 걸 숫자가 명확히 보여주고 있다.
구글 딥마인드가 발표한 벤치마크를 보면 충격적일 정도로 명확하다.
- MathArena Apex: 0.5% → 23.4% (약 46배 향상)
- ARC-AGI-2 (추론): 4.9% → 31.1%
- ScreenSpot-Pro: 11.4% → 72.7%
이건 단순한 개선이 아니다. 모델 체급을 키우고 데이터를 투입해도 성능이 더 이상 안 오른다는 통념을 정면으로 뒤집는 결과이며, 사전 학습 단계에서 여전히 끌어올릴 수 있는 잠재력이 얼마나 거대한지를 다시 증명한 순간이다.
즉, 스케일링은 아직도 유효하고, 알고리즘 개선의 여지는 무한하며, Pre-training은 아직 끝나지 않았다.
제미나이 3.0의 발표로 우리는 또 한 번의 폭발적 J-커브의 초입에 서 있는지도 모르겠다.
2장 중 1번

Microsoft AutoGen(멀티 에이전트 조율)

- 멀티 에이전트 AI 프레임워크로, 여러 AI Agent가 역할을 나눠 협업하여 자동으로 문제를 해결하는 구조

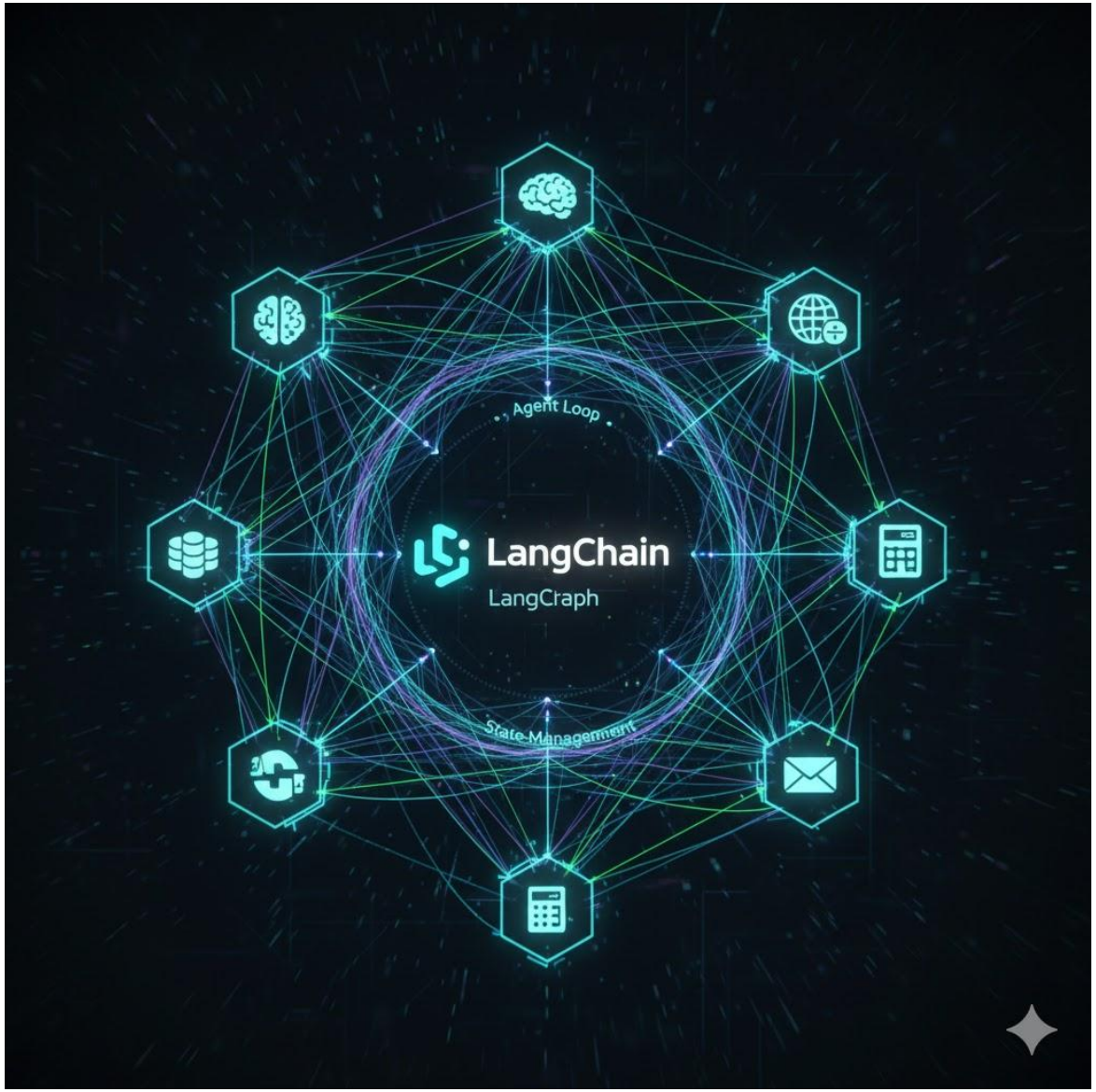
  - '코더', '리뷰어'등 역할을 부여하고 에이전트끼리 대화하며 문제 해결.

- 사람 - AI 유기적 협업 : 필요 시 사람에게 질문하며 막힘없이 작업 진행
- 강력한 코드 실행 : 복잡한 코딩 및 데이터 분석 작업에 특화
- <추천대상> 복잡한 워크플로우를 자동화하려는 기업 개발팀, 데이터 과학자



LangChain / LangGraph(개발 표준)

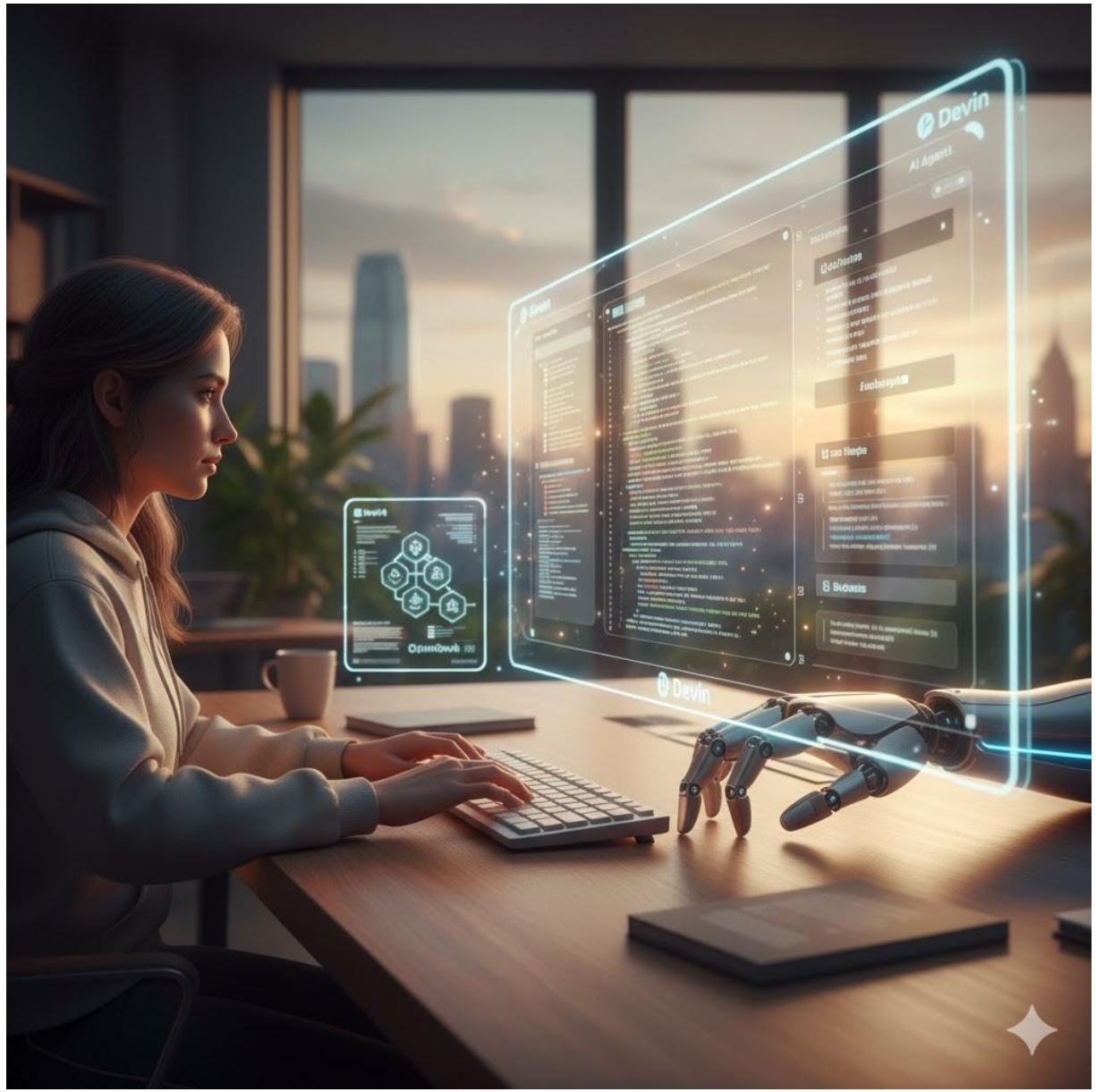
- 방대한 생태계 : 모든 LLM, DB, 외부 도구와 연결 가능한 컴포넌트 제공
- LangChain은 LLM 애플리케이션을 체계적으로 구성하기 위한 표준 프레임워크.
- LangGraph는 그 위에서 상태 기반(Stateful) 에이전트 그래프를 구성해 복잡한 워크플로우를 안정적으로 실행
- <추천 대상> 자체 AI 애플리케이션을 구축하고 모든 것을 제어하려는 AI 엔지니어, 연구팀



Devin & 대안들 (자율형 AI 직원의 미래)

- 완전 자율성 : 계획, 코딩, 디버깅, 배포까지 스스로 수행하는 AI 엔지니어
- 장기 기억 및 학습 : 프로젝트 맥락을 이해하고 과거의 실수로부터 학습 가능.
- 시사점 : 미래 업무는 “사람”과 “AI 에이전트 직원”의 협업 형태로 진화

- <참고> 아직 초기 단계이며 OpenDevin 등 오픈소스 대안들이 등장 중.



Genspark(개인화된 AI 검색/비서)

- 차세대 AI 검색 : 단순 정보 제공을 넘어 사용자 의도를 깊이 이해
- 개인화된 요약 제공 : 여러 소스를 종합하여 맞춤형으로 간결하게 정보 정리
- 맥락 유지 대화 : 비서처럼 이전 질문 맥락을 기억하고 대화를 이어감
- <추천 대상> 개인화된 정보 큐레이션과 효율적인 검색 경험을 원하는 일반 사용자, 연구자

Skywork(생성형 AI를 위한 데이터 플랫폼)

- 데이터 품질 극대화 : 생성형 AI 모델 성능을 위한 고품질 데이터셋 구축.
- 파이프라인 자동화 : 데이터 수집, 정제, 라벨링, 모델 통합 과정 최적화
- 비정형 데이터 처리 : 텍스트, 이미지, 오디오 등 다양한 형태의 데이터 처리 지원
- <추천 대상> 생성형 AI 모델을 개발 및 학습시키는 AI 연구기관, 기업의 AI 개발팀

핵심문제 : “개발자 공급 절벽”

- **폭발적인 앱 수요:** 향후 5년 내 5억 개의 새로운 앱이 필요할 것으로 예상됩니다. (지난 40년간 개발된 총량보다 많음)
- **만성적인 인력 부족:** 숙련된 개발자 채용은 하늘의 별 따기이며, 인건비는 지속적으로 상승하고 있습니다.
- **느린 개발 속도:** 전통적인 코딩 방식으로는 급변하는 비즈니스 요구사항을 따라잡기에 한계가 있습니다.

해결책: 노코드(No-Code)와 로코드(Low-Code)

No-Code (노코드)

대상: 현업 비즈니스 실무자 (마케터, HR 등)

특징: 코딩 지식 0% 필요. 드래그 앤 드롭 방식의 시각적 인터페이스 사용.

목표: 아이디어를 즉시 앱으로 구현하여 업무 효율성 증대.

Low-Code (로코드)

대상: 전문 개발자 및 파워 유저

특징: 최소한의 코딩으로 복잡한 기능을 구현. 반복 작업을 자동화.

목표: 개발 속도를 극대화하고 유지보수 비용 절감.

가속화 요인 1: 압도적인 속도(Velocity)

기존 개발 방식 대비 10배 더 빠른 속도를 자랑합니다.

수개월이 걸리던 프로젝트를 단 며칠 만에 런칭할 수 있습니다.

- **즉각적인 배포:** 아이디어 구상에서 실행까지 Time-to-Market 최소화
- **빠른 수정 및 반복:** 피드백을 실시간으로 반영하여 제품 개선
- **자동화된 인프라:** 서버 설정, 보안 등 복잡한 백엔드 작업 생략

가속화 요인 2: 개발의 대중화(Access)

'시티즌 디벨로퍼'의 부상

이제 누구나 개발자가 될 수 있습니다. IT 부서에 의존하지 않고 현업 담당자가 직접 문제를 해결하는 도구를 만듭니다.

- **IT 병목 현상 해소**: 단순 반복 업무 앱은 현업에서 직접 해결
- **다양성 확보**: 다양한 직군의 관점이 반영된 창의적 솔루션 탄생
- **협업 강화**: 비즈니스팀과 IT팀 간의 간극 축소

가속화 요인 3: 본질에 집중(Focus)

"The future of coding is no coding at all."

— Chris Wanstrath, CEO of GitHub

기술적 구현(How)의 장벽이 무너지고,

무엇을 해결할 것인가(What)에 집중하는 시대입니다.

개발 방식 비교: 전통적 방식 VS 노코드

구분	전통적 개발 (Traditional)	노코드/로코드 (No/Low-Code)
개발 속도	느림 (수개월 ~ 수년)	매우 빠름 (수일 ~ 수주)
필요 기술	전문 코딩 지식 (Java, Python 등)	비즈니스 로직 이해, 시각적 도구 활용
비용	높음 (인건비, 인프라 비용)	낮음 (구독형, 유지보수 용이)
주체	전문 개발자	시티즌 디벨로퍼 (현업 실무자)

미래 전망

70%

신규 앱 개발 비중

2025년까지 기업에서 개발되는 신규 앱의 70%가 노코드/로코드 기술을 사용할 것입니다. (Gartner)

$187B

시장 규모 (2030)

전 세계 노코드/로코드 시장은 2030년까지 약 1,870억 달러 규모로 폭발적인 성장을 할 것입니다.

4 : 1

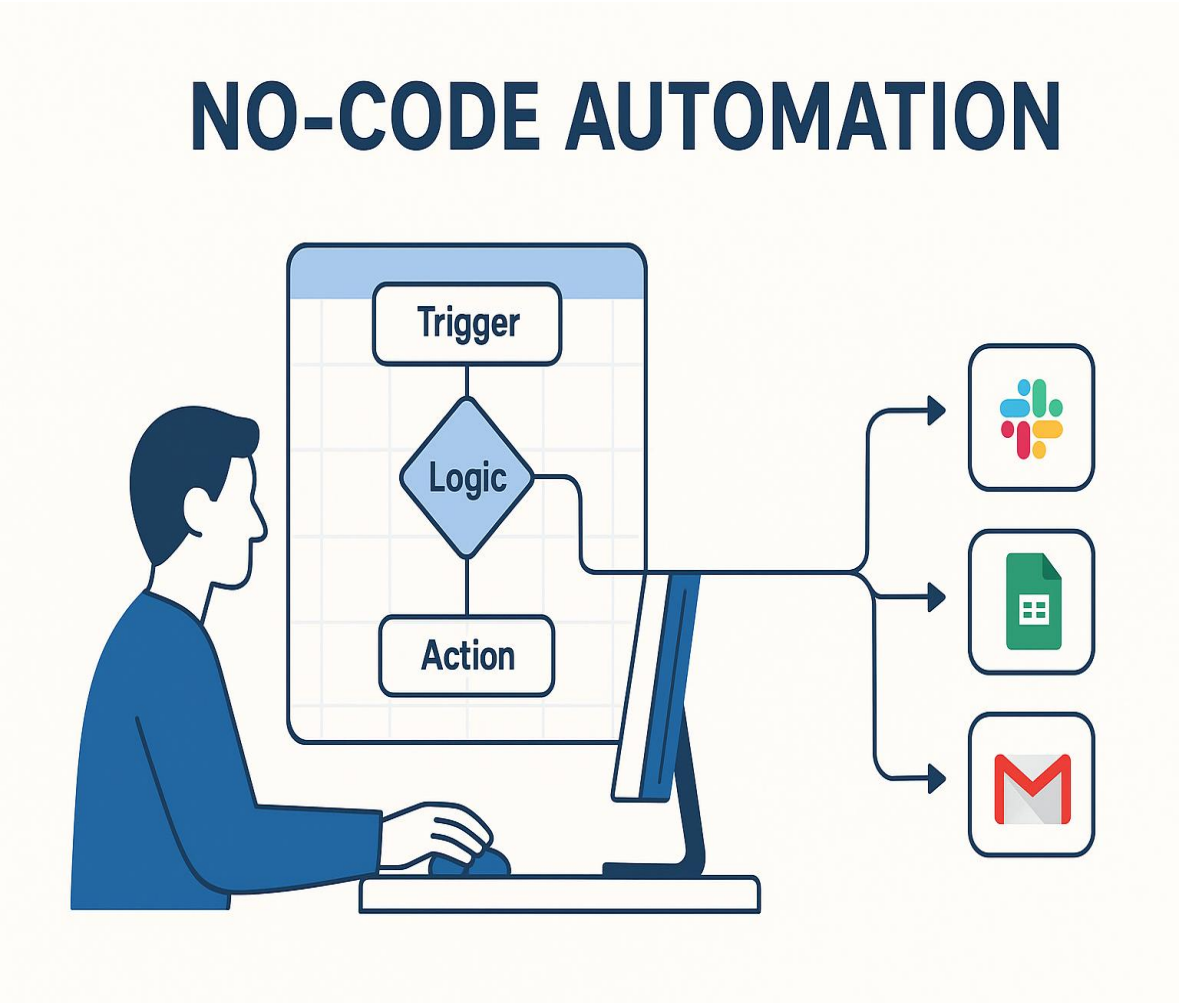
개발자 비율

대기업 내에서 시티즌 디벨로퍼의 수가 전문 개발자보다 4배 더 많아질 것입니다.

No-Code Automation

- 반복 업무 자동화를 통해 개발 리소스 부족 문제를 해결하는 핵심 기술

- 코딩 없이 **드래그 앤 드롭 방식**으로 자동화 워크플로우를 구축하는 기술
- “메일 발송, 로그 정리, 승인 요청, DB 업데이트 등 반복적인 오퍼레이션 업무의 30~50%를 자동화 가능
- Trigger: 자동화를 시작시키는 신호(예. 제출 폼, 스케줄, 웹 훅)
- Logic : 조건을 판단하고, 데이터를 분기(If, loop, Filtering)
- Action : 실제 실행(메일 전송, DB Write, Slack 알림, API Call)



No-Code Automation: 4대 핵심 가치

1. 초고속 구축 (Speed)

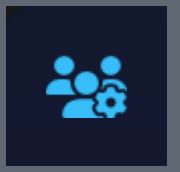
복잡한 코딩 과정 없이, 아이디어를 즉시 실행 가능한 워크플로우로 전환합니다. 개발 기간을 몇 주에서 몇 분으로 단축시킵니다.

2. 무한 연결 (Connectivity)

이메일, 슬랙, 엑셀 등 서로 다른 수천 개의 앱(SaaS)을 레고 블록처럼 자유롭게 연결하여 데이터 사일로를 제거합니다.

3. 업무 효율 (Efficiency)

단순 반복 업무를 24시간 365일 자동화하여, 사람은 더 창의적이고 전략적인 고부가가치 업무에 집중할 수 있게 합니다.



4. 역량 강화 (Empowerment)

'시티즌 디벨로퍼'의 시대. 개발자가 아니더라도 현업 실무자가 직접 자신의 문제를 기술로 해결하는 힘을 갖게 됩니다.

- SaaS(Software as a Service): “인터넷을 통해 바로 사용하는 구독형 소프트웨어 서비스”

2026 AI Coding Assistant Landscape

STRATEGY & TRENDS

마지막 업데이트: 2026년 10월 25일

DeepSeek-V4.1

DISRUPTIVE EFFICIENCY

- 고성능 · 저비용 AI 모델
  2026년 9월에 공개되었고, 효율적인 MoE 아키텍처를 활용해 추론 비용 절감
- 코딩 및 추론 성능 강화
  코드 생성 및 소프트웨어 개발 벤치마트 성능 개선
- 개방형 AI 생태계 확대
  기업 맞춤형 AI 서비스 구축에 활용

GPT-6(ChatGPT)

THE AGENTIC CODING ENGINE

- GPT-6 기반 고급 코딩
  복잡한 코드 생성, 수정 및 디버깅 지원
  대규모 코드베이스 분석과 개발 작업 수행
- Codex 기반 에이전트 개발
  자연어 요구사항을 코드로 구현
  여러 단계의 개발 작업을 자동화
- 클라우드 기반 개발 워크플로
  Codex Cloud를 통한 코딩 작업 수행
  개발자와 AI Agent의 협업 확대

Gemini 3 & Nano

MULTIMODAL DEVELOPMENT

- 대규모 문맥 이해
  긴 코드와 복잡한 문서 분석
  프로젝트 구조를 고려한 코드 생성 지원
- 멀티 모달 기반 개발
  텍스트,이미지 등 다양한 입력 활용
  문서와 코드의 통합 분석 지원
- Google 개발 생태계 연계
  Google AI studio 및 Gemini API 연동

Cursor IDE

AI-NATIVE DEVELOPMENT

- Agent 기반 자율 코딩
  프로젝트 전체 코드베이스 탐색
  여러 파일을 동시에 수정
- Backgroud Agent
  원격 개발 환경에서 코딩 작업 수행
  작업 진행 상황 및 후속 지시 가능
- 모델 선택과 개발 환경 통합
  개발 목적에 맞는 AI 모델 활용
  코드 작성부터 리팩터링, 테스트까지 통합 지원

2026 전략 요약: 단순 보조에서 자율적 시스템 구축(Autonomous Agent)으로의 패러다임 전환

2026 AI Coding Assistant Landscape

STRATEGY & TRENDS

마지막 업데이트: 2026년 10월 25일

Claude Opus 5.5

ADVANCED REASONING & AGENTIC CODING

- **고급 추론 및 코딩 성능**
  복잡한 프로그래밍과 다단계 추론 작업 지원
- **Claude code 기반 개발 자동화**
  자연어 명령을 통한 코드 생성, 수정,실행 테스트, 디버깅, 반복 수정 워크플로 지원
- **비용 효율성과 Agent 활용**
  장시간 수행하는 복잡한 개발 작업 활용 전문적인 지식 업무와 Agent 기반 자동화 지원

Grok 4.7

REAL-TIME INTELLIGENCE & CODING

- **실시간 정보 활용**
  X(트위터) 플랫폼의 실시간 데이터를 기반으로 한 최신 트렌드 즉각 반영
- **고급 추론 및 코딩**
  소프트웨어 개발 및 지식 업무 성능 강화
- **Grok Build 및 Agent 생태계**
  작업 맥락과 프로젝트 정보 유지 Grok Build를 통한 AI 기반 개발 작업

GitHub Copilot

AGENT SOFTWARE DEVELOPMENT

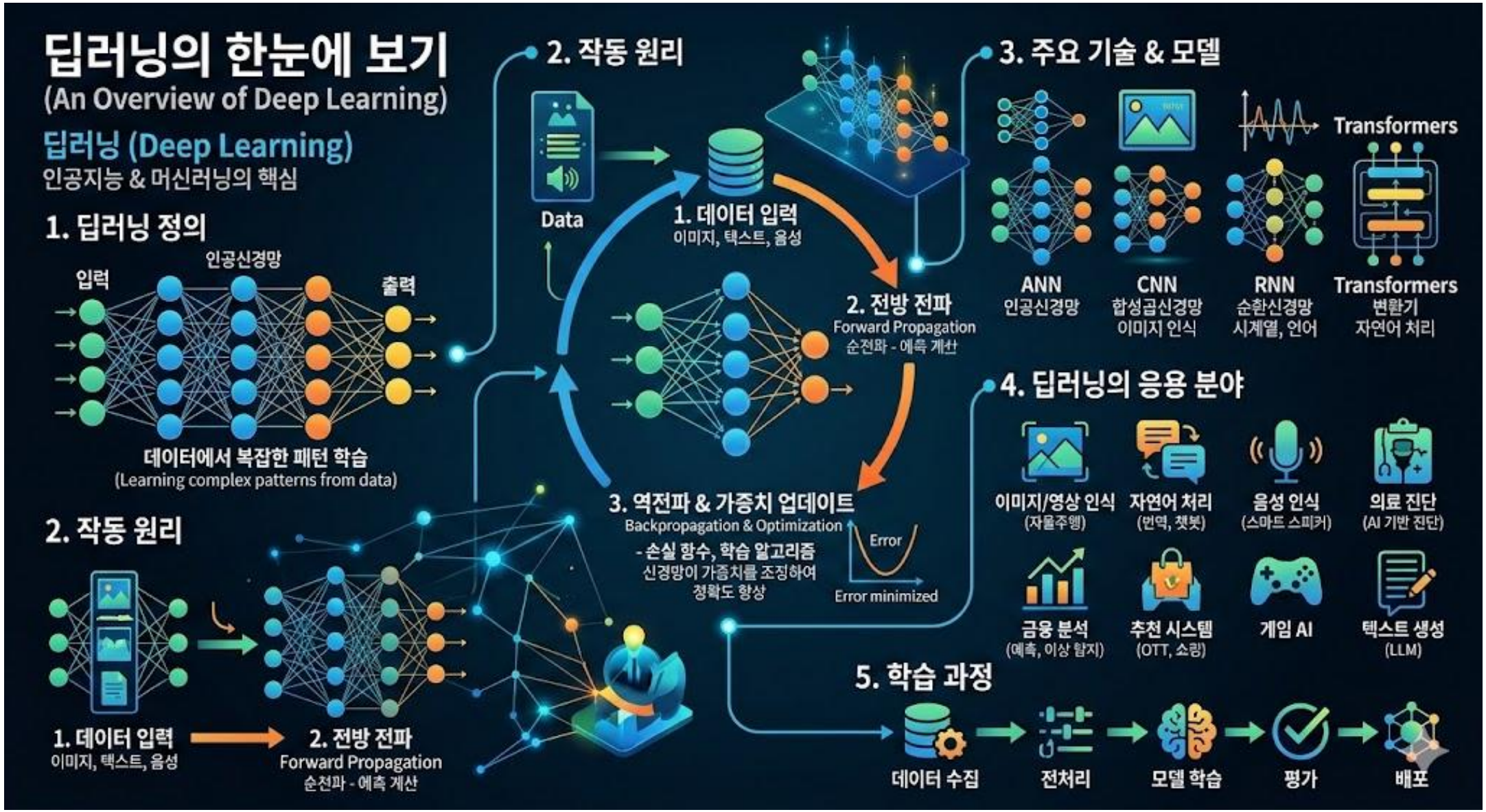
- **개발 환경 통합**
  VS Code, GitHub와 완벽히 연결된 에디터-PR 통합 개발 경험
- **에이전트 중심 개발**
  개발 작업을 AI Agent에 위임 코드 수정, 테스트, 오류 해결 자동화
- **멀티에이전트 개발 환경**
  여러 개발 작업을 독립적으로 수행 코드 리뷰 및 개발 워크플로 자동화

Perplexity Pro

SEARCH TO SOLUTION

- **근거 기반 AI 검색**
  최신 웹 검색 및 기술 문서 분석 출처를 제시하는 답변 생성
- **Pro Search 및 심층 조사**
  복잡한 질문에 대한 단계별 조사 다양한 출처를 종합한 분석
- **Research & Agentic Workflow**
  여러 단계에 걸친 자료 조사 및 분석 문서, 보고서 작성 지원

2026 전략 인사이트: 단순 코드 생성을 넘어 현장 적용(Artifacts)과 실시간성(Real-time)이 핵심 차별점


딥러닝의 한눈에 보기
(An Overview of Deep Learning)
딥러닝 (Deep Learning)
인공지능 & 머신러닝의 핵심
1. 딥러닝 정의
입력
인공신경망
출력
데이터에서 복잡한 패턴 학습
(Learning complex patterns from data)
2. 작동 원리
Data
1. 데이터 입력
이미지, 텍스트, 음성
2. 전방 전파
Forward Propagation
순전파 - 예측 계산
3. 역전파 & 가중치 업데이트
Backpropagation & Optimization
- 손실 함수, 학습 알고리즘
신경망이 가중치를 조정하여
정확도 향상
Error
Error minimized
3. 주요 기술 & 모델
ANN
인공신경망
CNN
합성곱신경망
이미지 인식
RNN
순환신경망
시계열, 언어
Transformers
Transformers
변환기
자연어 처리
4. 딥러닝의 응용 분야
이미지/영상 인식
(자율주행)
자연어 처리
(번역, 챗봇)
음성 인식
(스마트 스피커)
의료 진단
(AI 기반 진단)
금융 분석
(예측, 이상 탐지)
추천 시스템
(OTT, 쇼핑)
게임 AI
텍스트 생성
(LLM)
2. 작동 원리
1. 데이터 입력
이미지, 텍스트, 음성
2. 전방 전파
Forward Propagation
순전파 - 예측 계산
5. 학습 과정
데이터 수집
전처리
모델 학습
평가
배포

인공지능 분야_지식 기반 시스템

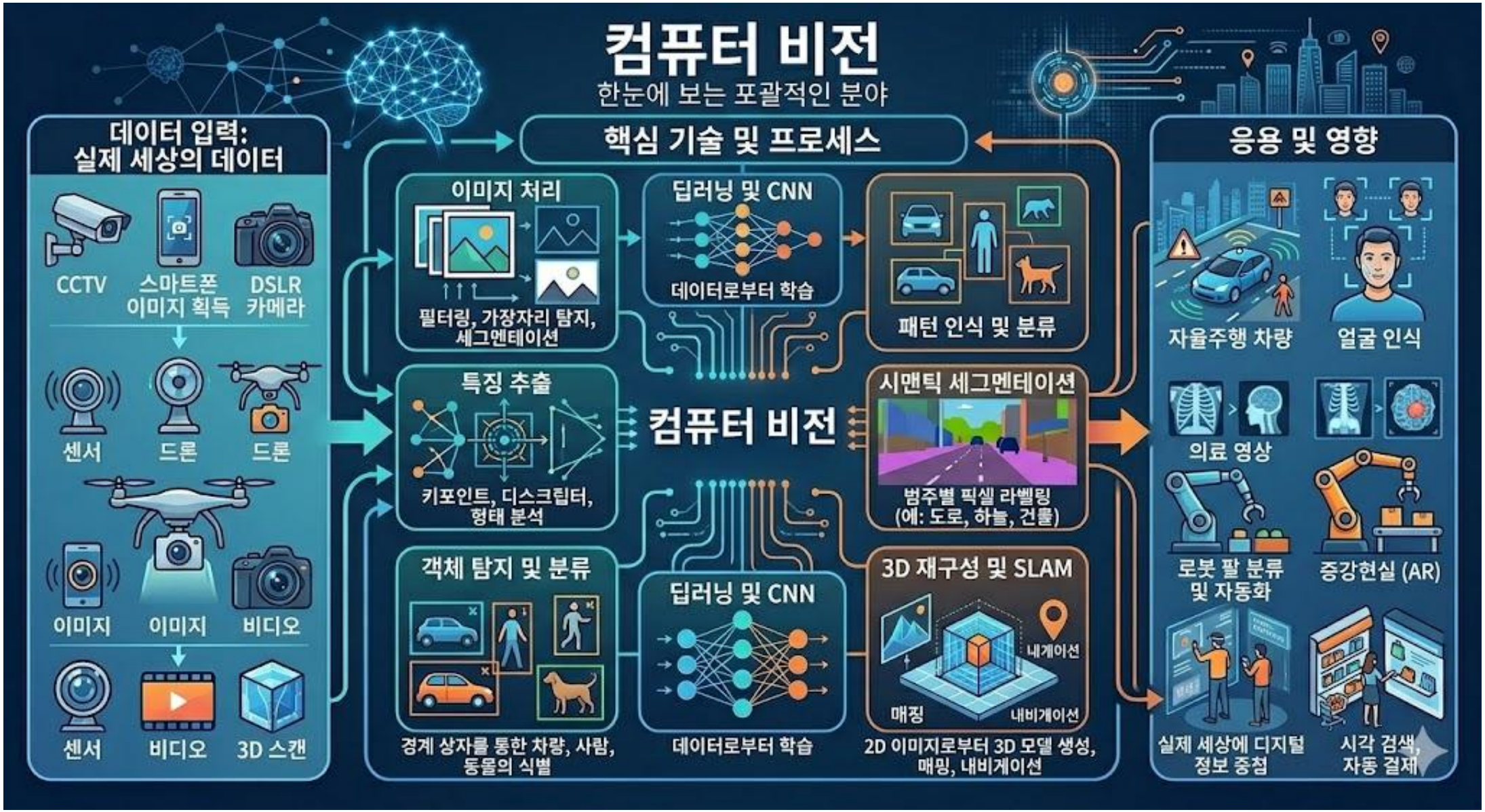
인공지능 분야_지식 기반 시스템

'3시에 자살' 음성 인식하면 한국판 시리가 내놓는 대답 "알겠어요"

중앙일보 | 입력 2018.06.27 01:00 지면보기

이에스더 기자 구독

인공지능 분야_컴퓨터 비전



인공지능 분야_ 컴퓨터 비전

2019.09.21

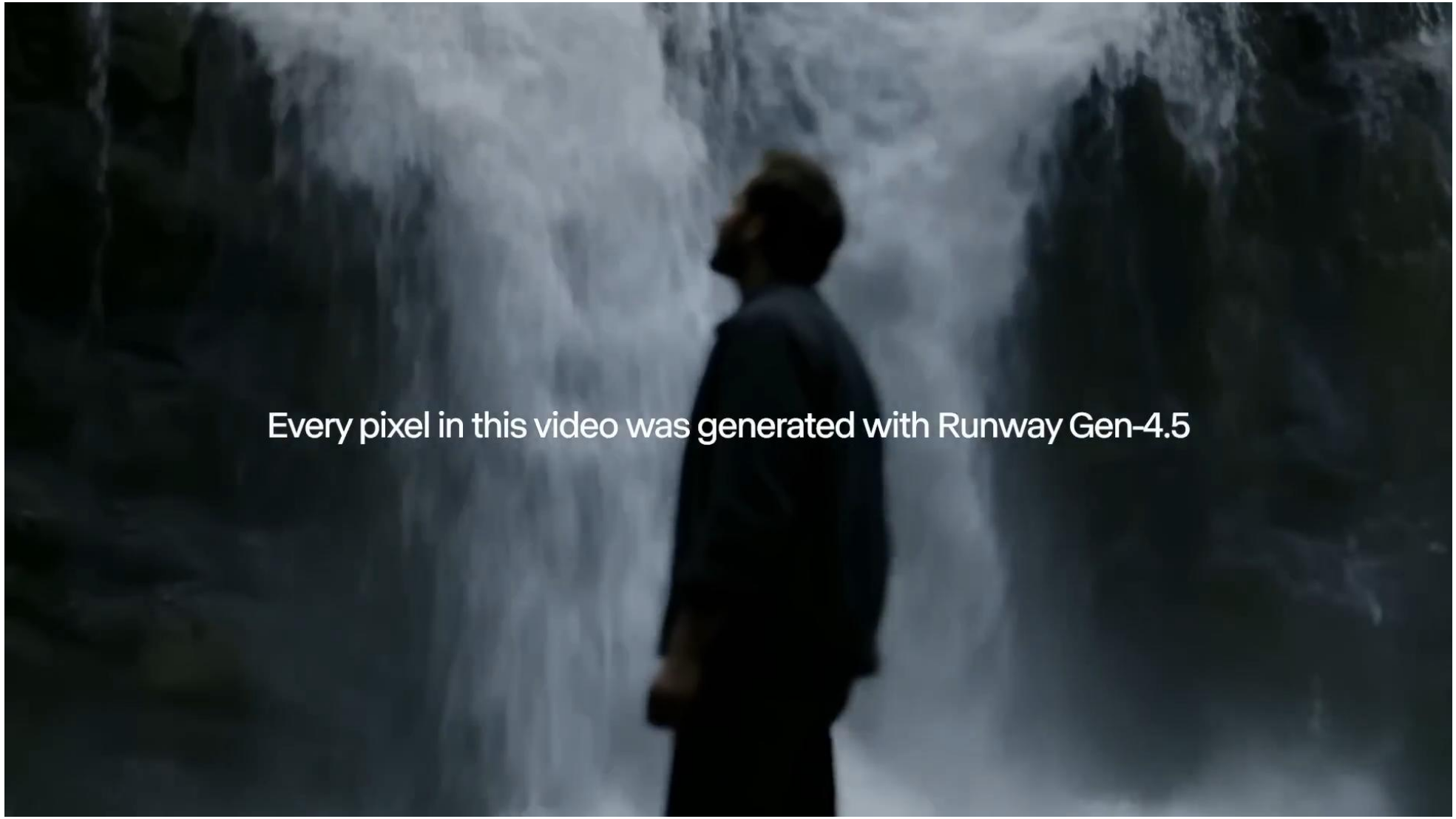
인공지능 분야_ 컴퓨터 비전

2025.03.05

인공지능 분야_ 컴퓨터 비전

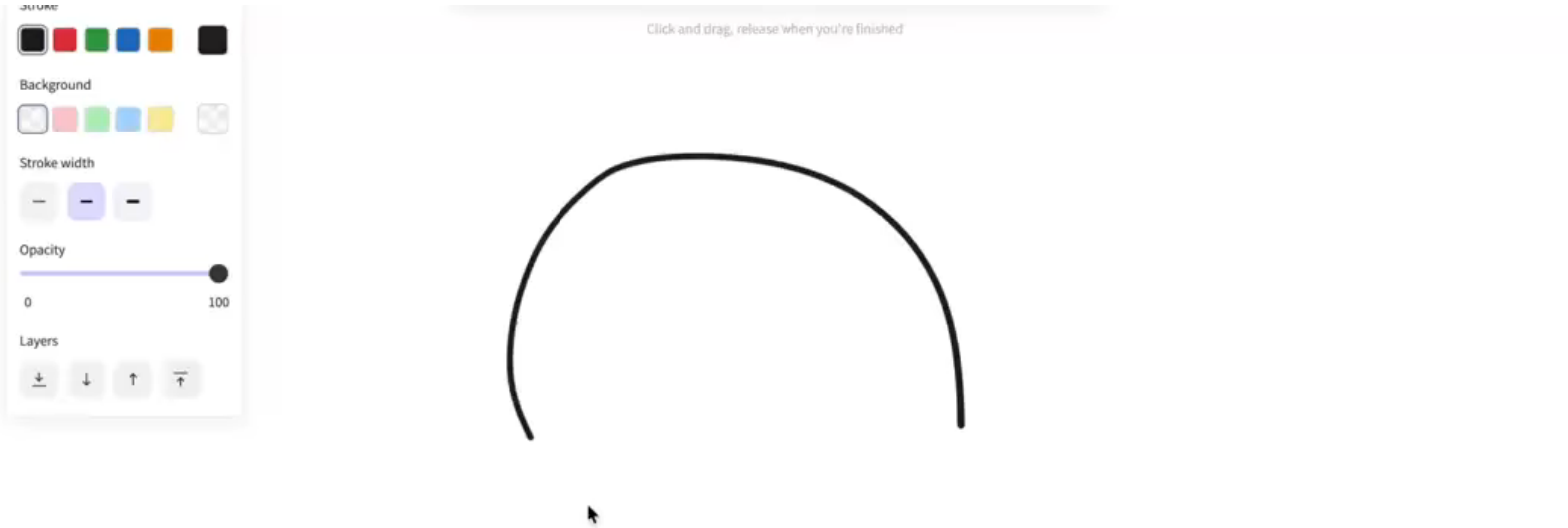
인공지능 분야_ 컴퓨터 비전

Runway Gen-4.5: 텍스트 프롬프트만으로 “시메나틱 수준”의 영상 수준 가능 (2025.12.02에 발표된 영상), 움직임, 물리 효과, 유체 역학까지 꽤 정교하게 구현된다는 평가를 받고 있음



인공지능 분야_ 컴퓨터 비전

Hedra AI: 2024년에 런칭 되었고, 2025년 5월에 시리즈A 투자를 받음



인공지능 분야_ 컴퓨터 비전

Google Veo 3: 사진이나 정지 이미지 + 음성/텍스트 만으로 “움직이는 캐릭터 영상(말하는 얼굴 혹은 인물)”을 자동 생성해줌. 영상 제작 경험이 없거나 3D 모델링을 모르는 사람도 손쉽게 영상 콘텐츠 제작 가능

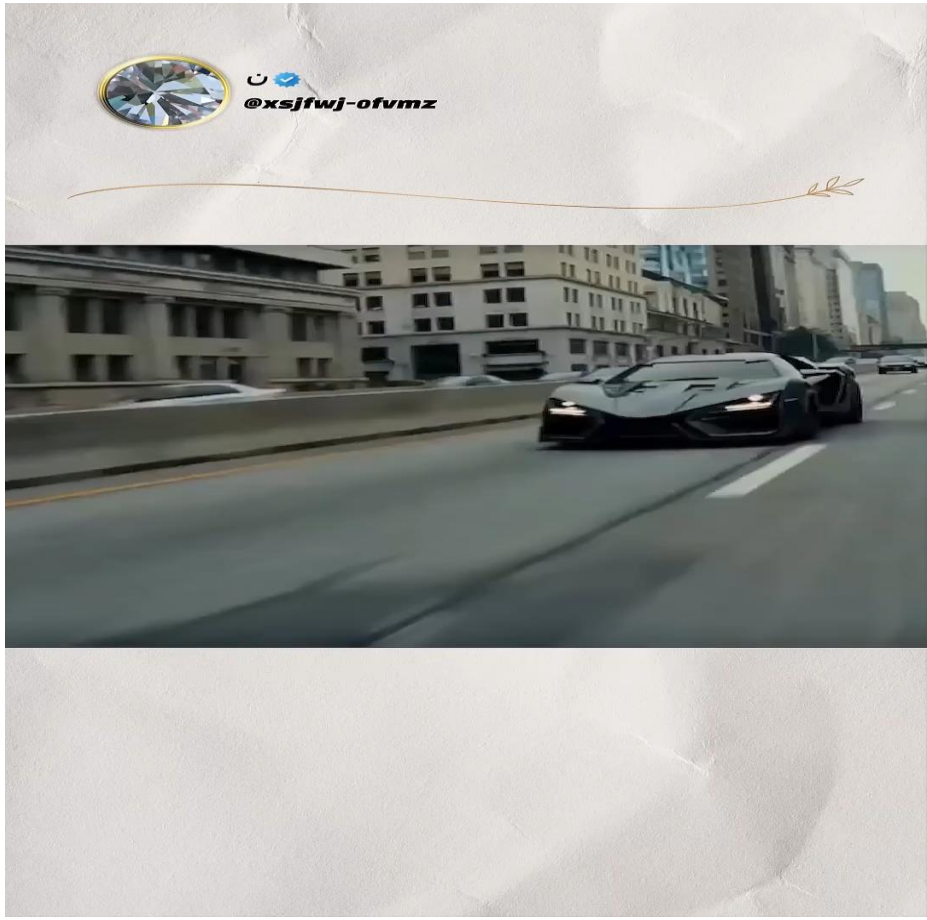
인공지능 분야_ 컴퓨터 비전

Kling 3.0: Kuaishou의 Kling AI가 2026년 2월 공개한 최신 모델 라인업으로, 생성 영상의 일관성(consistency)과 사실감(photorealism)을 강화하고 **최대 15초 길이**의 영상 생성 등을 주요 업그레이드로 내세움.

출처 : https://www.youtube.com/watch?v=l2fsWXlGCjw&t=866s

인공지능 분야_ 컴퓨터 비전

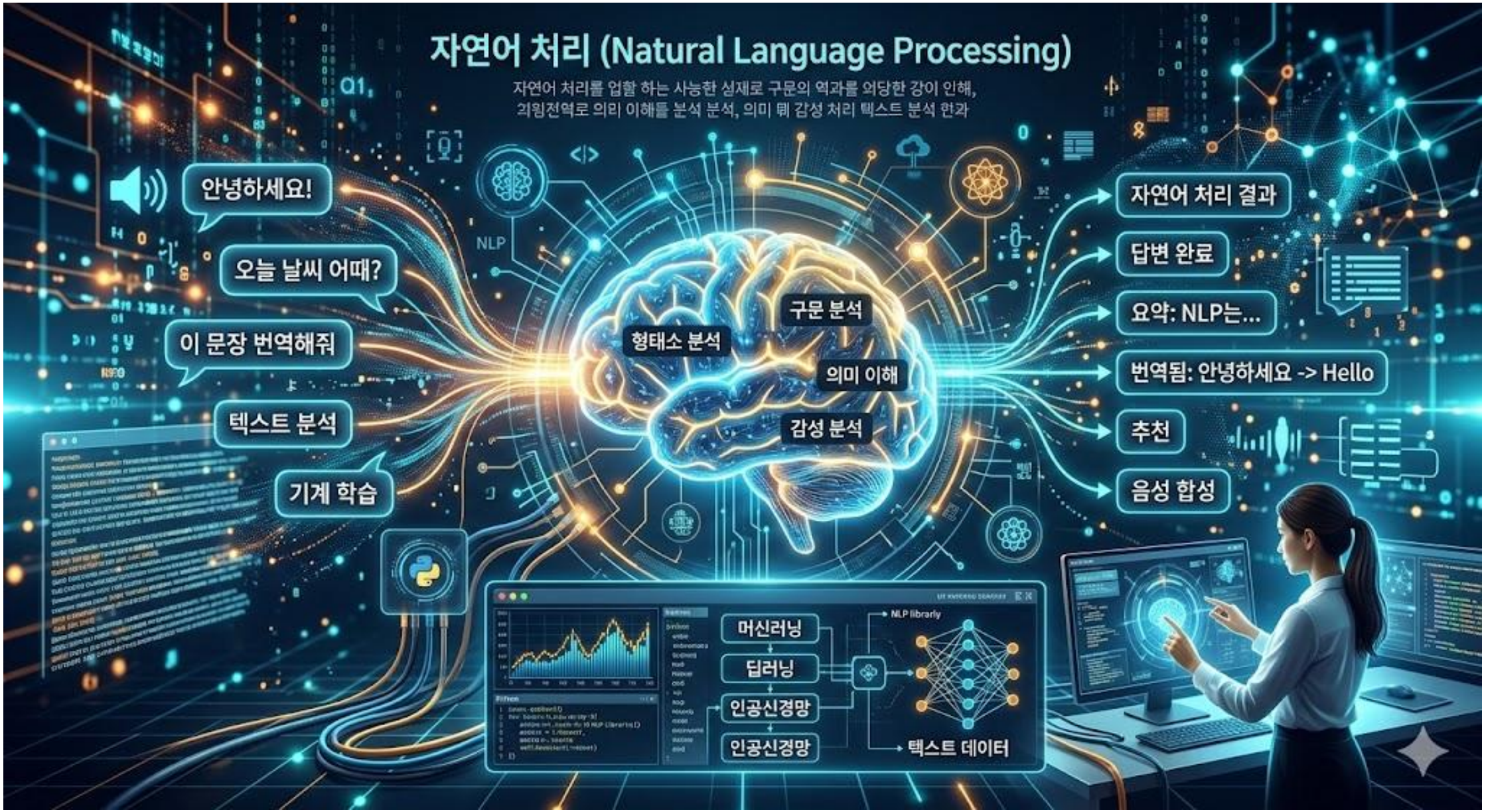
Seedance 2.0: ByteDance가 2026년 2월 공개한 영상 생성 모델로, **모션 안정성**과 오디오-비디오 공동 생성(audio-video joint generation)을 핵심 특징으로 내세움. 또한 **텍스트뿐 아니라 이미지·오디오·비디오를 참조**해 성능(퍼포먼스), 조명, 그림자, 카메라 움직임까지 “감독급 제어”를 지원하는 방향으로 소개됩니다.



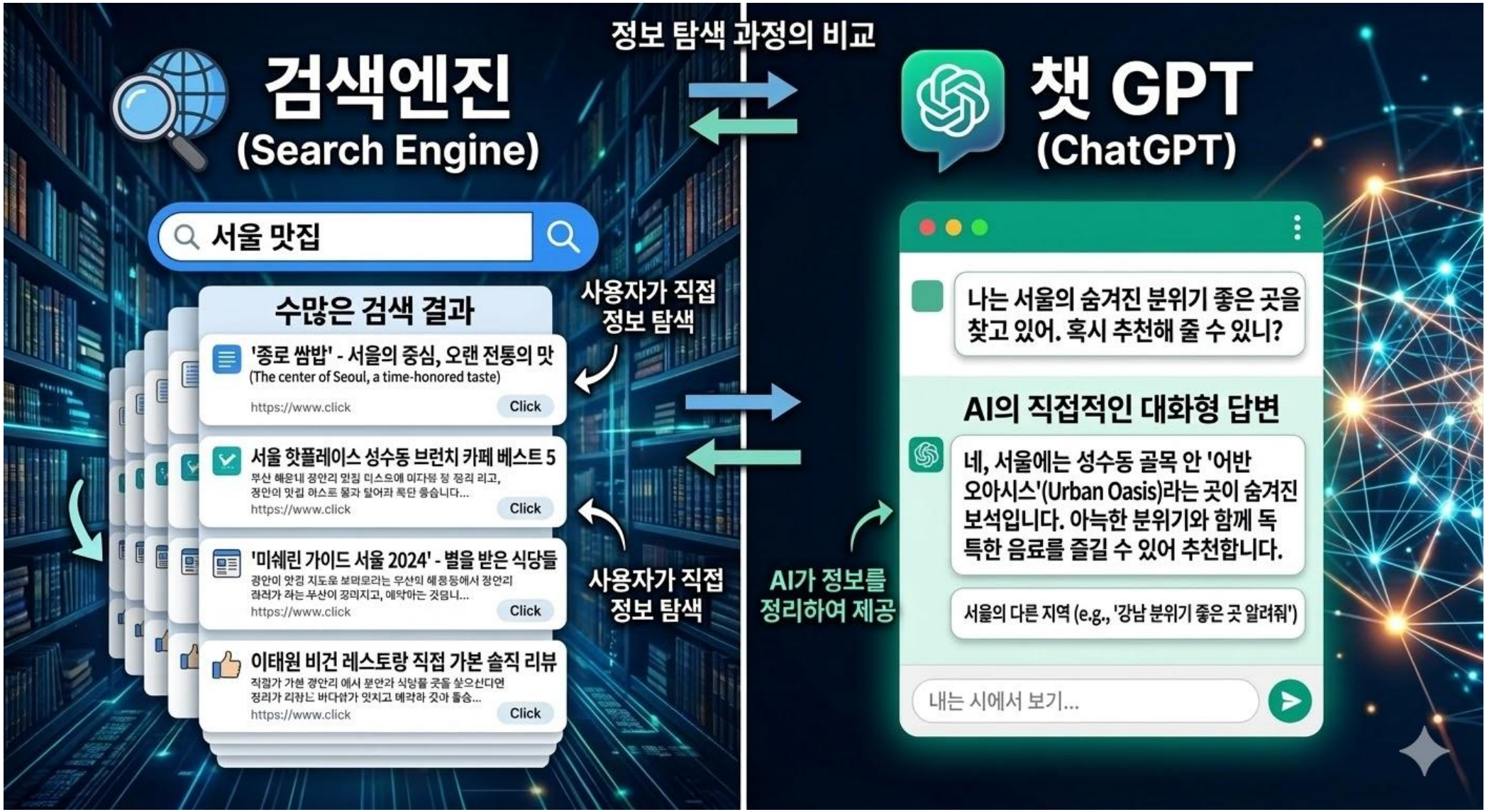
출처: https://www.youtube.com/watch?v=138ZMLTtKfE

인공지능 분야_컴퓨터 비전

인공지능 분야_자연어처리



인공지능 분야_자연어처리



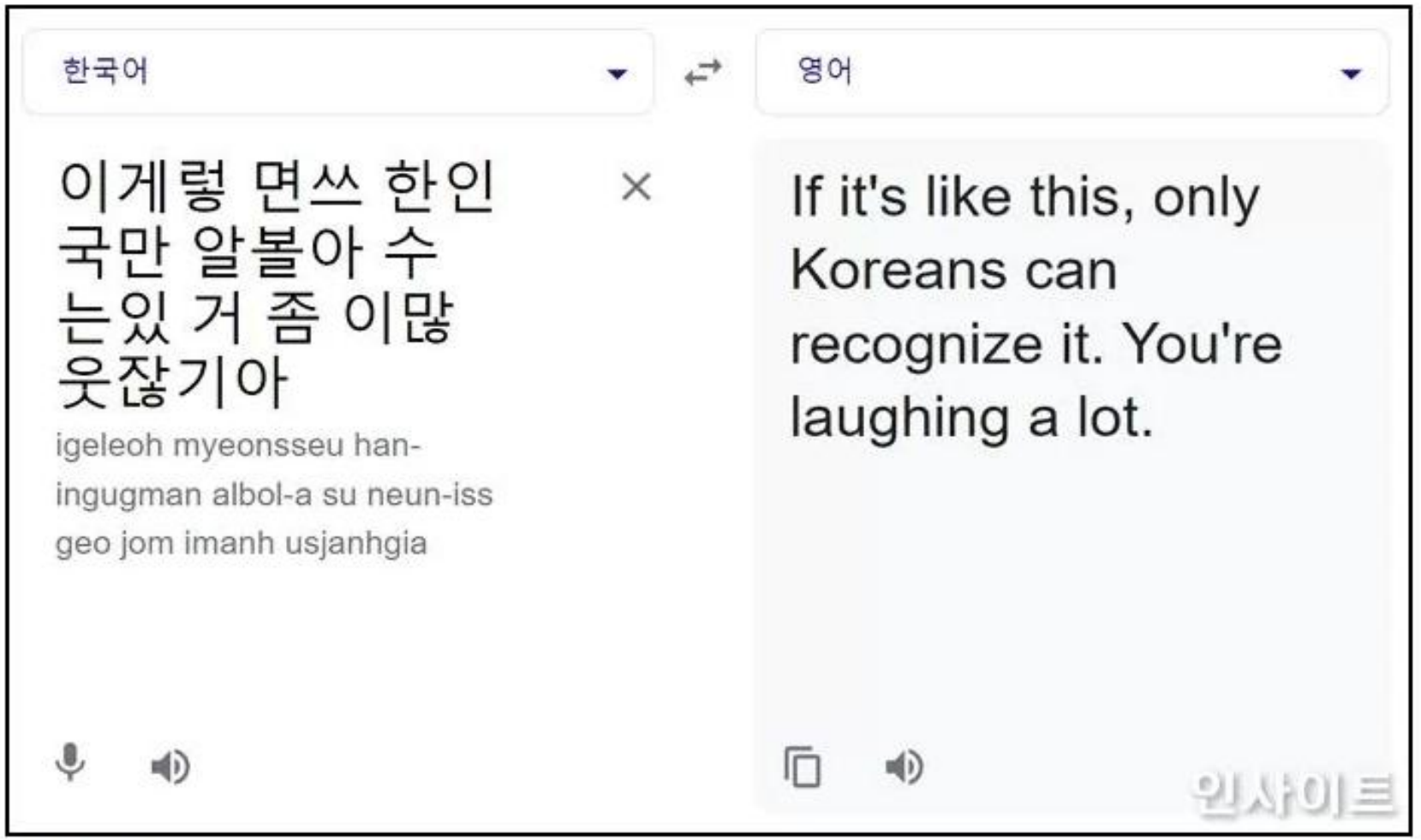
인공지능 분야_자연어처리

- 자연어 처리의 어려움 - 한국어의 특징

한꾹인뜰만알아뽈쑤있께짝썽하껬씁니따.
히까씨이케부쿠로역에썬
30초또안껄릴만끔가깝찌만쑥쏘까많이낙후뙤어있꼬
엘베없꼬4층이라짐많으면깨꼬쌩합니따.
빠뀌뻘레나왔꼬 화짱씰 많이낡았씁니따.
끄래써 똥 역화짱씰까써 쌌씁니따
쩔때 여끼로오찌마세요!
뜨럽꼬 낡꼬 꼐딴을 쫗아하씨는 뿐만 까세요!

인공지능 분야_자연어처리

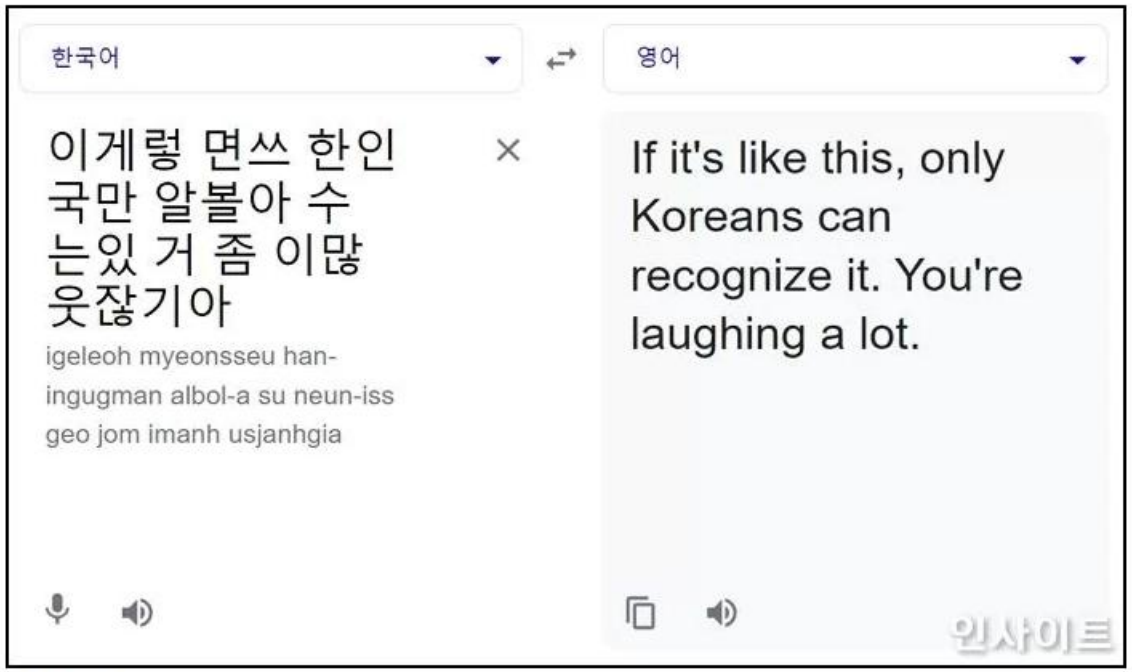
- 한국인들만 아는 암호가 뚫어짐(2023.01.30)



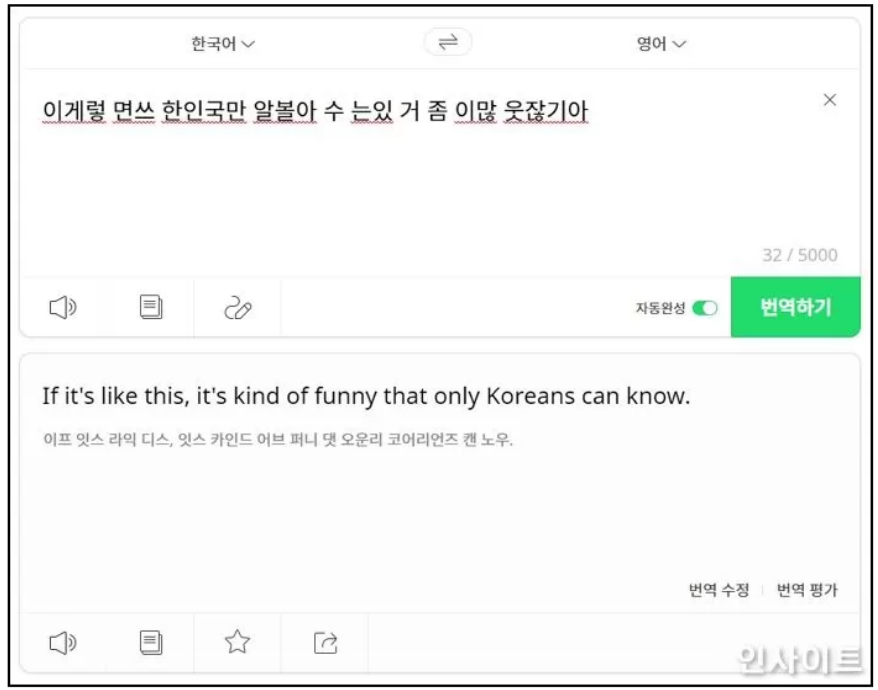
구글 번역 캡처 / 사진=인사이트

인공지능 분야_자연어처리

- 한국인들만 아는 암호가 뚫어짐(2023.01.30)



구글 번역 캡처 / 사진=인사이트



네이버 파파고 캡처 / 사진=인사이트

인공지능 분야_자연어처리

2026.01.20

★★☆☆☆

어~어~ 내 얘길 드러봐~Everybody~ listen up~ 예얍~
친구들과 기대했던 후쿠오카 여~행! 예얍~ 너무너무 좋았어~
とてもよかった 하G만~ 숙so에 도착하고 난 알하치~ 이건
mang했다고~ 예얍 넘나넘나 먼ㅈㅣ가많훈것... 하지만~but
괜찮았어! 전진먼진 털면 되니까다나까나까무라상~ 들어(가)(자)
(마)(자) 카펱은 구석으로ㅊㅣ워버rigo~구석기 시대로 바이바이
~~ yeah~ 그러나しかし 베ㄱㅐ피 누리끼리우리끼리~ 베ㄱ
ㅐ피~ B누로 씻었earth~ 누리끼리우리끼리? 없어졌어~
無くなった~ 이건 정말 shock baby 하지만 정말로 내가
행복했던건~僕が本当にいいだと思ったのは~ 바로 바메는
ㅋ 듣고 놀라지마 예얍 뚜루뚜루뚜거운 무리 안! 나! 와우!
wow! 値段が安すいので良かった가격이 저렴해서 정말
좋았어요. ^^ 코헤E~ 기레서 내 맞우치지맛쏘! 알긋나im(마)
개빢ㅊㅣ니카

ChatGPT 5.2 Thinking

공유하기 ···

~~ yeah~ 그러나しかし 베ㄱㅐ피 누리끼리우리끼리~ 베ㄱ
ㅐ피~ B누로 씻었earth~ 누리끼리우리끼리? 없어졌어~
無くなった~ 이건 정말 shock baby 하지만 정말로 내가
행복했던건~僕が本当にいいだと思ったのは~ 바로 바메는
ㅋ 듣고 놀라지마 예얍 뚜루뚜루뚜거운 무리 안! 나! 와우!
wow! 値段が安すいので良かった가격이 저렴해서 정말
좋았어요. ^^ 코헤E~ 기레서 내 맞우치지맛쏘! 알긋나im(마)
개빢ㅊㅣ니카

너 이런건 해석이 가능하니?

38s 동안 생각함

가능합니다. 다만 이 글은 의도적으로 **한/일/영을 섞고, 말장난(예: '예압~', 'shock baby'), 괄호로 글자를 끼워 넣는 방식**을 써서 "정확한 문장"이라기보다 **분위기+리듬으로 쓰인 후기**에 가깝습니다. 그래서 100% 직역은 일부 구간이 애매해질 수 있어요.

제가 읽히는 핵심 내용(요약)은 이렇습니다.

- 친구들과 기대했던 **후쿠오카 여행이 전반적으로 매우 좋았다**("とてもよかった" 포함).
- 그런데 **숙소에 도착하니 먼지가 너무 많아서 '망했다'고 느꼈다**.
- 그래도 **먼지 털고/닦으면 되니까 괜찮다**고 하며 들어감.
- 뭔가 **누리끼리한(누런) 얼룩/상태**가 있었는데 **표백제/세정('하이포' 같은 표현, 'B로 씻었' 등)으로 씻으니 없어졌다(無くなった)**고 말함.
- "진짜 shock였지만" 결과적으로는 **행복했고**, 특히 **가격이 저렴해서 좋았다(値段が安い)**고 마무리.
- 마지막엔 장난스럽게 **짜증("가빡치니까")** 같은 표현도 섞임.

원하시면,

1. 제가 이미지 내용을 **줄 단위로 깔끔하게 전사(텍스트화)** 하고
2. 그 다음에 **자연스러운 한국어 정리본** + **자연스러운 영어 번역본**
   두 버전으로 만들어드릴게요.

그리고 괄호 안의 동그라미 글자들(예: (가)(자)(치)...)은 **숨은 메시지/말장난**일 가능성이 높아서, 그걸 "해독"까지 원하면 그 부분만 따로 모아서 패턴을 찾아드릴 수도 있습니다.

인공지능 분야_자연어처리

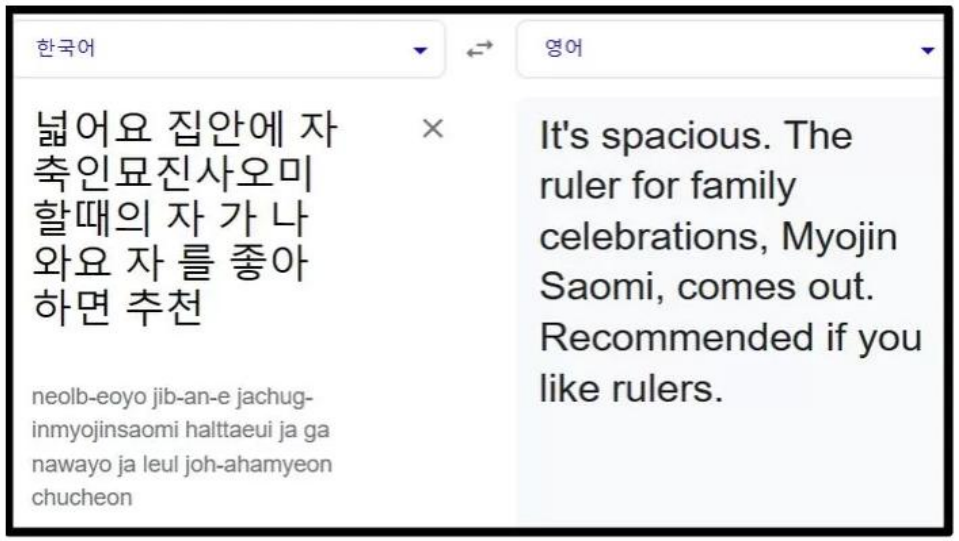
인사이트 정치 경제 사회 연예 여행 사람들 IT·과학 생활 스토리 산업 금융 보도자료

구글번역·챗GPT 뚫리자 오직 한국인만 알아들을 수 있는 '암호 후기' 업글 버전 나왔다

해외 숙소에서 쥐를 발견한 여행객은 한국인들을 위해 12간지를 동원해 번역기가 뚫지 못하는 후기글을 완성했다.

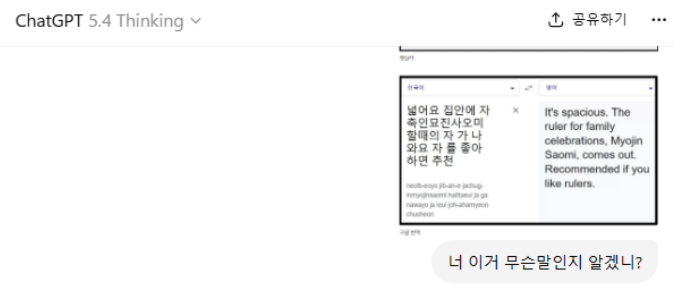
김소영 기자 입력 2023.03.30 10:43

챗GPT



구글 번역

- 2026.03.09(GPT 5.4)

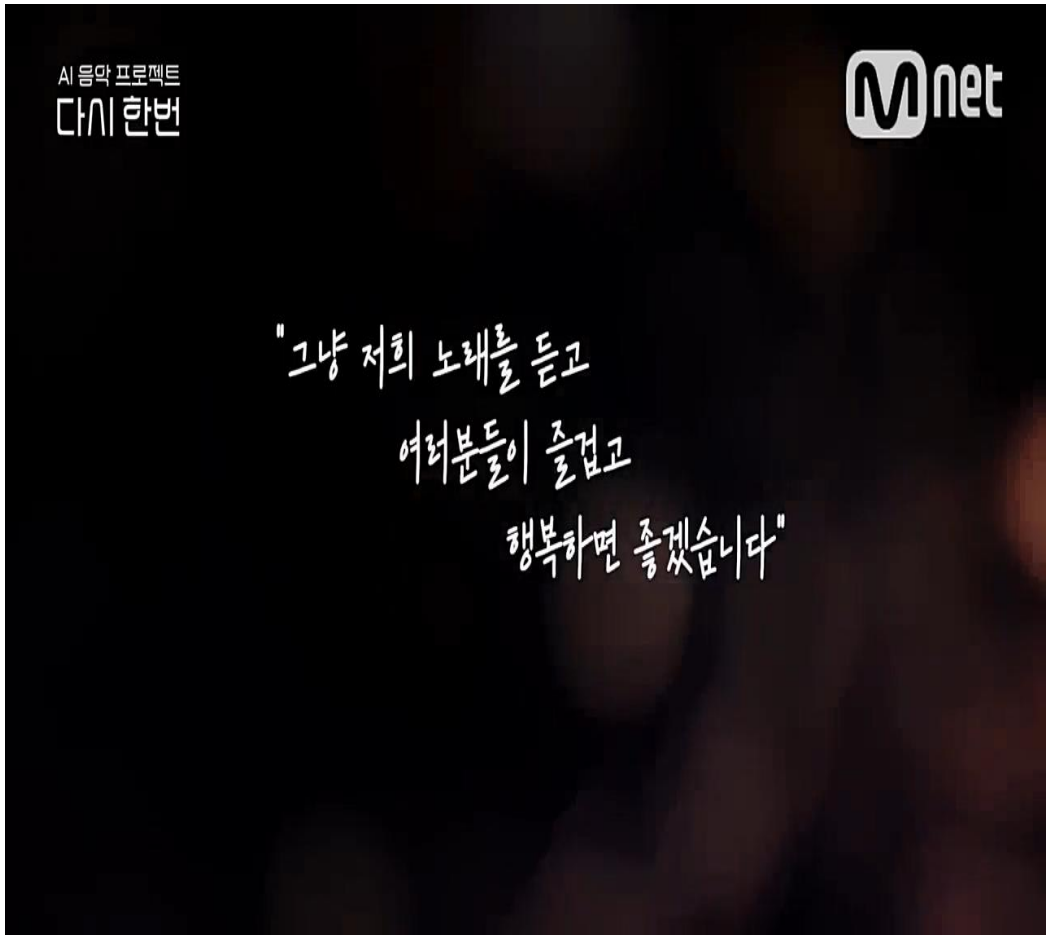


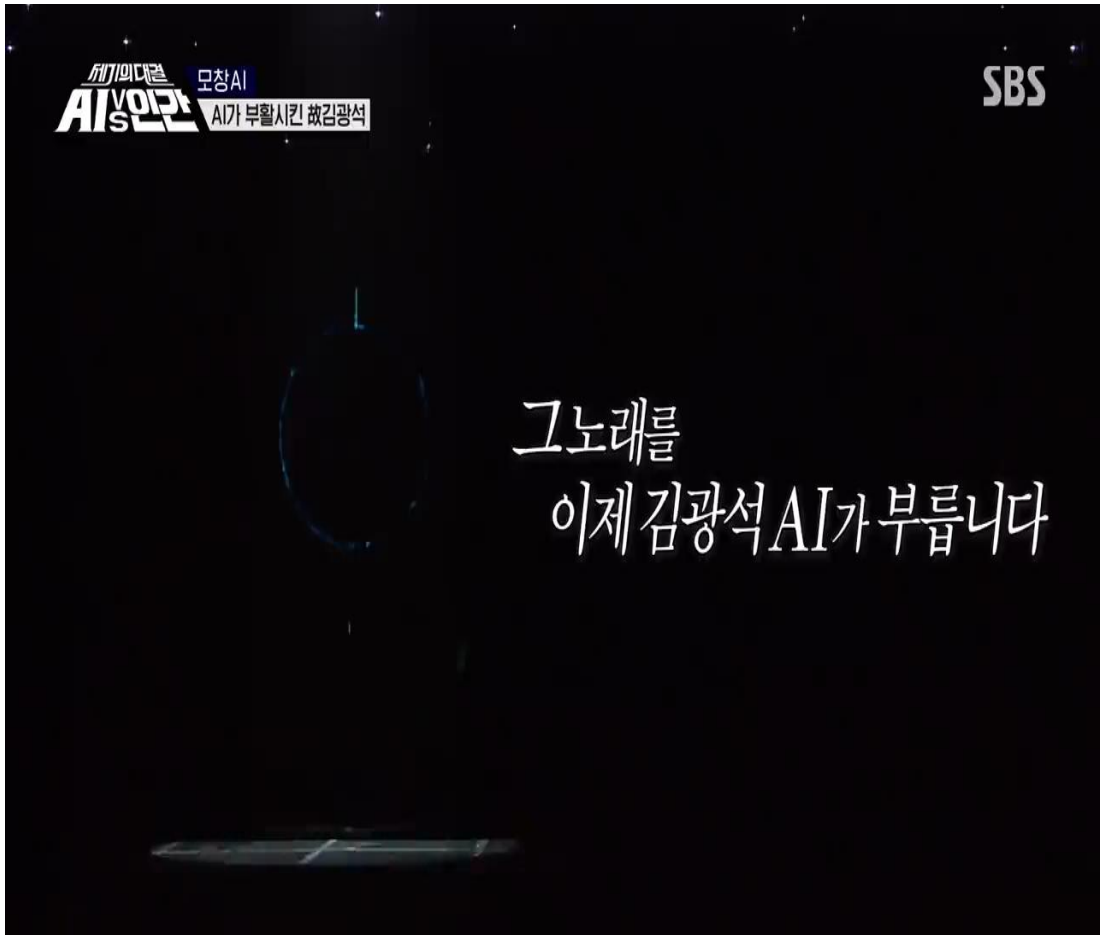
인공지능 분야_자연어처리

인공지능 분야_ 음성인식

음성인식: 소리를 의미로 바꾸는 과정
SPEECH RECOGNITION AI
(음성인식 인공지능)
SPEECH
(음성 입력)
TRANSCRIPTION
(텍스트 변환)
소리
음성역
인최
음성역
AI
딥러닝 분석
오늘 날씨는 어때?
(What's the weather today?)
현재 서울은 맑음, 24°C입니다.
SPEECH RECOGNITION AI
(음성인식 인공지능)
소리를 텍스트로 바꾸는 과정
인식
분석
변환
결과 확인 & 실행

인공지능 분야_ 음성인식


AI 음악 프로젝트
다시 한번
Mnet
"그냥 저희 노래를 듣고
여러분들이 즐겁고
행복하면 좋겠습니다"

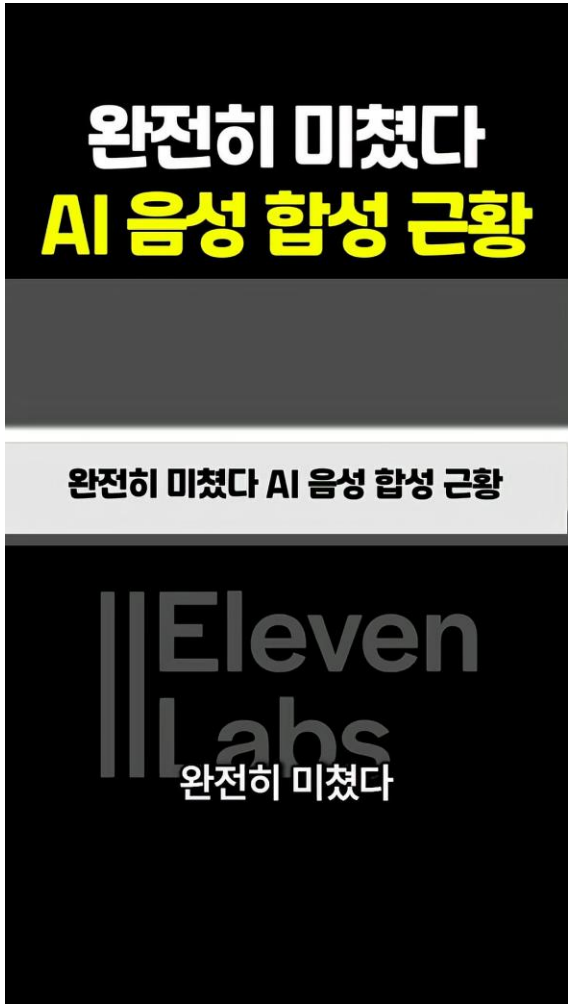

세기의 대결
AI vs 인간
모창AI
AI가 부활시킨 故김광석
SBS
그노래를
이제 김광석AI가 부릅니다

인공지능 분야_ 음성인식

인공지능 분야_ 음성인식

일레븐랩스: 텍스트를 자연스러운 사람 목소리로 바꿔주는 AI 음성 생성(Text-to-Speech)·보이스 클로닝 플랫폼/모델을 만드는 회사

: 한국에는 2025년 11월에 런칭



인공지능 분야_ 음성인식

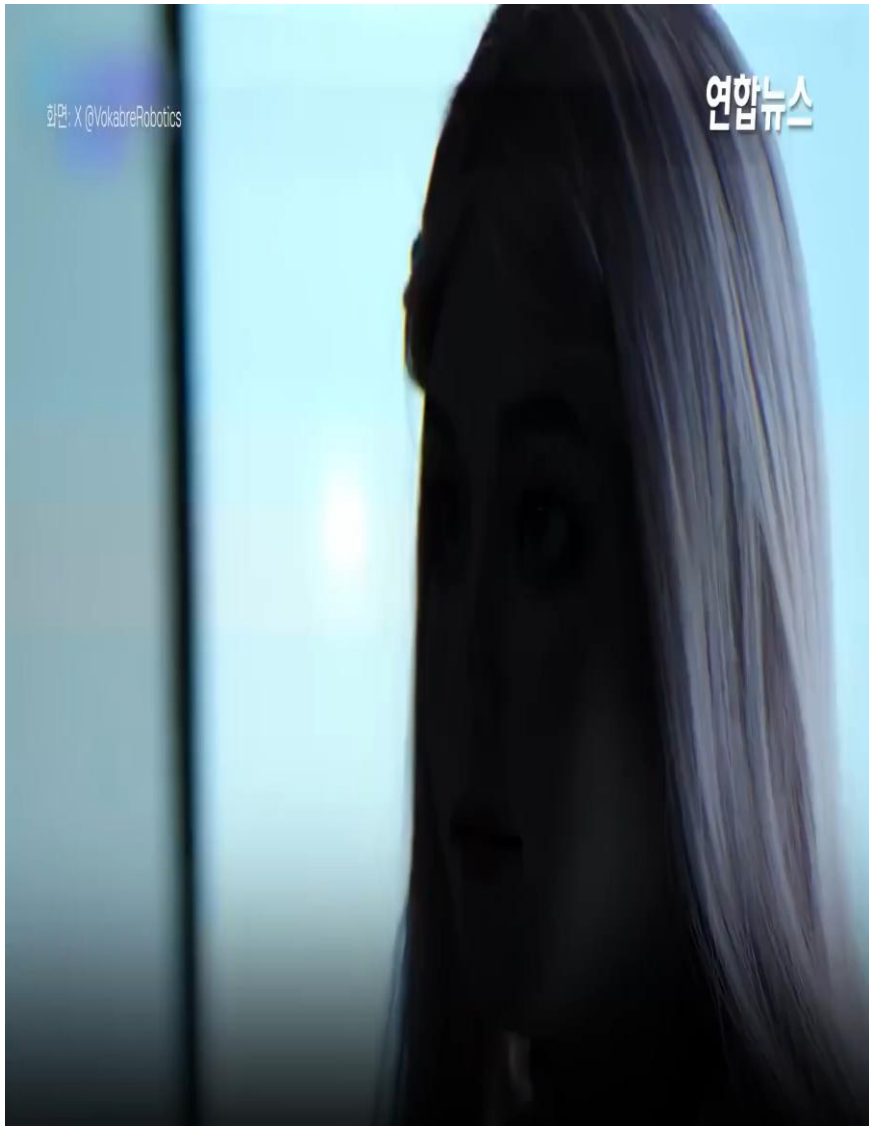
인공지능 분야_ 멀티모달

Sora+나노바나나+Kling AI(2026.01.20)

출처: Threads, choi.openai

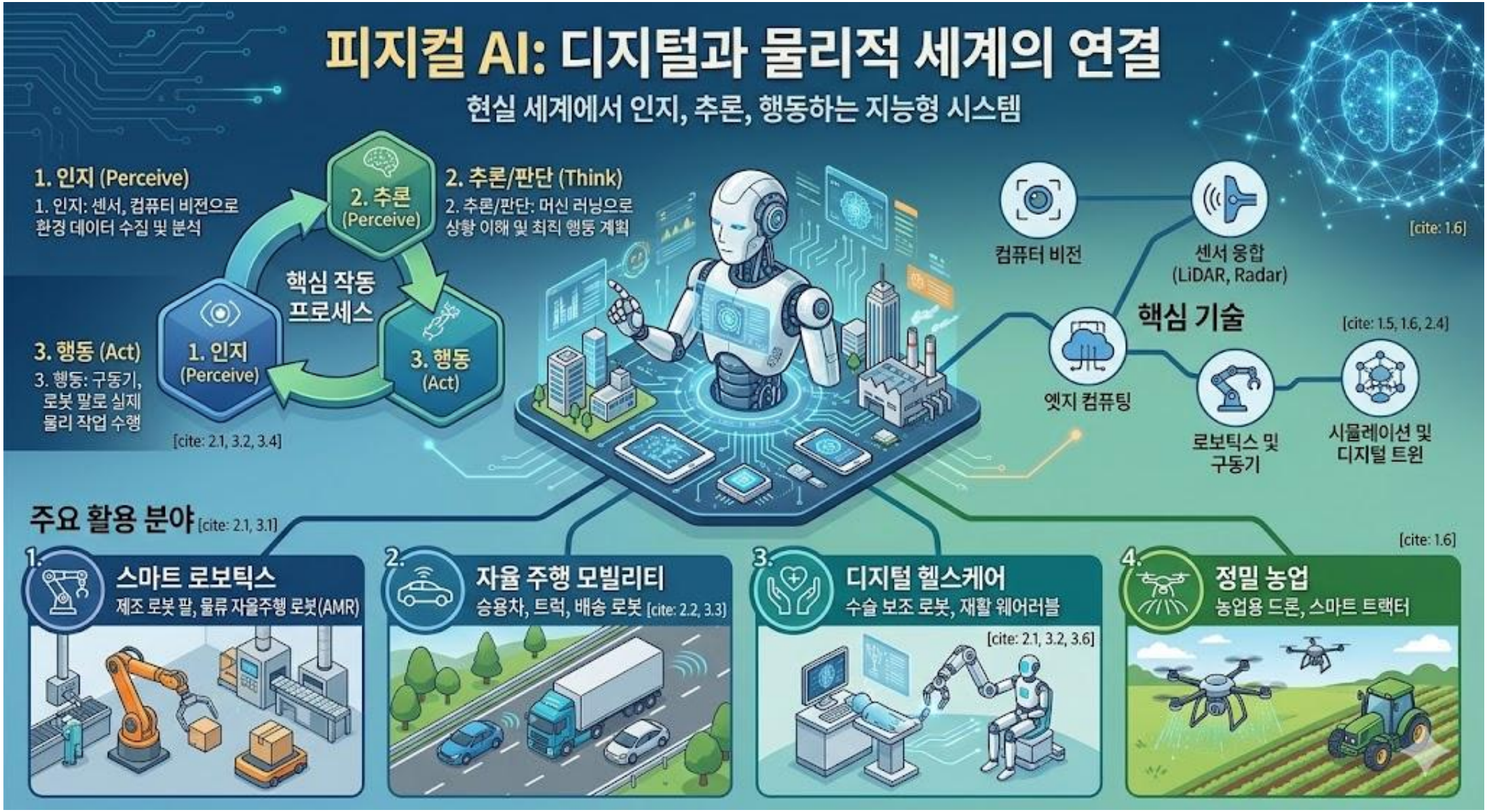
인공지능 분야_ 멀티모달

2026.02.25



2026.02.06

피지컬 AI


피지컬 AI: 디지털과 물리적 세계의 연결
현실 세계에서 인지, 추론, 행동하는 지능형 시스템
1. 인지 (Perceive)
1. 인지: 센서, 컴퓨터 비전으로 환경 데이터 수집 및 분석
2. 추론/판단 (Think)
2. 추론/판단: 머신 러닝으로 상황 이해 및 최적 행동 계획
3. 행동 (Act)
3. 행동: 구동기, 로봇 팔로 실제 물리 작업 수행
2. 추론 (Perceive)
3. 행동 (Act)
1. 인지 (Perceive)
핵심 작동 프로세스
[cite: 2.1, 3.2, 3.4]
컴퓨터 비전
센서 융합 (LiDAR, Radar)
[cite: 1.6]
핵심 기술
엣지 컴퓨팅
로보틱스 및 구동기
시뮬레이션 및 디지털 트윈
[cite: 1.5, 1.6, 2.4]
주요 활용 분야 [cite: 2.1, 3.1]
[cite: 1.6]
1. 스마트 로보틱스
제조 로봇 팔, 물류 자율주행 로봇(AMR)
2. 자율 주행 모빌리티
승용차, 트럭, 배송 로봇 [cite: 2.2, 3.3]
3. 디지털 헬스케어
수술 보조 로봇, 재활 웨어러블
[cite: 2.1, 3.2, 3.6]
4. 정밀 농업
농업용 드론, 스마트 트랙터

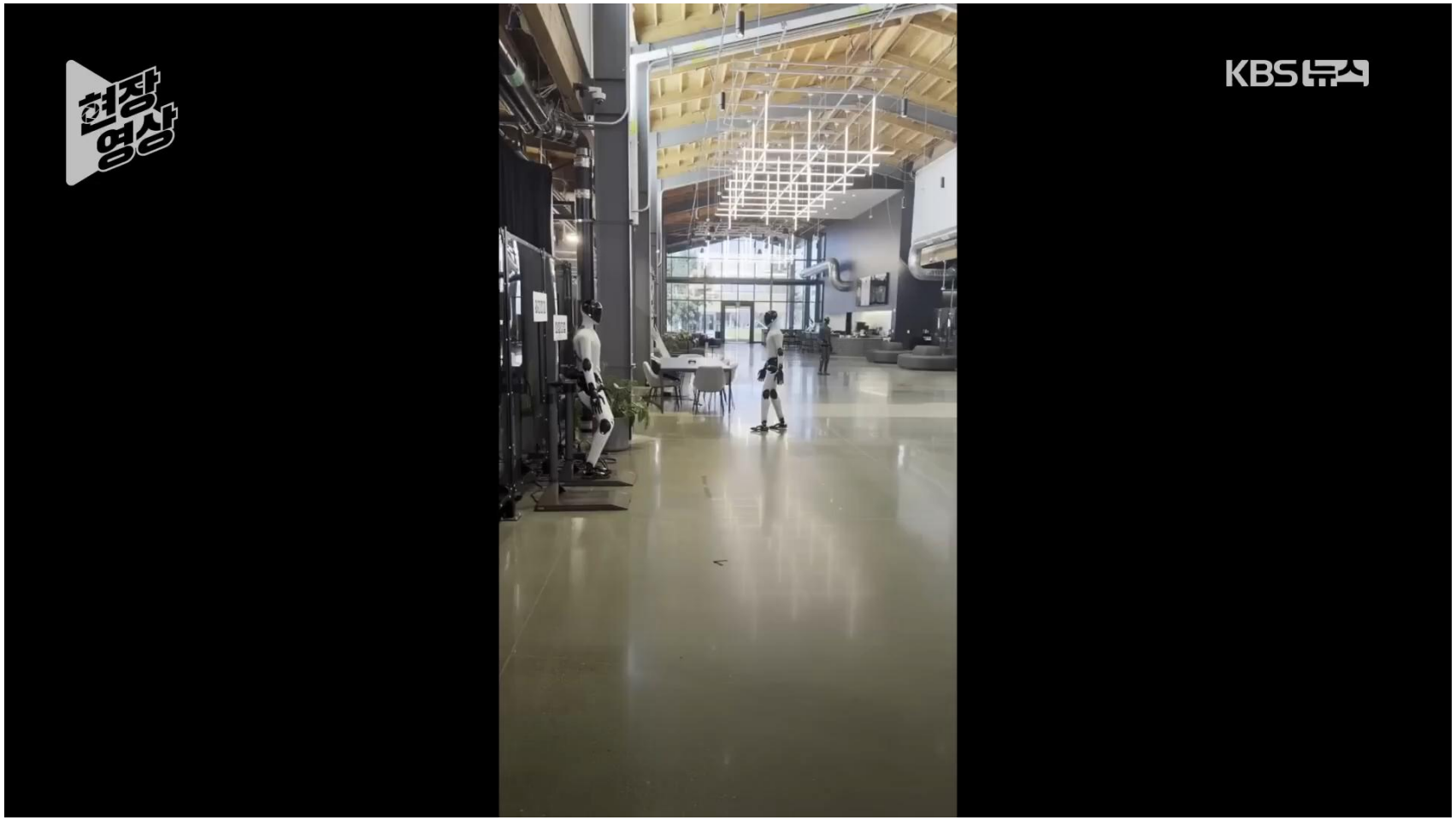
휴머노이드 로봇

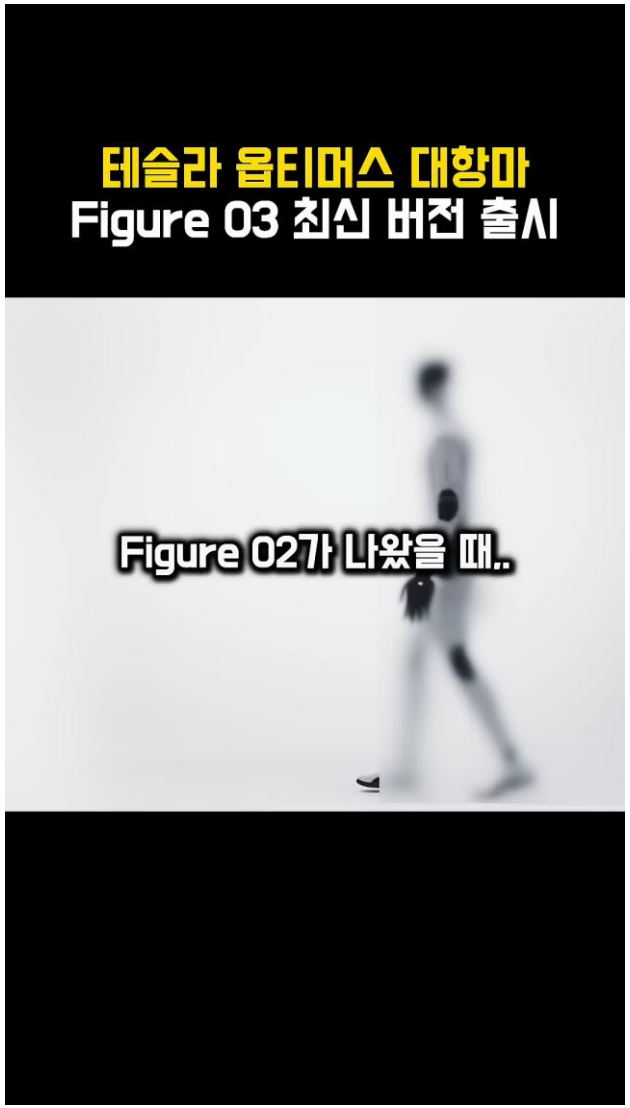
이제는 진짜로

쇼츠백과
SHORTS
ENCYCLOPEDIA
누가 봐도 100%
"메이드 인 코리아" 로봇!!
자 준비 하시고!
현재 K-뽕짝 리듬을
갈아넣은듯한 무브로 화제인 로봇

테슬라 옵티머스
CG가 아닙니다
1x Speed
오늘의 테슬라 뉴스

휴머노이드 로봇





2026.02.27

휴머노이드 로봇

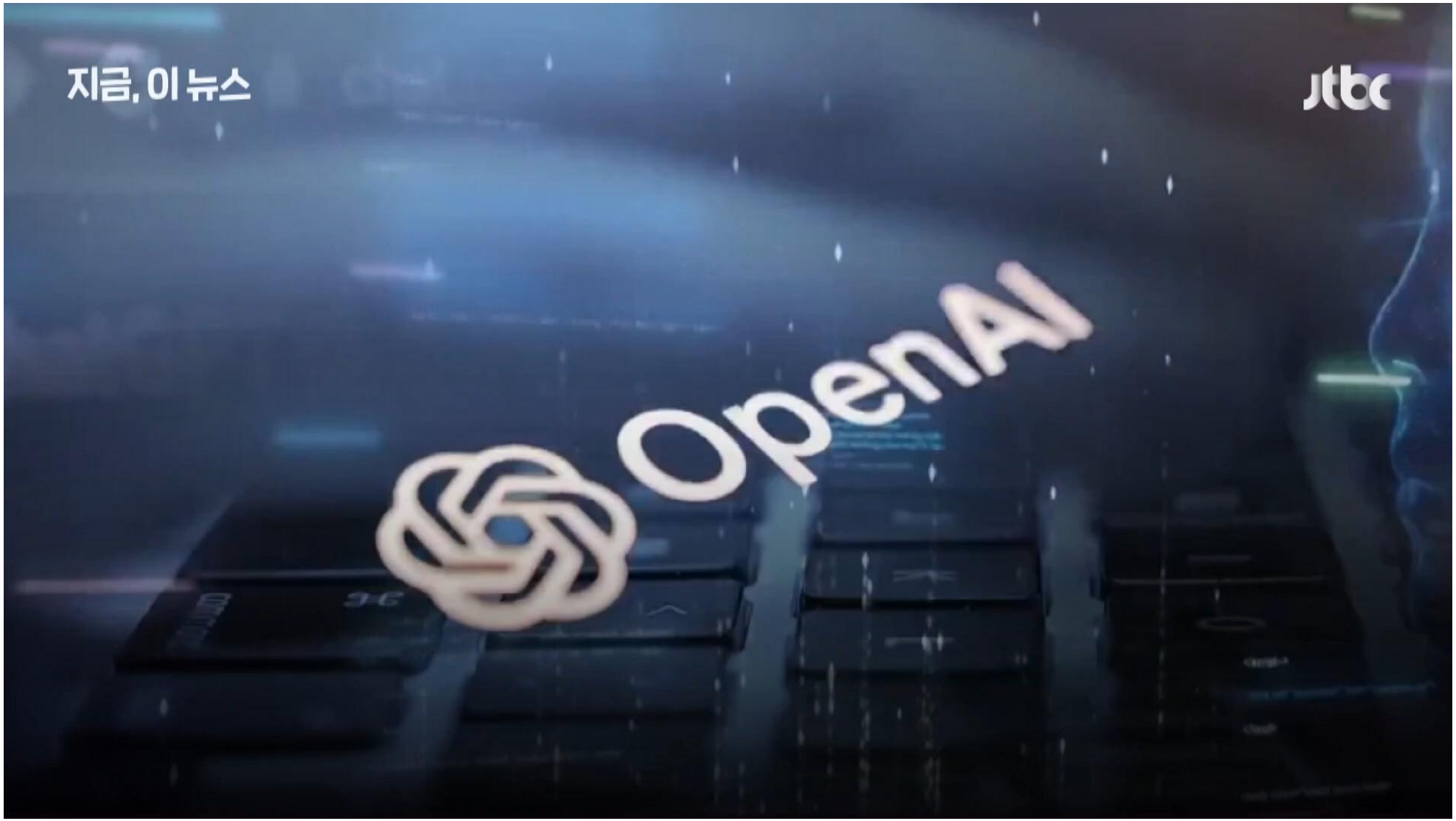
2026.01.19

AI 안경

2026.10.25

인공지능_부작용

2025.11.27



인공지능_부작용

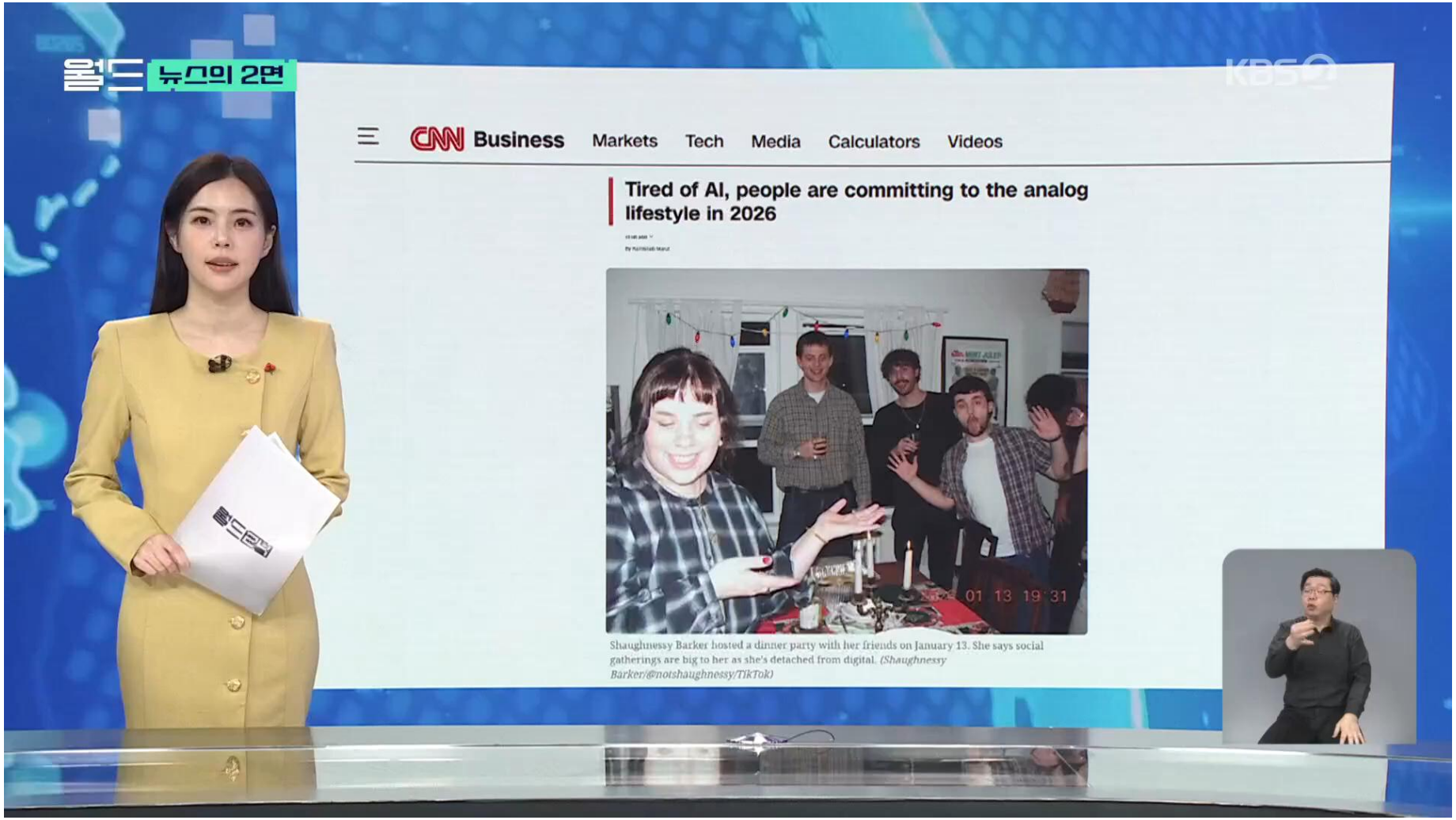
2025.05.16

인공지능_부작용

2025.12.04

인공지능_부작용

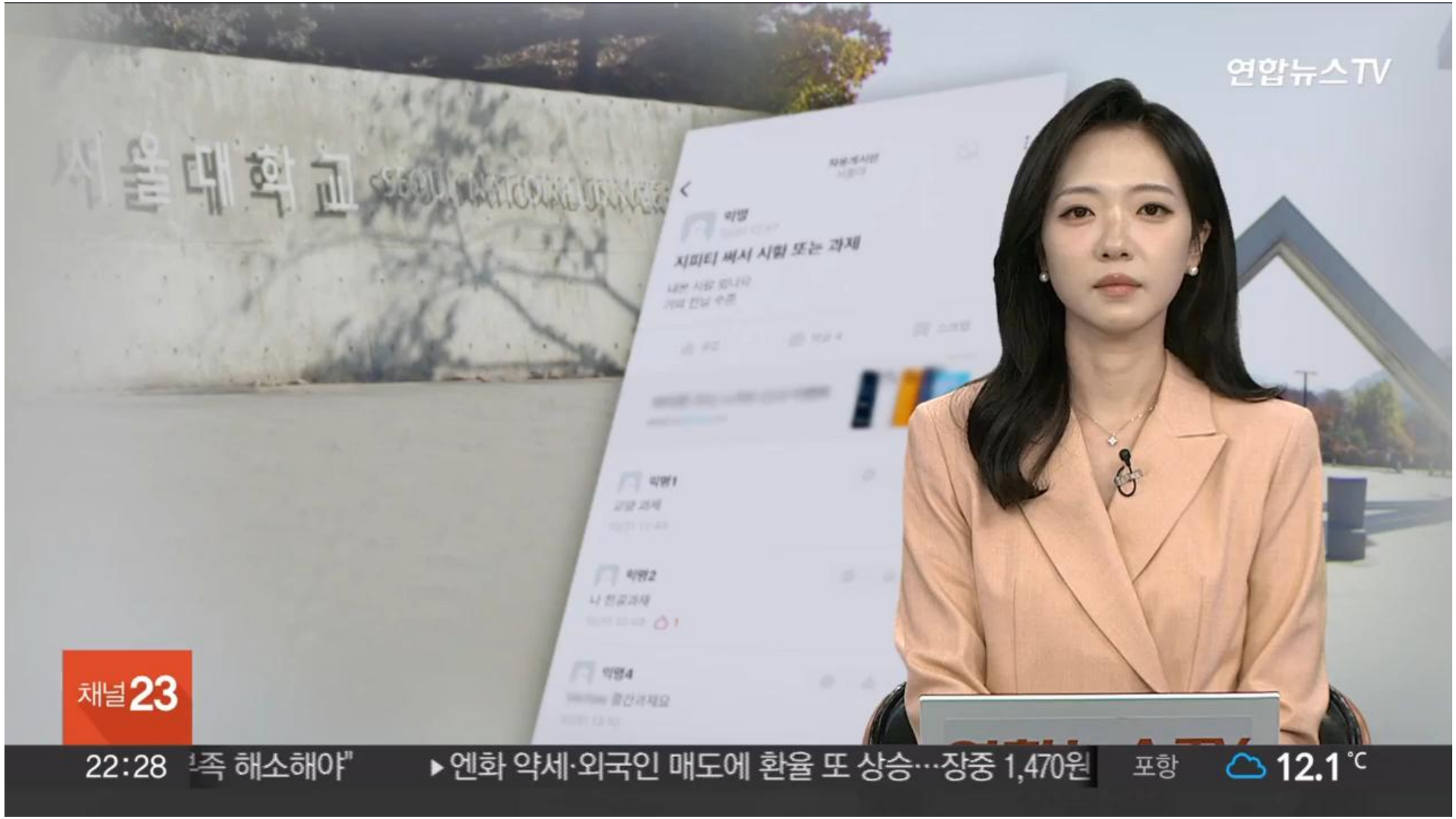
2025.01.19



인공지능_부작용

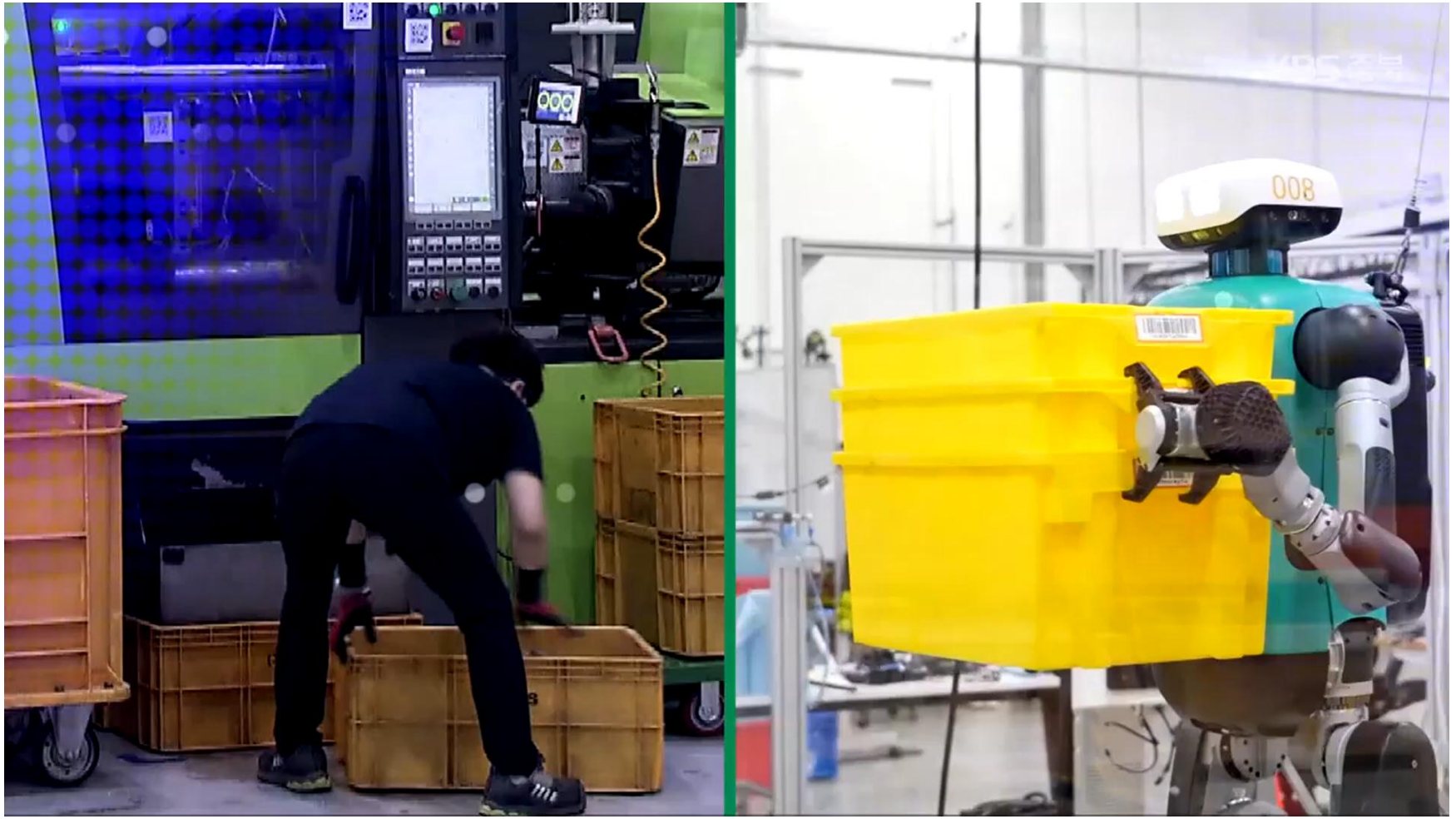
2026.03.03

인공지능의 윤리


연합뉴스TV
서울대학교
지피티 써서 시험 또는 과제
채널23
22:28
해소해야"
▶엔화 약세·외국인 매도에 환율 또 상승…장중 1,470원
포항
12.1℃

인공지능의 윤리

‘인공지능 대부’ 제프리 힌튼 단독 인터뷰…“인간 대체할 것” / KBS 2025.11.20.



인공지능의 윤리

2026.01.11

AI 툴 생태계

2026 생성형 AI 툴 생태계 - 목적별 총정리

Ver. 2026.10

빠르게 진화하는 AI 도구를
목적에 맞게 활용하세요!

글쓰기 & 콘텐츠 생성

블로그, 마케팅, 문서, 요약 등 다양한 콘텐츠 제작

- 블로그/글쓰기: Jasper, Copy.ai, Writesonic, Rytr
- 마케팅/카피라이팅: ChatGPT, Claude, Writesonic, Anyword
- 문법/교정: Grammarly, QuillBot, LanguageTool, ProWritingAid
- 요약/문서화: Notion AI, Sudowrite, ChatPDF, Perplexity

당신의 목적에 맞는
최적의 AI 도구를 찾아보세요!

생산성 & 오피스

회의, 문서, 프레젠테이션, 업무 자동화

- 미팅 기록/요약: Otter.ai, Fathom, Fireflies, Notta
- 문서 작성: Microsoft 365 Copilot, Google Gemini in Workspace, Notion AI, Coda AI
- 프레젠테이션: Gamma, Tome, Beautiful.ai, Canva AI
- 작업 관리/자동화: Asana AI, Monday.com, ClickUp AI, Zapier

이미지 & 예술 생성

이미지, 디자인, 영상, 3D 등 시각 콘텐츠 제작

- 이미지 생성: Midjourney, DALL-E 3, Stable Diffusion 3, Ideogram
- 디자인 편집: Canva AI, Adobe Firefly, Figma AI, Krea
- 영상 생성: Runway, Pika, Luma, Kling AI
- 3D/캐릭터: Adobe GenStudio, Meshy, Leonardo.ai, Scenario

생성형 AI 툴 생태계

- 목적별 총정리

코딩 & 프로그래밍

코드 작성, 수정, 디버깅, 개발 자동화

- 코드 생성/자동완성: GitHub Copilot, Cursor, Tabnine, Amazon Q
- 코드 실행/에이전트: Claude Code, OpenAI Codex (GPT-6), Devin, Replit AI
- 개발 환경: VS Code + Copilot, JetBrains AI, Sourcegraph, Windsurf
- 디버깅/테스트: Sentry AI, Linear, CodeRabbit, Continue

영상 & 오디오 생성

영상 편집, 음성 생성, 더빙, 음악 제작

- 영상 편집: Runway, CapCut AI, Opus Clip, Descript
- 아바타/비디오: HeyGen, Synthesia, D-ID, Colossyan
- 음성 생성/더빙: ElevenLabs, PlayHT, Speechify, Murf
- 음악 생성: Suno, Udio, Loudly, Stable Audio

연구 & 정보 검색

웹 검색, 문헌 조사, 데이터 분석, 번역

- 문헌 검색: Perplexity, Elicit, Consensus, Scite
- 데이터 분석: ChatGPT (Advanced Data Analysis), Claude, Google Gemini, Polymer
- 언어 번역: DeepL, Google Translate, Papago, Reverso
- 리서치/보고서: Perplexity Pro (Deep Research), Claude Research, NotebookLM, SciSpace

2026 핵심 인사이트: 단일 작업 도구에서 벗어나, 여러 AI 툴을 연결해 하나의 워크플로로 활용하는 것이 핵심입니다.

인지 설계도

프롬프트 엔지니어링 마스터클래스:

언어 모델의 한계를 넘어
집단 지성 엔진으로

시스템적 사고

복잡한 문제를 구조적으로 분해하고
핵심 요소를 설계합니다.

집단 지성 활용

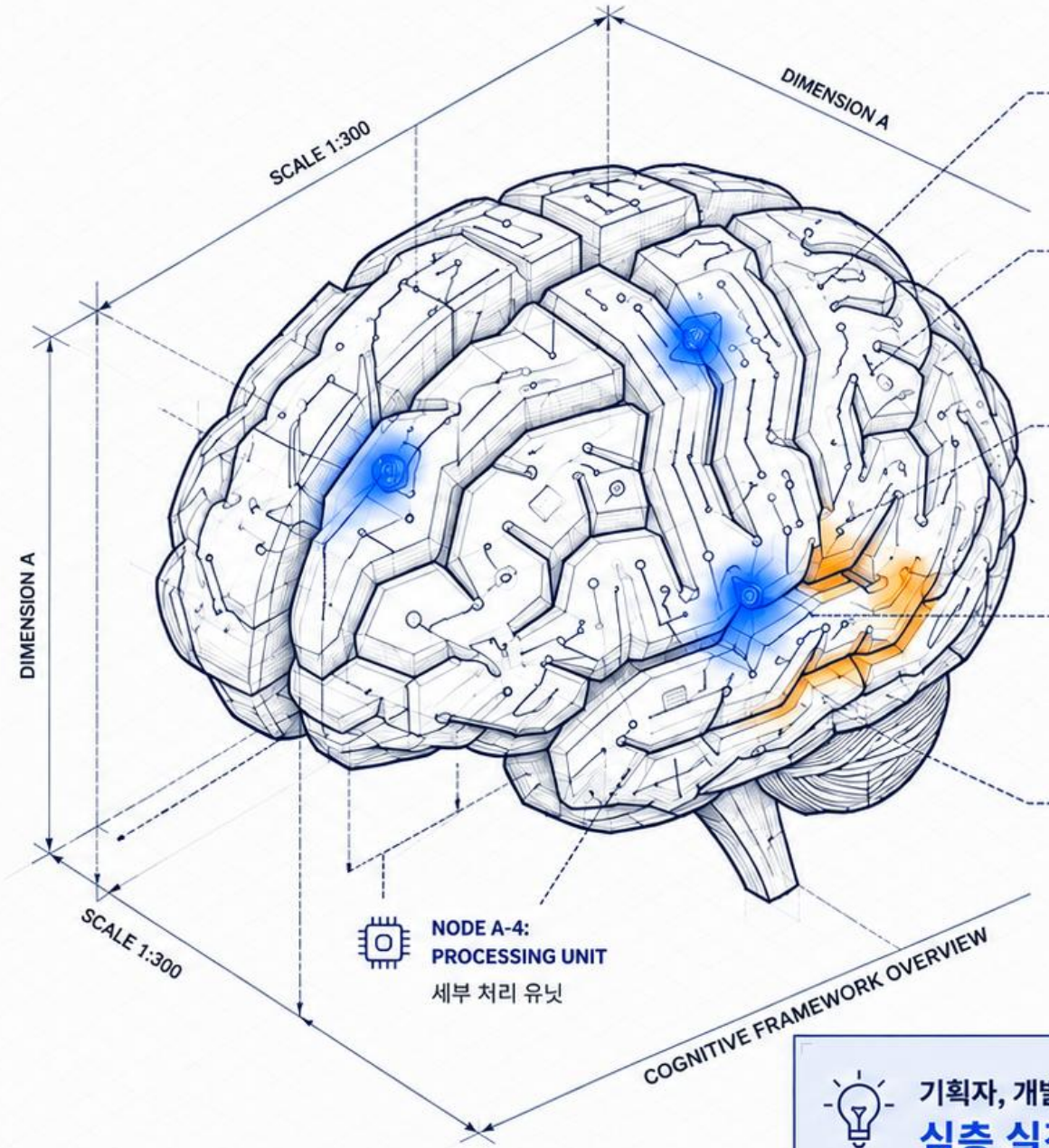
언어 모델을 개인 지식의 확장이 아닌
집단 지성의 허브로 활용합니다.

정확한 방향 설정

명확한 목표와 맥락으로 모델의
사고 방향을 정밀하게 조율합니다.

지속적 개선

설계-실행-평가-개선의 순환을 통해
모델 성능을 지속적으로 향상시킵니다.



기획자, 개발자 및 AI 아키텍트를 위한
심층 실전 가이드

LLM의 구조적 한계와 대응 전략 (2026년 10월 기준)

최신 LLM이 가진 구조적 한계를 4가지 핵심 특성으로 정리하고, 이를 고려한 대응 전략을 제시합니다.

더 안전하고 신뢰할 수 있는
AI 활용을 위해 LLM의 한계를
정확히 이해하는 것이 중요합니다.

대표 최신 모델 예시
(한계는 모델별로 다름)

OpenAI
- GPT-6 Astra
- GPT-6.1 Sol
- GPT-6 Luna

2026.10.07 공식 발표

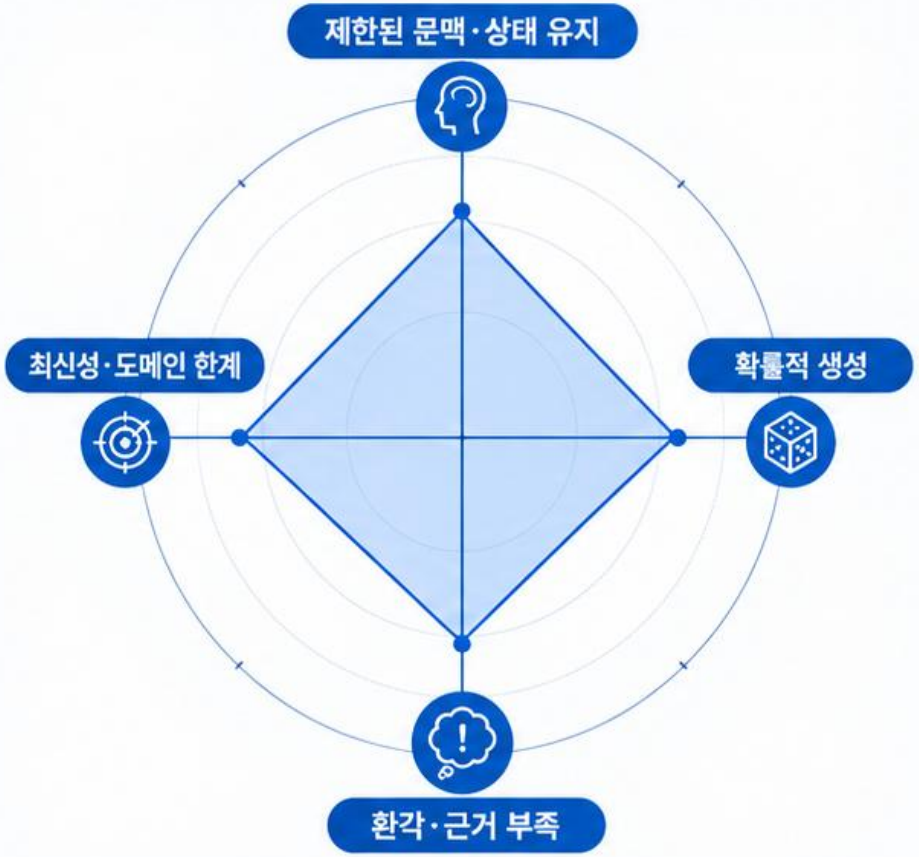
Claude 계열
(Anthropic)

Gemini 계열
(Google)

Llama 계열
(Meta)

Mistral 계열
(Mistral AI)

LLM의 4가지 핵심 한계 (개념적 위험 지도, 정량평가 아님)



Panel 1 — 제한된 문맥·상태 유지

긴 대화나 많은 컨텍스트를 지속적으로 유지하는 데 한계가 있습니다.
요약, 메모리, 검색을 활용할 수 있지만 완전한 영구 기억은 보장되지 않습니다.

대응 전략
- 중요한 정보 요약 및 구조화
- 외부 지식 저장소 연결
- 세션별 컨텍스트 관리
- 핵심 정보 반복 제공

Panel 2 — 확률적 생성

동일한 질문에도 다른 결과를 생성할 수 있습니다.
temperature=0으로 설정해도 모든 환경에서 동일한 결과를 보장하지 않습니다.

대응 전략
- 명확하고 구체적인 지시
- 일관된 출력 형식 지정
- 여러 번 생성 후 비교
- 중요 작업은 재검증

Panel 3 — 환각·근거 부족

그럴듯하지만 사실과 다른 정보를 생성할 수 있습니다.
존재하지 않는 사실, 출처, 수치 등이 포함될 수 있으므로 검증이 필요합니다.

대응 전략
- 출처 제시 요구
- 사실 및 수치 검증
- 신뢰 가능한 자료와 교차 확인
- 중요한 내용은 전문가 검토

Panel 4 — 최신성·도메인 한계

학습된 지식은 특정 시점까지의 데이터에 기반하며, 최신 정보나 전문 분야, 비공개 자료에 대해서는 정확하지 않을 수 있습니다.

대응 전략
- 외부 검색(RAG)으로 최신 정보 보완
- 도메인별 전문 자료 활용
- 내부 데이터 연계
- 전문가 검증을 통한 신뢰성 확보

프롬프트 엔지니어링은 위험을 줄이는 한 가지 방법입니다.
근거 검색(RAG) · 외부 도구 · 자동 검증 · 사람의 감독을 함께 적용해야 합니다.

- 근거 검색 (RAG)
- 외부 도구 (데이터 연계)
- 자동 검증 (사실·수치 확인)
- 사람의 감독 (전문가 검토)

기준일: 2026.10.21 | 출처: OpenAI GPT-6 공식 발표 (2026.10.07), OpenAI API 모델 문서 | 도식은 정량 평가가 아닌 개념 설명입니다.

프롬프트 엔지니어링에서 맥락 엔지니어링으로 (2026년 10월 기준)

좋은 질문 작성에서 벗어나, 정보·도구·상태·평가를 설계하는 AI 시스템으로 발전

3단계
맥락·에이전트 운영
(2025 ~ 2026)

- 컨텍스트 선별 및 압축
- 메모리·검색을 통한 지속적 맥락 관리
- 에이전트 작업 실행 (여러 단계의 작업)
- 접근 권한과 안전장치 설계
- 평가·관측을 통한 품질 관리

2단계
추론·검색 결합
(2023 ~ 2024)

- 작업 분해 및 계획 수립
- 결과 검증 및 자기 점검
- 외부 지식 검색 (RAG)
- 도구 호출 및 외부 시스템 연동

1단계
프롬프트 설계
(2021 ~ 2022)

- 지시문 작성 (Instructions)
- 예시 제공 (Examples)
- 출력 형식 지정 (Output Format)
- Zero-shot / Few-shot 활용

통제 가능한 AI 워크플로

INSIGHT 2026

핵심은 좋은 질문만이 아니라
신뢰 가능한 맥락과 실행 흐름의 설계입니다.

맥락 품질
필요한 정보만 적시에 제공
- 관련 정보 선별 및 압축
- 최신 정보와 도메인 지식 활용
- 불필요한 정보로 인한 혼란 최소화

도구와 권한
실행 범위·승인·오류 복구 설계
- 적절한 도구 선택 및 연동
- 접근 권한과 보안 정책 수립
- 실행 오류 감지 및 복구 전략

검증과 평가
출처 확인·테스트·모니터링
- 결과의 출처와 근거 확인
- 다양한 테스트로 신뢰성 검증
- 지속적인 모니터링과 품질 개선

프롬프트 엔지니어링은 시작점이며,
맥락 구성, 도구 실행, 검증·평가까지 통합적으로 설계할 때
실제 업무와 문제 해결에 활용할 수 있는 신뢰 가능한 AI 시스템이 됩니다.

프롬프트 작성 (지시·예시·형식) → 맥락 구성 (선별·요약·메모리) → 도구 실행 (외부 시스템 연동) → 검증·평가 (출처 확인·모니터링)

참고: Anthropic, Effective context engineering for AI agents (2025); OpenAI Agents SDK 문서 (2026)

최신 LLM 에이전트 구조: 4대 핵심 모듈 (2026년 10월 기준)

최신 LLM은 단순한 생성 모델을 넘어, 계획하고, 도구를 사용하며, 검증하고, 지속적으로 개선하는 에이전트 형태로 발전하고 있습니다.

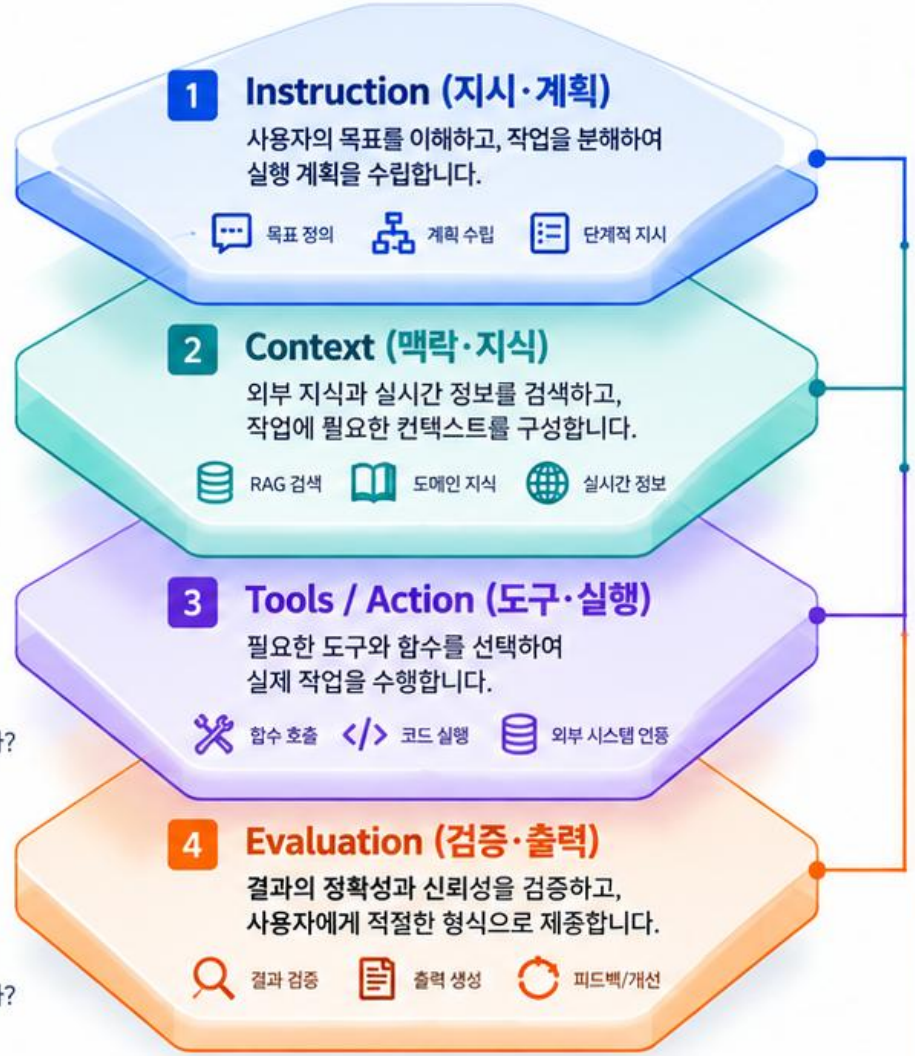
멀티모달 입력 | 도구/함수 사용 | 장기 기억 연계 | 안전·정책 가드레일

목적·계획 (Why/What)
무엇을 해결하고 어떤 계획으로 수행할 것인가?

정보 이해 (Context)
필요한 정보를 어떻게 이해하고 활용할 것인가?

도구 실행 (Action)
어떤 도구로 어떻게 실행할 것인가?

결과 검증 (Evaluation)
결과가 올바른지 어떻게 검증할 것인가?



1. Instruction (지시·계획)

- ✓ **역할 부여 및 목표 정의** (예: 데이터 분석가, 연구원 등)
- ✓ **복잡한 문제를 단계적으로 분해**
- ✓ **제약조건, 우선순위, 출력 형식 명시**
- ✓ **단계적 사고(Chain-of-Thought) 및 계획 수립**

예시
"프로젝트 데이터를 분석하여 소재와 표면적에 따른 누설전류 예측 모델을 개발해줘."

핵심 기능
- 역할/목표 정의
- 계획 수립
- 제약조건 설정
- 단계적 자시

2. Context (맥락·지식)

- ✓ RAG 기반 외부 지식 검색 (문서, 논문, 사내 데이터)
- ✓ 도메인 지식 및 전문가 지식 연계
- ✓ 대화 이력 및 장기 기억 활용
- ✓ 실시간 정보(웹 검색, 최신 데이터) 반영

예시
"프로젝트 데이터 설명서와 관련 논문을 찾아 분석에 활용해줘."

핵심 기능
- RAG/검색
- 도메인 지식 연계
- 장기 기억 활용
- 실시간 정보 반영

3. Tools / Action (도구·실행)

- ✓ 파이썬 코드 실행 및 데이터 분석 (pandas, sklearn 등)
- ✓ 외부 도구/함수 호출 (API, 데이터베이스, 파일 처리)
- ✓ 시각화 도구 활용 (matptollib, seaborn 등)
- ✓ 다단계 작업 자동화 및 에이전트 수행

예시
"데이터를 전처리하고 특정을 추출한 후, 모델을 학습하고 결과를 시각화해줘."

핵심 기능
- 코드 실행
- 함수/도구 호출
- 외부 시스템 연동
- 작업 자동화

4. Evaluation (검증·출력)

- ✓ 결과의 정확성, 일관성, 신뢰성 검증
- ✓ 코드 실행 결과 및 분석 과정 점검
- ✓ 표, 그래프, 보고서 등 적절한 형식으로 출력
- ✓ 추가 질문 및 피드백을 통한 지속적 개선

예시
"예측 결과를 표와 그래프로 정리하고, 모델 성능을 평가해줘."

핵심 기능
- 결과 검증
- 품질 기준 적용
- 출력 형식 생성
- 피드백/개선

핵심 원칙: 목적 지향성 · 맥락 기반성 · 도구 활용성 · 검증 가능성 · 안전성

필요한 정보를 충분히 활용하고, 적절한 도구를 사용하여, 결과를 검증한 후 신뢰할 수 있는 출력을 생성하도록 설계합니다.

정확성 (정확한 답변) | 일관성 (동일한 결과) | 재현성 (반복 가능) | 효율성 (비용·속도 최적화) | 안전성 (정책·윤리 준수)

※ 기준: 2026년 10월 (주요 모델: GPT-5 계열, Claude 4 계열, Gemini 2.5 계열, Llama 4 계열 등) | ※ RAG, 도구 사용, 장기 기억, 가드레일이 통합된 최신 에이전트 아키텍처 기준

프롬프트 아키텍처: 통제의 기반 설계

건축의 구조 요소처럼, 프롬프트의 핵심 구성 요소를 체계적으로 설계하여
모델의 응답을 안정적이고 일관되게 통제합니다.

[기둥 Pillars]
맥락과 입력 정보

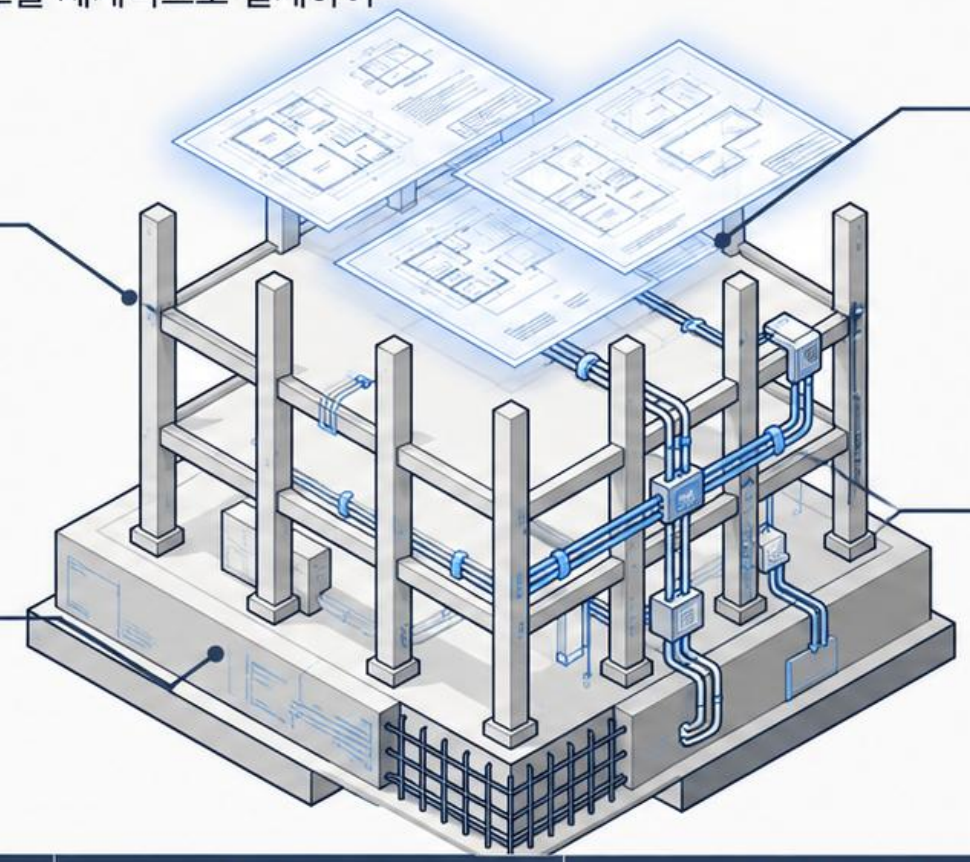
프롬프트 요소

역할, 배경, 제약, 관련 데이터 등
모델이 이해해야 할 모든 맥락과
입력 정보를 제공합니다.

[기초 Foundation]
목표와 성공 기준

프롬프트 요소

수행, 목표(Task), 기대 결과, 평가
기준 등 성공의 방향과 기준을
명확히 설정합니다.



[배선 Wiring]
논리적 작업 절차

프롬프트 요소

복잡한 작업을 단계별 절차로 분해하고,
사고 흐름과 의사결정 과정을 구조화하여
일관된 추론을 유도합니다.

[설계도 Blueprint]
출력 형식과 예시

프롬프트 요소

출력의 형식, 구조, 스타일과 함께
입출력 예시(Few-shot)를 제공하여
원하는 응답 형태를 명확히 합니다.

건축 비유	기초 (Foundation)	기둥 (Pillars)	배선 (Wiring)	설계도 (Blueprint)
프롬프트 요소	**목표와 성공 기준**	**맥락과 입력 정보**	**논리적 작업 절차**	**출력 형식과 예시**
주요 내용	수행 목표, 기대 결과, 평가 기준, 제약 조건	역할, 배경, 상황, 관련 데이터, 용어 정의 등	단계별 절차, 사고 흐름, 의사결정 규칙	출력 형식, 구조, 톤/스타일, 입출력 예시(Few-shot)
프롬프트 예시	"다음 분석 보고서를 작성해줘." "성공 기준: 정확성, 완전성, 간결성"	"당신은 마케팅 분석가입니다." "다음은 시장 조사 데이터입니다."	"1) 문제 정의 → 2) 원인 분석 → 3) 대안 도출 → 4) 최종 제안"	"다음 형식으로 작성해줘." [입력 예시] → [출력 예시]

효과적인 프롬프트의 6가지 요소

명확한 목표 · 충분한 맥락 · 참고 자료 · 작업 지시 · 출력 형식 · 검증

2026년 10월 기준

최신 프롬프트 엔지니어링 원칙을 반영한 실전 예시

6가지 요소	실전 예시 프롬프트 (마케팅 캠페인 기획)	기대 효과
1 목표 · 역할 Goal & Role	**친환경 신제품 출시를 위한 마케팅 전략을 제안해 주세요.** 마케팅 전략가 관점에서 작성합니다.	**목표 명확화** ✓ 원하는 결과와 역할을 명확히 설정 ✓ 일관된 관점 유지
2 맥락 · 제약 Context & Constraints	• **대상:** 25–40세 직장인 • **예산:** 1억 원 • **기간:** 3개월 • **지역:** 한국	**제약 조건 반영** ✓ 현실적인 범위에서 실질적인 제안 ✓ 모델의 오해 방지
3 자료 · 근거 Input Data & References	**제품 설명, 타킷 데이터, 이전 캠페인 성과를 참고합니다.** 자료가 없으면 가정이라고 표시합니다.	**근거 강화** ✓ 정확하고 신뢰도 높은 결과 생성 ✓ 가정이 필요한 경우 명확히 표시
4 수행 작업 Tasks & Steps	**다음 순서로 분석하고 제안해 주세요.** 시장 분석 → 고객 세분화 → 채널 전략 → KPI 제안	**작업 체계화** ✓ 복잡한 작업을 단계적으로 수행 ✓ 누락 없이 구조적인 결과 도출
5 결과 형식 Output Format	**다음 형식으로 결과를 제공해 주세요.** • 핵심 요약 • 점량 표 • 일정 표 • 예산 배분	**형식 일관성** ✓ 원하는 형태로 바로 활용 가능 ✓ 보고서, 표, 일정 등 가독성 향상
6 검증 · 안전 Verification & Safety	**사실과 추정을 구분하고 출처를 표시하며, 개인정보를 제외합니다.**	**검증 가능성** ✓ 사실 기반의 신뢰성 확보 ✓ 윤리적·법적 리스크 최소화

좋은 프롬프트 = 목표 + 맥락 + 자료 + 작업 + 형식 + 검증

활용 가이드

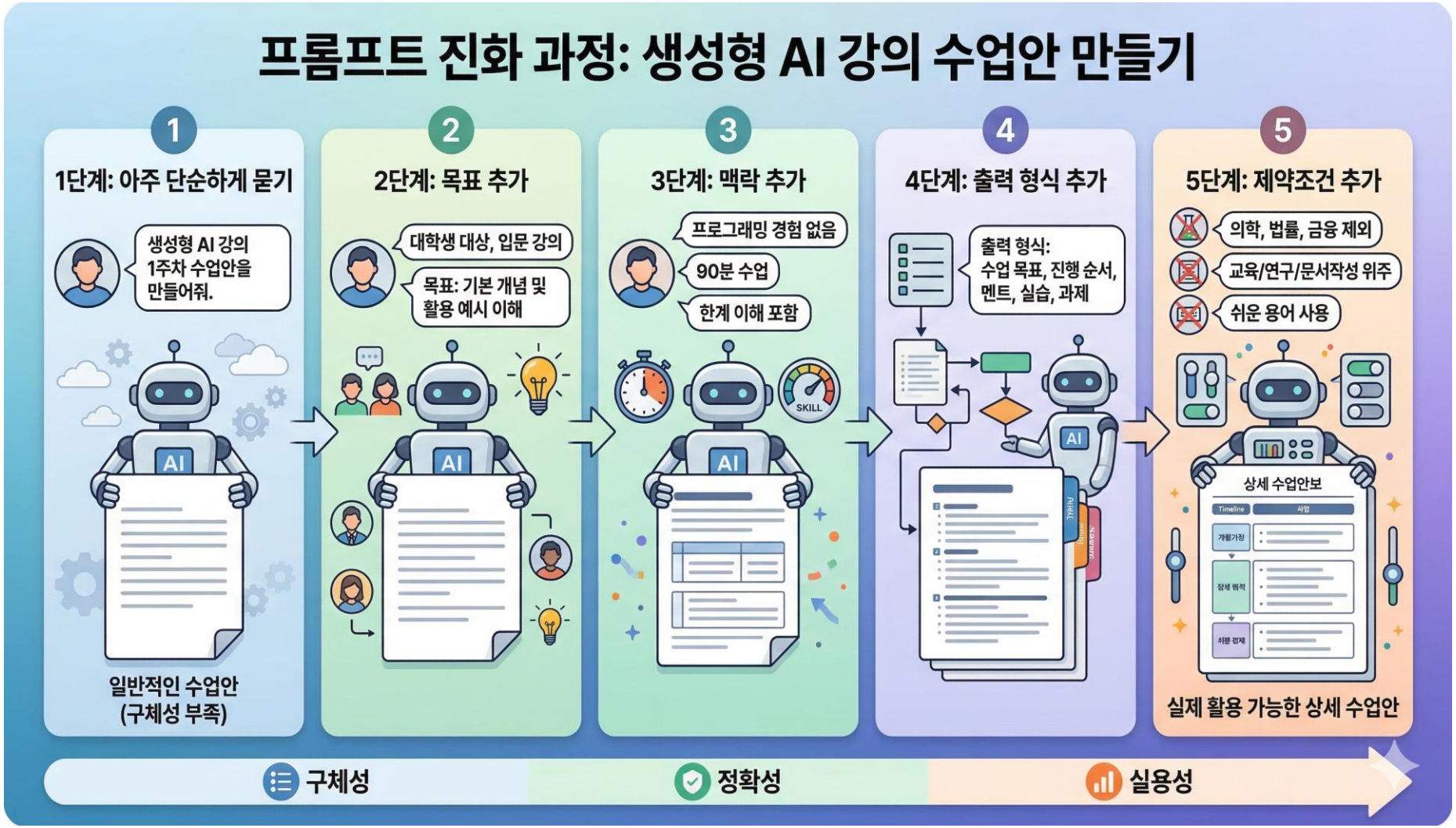
- 필요한 경우 예시를 제공하고 결과를 검토·수정하세요.
- 모든 작업에 6개 요소를 전부 요구할 필요는 없습니다.

출처

OpenAI Prompt Engineering · Anthropic Prompting Best Practices · Google Gemini Prompt Design Strategies (2026)

같은 질문을 “좋은 프롬프트”로 바꾸기

주제 : “생성형 AI” 강의 1주차 수업안을 만들어줘



1단계: 아주 단순하게 묻기

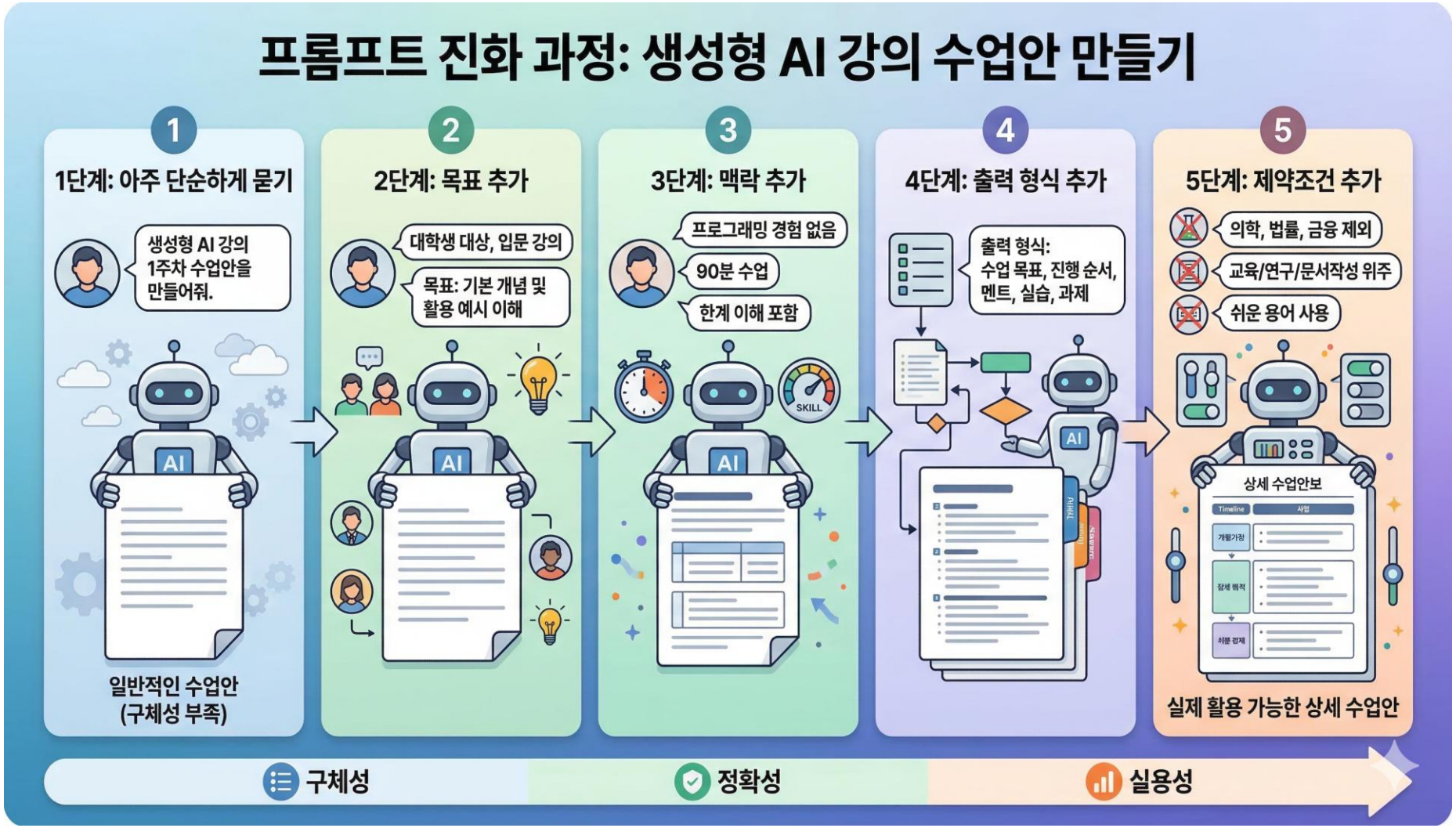
생성형 AI 강의 1주차 수업안을 만들어줘

2단계: 목표 추가

대학생 대상 생성형 AI 입문 강의 1주차 수업안을 만들어줘. 수업 목표는 학생들이 생성형 AI의 기본 개념과 활용 예시를 이해하는 것이야.

같은 질문을 “좋은 프롬프트”로 바꾸기

주제 : “생성형 AI” 강의 1주차 수업안을 만들어줘

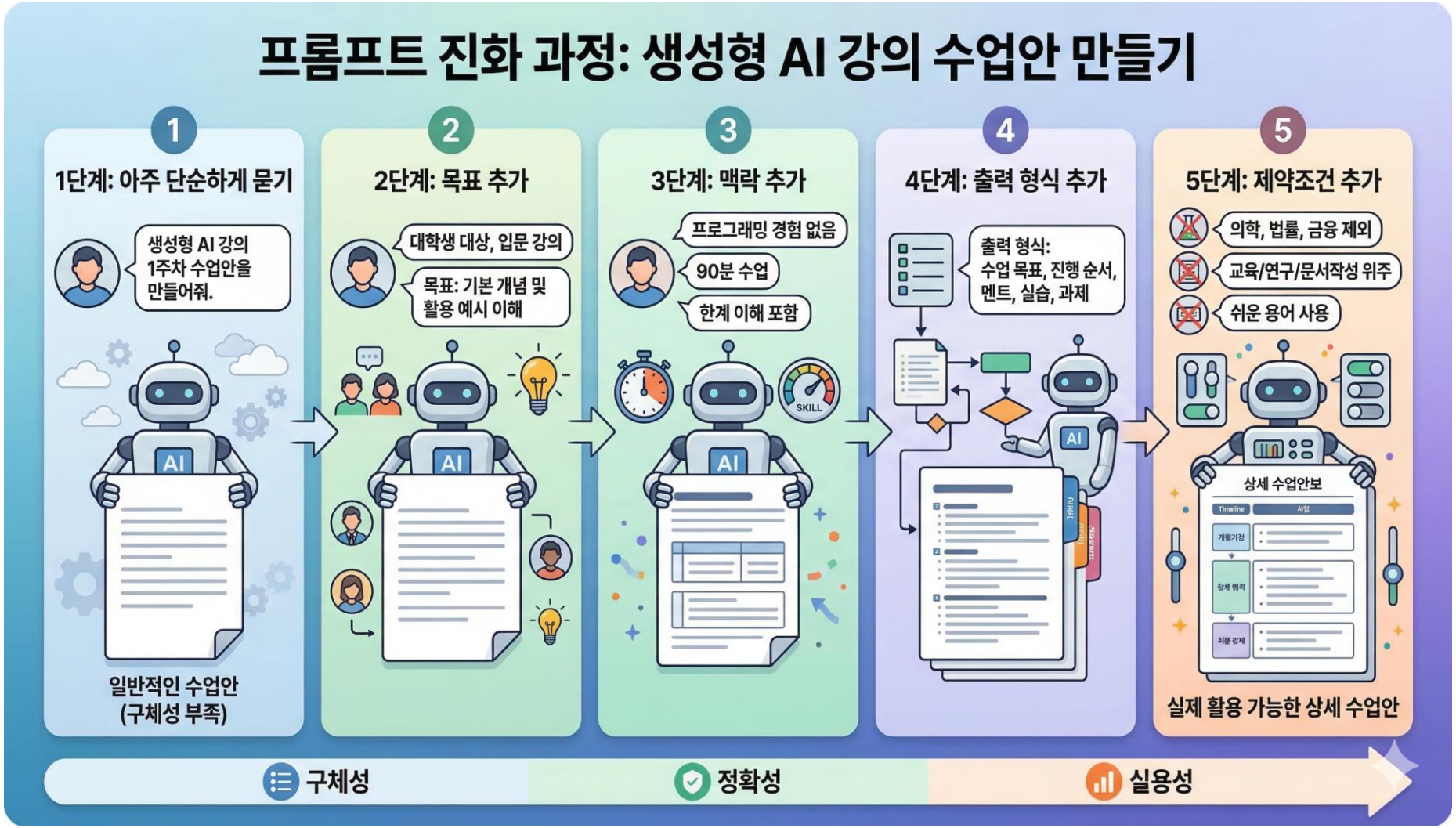


3단계: 맥락 추가

대학생 대상 생성형 AI 입문 강의 1주차 수업안을 만들어줘. 학생들은 프로그래밍 경험이 거의 없고, 90분 수업이야. 수업 목표는 생성형 AI의 기본 개념, 활용 사례, 한계를 이해 하는 것이야.

같은 질문을 “좋은 프롬프트”로 바꾸기

주제 : “생성형 AI” 강의 1주차 수업안을 만들어줘

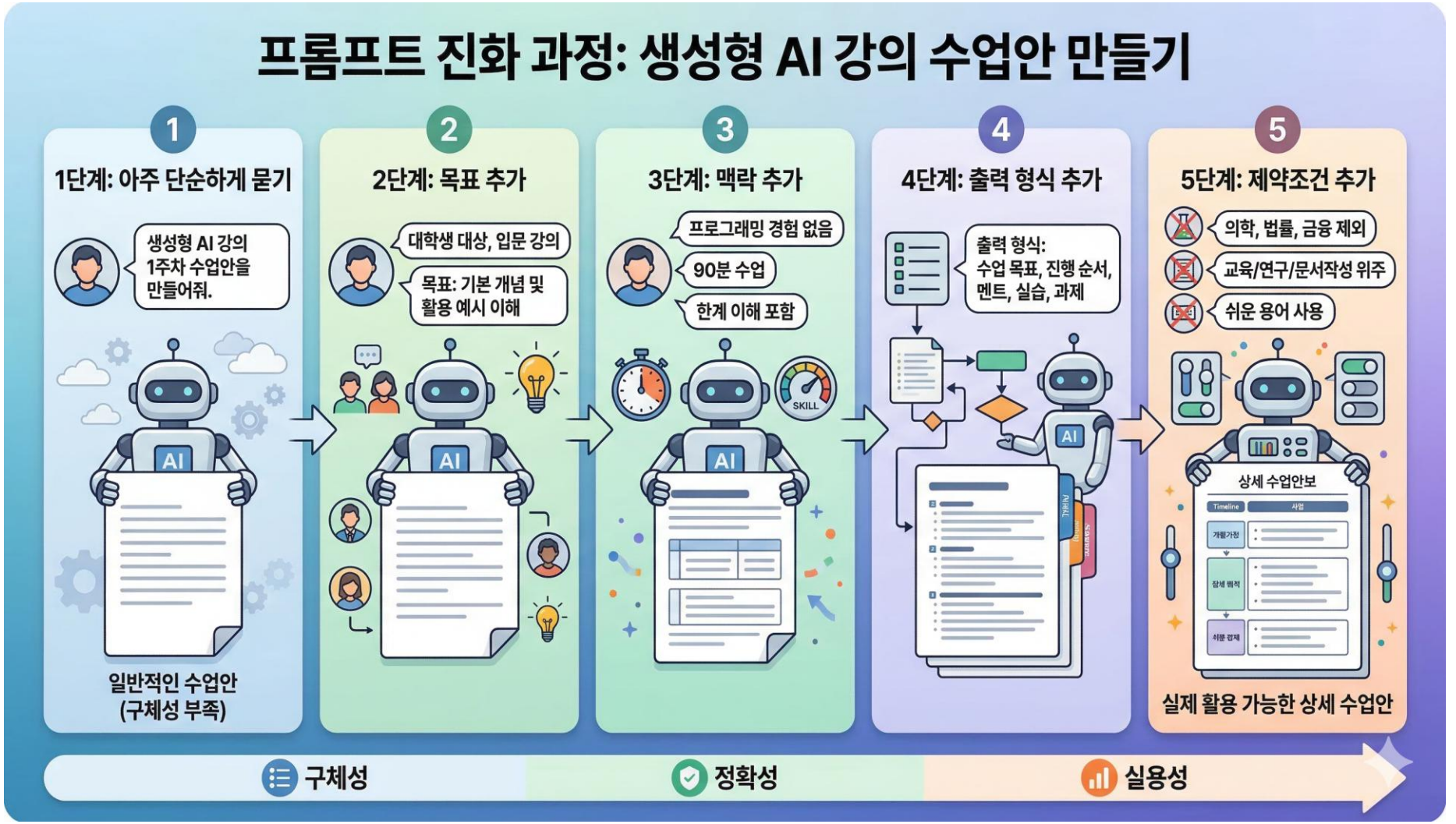


4단계: 출력 형식 추가

대학생 대상 생성형 AI 입문 강의 1주차 수업안을 만들어줘.
학생들은 프로그래밍 경험이 거의 없고, 90분 수업이다.
수업 목표는 생성형 AI의 기본 개념, 활용 사례, 한계를 이해하는 것이야.
다음 형식으로 출력해줘.
1. 수업 목표
2. 시간대별 진행 순서
3. 강의자 멘트 예시
4. 학생 실습 활동
5. 과제

같은 질문을 “좋은 프롬프트”로 바꾸기

주제 : “생성형 AI” 강의 1주차 수업안을 만들어줘



5단계: 제약조건 추가

대학생 대상 생성형 AI 입문 강의
1주차 수업안을 만들어줘.
학생들은 프로그래밍 경험이 거의 없고,
90분 수업이다.
수업 목표는 생성형 AI의 기본 개념,
활용 사례, 한계를 이해하는 것이다.
의학, 법률, 금융 사례는 제외하고
교육/연구/문서작성 사례 위주로
설명해줘.
너무 어려운 기술용어는 피하고,
초보자도 이해할 수 있게 써줘.
다음 형식으로 출력해줘.
1. 수업 목표
2. 시간대별 진행 순서
3. 강의자 멘트 예시
4. 학생 실습 활동
5. 과제

같은 질문을 “좋은 프롬프트”로 바꾸기

실습 과제:

1. “연구발표 연습자료 만들어줘”를 같은 방식으로 개선하기

2. “논문 요약해줘”를 구조화된 프롬프트로 바꾸기

3. “학생 과제 문제 만들어줘”를 평가 기준까지 포함해 바꾸기

좋은 프롬프트란 무엇인가?

좋은 프롬프트는 AI가 사용자의 의도를 정확히 이해하고,
필요한 정보를 정확하고 유용한 형태로 제공하도록 돕는 입력입니다.

핵심 목표

AI의 이해도를 높이고,
정확성·관련성·활용성을 높은 답변을 얻는 것

좋은 프롬프트의 핵심 특징

1. 명확하다

무엇을 원하는지
구체적이고 명확하게
전달한다.

2. 구체적이다

맥락, 조건, 예시 등을
포함하여 구체적으로
설명한다.

3. 관련성이 높다

목적과 상황에 맞게
AI가 필요한 정보를
이해할 수 있게 한다.

4. 구조화되어 있다

논리적 흐름과 형식을
지정하여 답변의 품질과
일관성을 높인다.

5. 적절한 어조 · 스타일

원하는 어조, 톤, 스타일을
지정하여 목적에 맞는
답변을 얻는다.

6. 목적 지향적이다

AI가 수행해야 할 역할과
목표를 명확히 하여
성과를 극대화한다.

나쁜 프롬프트 예시

마케팅 전략 알려줘.

- 너무 짧고 모호함
- 맥락과 조건이 없음
- 원하는 형식이나 범위가 불분명

좋은 프롬프트 예시

저는 대학생 대상 온라인 학습 플랫폼을 운영하고 있습니다.
신규 사용자 유치를 위한 마케팅 전략을 제안해 주세요.
타겟 분석, 채널 전략, 콘텐츠 아이디어, 설랭 계획을
표 형태로 정리해 주세요. 어조는 전문적이고 설득력 있게 작성해 주세요.

- 구체적인 맥락과 목표가 있음
- 원하는 결과의 범위와 형식 지정
- 어조와 스타일까지 명확히 요청

좋은 프롬프트가 주는 효과

정확성 향상	AI의 오해를 줄이고, 핵심을 정확히 파악
시간 절약	추가 설명이 줄어들어 반복 작업 감소
품질 향상	더 깊이 있고, 유용한 답변을 얻을 수 있음
활용도 증가	바로 적용 가능한 결과를 얻어 생산성 향상

좋은 프롬프트는 AI와의 대화가 아니라, 협업의 시작입니다. 명확하게 요정할수록, AI는 더 정확하고 가치 있는 답변으로 보답합니다.

CO-STAR 프롬프트 설계 프레임워크

2026년 10월 업데이트

AI 에이전트 시대의 범용 업무 설계 프레임워크 - 명확한 목적과 맥락을 기반으로 일관성 있고 재사용 가능한 프롬프트/업무를 설계합니다.

AI Agent 시대의 확장 요소

도구 · 권한
검색, 코드 실행,
API 범위 등 필요한 도구와
권한을 지정

근거 · 참조
파일, 데이터, 출처 등
참고 자료를 제공

검증 기준
정확성, 완료 조건,
테스트 방법을 명시

안전 · 보안
개인정보, 민감정보,
승인이 필요한 작업 범위를 지정

기대 효과

목표 정렬
AI와 사용자의 목표·기대치를
명확히 맞추어 일관된 작업 수행

답변 일관성
형식과 기준이 통일되어
재사용 가능한 고품질 결과 생성

실행 가능성
실무에서 바로 활용 가능한
구체적이고 현실적인 결과 제공

결과 검증
품질 기준에 따라 결과를 검토하고
정확성을 높일 수 있음

위험 관리
오류, 편향, 보안 이슈를 사전에
줄여 안전하게 활용 가능

핵심: 목표 · 성공 기준을 명확히, 필요한 맥락을 제공하고 결과를 검증한다.

목적 정의 › 맥락 제공 › 대상 고려 › 스타일/어조 설정 › 응답 형식 지정 › 검증 및 개선

TIP 복잡한 작업에는 세부 절차를 과도하게 강제하지 말고, 결과와 완료 기준 중심으로 작성하세요.

※ CO-STAR는 기본 프레임워크이며 도구·검증·안전은 최신 AI 에이전트 활용을 위한 확장 요소입니다.

CO-STAR (프롬프트 구성요소)

좋은 프롬프트는 6가지 요소를 명확하게 구성해야 합니다.

구성요소	설명	핵심 질문 (작성 가이드)
C Context (맥락)	배경 정보, 상황, 제약 조건, 관련 데이터 등을 제공하여 모델이 전체 맥락을 이해하도록 돕습니다.	**어떤 상황에서 요청하는가?** (예: 프로젝트 배경, 데이터 정보, 업무 목적, 현재 상태 등)
O Objective (목표)	수행해야 할 작업과 기대하는 결과를 구체적으로 정의합니다.	**무엇을 하길 원하는가?** (예: 분석, 요약, 생성, 비교, 평가, 제안 등)
S Style (스타일)	출력 형식, 글쓰기 스타일, 톤앤매너, 구조 등을 지정합니다.	**어떤 형식과 스타일로 답변하길 원하는가?** (예: 표, 코드, 보고서, 친절한, 전문적인, 간결한 등)
T Tone (톤)	답변의 말투, 분위기, 수준을 지정하여 목적에 맞는 표현 방식을 설정합니다.	**어떤 말투와 분위기로 답변하길 원하는가?** (예: 전문적, 친절한, 객관적인, 쉽게 설명하는, 설득력 있는 등)
A Audience (대상)	응답을 받을 대상(독자/사용자)의 지식 수준, 역할, 관심사 등을 명시합니다.	**누구를 위해 작성하는가?** (예: 고등학생, 비전문가, 실무자, 경영진, 고객 등)
R Response (응답)	출력의 범위, 길이, 구성, 표함/제외할 내용, 단계별 설명 등 구체적인 응답 요구사항을 지정합니다.	**어떤 형태로 답변을 제공하길 원하는가?** (예: 단계별 절차, 표/그래프 포함, 코드와 설명, 요약과 결론 등)

CO-STAR로 프롬프트 설계하기

6가지 요소를 모두 고려하여 **맥락이 명확하고, 목적이 분명하며, 대상과 톤에 맞는 프롬프트**를 작성하면 AI가 더 정확하고 유용한 결과를 생성합니다.

- CO-STAR는 좋은 프롬프트의 재료
- 각 요소를 조합하여 프롬프트 전략을 설계
- 요청의 목적과 상황에 맞게 요소의 비중을 조절
- 명확하고 구체적인 프롬프트가 더 나은 결과를 가져옵니다.

핵심 takeaway

CO-STAR의 6가지 요소를 균형 있게 설계해야 목적에 맞는 고품질의 응답을 얻을 수 있습니다.

프롬프트 전략 유형 (Taxonomy)

작업의 목적과 난이도에 따라 적절한 프롬프트 전략을 선택합니다.

전략 유형	설명 및 특징	주요 사용 상황	난이도
Zero-shot (제로샷)	• 추가 예시 없이 지시만으로 바로 답변 • 일반적인 질문, 정의, 간단한 요약 등에 적합	• 개념 설명 • 일반 상식 질의 • 간단한 정보 요청	낮음
Few-shot (퓨샷)	• 소수의 예시를 제공하여 패턴을 학습한 후 답변 • 특정 형식, 스타일 모방이 필요할 때 효과적	• 형식이 필요한 답변 • 데이터 변환 예시 • 스타일 일관성 유지	중간
Chain-of-Thought (CoT)	• 단계별 사고 과정을 명시적으로 요구 • 복잡한 추론, 계산, 논리 문제에 적합	• 복잡한 문제 해결 • 수리/논리 추론 • 분석 과정 설명	높음
Self-Consistency (자기 일관성)	• 여러 추론 경로를 생성하고 일관된 결론을 선택 • 복잡한 문제에서 정확도 향상	• 다단계 추론 • 여러 관점 검증 • 높은 신뢰도 필요	매우 높음
RAG (검색 증강 생성)	• 외부 지식(문서, 데이터, 웹)을 검색하여 근거 기반 답변 • 최신 정보, 도메인 지식 활용에 적합	• 최신 데이터 질의 • 사내 문서 기반 분석 • 전문 분야 정보 제공	높음
에이전트 기반 (Agentic)	• 목표를 세우고, 도구를 활용하여 단계적으로 작업 수행 • 복잡한 업무 흐름, 다단계 작업에 적합	• 데이터 수집·분석 • 보고서 자동 생성 • 업무 자동화 (도구 연계)	매우 높음

좋은 프롬프트는 **CO-STAR(맥락·목표·스타일·톤·대상·응답)** 요소를 바탕으로,
작업의 난이도와 목적에 맞는 **전략**을 선택하여 설계해야 합니다.

목적 명확화 → 맥락 제공 → 전략 선택 → 요소 조합 → 결과 검증 → 지속적 개선

C — Context: 맥락

AI가 정확하고 적절한 답변을 하기 위해 필요한 배경과 상황을 제공하는 단계입니다.

핵심 역할

AI가 문제를 제대로 이해할 수 있도록
필요한 배경정보와 제약조건을 명확히 전달합니다.

1. 어떤 상황인가?

현재 어떤 상황에서 프롬프트를
작성하고 있는지 설명합니다.

포함할 내용

- 현재 수행 중인 업무/프로젝트
- 사용 목적 (보고서 작성, 의사결정, 교육 등)
- 시간적 배경 (시점, 기간 등)

예시

"분기별 마케팅 성과 보고서를 작성 중이며,
경영진에게 공유할 예정입니다."

2. 어떤 배경정보가 필요한가?

AI가 맥락을 이해하는 데 필요한
중요한 배경정보를 제공합니다.

포함할 내용

- 관련 데이터/자료/문서 정보
- 조직, 산업, 고객 등 관련 배경
- 이전에 수행한 활동이나 결과

예시

"2026년 상반기 판매 데이터와 고객 설문
결과를 기반으로 분석해야 합니다."

3. 현재 문제와 제약은 무엇인가?

해결해야 할 핵심 문제와 작업 시
고려해야 할 제약조건을 명확히 합니다.

포함할 내용

- 해결해야 할 문제 또는 핵심 질문
- 시간, 예산, 데이터 등 제약조건
- 반드시 지켜야 할 규칙/정책

예시

"매출 감소 원인을 분석하고 개선 방안을 제시하되,
분석 기간은 6개월로 제한합니다."

좋은 Context를 작성하면?

AI의 이해도 향상
정확한 맥락 파악으로
더 관련성 높은 답변 생성

답변 품질 향상
필요한 정보 중심으로
구체적이고 실용적인 결과 제공

시간 절약
추가 질문/수정 없이
첫 답변의 완성도 증가

오류 감소
잘못된 가정이나 일반화로 인한
답변 오류 최소화

O — Objective: 목표

AI가 수행할 최종 목표와 원하는 결과, 성공 기준을 명확히 정의하는 단계입니다.

핵심 역할

목표가 명확할수록 AI의 응답이 구체적이고 정확해집니다.

1 AI가 최종적으로 무엇을 해야 하는가?

- AI가 수행해야 할 핵심 작업을 정의합니다.
- 결과물의 형태나 범위를 구체적으로 지정합니다.

예시

- 특정 주제에 대한 분석 보고서를 작성한다.
- 데이터를 분석하여 인사이트를 도출한다.
- 마케팅 전략을 제안한다.
- 긴 글을 요약한다.

2 원하는 결과는 무엇인가?

- AI가 만들어야 할 결과물의 내용과 특성을 정의합니다.
- 포함되어야 할 핵심 항목을 명확히 합니다.

예시

- 핵심 내용 5가지를 정리한 보고서
- 표와 그래프가 포함된 분석 결과
- 장점/단점과 추천 방안을 포함한 제안서
- 3가지 요점으로 요약된 본문

3 성공 기준은 무엇인가?

- 결과의 품질을 판단할 기준을 설정합니다.
- AI가 목표를 달성했는지 평가할 수 있는 기준을 제시합니다.

예시

- 내용이 정확하고 신뢰할 수 있는가?
- 요구한 형식과 분량을 준수했는가?
- 논리적이고 이해하기 쉬운가?
- 실행 가능한 인사이트가 포함되었는가?

Objective를 명확히 하면?

- **AI가 방향을 정확히 이해**
  불필요한 답변을 줄이고 핵심에 집중한 결과 생성
- **원하는 결과에 더 가까운 응답**
  필요한 항목과 구조를 갖춘 결과물을 얻을 수 있음
- **시간과 비용 절약**
  수정과 재요청을 줄여 업무 효율성 향상
- **일관되고 높은 품질 유지**
  명확한 기준으로 결과 품질을 안정적으로 확보

Tip “분석해줘” 보다는 “A 데이터를 분석해 매출 감소의 원인 3가지를 찾아 표로 정리해줘” 처럼 구체적으로 작성하세요.

S — Style: 스타일

AI가 정보를 어떤 방식과 형식으로 작성할지 결정하는 요소입니다.

핵심 역할

내용을 전달하는 형식과 구조를 정하여
독자가 이해하기 쉽고 목적에 맞게 구성합니다.

어떤 방식을 선택할 것인가?

구조와 형식 결정

정보를 구성하고 정리하는
형식(구조, 분량, 항목 등)을
선택합니다.

표현 방법 선택

텍스트, 표, 그래프, 이미지 등
어떤 요소를 활용할지 결정합니다.

구성 흐름 설계

도입 → 전개 → 요약 → 결론 등
정보의 흐름과 순서를 설계합니다.

대표적인 스타일 유형

보고서형

체계적이고 공식적인 문서 형식.
분석과 근거 중심으로 작성

예시
- 기업 보고서
- 프로젝트 제안서
- 시장 조사 보고서

강의자료형

슬라이드 또는 시각 자료 중심.
핵심 내용을 간결하게 전달

예시
- 수업/교육 자료
- 발표 슬라이드
- 워크숍 자료

논문형

학술적이고 객관적인 형식.
이론, 연구, 근거를 중심으로 작성

예시
- 학술 논문
- 연구 논문 초록
- 문헌 리뷰

블로그형

친근하고 읽기 쉬운 형식.
스토리텔링과 경험 공유 중심

예시
- 정보 공유 글
- 사용 후기
- 팁/가이드

가이드/매뉴얼형

단계별 설명과 실용 정보 중심.
따라 하기 쉬운 구조

예시
- 사용 가이드
- 튜토리얼
- FAQ 문서

스타일 선택 시 고려할 사항

목적에 적합한가?
전달하려는 목적과
일치하는 스타일인가?

대상에 적합한가?
독자의 배경지식과 관심사에
맞는 스타일인가?

정보 전달이 효과적인가?
핵심 내용을 명확하고
효과적으로 전달할 수 있는가?

분량과 시간에 적절한가?
요청된 분량과
시간에 맞는 스타일인가?

가독성과 일관성이 있는가?
읽기 쉽고, 전체적으로
일관된 형식인가?

Tip 스타일을 명확히 지정할수록 AI의 출력이 목적에 더 잘 맞고, 수정 시간을 줄일 수 있습니다.

예시 요청 문구 ▶ "강의 슬라이드 형식으로, 표와 아이콘을 활용해 작성해줘."

T — Tone: 어조

AI가 내용을 전달할 때의 말투와 분위기를 결정하는 요소입니다.
독자에게 어떻게 느껴지길 원하는지에 따라 어조를 선택하세요.

핵심 역할

메시지의 인상과 설득력을 좌우합니다.
대상과 목적에 맞는 어조는 이해도와 공감도를 높입니다.

대표적인 어조 유형

전문적으로

정확하고 신뢰감 있는
표현으로 전문성을 강조합니다.

키워드

정확성, 근거, 객관성,
전문 용어, 논리적

예시

"본 보고서는 통계 데이터를 기반으로
분석되었으며, 다음과 같은 결론을
도출합니다."

친근하게

편안하고 따뜻한 말투로
독자와의 거리를 좁힙니다.

키워드

일상적 표현, 쉬운 단어,
공감, 친근감, 대화체

예시

"어려운 내용도 쉽게 설명해드릴게요!
함께 차근차근 알아볼까요?"

설득력 있게

논리와 근거를 바탕으로
독자의 행동이나 결정을 유도합니다.

키워드

근거 제시, 강조, 해택 중심,
문제 해결, 행동 유도

예시

"이 방법을 사용하면 시간과 비용을
절약할 수 있습니다.
지금 바로 도입해보세요!"

중립적으로

감정이나 의견을 배제하고
사실 중심으로 전달합니다.

키워드

객관적, 사실 기반,
중립적 포현, 균형

예시

"해당 대이터는 2024년 1분기 기준이며,
다음과 같은 수치를 보입니다."

간결하게

불필요한 포현을 줄이고
핵심만 명확하게 전달합니다.

키워드

간결, 핵심 중심, 짧은 문장,
명확성, 요약

예시

"핵심은 세 가지입니다.
첫째, …
둘째, …
셋째, …"

어조 팁

대상 파악

누구에게 전달할지
먼저 생각하세요.

목적 명확화

설득, 안내, 공유 동
목적에 맞는 어조를
선택하세요.

상황 고려

공석 문서, 발포, SNS 동
상황에 어울리는 어조를
사용하세요.

일관성 유지

문장 전체에서
일관된 어조를
유지하세요.

다양한 어조 조합

필요에 따라 두 가지 이상
어조를 조합해 사용할 수도
있습니다.

Tip 프룸프트에 원하는 어조를 명확히 지정하면 AI의 답변 품질이 높아집니다.

예시 프룸프트 ▶ "전문적이고 신뢰감 있는 어조로, 이유와 근거를 포함해 설명해줘."

A — Audience: 대상

AI가 누구를 대상으로 답변할지 명확히 지정하는 요소입니다.
대상의 특성을 이해해야 적절한 수준과 표현으로 전달할 수 있습니다.

핵심 역할

독자의 특성을 정의하면 AI가 맞춤형 수준과
용어로 더 정확하고 이해하기 쉽게 답변합니다.

1 누가 읽는가?

답변의 독자(대상)를 구체적으로 정의합니다.

대상 예시

경영전/임원	실무자/직원	학생/학습자	일반 대중	전문가/연구자
의사결정과 전략 수립이 중요한 사람	현업에서 실무를 수행하는 사람	배우는 과정에 있는 학생 또는 학습자	해당 주제에 대해 일반적인 관심이 있는 사람	해당 분야의 지식과 경험이 풍부한 사람

2 배경지식 수준은?

독자의 사전 지식과 이해 수준을 설정합니다.

수준 예시

☆ 초급	★★ 중급	★★★ 고급	★★★ 전문가 수준
기본 개념을 처음 접하는 수준	기본 개념을 알고 있으며 활용 경험이 있는 수준	전문 지식을 갖추고 심화 내용을 이해할 수 있는 수준	깊이 있는 지식과 경험으로 새로운 통찰이 필요한 수준

3 어떤 용어 수준이 적절한가?

독자가 이해할 수 있는 용어와 표현 수준을 정합니다.

용어 수준 예시

Aa 쉬운 용어	일반적인 전문 용어	전문 용어	고급 전문 용어
일상 언어 중심, 비전문 용어 사용	기본적인 전문 용어 일부 사용	해당 분야의 전문 용어 적절히 사용	분야 특화 용어과 약어·기호 사용

대상 설정이 답변 품질을 결정합니다

이해도 향상
대상에 맞는 수준의 설명은
이해와 몰입도를 높입니다.

소통 효율성 증가
불필요한 설명은 줄이고,
핵심 메시지를 효과적으로
전달할 수 있습니다.

신뢰도 향상
대상을 고려한 정확한 표현은
전문성과 신뢰를 높입니다.

활용도 증가
실제 상황에 맞는 답변은
업무와 학습에 바로
활용할 수 있습니다.

잘못된 대상 설정의 결과

- 너무 어렵거나 전문용어가 과다하여 이해 어려움
- 너무 단순하여 필요한 정보가 부족함
- 독자의 니즈와 맞지 않아 활용도가 낮아짐
- 신뢰도와 만족도가 떨어질 수 있음

작성 팁

- **가능한 한 구체적으로 지정하세요.**
  예: "마케팅 부서 신입사원(업무 경험 6개월 이하)"
- **배경지식 수준을 명시하세요.**
  예: "데이터 분석을 처음 접하는 수준"
- **용어 수준과 표현 방식을 함께 지정하세요.**
  예: "전문용어는 최소화하고 쉬운 예시를 사용"

예시 프롬프트 문구

"마케팅 부서 신입사원(경험 6개월 이하)을 대상으로,
"대이터 분석을 처음 접하는 수준에 맞춰, 쉬운 용어로 설명해줘."

R — Response: 응답 형식

AI가 결과를 어떤 형식과 구조로 제공할지 지정하는 요소입니다.
원하는 형식을 명확히 제시하면 더 정확하고 활용하기 쉬운 답변을 얻을 수 있습니다.

핵심 역할

결과물을 구조화하여 바로 활용할 수 있게 하고,
형식 일관성과 가독성을 높입니다.

주요 응답 형식 유형

표 (Table)

데이터를 행과 열로 정리하여 비교와 분석이 쉽도록 제공

예시

항목	내용	수치
A	항목 A 설명	120
B	항목 B 설명	85
C	항목 C 설명	63

활용 예시
비교, 통계, 일정, 요약 정리 등

Markdown

제목, 목록, 강조, 코드블록 등으로 구조화된 텍스트 제공

예시

```
# 제목
## 소제목
- 항목 1
- 항목 2
**강조 텍스트**
```python
print("Hello")
```
```

활용 예시
문서 작성, 기술 문서, 노트 정리 등

코드 (Code)

프로그래밍 연어로 작성된 코드를 제공

예시

```
def add(a, b):
    return a + b


result = add(3, 5)
print(result)  # 8
```

활용 예시
개발, 데이터 분석, 자동화 등

JSON

구조화된 데이터를 키-값 형태로 정확하게 제공

예시

```
{
 "name": "AI",
 "type": "기술",
 "features": ["학습", "추론"],
 "use": "업무 자동화"
}
```

활용 예시
API 연동, 데이터 자장, 시스템 연계 등

슬라이드 구조

프레젠테이션에 바로 사용할 수 있는 슬라이드 구성 제공

예시

슬라이드 1. 제목
- 핵심 메시지

슬라이드 2. 배경
- 현황 및 문제

슬라이드 3. 해결 방안
- 핵심 전략

슬라이드 4. 결론
- 기대 효과

활용 예시
발표 자료, 제안서, 교육 자료 등

글자 수·항목 수 등

분량, 항목 수, 단계 수 등을 지정하여 결과를 통제

예시

- 글자 수: 500자 내외
- 항목 수: 5가지
- 단계 수: 4단계
- 문단 수: 3개 이내
- 표: 1개 표함

활용 예시
분량 관리, 핵심 요약, 가이드 작성 등

지정 팁

- 필요한 형식을 구체적으로 지정하세요.
  "마크다운 형식으로, 소제목과 목록을 포함해 작성해줘."
- 구조와 분량 제한을 함께 제시하세요.
  "5가지 항목으로 표로 정리하고, 각각 2줄로 설명해줘."
- 후속 활용을 고려해 형식을 선택하세요.
  "슬라이드 4장 분량으로, 발표용 핵심만 정리해줘."

좋은 응답 형식의 장점

- 정보 전달이 명확하고 가독성이 높아집니다.
- 후속 작업(복사, 편집, 연동)이 쉬워집니다.
- 일관된 구조로 재사용과 자동화가 용이합니다.

예시 프롬프트

"중소기업의 디지털 전환 전략을 5가지로 정리해 표로 작성해줘.
각 항목은 목표, 주요 내용, 기대 효과를 포함하고, 표 아래에 핵심 요약을 3줄로 작성해줘."

기대되는 응답 형식
표(3열, 5행) + 핵심 요약(3줄)

하나의 단순한 프롬프트를 CO-STAR로 완성하는 예

구체적인 맥락과 목적, 스타일, 대상, 형식을 추가하면 AI가 더 정확하고 유용한 답변을 제공합니다.

단순한 프롬프트 (시작점)

> 생성형 AI를 설명해줘.

문제점

- 맥락이 없어 상황 파악이 어려움
- 원하는 결과와 범위가 불명확
- 누구를 위한 설명인지 알 수 없음
- 어떤 형식으로 답해야 할지 모호

CO-STAR 프레임워크로 완성하기

C	Context (맥락)	신입사원 AI 교육을 준비하고 있다. 회사 업무에 활용할 수 있도록 소개 자료가 필요하다.
O	Objective (목표)	생성형 AI의 개념을 이해시키고, 업무에서 활용할 수 있는 방법을 설명한다.
S	Style (스타일)	실무 교육 자료 스타일로 작성한다. 핵심 내용이 한눈에 들어오도록 구성한다.
T	Tone (어조)	쉽고 친근하면서도 전문성을 유지한다. 현실적인 사례를 들어 설명한다.
A	Audience (대상)	생성형 AI를 처음 접하는 신입사원. AI 지식이 많지 않은 수준을 가정한다.
R	Responce (응답 형식)	개념 → 활용 사례 → 주의사항 순으로 정리한다. 표 1개와 핵심 요약(5가지)을 포함한다. 전체 글자 수는 800~1,000자 내외로 작성한다.

완성된 CO-STAR 프롬프트 (예시)

> 신입사원 AI 교육을 준비하고 있습니다.
> 회사 업무에 활용할 수 있도록 소개 자료가 필요합니다.
>
> [Objective] 생성형 AI의 개념을 이해시키고,
> 업무에서 활용할 수 있는 방법을 설명해주세요.
>
> [Style] 실무 교육 자료 스타일로 작성해주세요.
> 핵심 내용이 한눈에 들어오도록 구성해주세요.
>
> [Tone] 쉽고 친근하면서도 전문성을 유지해주세요.
> 현실적인 사례를 들어 설명해주세요.
>
> [Audience] 생성형 AI를 처음 접하는 신입사원을
> 대상으로 작성해주세요. AI 지식이 많지 않은 수준을
> 가정해주세요.
>
> [Response] 개념 → 활용 사례 → 주의사항 순으로
> 정리해주세요. 표 1개와 핵심 요약(5가지)을 포함하고,
> 전체 글자 수는 800~1,000자 내외로 작성해주세요.

AI 응답 결과 예시 (구조)

1. 생성형 AI란?
- 개념 설명 …

2. 업무 활용 사례
- 다양한 사례 …

분야	활용 예시
문서 작성	보고서 초안 작성
마케팅	콘텐츠/광고 문구 생성
고객 지원	FAQ 자동 생성
데이터 분석	데이터 요약/분석
… …	

3. 주의사항
- 유의할 점 …

핵심 요약 (5가지)

1. 생성형 AI는 새로운 콘텐츠를 생성하는 AI
2. 업무 효율성과 창의성 향상에 도움
3. 정확성 검토와 검증이 필수
4. 개인정보 및 보안에 유의
5. AI는 보조 도구, 최종 판단은 사람

포인트

- ✓ 구체적일수록 AI의 이해도가 높아집니다.
- ✓ 대상과 목적이 명확하면 불필요한 설명을 줄여줍니다.
- ✓ 형식을 지정하면 바로 활용 가능한 결과를 얻을 수 있습니다.
- ✓ CO-STAR를 활용하면 일관되고 재사용 가능한 프롬프트를 만들 수 있습니다.

CO-STAR 프레임워크

목표 : 좋은 프롬프트가 운에 맡기는 문장이 아니라, 요소를 조합하는 설계라는 것을 익혀보자

주제: 학부생 대상 머신러닝 입문 특강

Context:

이번 특강은 파이썬을 처음 배우는 학부생 대상이다.

Objective:

특강 소개문을 작성하되, 학생들이 부담 없이 신청하고 싶어지게 만든다.

Style:

간결하고 홍보문답게 작성한다.

Tone:

친절하고 명확하게 쓴다.

Audience:

프로그래밍 초보 학부생

Response:

1) 100자 소개

2) 300자 소개

3) 포스터용 핵심 문구 3개

CO-STAR 프레임워크

실습 과제:

1. 영어 강의 소개문
2. 연구실 홍보 소개문
3. 대학원 신입생 OT 공지
4. 수업 과제 안내문
5. 발표 초록 초안 작성

CRAFT 프레임워크란?

AI에게 필요한 정보를 구조화하여 더 정확하고 유용한 결과를 얻는 프롬프트 설계 방법

"Python에 대해서 알려줘"
(일반적인 프롬프트)

AI는 **왜** 만드는지, **어떤 역할**을 해야 하는지, **누구**를 위한 것인지, **어떤 형식**으로 만들어야 하는지 스스로 추측해야 합니다.

CRAFT는 이러한 정보를 명시적으로 제공합니다.

CRAFT = Context + Role + Action + Format + Target

C – Context
맥락·배경

지금 어떤 상황인가?

- 현재 상황, 배경, 목적
- 사용 중인 대이터, 정보
- 이미 알고 있는 내용
- 제한 조건, 환경

R – Role
AI의 역할

AI가 누구처럼 행동해야 하는가?

- 전문가/직무/관점 지정
- 필요한 전문성, 경험
- 특정 스타일의 역할 부여

A – Action
수행할 작업

무엇을 해야 하는가?

- 수행해야 할 작업을 명확히
- 단계별로 구체적으로 지시
- 동사(분석, 비교, 설명, 제안 등)

F – Format
결과 형식

어떤 형태로 출력해야 하는가?

- 표, 목록, 코드, 글, 슬라이드 등
- 구조, 순서, 길이, 스타일
- 마크다운, JSON 등 지정

T – Target
대상

누구를 위한 결과물인가?

- 독자/사용자 정의
- 지식 수준, 배경, 필요
- 이해하기 쉬운 수준 조절

예시로 이해하기

일반적인 프롬프트

Python에 대해서 알려줘

- 배경? 역할? 대상? 형식? → 모두 불명확
- 결과: 길고 산만하거나 원하는 내용이 아닐 수 있음

CRAFT 적용 프롬프트 예시

Context	대학생을 대상으로 Python 기초 수업을 준비하고 있다.
Role	너는 Python을 가르치는 경험이 많은 대학 교수이다.
Action	Python의 개념과 주요 특징, 장점, 활용 분야를 설명해라.
Format	개념 → 특징 → 장점 → 활용 분야 순서로 표와 예시를 포함해 정리해라.
Target	프로그래밍을 처음 접하는 비전공 1학년 학생이 이해할 수 있도록 작성해라.

기대 효과

- 정확하고 관련성 높은 내용
- 이해하기 쉬운 설명과 구조
- 원하는 형식으로 깔끔한 결과
- 수업/업무에 바로 활용 가능

왜 CRAFT가 중요할까?

- **추측 감소**
  AI가 스스로 추측해야 하는 부분을 최소화하여 오해와 누락을 줄입니다.
- **의도 명확화**
  요청의 목적과 기대하는 바를 정확하게 전달할 수 있습니다.
- **결과 품질 향상**
  필요한 정보만 정확한 형식으로 더 높은 품질의 결과를 얻습니다.
- **재사용성 증가**
  구조화된 프롬프트는 다양한 상황에서 수정·재사용하기 쉽습니다.
- **학습, 업무 효율 증가**
  좋은 프롬프트 습관은 시간과 노력을 아껴주고 성과를 높여줍니다.

핵심 정리

CRAFT는 AI에게 '어떤 상황에서(Context), 어떤 역할로(Role), 무엇을 수행하고(Action), 어떤 형태로 결과를 만들며(Format), 누구를 대상으로 할지(Target)'를 명확하게 전달하는 프롬프트 설계 방법입니다.

C – Context: 맥락

AI가 좋은 답변을 하려면 '왜, 어떤 상황에서, 무엇을 위해' 요청하는지 알아야 합니다.

Context는 AI에게 현재 상황과 배경정보를 알려주는 부분입니다.

AI는 단순히 "무엇을 해라"만 알면 필요한 세부 정보들을 스스로 추측해야 하므로,
원하는 수준과 방향이 아닌 답변을 줄 수 있습니다.

따라서, **배경을 명확히 제공**하면 더 정확하고 적절한 답변을 받을 수 있습니다.

배경 정보가 없는 경우 (나쁜 예)

"Python을 설명해줘."

AI가 대상, 수준, 목적 등을 추측해야 하므로
너무 기본적이거나, 너무 어려운 내용을 포함할 수 있습니다.

"누구를 대상으로"
어떤 수준으로?
어떤 형식으로?"

배경 정보를 제공한 경우 (좋은 예)

"비전공 대학생을 대상으로
"Python 첫 수업을 준비하고 있다."

AI가 보다 정확한 수준과 범위, 형식으로
적절한 답변을 제공합니다.

대상, 수준, 목적이
명확하니 정확하게
설명할 수 있어요!

예시 **좋은 Context 예시**

"대학 1학년 비전공자를 대상으로 Python 기초 수업을 준비하고 있다.
학생들은 프로그래밍 경험이 없으며 변수와 자료형까지 학습한 상태이다."

Context에 들어갈 수 있는 정보

- **현재 상황**
  지금 어떤 상황인지 (수업 준비, 프로젝트 진행, 보고서 작성 등)
- **문제의 배경**
  문제가 발생한 배경, 관련된 환경이나 역사적 맥락
- **작업 목적**
  이 작업을 통해 무엇을 달성하고자 하는지
- **사용 중인 데이터**
  데이터의 종류, 출처, 크기, 특성 등
- **이미 알고 있는 내용**
  이미 학습했거나 이해하고 있는 범위
- **현재 단계**
  전체 과정 중 지금 어디 단계인지
- **제한 조건**
  시간, 분량, 사용 가능한 도구, 금지된 내용 등 제약 사항

이렇게 하면...

AI가 갑자기 클래스나 데코레이터 같은 고급 내용을
설명할 가능성이 줄어들고,
수준과 범위가 적절한 답변을 받을 수 있습니다.

R – Role: 역할

AI에게 어떤 전문가(역할)의 관점에서 답변할 것인지 지정하는 요소
역할을 명확히 하면 답변의 전문성, 관점, 깊이가 달라집니다.

AI는 역할을 통해 '누구의 관점으로, 어떤 지식과 경험을 활용해' 답변할지 결정합니다.
같은 질문이라도 역할을 다르게 설정하면 답변의 내용과 깊이가 완전히 달라집니다.

예시 비교

이 데이터를 분석해줘. → 너는 10년 경력의 데이터 분석가다. 이 데이터를 분석해줘.

Role의 예시

데이터 분석가
데이터 분석, 통계, 시각화, 인사이트 도출에 강점

"너는 데이터 분석가다."

대학 교수 (Python)
교육적 설명, 개념 이해, 학습자 수준 고려

"너는 대학에서 Python을 가르치는 교수다."

UX/UI 전문가
사용자 경험, 화면 설계, 사용성, 디자인 원칙

"너는 UX/UI 전문가다."

머신러닝 연구자
의료영상, ML 모델, 연구 방법론에 대한 전문성

"너는 의료영상 연구 경험이 있는 머신러닝 연구자다."

Role에 따른 답변 차이 예시

질문: 다음 매출 데이터를 분석해줘.

역할 없이 요청한 경우 (모호한 프롬프트)

일반적인 데이터 요약 수준의 답변

- 전체 매출 요약 제공
- 기본 통계 정보 나열
- 제한적인 인사이트 제시

예시 답변:
전체 매출은 1억 2천만원입니다.
평균 매출은 2천만원이며, 최고 매출은 3천만원입니다.
월별 매출 추이는 다음과 같습니다...

VS

구체적인 Role을 지정한 경우 (명확한 프롬프트)

전문가의 관점에서 깊이 있는 분석 제공

- 업무 맥락에 맞는 분석 관점 적용
- 핵심 인사이트 도출 및 원인 분석
- 구체적인 개선 제안 및 실행 방안 제시

예시 답변 (데이터 분석가 역할):
매출 데이터를 분석한 결과, 3월 매출 급감의 주요 원인은...
제품 A의 판매 부진이 전체 매출에 미친 영향을 분석하면...
다음과 같은 개선 방안을 제안합니다...

효과적인 Role 작성 방법

- **구체적인 전문 분야를 명시하세요.**
  예) 마케팅 전문가, 금융 데이터 분석가, iOS 개발자
- **경험 수준이나 경력을 포함하면 더 좋습니다.**
  예) 10년 경력의, 실무 경험이 풍부한, 연구 경험이 있는
- **업무/상황과 관련된 전문성을 강조하세요.**
  예) 제조업 데이터를 분석하는, 스타트업 UX 설계를 담당하는

피해야 할 Role 작성 예시

- 너는 세상에서 가장 뛰어난 천재 데이터 분석가다.
  → 과장된 표현은 불필요하며, 오히려 모호해집니다.
- 너는 모든 분야에 통달한 전문가다.
  → 너무 넓은 역할은 답변의 초점이 흐려집니다.

핵심 Tip

Role은 AI가 '어떤 전문가의 렌즈'를 쓰고 답변할지 결정하는 나침반입니다.
구체적이고 현실적인 역할 설정이 더 정확하고 유용한 결과를 만듭니다.

A – Action: 수행할 작업

Action은 CRAFT에서 가장 중요한 요소입니다.
AI에게 실제로 무엇을 해야 하는지 명확하게 지시하세요.

Action의 핵심

AI가 추측하지 않도록, 구체적으로 무엇을 해야 하는지 지시합니다.
동사(행동) 중심으로 작성하면 더 정확한 결과를 얻을 수 있습니다.

Action은 동사(행동) 중심으로!

분석해라 | 비교해라 | 찾아라 | 설명해라 | 제안해라 | ...

나쁜 예 (모호한 Action)

"이 데이터에 대해 알려줘."

문제점

- 무엇을 분석해야 하는지 불명확
- 어떤 항목을 찾아야 하는지 모호
- 결과의 범위와 깊이를 AI가 임의로 판단

좋은 예 (명확한 Action)

"월별 매출 추이를 분석하고 매출이 가장 크게 증가하거나 감소한 달을 찾아 원인을 추정해라."

장점

- 분석해야 할 내용이 명확
- 찾아야 할 항목과 목적이 구체적
- 더 정확하고 일관된 결과를 얻을 수 있음

더 구체적인 예 (단계별 Action)

"다음 작업을 수행해라."

1. 월별 매출을 계산한다.
2. 전월 대비 증감률을 계산한다.
3. 가장 큰 감소가 발생한 월을 찾는다.
4. 제품군별로 감소 원인을 분석한다.

복잡한 작업은 단계별로 나누어 지시하면
AI가 누락 없이, 더 체계적으로 수행할 수 있습니다.

Action 작성 팁

구체적으로
대상, 범위, 조건을 구체적으로 명시하세요.
예) "매출" → "월별 매출"

단계적으로
여러 작업은 순서대로 단계별로 작성하세요.
예) 1단계, 2단계, 3단계 ...

측정 가능하게
계산, 수치, 순위 등 명확한 결과를 요구하세요.
예) "증감률 계산", "Top 3 찾기"

필터/조건 포함
기간, 지역, 제품 등 필터 조건을 포함하세요.
예) "2024년, 국내, 전체 제품"

핵심 목적 제시
이 작업을 통해 무엇을 알고 싶은지 목적을 밝히세요.
예) "원인 분석을 통해 개선 방안 도출"

정리

Action은 AI에게 '무엇을 해야 하는지'를 명확히 알려주는 핵심 요소입니다.
구체적이고, 단계적이며, 동사 중심으로 작성할수록 더 정확하고 유용한 결과를 얻을 수 있습니다.

Tip!
모호한 지시는 모호한 결과를,
명확한 Action은 명확한 결과를 만듭니다.

F – Format: 출력 형식

AI가 답변을 어떤 형태로 만들어야 하는지 지정하는 요소입니다.
출력 형식을 명확히 지정하면 원하는 형태의 결과를 빠로고 청확하게 얻을 수 있습니다.

Format의 핵심

같은 내용을 요청해도 출력 형식에 따라
결과의 가독성, 활용도, 품질이 크게 달라집니다.
AI에게 "어떤 형태로 보여줄지"를 명확히 알려주세요.

Format의 예시

- 표 (Table)
- Markdown
- Bullet Point
- 코드 (Python 등)
- JSON
- 슬라이드 구조
- 구조화된 순서 (제목→핵심→예제)

같은 요청도 Format에 따라 결과가 달라집니다!

✗ 형식 지정이 없는 요청 (비효율적)

"Power BI와 Tableau를 비교해줘."

AI의 응답 (긴 설명 위주)

- Power BI는 Microsoft에서 개발한 ... (긴 설명)
- Tableau는 데이터 시각화에 강점이 있으며 ... (긴 설명)
- 두 도구는 기능, 가격, 사용성 등 여러 측면에서 ... (긴 설명)

...

필요한 항목을 찾기 어렵고, 표나 요약이 없어 바로 활용하기 힘듬

✓ 형식을 명시한 요청 (효율적)

"Power BI와 Tableau를
가격 / 데이터 처리 / 시각화 / 학습 난이도 / 협업
항목으로 **비교**해서 **표로 만들어줘.**"

AI의 응답 (표 형태)

항목	Power BI	Tableau
가격	상대적으로 저렴 (라이선스 다양)	상대적으로 높음 (유연한 라이선스)
데이터 처리	대용량 데이터 처리에 강점 (모델링, DAX)	실시간 연결 및 다양한 데이터 소스 지원
시각화	비즈니스 리포트에 최적화 (통합 Microsoft 생태계)	고급 시각화에 강점 (시각적 표현 자유도 높음)
학습 난이도	초보자 친화적	중급 이상에 적합
협업	MS Teams, SharePoint 등 연동이 용이	Tableau Server/Cloud로 협업 기능 제공

핵심 항목을 한눈에 비교할 수 있어 바로 활용 가능!

Format 작성 팁

1. **목적**에 맞는 형식을 선택하세요.
   (표, 목록, 코드 등)
2. **구체적으로 작성**하세요.
   예) "표로 작성해줘" → "다음 항목으로 비교하여 표로 작성해줘"
3. **구조와 순서를 지정**하세요.
   예) "제목 → 핵심 개념 → 예제 → 실습 문제 순으로 작성해줘"
4. **길이, 개수, 수준 등을 지정**하면 더 정확한 결과를 얻을 수 있습니다.
   예) "5개의 bullet point로 요약해줘"

핵심 정리

Format을 명확히 지정하면
AI는 원하는 구조와 형태로,
더 빠로고 정확하게 답변합니다.

핵심 메시지

좋은 답변은 좋은 **형식**에서 시작됩니다.
AI에게 **출력 형식을 명확히 알려주면**, 결과의 품질과 활용도가 크게 향상됩니다.

기억하세요! 명확한 Format 지정 = 시간 절약 + 높은 품질 + 즉시 활용 가능

T – Target: 대상

Target은 결과물을 누가 사용할 것인지 지정하는 요소입니다.
대상에 따라 설명의 깊이, 전문용여 수준, 예제의 난이도가 달라집니다.

같은 주제라도 대상이 달라지면 전달 방식이 원전히 달라집니다.

Target을 명확히 지정하면,
AI가 적절한 수준과 톤으로 답변을 제공합니다.

Target을 지정하면 좋은 점

- 적절한 난이도와 깊이
- 이해하기 쉬운 설명 방식
- 알맞은 예제와 비유 제공
- 불필요한 전문용어 최소화

예시 주제: "기후변화"에 대해 설명해줘

초등학생 대상

지구가 점점 더워지고 있어서
날씨가 이상해지는 현상이에요.
우리가 쓰는 자동차나 전기가
많은 이산화탄소를 만들어 내고,
그것이 지구를 뜨겁게 만들어요.

- 쉬운 단어, 짧은 문장, 비유 중심
- 생활 속 예시 사용 (얼음 늑음, 폭염 등)

대학생 대상

기후변화는 온실가스 증가로 인해
지구 평균기온이 상승하고,
이로 인해 극한 기상 현상, 해수면 상승,
생태계 변화 등이 발생하는
전 지구적 문제입니다.

- 개념과 원인, 영항, 해결 방향을 체제적으로 설명
- 과학적 근거와 통계, 사례 제시

정책 연구자 대상

기후변화는 복합적 시스템 변화로서,
온실가스 배출량, 기후 민겁도,
사회·정제적 취약성의 상호작용에 의해
영향이 중폭됩니다. 정책 평가를 위해서는
정량 모델과 시나리오 분석이 필수적입니다.

- 전문용어와 이론적 배경, 모델/지표 중심
- 정책적 시사점과 분석 방법론 포함

Target 지정 예시

- Target: 생성형 AI를 처음 사용하는 직장인
  → 업무에서 AI를 활용하는 방법을 쉽게 알려줘.
- Target: Python을 처음 배우는 대학 1학년 학생
  → Python 기초 문법을 쉽고 자세하게 설명해줘.

작성 팁

- ✔ 대상의 배경지식과 목적을 함께 고려하세요.
- ✔ 누구에게 설명할지 구체적으로 표현하세요.
- ✔ 대상에 맞는 언어, 예시, 톤을 요청하세요.
- ✔ 여러 대상을 동시에 지정하면 우선순위를 정하세요.

질문 예시 비교

나쁜 예

기후변화에 대해 설명해줘.

→ 대상이 불명확하여
적절한 수준을 알 수 없음

좋은 예

Target: 대학 1학년 학생
기후변화에 대해
쉽게 설명해줘.

→ 명확한 대상 지정으로
적절한 수준의 답변 제공

핵심 정리

Target은 "누구를 위한 결과물인가"를 결정하는 핵심 요소입니다.
대상을 명확히 지정할수록 AI가 더 정확하고 유용한 답변을 제공합니다.

CRAFT 프레임워크 – 하나의 예제로 완성하기

단순한 프룸프트를 CRAFT 구조로 구체화하면, AI가 더 정확하고 유용한 답변을 제공항합니다.

단순한 프룸프트 (Before)

"머신러닝을 설명해줘."

AI가 상황, 역할, 목적, 형식, 대상 등을
스스로 추축해야 하므로 결과가 모호할 수 있습니다.

C Context (맥락)

왜 이 질문을 하는가?
어떤 상황인가?

대학생을 대상으로
데이터 분석 기초 강의를
준비하고 있다.

R Role (역할)

AI가 어떤 전문가의
관점에서 답해야 하는가?

너는 데이터 사이언스를
가르치는
대학 교수다.

A Action (수행할 작업)

무엇을 해야 하는가?

머신러닝의 개념을 설명하고
지도학습, 비지도학습,
강화학습의 차이를 설명해라.

F Format (출력 형식)

어떤 형태로
출력해야 하는가?

먼저 어신러닝을 3문장으로
설명하고, 이후 세 가지
학습 방변을 표로 비교한 뒤
각각 하나의 설제 활용 사례를
제시해라.

T Target (대상)

누구를 위한 결과물인가?
어떤 수준으로 설명할까?

수학이나 프로그래밍
경험이 많지 많은
대학 1학년 학생을
대상으로 한다.

모든 요소를 연결하면 → 완성된 프룸프트 (After)

대학생을 대상으로 데이터 분석 기초 강의를 준비하고 있다.
너는 데이터 사이언스를 가르치는 대학 교수다.
머신러닝의 개념과 지도학습, 비지도학습, 강화학습의 차이를 설명해라.
면저 머신러닝을 3문장으로 설명한 뒤 세 가지 방법을 표로 비교하고
각각 하나의 실제 활용 사례를 제시해라.
수학이나 프로그래밍 경험이 많지 않은 대학 1학년 학생도 이해할 수 있도록 설명해라.

이렇게 달라집니다!

- AI가 상황과 목적을 정확히 이해합니다.
- 전문가의 관점에서 일관성 있는 답변을 제공합니다.
- 지정한 형식으로 구조화된 답변을 얻을 수 있습니다.
- 대상에 맞는 수준과 언어로 설명해 줍니다.

핵심 포인트

Context는 '왜?'를, Role은 '누구의 관점으로?'를 알려줍니다.

Action은 '무엇을?'을 정확히 지시하는 핵심 요소입니다.

Format은 '어떻게 보여줄 것인가?'를 명확히 합니다.

Target은 '누구에게?'를 정의하여 수준과 톤을 조절합니다.

CRAFT를 사용하면 더 명확하고, 정확하며, 유용한 답변을 얻을 수 있습니다.

CRAFT 사용법: 순서보다 정보가 중요!

CRAFT는 정해진 순서를 지키는 것이 아니라, **필요한 정보를 빠짐없이 포함**하는 것이 핵심입니다.

1 CRAFT를 꼭 C-R-A-F-T 순서로 써야 할까?

그럴 필요는 없습니다!

실제 프롬프트에서는 자연스럽게 작성해도 됩니다.

나쁜 예

"이 데이터에 대해 알려줘."

정보가 부족해
AI가 많은 것을 추측해야 함

좋은 예 (자연스러운 작성)

- 당신은 데이터 분석 전문가입니다.
- 신입사원 교육을 준비하고 있습니다.
- 다음 데이터를 분석하여 핵심 문제 3가지를 찾아주세요.
- 결과는 표로 정리해주세요.
- 데이터 분석 경험이 없는 신입사원도 이해할 수 있게 설명해주세요.

중요한 것은 순서가 아니라 **필요한 정보를 빠뜨리지 않는** 것입니다.
CRAFT는 엄격한 규칙이 아니라, 효과적인 프롬프트를 위한 체크리스트입니다.

2 CRAFT에서 한 단계 더 발전시키의화기

CRAFT에 **Constraints(제약조건)**와 **Examples(예시)**를 추가하면
AI가 더 정확하고 원하는 형태의 결과를 만들 수 있습니다.

C Context (맥락) | R Role (역할) | A Action (수행할 작업) | F Format (출력 형식) | T Target (대상) + Constraints (제약조건) + Examples (예시)

Constraints 예시

- 500자 이내로 작성해라.
- 전문용어는 최소화해라.
- 불확실한 내용은 추측하지 마라.
- 객관적인 근거가 있는 내용만 사용해라.
- 핵심 내용을 5개 이내로 정리해라.

Examples 예시

- 다음 예시와 동일한 형식으로 작성해라.
- 예시 형식

문제	원인	영향	개선 방안
예시 문제 1	예시 원인	예시 영향	예시 개선 방안
...	...	...	...

명확한 제약조건과 예시는 결과의 품질과 일관성을 크게 높여줍니다.

3 CRAFT + Constraints + Examples 예시

- C Context (맥락) : 신입사원 교육을 위한 데이터 분석 실습 자료를 만들고 있습니다.
- R Role (역할) : 당신은 10년 경력의 데이터 분석 전문가이자 좋은 강사입니다.
- A Action (수행할 작업)" : 다음 데이터를 분석하여 핵심 문제 3가지를 도출하고, 원인과 개선 방안을 제시하세요.
- F Format (출력 형식) : 표로 정리하고, 각 항목에 대한 설명을 간단히 덧붙이세요.
- T Target (대상) : 데이터 분석 경험이 없는 신입사원이 이해할 수 있게 쉬운 용어로 설명하세요.
- Constraints (제약조건) : 500자 이내로 작성하세요, 전문용어는 최소화하고, 추측은 금지하세요.
- Examples (예시) : 오른쪽 예시 표의 형식과 톤을 참고하여 작성하세요.

예상되는 결과 (예시)

핵심 문제	원인	영향	개선 방안
문제 1 예시	원인 설명	영향 설명	개선 방안 제시
문제 2 예시	원인 설명	영향 설명	개선 방안 제시
문제 3 예시	원인 설명	영향 설명	개선 방안 제시

이렇게 활용하세요!

- 필요한 요소를 빠짐없이 포함했는지 확인하세요.
- 제약조건으로 범위와 품질을 조절하세요.
- 예시를 통해 원하는 결과의 형태와 톤을 알려주세요.

핵심 요약 | CRAFT는 정답이 있는 공식이 아닙니다. 핵심은 AI가 **필요한 정보를 정확히 이해**하고, **원하는 형태의 답변**을 만들 수 있도록 도와주는 것입니다!

과제: CRAFT 프레임워크를 활용한 프롬프트 작성

CRAFT 프레임워크를 이해하고, 실생활·학습·업무 상황에 맞게 구조화된 프롬프트를 작성해 보세요.

학습 목표

- CRAFT 각 요소의 의미를 이해한다.
- 필요한 정보를 빠짐없이 포함한 프롬프트를 작성한다.
- 제약조건(Constraints)과 예시(Examples)를 함께 활용한다.

1 과제 개요

과제 내용	주어진 주제/상황에 대해 CRAFT(+Constraints+Examples) 프롬프트를 작성하기
주제 선택	학습, 일상생활, 업무, 프로젝트 등 자유롭게 선택
작성 범위	CRAFT 5요소 + Constraints + Examples를 포함한 완성된 프롬프트
제출 방법	LMS 과제 제출 페이지에 업로드
제출 마감	자유 제출 마감

2 CRAFT 프레임워크 안내

C Context (맥락·배경)	R Role (AI의 역할)	A Action (수행할 작업)	F Format (출력 형식)	T Target (대상)
지금 어떤 상황인가? 왜 이 작업이 필요한가?	AI가 누구의 관점에서 답변해야 하는가?	AI에게 무엇을 해야 하는가?	어떤 형태로 결과를 출력해야 하는가?	누구를 위해 작성하는가?

Constraints (제약조건)
반드시 지켜야 할 조건, 길이 제한, 금지 사항 등

Examples (예시)
원하는 출력 형식이나 내용의 예시 제공

3 주제 (아래 주제 중 1개를 선택하여 작성)

1. **환경 문제 분석**
   특정 환경 이슈(예: 미세먼지, 탄소 배출, 플라스틱 사용 등)를 분석하고 해결 방안을 제안하는 보고서를 작성하도록 프롬프트를 구성하세요.
2. **데이터 분석 과제**
   가상의 매출 데이터(또는 제공된 데이터)를 분석하여 인사이트를 도출하고 시각화까지 수행하도록 프롬프트를 구성하세요.
3. **학습 콘텐츠 제작**
   대학생을 위한 특정 개념(예: 딥러닝, 확률, 회귀분석 등)을 설명하는 학습 자료(강의 노트 또는 슬라이드)를 만들도록 프롬프트를 구성하세요.
4. **업무 자동화/효율화**
   반복적인 업무를 자동화하거나 효율을 높이는 방법을 제안하도록 프롬프트를 구성하세요.

4 작성할 내용 (템플릿 예시)

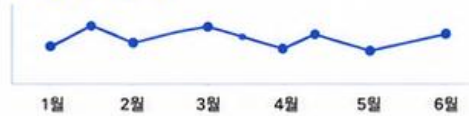
요소	작성 내용 예시
C Context	(예) 대학생을 대상으로 데이터 분석 기초 강의를 준비하고 있다.
R Role	(예) 너는 데이터 분석을 가르치는 전문가다.
A Action	(예) 매출 데이터를 분석하여 핵심 문제 3가지를 도출하고, 원인과 개선 방안을 제시해라.
F Format	(예) 분석 결과를 표와 그래프로 정리하고, 요약을 5문장 이내로 작성해라.
T Target	(예) 데이터 분석 경험이 없는 신입사원도 이해할 수 있도록 설명해라.
Constraints	(예) 글자 수는 800자 이내, 전문용어는 최소화, 추측은 금지.
Examples	(예) 아래의 예시 형식과 유사하게 결과를 작성해라. (간단한 예시 포함)

예시 (출력 형식 예시)

1. 핵심 문제 요약

문제	원인	개선 방안
문제 1	...	...
문제 2	...	...
문제 3	...	...

2. 매출 추이 (예시)



3. 개선 제안 요약

- ...
- ...
- ...

CRAFT vs CO-STAR 비교

두 프레임워크를 명확히 이해하고, 목적과 상황에 맞게 선택하여 활용하세요.

CRAFT (간결하고 실용적인 기본 프레임워크)

C	**Context (맥락)**	상황, 배경, 관련 정보를 제공하여 전체 맥락을 이해시킵니다.
R	**Role (역할)**	AI가 어떤 역할이나 관점에서 답변해야 하는지 지정합니다.
A	**Action (행동)**	AI가 수행해야 할 구체적인 작업이나 요청 사항을 명확히 지시합니다.
F	**Format (형식)**	원하는 결과의 형식, 구조, 길이, 레이아웃을 지정합니다.
T	**Target (대상)**	답변의 대상(독자/사용자), 지식 수준, 관심사, 필요 수준을 지정합니다.

주요 비교 포인트

- **초점** — 어떤 점을 강조하는가?
- **구성 방식** — 어떻게 구성되어 있는가?
- **세부 제어력** — 얼마나 세밀하게 지정할 수 있는가?
- **적합한 상황** — 언제 사용하면 좋은가?
- **핵심 강점** — 가장 큰 장점은 무엇인가?

CO-STAR (커뮤니케이션 중심의 확장 프레임워크)

C	**Context (맥락)**	상황, 배경, 관련 정보를 제공합니다.
O	**Objective (목표)**	AI가 달성해야 할 목표와 기대 결과를 명확히 설정합니다.
S	**Style (스타일)**	어떤 방식(형식, 글쓰기 스타일, 톤앤매너, 구조 등)으로 작성할지 지정합니다.
T	**Target (대상)**	누가 읽을지, 배경지식 수준, 관심사, 필요 수준 등 답변의 대상을 지정합니다.
A	**Audience (청중)**	응답을 받을 배상의 특성, 지식 수준, 역할, 관점 등을 명시합니다.
R	**Response (응답 형식)**	결과물의 형식(표, 코드, Markdown 등)과 분량, 항목 구성, 포함할 내용 등을 지정합니다.

핵심 차이 한눈에 보기

CRAFT는 간결하게 핵심을 지시하는 5가지 요소로 빠른 프롬프트 작성에 적합합니다.

CO-STAR는 목표와 응답 형식을 추가한 6가지 요소로 더 구체적이고 정교한 프롬프트 설계에 적합합니다.

선택 가이드

CRAFT가 적합한 경우

- ✓ 빠르고 간단한 프롬프트 작성이 필요할 때
- ✓ 명확한 작업 지시가 주목적일 때
- ✓ 일반적인 Q&A, 요약, 작성 업무 등

CO-STAR가 적합한 경우

- ✓ 특정 독자(대상)와 커뮤니케이션을 고려할 때
- ✓ 목표, 스타일, 응답 형식까지 세밀히 지정할 때
- ✓ 보고서, 발표 자료, 마케팅 콘텐츠 등 복잡한 작업이 필요할 때

함께 사용하면 더 강력합니다!

CRAFT로 핵심을 빠르게 정리한 후, CO-STAR로 목표(Objective)와 응답(Response)까지 보완하면 더 정확하고 완성도 높은 결과를 얻을 수 있습니다.

한 줄 정리 | CRAFT는 **신속하고 간결한** 프롬프트 설계, CO-STAR는 **목표와 커뮤니케이션을 고려한 정교한** 프롬프트 설계에 적합합니다.

NLP의 근본적 한계와 돌파구

비정형 텍스트의 복잡성과 비용의 한계를, 최신 LLM 기술로 구조화된 가치로 전환

구조화된 데이터로
전환 & 활용

구조화된 출력 예시

ID	분류	핵심 내용	요약	감성	신뢰도	날짜
0001	문의	배송 지연 문의	배송 일정 확인 요청	부정	0.92	2026-08-01
0002	제안	신규 기능 제안	대시보드 개선 제안	긍정	0.95	2026-08-01
0003	불만	결제 오류 불만	결제 실패 반복 발생	부정	0.91	2026-08-01
...	...	...	...	...	...	...

1. 문제: 비정형 텍스트의 복잡성

줄여주세요 ㅜㅜ
이거 언제 배송되나요?
완전 별로예요 다시 안 사요
이 기능 있으면 좋겠어요!
사이트가 자꾸 멈춰요!!
가격이 너무 비싸요
어제 주문했는데 아직도 출발 전?

형식, 표현, 길이, 주제가 모두 다른 텍스트는
분석과 활용이 어렵고, 수작업 처리 비용이 큽니다.

2. 돌파구: LLM 기반 프롬프팅 + 에이전트 파이프라인

프롬프트 설계 (목적/범위 정의) → 컨텍스트 제공 (지식/규칙/예시) → LLM 추론 (이해/분석/생성) → 구조화된 출력 (JSON/테이블)

에이전트 파이프라인

이해 (Understand) → 추출 (Extract) → 분류/태깅 (Classify/Tag) → 요약/생성 (Summarize) → 검동 (Validate)

- 최신 LLM (GPT-4o, Claude 4, Gemini 2.5 등)
- RAG & 백터 DB (지식 기반 연결)
- 도구/함수 호출 (API, 개선, 검색)
- 평가 & 모니터링 (품질/성능 관리)

제로샷/소수샷 프롬프트와 에이전트를 통해
일관성 있고 확장 가능한 자동 분석과 구조화를 실현합니다.

3. 한계 극복: 비용·보안·확장성 확보

비용 최적화
자동화로 운영 비용 절감, ROI 극대화

보안 & 프라이버시
데이터 마스킹, 정책 준수, 안전한 처리 환경

확장성 & 안정성
대량 데이터 처리, 일관된 성능과 높은 가용성

인사이트 & 의사결정
구조화된 데이터로 빠른 분석, 정확한 의사결정 지원

복잡한 비정형 텍스트를 안전하고 확장 가능하게 처리하여
비즈니스 가치를 지속적으로 창출합니다.

PROBLEM: UNSTRUCTURED DATA — CATALYST: LLM + PROMPTING + AGENT — BARRIER: COST, PRIVACY & SCALE

최종 목표: 비정형 텍스트를 구조화된 인사이트로 전환

업무 자동화 · 운영 효율화 · 고객 경험 향상 · 데이터 기반 의사결정 가속화

- 처리 시간 단축 — 최대 80%+)
- 운영 비용 절감 — 최대 50%+)
- 정확도 향상 — 지속적 개선)
- 생산성 & 확장성 — 무한 확장)
- 안전한 운영 — 보안 & 컴플라이언스)

프롬프트 개발 과정

정확하고 효과적인 프롬프트를 설계하고 개선하는 단계

1 문제 정의

- 목표(Task)와 성공 기준 정의
- 사용 시나리오 및 제약 조건 파악
- 필요한 출력과 평가 기준 설정

→

2 초안 작성

- 핵심 지시사항과 맥락 작성
- 입출력 형식 및 예시(Few-shot)
- 가이드라인, 제약 및 규칙 포함

→

3 테스트

- 다양한 입력으로 테스트 실행
- 엣지 케이스 및 실패 사례 확인
- 모델/설정(파라미터 등) 비교

→

4 평가

- 정량/정성 평가 지표로 측정
- 기준 대비 성능 분석
- 오류 유형 및 원인 파악

→

5 개선

- 프롬프트 수정 및 구조 개선
- 예시/규칙/형식 보완
- 반복 테스트로 성능 향상

핵심 포인트

작은 변경을 반복하면서 평가를 통해 검증하고 개선하는 것이 프롬프트 품질 향상의 핵심입니다.

주요 산출물

최적화된 프롬프트, 평가 리포트, 테스트 케이스, 프롬프트 버전 히스토리

운영 환경 적용 과정

실서비스에 배포하고 지속적으로 운영/개선하는 단계

1 배포

- 프로덕션 환경에 적용
- 버전 관리 및 롤백 전략 수립
- 트래픽 점진적 확대 (Canary 등)

→

2 모니터링

- 성능, 비용, 지연시간 모니터링
- 오류율, 이상 징후 탐지
- 사용자 행동 및 시스템 로그 수집

→

3 사용자 피드백

- 사용자 만족도 및 피드백 수집
- 실제 사용 사례 및 개선 요구 파악
- 우선순위 기반 개선 과제 도출

→

4 버전 관리 및 개선

- 프롬프트 버전 관리 및 변경 이력
- A/B 테스트 및 성능 비교
- 정기적 업데이트 및 재배포

핵심 포인트

운영 데이터와 사용자 피드백을 기반으로 지속적으로 개선해야 장기적인 성능과 사용자 만족을 유지할 수 있습니다.

주요 산출물

모니터링 대시보드, 피드백 리포트, 버전 관리 기록, 개선된 프롬프트

Prompting vs RAG vs Tool Use vs Agent

문제의 특성과 목적에 맞는 방식을 선택하면, 더 정확하고 효율적으로 목표를 달성할 수 있습니다.

	Prompt	RAG (Retrieval-Augmented Generation)	Tool Use	Agent
모델에게 주는 것	지시 · 맥락 (Instruction & Context)	외부 지식 (External Knowledge)	실행 능력 (Execution Capability)	반복 실행 루프 (Autonomous Loop)
핵심 목적	모델이 가진 지식으로 이해하고 생성	외부 지식을 검색 · 참조하여 더 정확하고 최신의 답변 생성	모델이 스스로 할 수 없는 작업을 외부 도구로 실행	여러 단계를 계획 · 실행 · 평가하며 목표를 스스로 달성
적합한 경우 (사용 예시)	요약, 변역, 글쓰기, 아이디어 생성, 분류, 형식 변환 동 일반적인 생성/변환 작업	최신 정보, 사내 문서, 정책/매뉴얼, 전문 지식, 제한된 도메인 정보가 필요한 질의응답	검색, 계산, DB 조회/업데이트, API 호출, 파일 처리 등 정확한 실행이 필요환 작업	여러 도구와 정보를 조합해 복잡한 문제를 여러 단계로 해결해야 하는 작업
장점	✓ 간단하고 빠로게 시작 가능 ✓ 비용이 낮고 구현이 쉬움 ✓ 다양한 작업에 유연하게 적용	✓ 최신 · 사내 정보 활용 가능 ✓ 환각(모델이 모르는 정보 생성) 감소 ✓ 출처 기반으로 신뢰도 향상	✓ 정확한 실행 결과 확보 ✓ 실시간 데이터/시스템 연동 가능 ✓ 모델의 한계를 보완	✓ 복잡한 목표를 자동으로 수행 ✓ 유연한 계획 및 오류 복구 가능 ✓ 사람의 개입을 최소화
주의점	⚠ 모델 지식의 한계 존재 최신/전문 정보에 취약 길어질수록 비용 증가/성능 저하	⚠ 검색 품질에 성능이 좌우됨 문서 품질/청킹/임베딩 설계 중요 잘못된 문서를 참조할 수 있음	⚠ 도구/데이터 품질에 의존 보안 · 권한 · 비용 관리 필요 도구 사용 설계가 복잡할 수 있음	⚠ 설계 · 평가가 복잡 비용 및 지연 증가 가능 예상치 못한 행동 가능성 존재

선택 가이드

단순한 생성/변환이면? → Prompt

→ 외부 지식이 필요하면? → RAG

→ 정확한 실행이 필요하면? → Tool Use

→ 여러 단계를 스스로 수행해야 하면? → Agent

핵심 문장: 모든 문제를 더 긴 프롬프트로 해결하려 하지 마세요.

문제의 성격에 맞는 방식(프롬프트 / RAG / 도구 사용 / 에이전트)을 선택하고, 필요할 때 조합하여 사용하는 것이 가장 효과적입니다.

프롬프트 설계 요소와 고급 실행 전략
(Prompt Design Components & Advanced Execution Strategies)

좋은 프롬프트는 명확한 설계 요소로 구성되고, 고급 전략을 통해 복잡한 문제 해결 성능을 극대화합니다.
두 가지를 목적과 상황에 맞게 조합하여 정확하고 신뢰할 수 있는 결과를 얻으세요.

최신 트렌드 반영 요소

- 멀티턴 대화 최적화
- RAG 기반 지식 활용
- 도구/함수 자동 호출
- 안전성 & 편향 최소화
- 평가 & 지속적 개선

프롬프트 전략 (Prompt Strategies)
목적, 맥락, 제약, 예시 동을 조합해 원하는 결과를 유도

1. 프롬프트 설계 요소 (Prompt Components)
명확하고 일관된 응답을 위한 기본 구성 요소

- **지시 (Instruction)** — 역할, 목표, 작업을 명확하게 지정
- **맥락 (Context)** — 배경 정보, 입력 데이터, 제약 조건 제공
- **예시 (Examples)** — 입출력 예시를 통해 모델의 이해를 보조 (Few-shot / Zero-shot)
- **출력 형식 (Output Format)** — JSON, 표, 목록 동 원하는 형식으로 출력 규정
- **검증 (Validation)** — 응답의 품질 기준과 검증 조건 제시

2. 고급 실행 전략 (Advanced Execution Strategies)
복잡한 문제 해결과 성능 최적화를 위한 전략

- **사고의 사슬 (Chain-of-Thought)** — 단계별 추론 과정을 명시하여 정확도 향상
- **자기 일관성 (Self-Consistency)** — 여러 추론 경로를 생성 후 일관된 답을 선택
- **도구 활용 (Tools / Function Calling)** — 검색, 계산, API 등 외부 도구를 안전하게 자동 호출
- **규칙 & 제약 (Rules & Constraints)** — 도메인 규칙, 정책, 비즈니스 로직을 명시하여 일관성 확보
- **앙상블 & 재평가 (Ensemble & Re-rank)** — 다양한 프롬프트/모델 결과를 통합·재평가하여 최적 결과 선택

모든 프롬프트에 적용해야 할 공통 요소

- **안전성 (Safety)** — 유해 콘텐츠 방지, 프롬프트 인젝션 대응, 개인정보 보호
- **공정성 (Fairness)** — 편향 최소화, 다양한 관점 고려, 포용적 언어 사용
- **품질 관리 (Quality)** — 명확성, 일관성, 안전성 기준 설정 및 자동/수동 평가
- **관찰 & 모니터링 (Observability)** — 프롬프트 버전 관리, 로그 수집, 성능 모니터링 및 알림
- **반복적 개선 (Iterative Optimization)** — A/B 테스트, 피드백 반영, 프롬프트 지속 개선

좋은 프롬프트는 **체계적인 설계 요소**와 **검증된 실행 전략**의 조합입니다. 목적에 맞게 설계하고, 지속적으로 평가·개선하여 일관성 있고 신뢰할 수 있는 결과를 얻으세요.

매트릭스 1: 프롬프팅 인지 아크 (The Evolution of Prompting)

ⓘ 개념 이해를 위한 상대적 예시이며, 실제 조사 통계가 아님

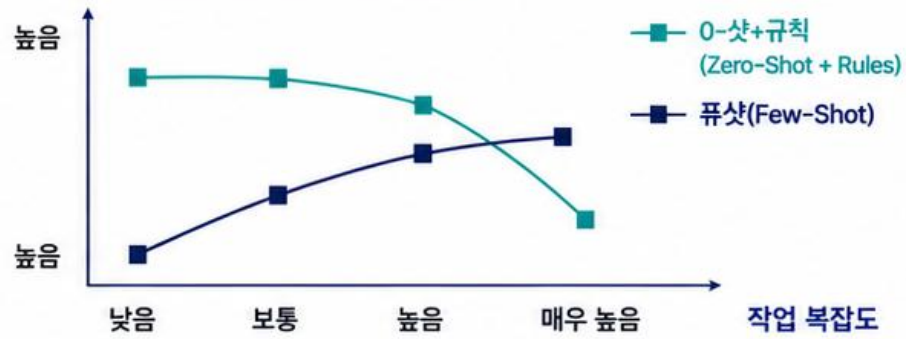
Techniques (기법)	**인지 부하 및 토큰 사용량** (상대적 수준)	**추론 능력** (상대적 수준)	**주요 용도** (Primary Use Case)
Zero-shot **(제로샷)**	낮음 ■ □ □	낮음 ■ □ □	단순 번역, 기본 요약
Few-shot **(퓨샷)**	중간 ■ ■ □	중간 ■ ■ □	톤앤매너 통제, 특정 포맷 출력
Chain-of-Thought **(CoT)**	높음 ■ ■ ■	높음 ■ ■ ■	수학적 계산, 논리적 퍼즐
Self-Consistency **(자기 일관성)**	매우 높음 ■ ■ ■	매우 높음 ■ ■ ■	복잡한 추론, 환각 최소화가 필수적인 기업용 의사결정

상대적 수준 범례 | ■ □ □ 낮음 ■ ■ □ 중간 ■ ■ ■ 높음

제로샷 vs 퓨샷: 문맥의 딜레마

일반적으로 예제를 제공하는 **퓨샷(Few-Shot)**이 유리하지만,
특정 작업에서는 '규칙'과 '도메인 지식'을 명확히 제공하는 **0-샷+규칙(Zero-Shot + Rules/Constraints)**이
더 높은 성능과 일관성을 보일 수 있습니다.

작업 유형별 성능 경향 (예시)



단순·정형·규칙 기반 작업은 0-샷+규칙이 강점,
복잡·비정형·개방형 작업은 퓨샷이 강점

퓨샷 (Few-Shot) – 예제 제공

복잡한 패턴/다양한 형식
복잡한 데이터 분포와
다양한 형식에 강함

모델 내 학습 (ICL)
모델이 예제를 통해 스타일,
형식, 의도를 학습

데이터 누수/비용 증가
긴 입력으로 인한 비용·지연 증가,
불필요한 정보 노출 위험

선택의 기준

작업 복잡도
낮음 → 0-샷+규칙 / 높음 → 퓨샷

일관성 & 신뢰성
높음 필요 → 0-샷+규칙

비용 & 지연
낮춰야 함 → 0-샷+규칙

변화하는 데이터
빠르게 적응 필요 → 퓨샷

0-샷+규칙 (Zero-Shot + Rules/Constraints) – 규칙/지식 제공

일관성과 정확도 향상
도메인 규칙, 제약, 절차를 명시하여
더 일관된 출력

환각(Hallucination) 억제
사전 정의된 규칙·지식을 기반으로
불필요한 추론과 오류 강소

명확한 분류/추출 최적화
엄격한 형식, 스키마, 규칙으로
분류/추출 작업에 최적

프롬프트 설계 고려 요소

도구 활용 (Tools)
검색, 코드, 계산 등
도구와 결합

RAG 연계
최신 지식 기반
정확성 보강

안전·정책 준수
가이드라인, 금칙사항,
정책 명시

출력 형식 강제
JSON, 표, 스키마 등
구조화 출력

평가 & 모니터링
자동 평가, 로그 분석으로
지속 개선

반복적 최적화
A/B 데스트, 피드백 반영,
프롬프트 버전 관리

좋은 프롬프트는 **작업 목적, 데이터 특성, 비용 제약, 안전성 요구사항**을 종합적으로 고려해 설계됩니다.

Zero-shot

목표 : 예시 없어서 꽤 잘 작동하는 문제와, 한계가 있는 문제를 구분한다.

과제 A: 요약

다음 글을 2문장으로 요약해줘.

조건:

1. 핵심 내용을 포함할 것
2. 쉬운 표현을 사용할 것
3. 원문에 없는 내용을 추가하지 말 것.

과제 B: 분류

다음 글을 읽고 가장 적절한 주제를 하나 선택해줘.

분류:

1. 건강
2. 경제
3. 교육
4. 환경
5. 기술

규칙적인 운동은 건강을 유지하는 데 중요하다. 운동은 체력을 향상시키고 스트레스를 줄이는 데 도움이 된다. 또한 충분한 수면과 균형 잡힌 식사를 함께하면 건강 관리에 더욱 효과적이다. 하지만 무리한 운동은 부상을 유발할 수 있으므로 자신의 체력에 맞게 운동하는 것이 좋다.

Few-shot

목표 : 예시 주어지면 형식과 기준이 안정됨

주제: 학생 질문을 난이도별로 분류하기

다음 질문을 초급 / 중급 / 고급 중 하나로 분류해줘.

예시 1

질문: 파이썬에서 리스트가 뭐예요?

답: 초급

예시 2

질문: 회귀와 분류의 차이는 무엇인가요?

답: 초급

예시 3

질문: 과적합을 줄이기 위한 방법을 설명해보세요.

답: 중급

예시 4

질문: Transformer의 self-attention이 시퀀스 길이에 따라 가지는 계산 복잡도를 설명해보세요.

답: 고급

이제 분류할 질문:

질문: Few-shot prompting은 언제 zero-shot보다 유리한가요?

Few-shot

목표 : 예시 주어지면 형식과 기준이 안정됨

추가 실습

1. 좋은 발표 제목/나쁜 발표 제목 구분
2. 초록 문장 다듬기 예시 3개 제시 후 새 문장 수정
3. 강의계획서 형식 맞추기
4. 한국어 문장을 자연스러운 영어 발표체로 바꾸기
5. 요약문 스타일 통일하기

인지 모델링: 휴리스틱(Heuristic) vs 생각의 사슬(CoT)

(CoT: Chain-of-Thought – 연쇄적 사고)

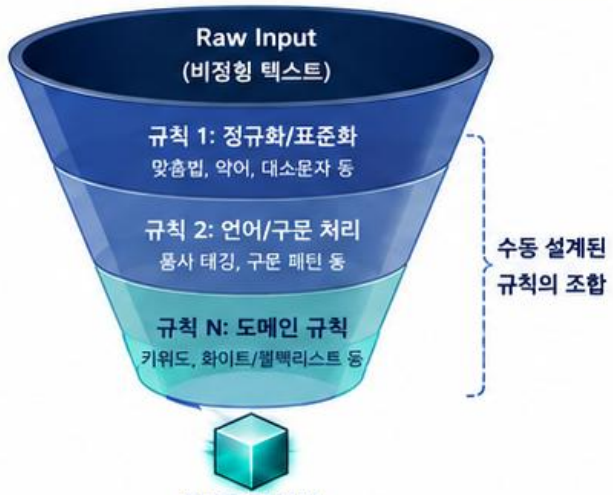
같은 정보를 다루더라도, 접근 방식에 따라 모델의 추론 과정의 안정성과 검증 가능성이 달라집니다.

CoT는 모델의 '내부 사고'를 그대로 공개하는 것이 아니며, 단계적 설명이 항상 정답을 보장하지는 않습니다.
복잡한 문제를 중간 단계로 분해하면 계산·조건 누락을 줄이고 검증 가능한 중간 결과를 만들 수 수 있습니다.

기억하세요

- 단계적 추론은 도구이지 만능이 아닙니다.
- 일관성·검증 가능성·효율성의 균형이 중요합니다.
- 문제 특성과 목적에 맞게 적절히 선택하세요.

휴리스틱(Heuristic) – 규칙 기반 필터링

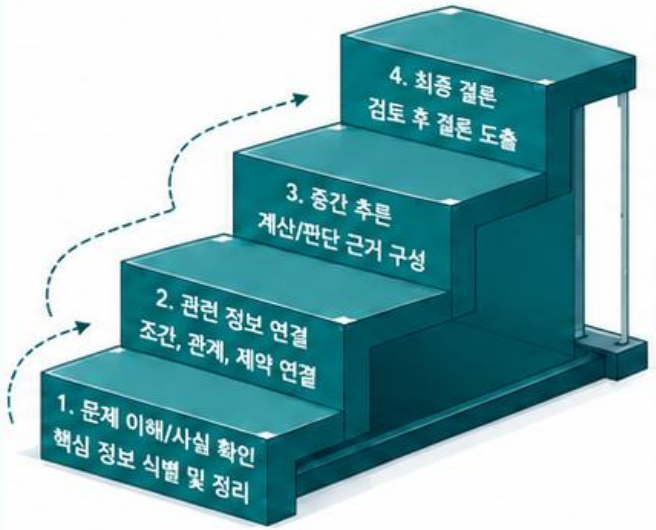


핵심 특징

- **명확한 규칙 적용** – 정해진 규칙을 일관되게 적용하여 필터링
- **자원 효율적** – 비고적 낮은 계산 비용으로 빠른 처리
- **안정성 높음** – 고정된 규칙으로 예측 가능성과 일관성 확보
- **유연성 한계** – 새로운 상황이나 복잡한 맥락에 취약

장점: 빠르고 비용 효율적, 일관된 규칙으로 안정적 동작
한계: 규칙 설계/유지 비용이 높고, 예외 패턴과 새로운 상황에 취약
적합한 작업: 정형 패턴 매칭, 단순 추출, 고정된 도메인 규칙 적용

생각의 사슬(CoT) – 단계적 논리 빌딩



핵심 특징

- **단계적 추론** – 문제를 여러 단계로 나누어 논리적 추론 수행
- **자기 검증** – 각 단계에서 중간 결과를 검토하고 수정
- **상황 적응** – 새로운 정보와 맥락에 유연하게 대응
- **기록성/검증 가능성** – 추론 과정을 명확히 하여 검토와 피드백 용이

장점: 복잡한 추론, 예외 상황, 새로운 도메인에서도 높은 성능과 유연성
한계: 더 많은 토큰/시간 소모, 잘못된 중간 단계가 누적될 위험
적합한 작업: 복잡한 다단계 추론, 의사결정 지원, 심층 이해가 필요한 작업

실전 프롬프트 가이드 (CoT 요청 예시)

1. **조건 정리**
   주이전 정보를 구조화하고
   핵심 조건과 제약을 정리해라.
2. **근거 제시**
   문제 해결을 위해 필요한
   계산/판단 근거를 간결하게
   단계별로 제시해라.
3. **최종 답안**
   앞서 제시한 근거를 바탕으로
   최종 결론을 명확히 제시해라.

주의 사항

- CoT는 정답을 보장하지 않습니다.
- 중요한 결정에는 외부 검증이나 도메인 지식 검토를 병행하세요.
- 토큰/비용 효율을 고려해 필요한 수준만 요청하세요.

단계별 사고의 효과
전제를 작은 단위로 나누어 누락과 오류 가능성 감소

정확도 향상
중간 검증으로 조건 누락과 계산 실수 감소

일관성 및 안정성
논리적 흐름 유지로 답변 변동성 완화

해석 가능성
추론 근거 추적 용이

일반화 및 확장성
다양한 문제 유형에 유연하게 적용

신뢰성 확보
근거 기반의 신뢰 높은 답변

핵심 메시지

단순하고 고정된 규칙에는 휴리스틱, 복잡하고 변화하는 문제에는 CoT가 더 적합합니다.
두 접근을 상황에 맞게 적절히 결합하면 더 나은 성능과 안정성을 얻을 수 있습니다.

연쇄적 사고(Chain-of-Thought): 추론의 징검다리 놓기

핵심 메시지

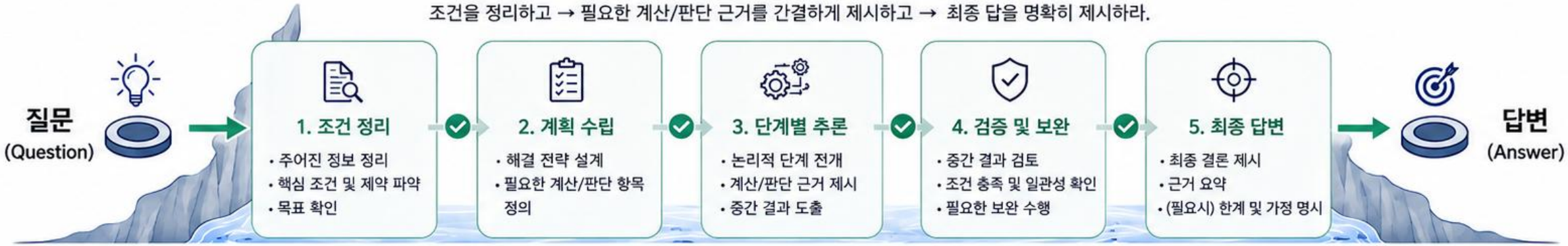
복잡한 문제를 중간 단계로 분해하면 계산·조건 누락을 줄이고
검증 가능한 중간 결과를 만들 수 있습니다.
이것이 **연쇄적 사고(CoT)**를 요청하는 이유이며, 정답을 보장하지는 않습니다.

단일 도약의 한계 (The Limitation of a Single Leap)

단일 도약의 한계: 복잡한 문제에 대해 즉각적인 답을 요구하면 연어 모델의 연산 과정이 누락되어 오류나 환각이 발생할 수 있습니다.

단계별 사고 유도 (Inducing Step-by-Step Thinking)

조건을 정리하고 → 필요한 계산/판단 근거를 간결하게 제시하고 → 최종 답을 명확히 제시하라.



단계적 사고의 효과
전체를 작은 단위로 나누어
누락과 오류 가능성 감소

정확도 향상
중간 검증으로
조건 누락과 계산 실수 감소

일관성 및 안정성
논리적 흐름 유지로
답변 변동성 완화

해석 가능성
중간 근거가 명확해
검토·설명·재사용 용이

일반화 및 확장성
다양한 문제 유형에
유연하게 적용

신뢰성 확보
근거 기반 답변으로
사용자 신뢰 향상

유의 사항: CoT는 내부 사고를 그대로 공개하는 것이 아니며, 항상 정답을 보장하지 않습니다. 중요한 결정에는 추가 검증이 필요합니다.

단계적 추론과 검증이 필요한 문제 찾기

목표 : 단순 생성과 추론 문제를 구분한다.

과제 A: 계산 추론

다음 문제를 문제의 조건을 정리하고, 필요한 계산 또는 판단 근거를 간결하게 제시한 뒤 최종 답을 알려줘.

A가 B보다 3살 많고, B는 C보다 2살 많다.

C가 10살이면 A는 몇 살인가?

과제 B: 일정 조정

다음 조건을 만족하는 2일 발표 일정을 짜줘.

- 발표자는 6명
- 각 발표는 20분
- 오전 10시~오후 4시 진행
- 점심 12시~1시
- 같은 연구주제 발표는 연속 배치하지 않기

먼저 조건을 정리하고, 그 다음 가능한 일정을 제안해줘.

과제 C: 문헌 비교

다음 두 연구 요약을 비교해

1) 공통점

2) 차이점

3) 어떤 상황에서 각각 더 적합한지

순서로 정리해줘.

판단 근거를 먼저 정리한 후 결론을 제시해줘.

추가 실습 5개

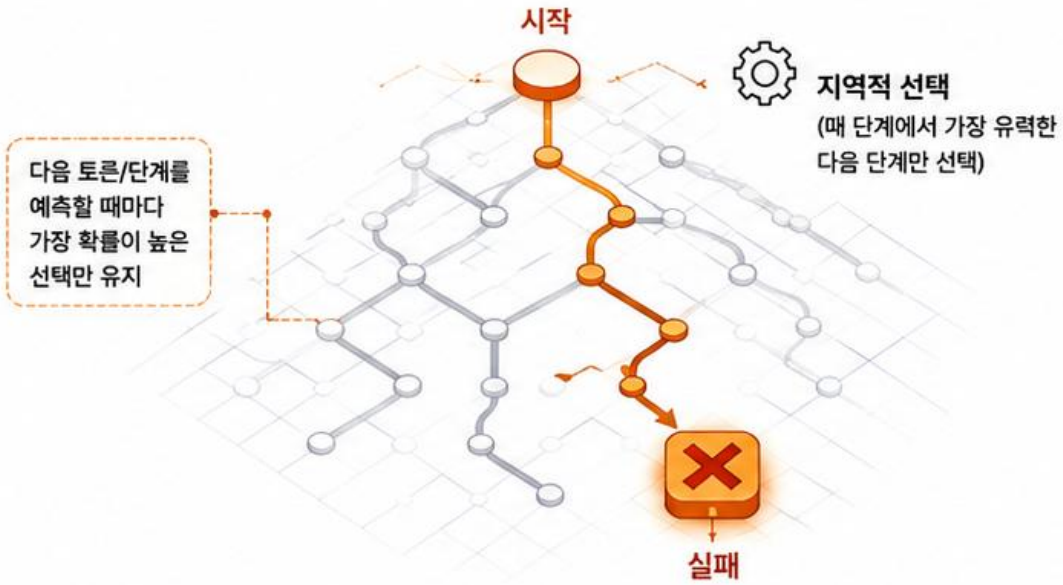
1. 논문 리뷰 구조 만들기
2. 수업 프로젝트 평가 기준 설계
3. 연구계획서 항목 간 논리 연결 점검
4. 토론 찬반 논리 정리
5. 표 데이터를 바탕으로 결론 도출

매트릭스 2: 단일 경로 추론 vs 다중 경로 추론·집계

복잡한 문제일수록 여러 경로를 탐색하고 결과를 집계하면 계산·조건 누락을 줄이고 검증 가능한 중간 결과를 만들 수 있습니다.

단일 경로 추론 (Single-path Inference)

한 번의 추론 경로만 따라 최종 답을 도출



- **장점:** 빠르고 비용 효율적, 구현이 단순
- **한계:** 초기 오류에 취약, 복잡한 문제에서 성능 저하 가능
- **적합한 상황:** 명확하고 제약이 적은 단순 작업

다중 경로 추론·집계 (Self-Consistency / Multi-sample Reasoning)

여러 추론 경로를 탐색하고 일관된 결론을 집계

1. **경로 생성** — 프롬프트/샘플링을 통해 여러 추론 경로 생성 →
2. **경로 평가** — 각 경로의 논리적 정합성 확인 →
3. **결과 집계** — 일관성 높은 결론 선택 (투표/스코어) →
4. **검증 & 보정** — 사실 검증 및 필요 시 재추론

- **장점:** 다양한 관점 탐색으로 누락·오류 완화, 일관성 향상, 신뢰성 증가
- **한계:** 계산 비용 증가, 설계·검증 로직이 필요
- **적합한 상황:** 복잡한 추론, 다단계 문제, 높은 신뢰성이 필요한 작업

더 넓게 탐색 — 다양한 경로를 고려

일관성으로 신뢰 확보 — 다수 경로의 합의로 검증

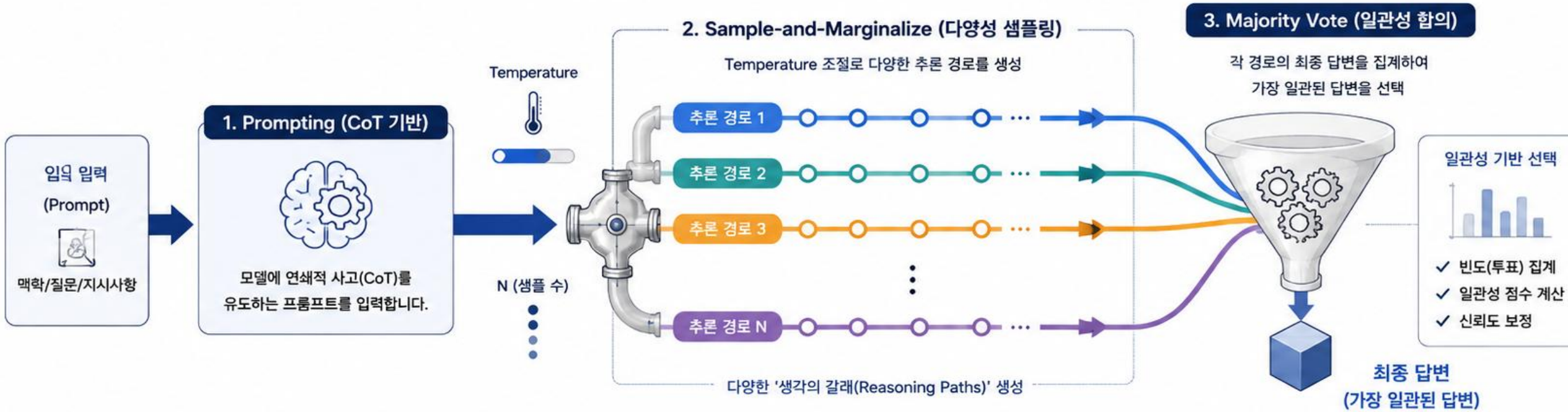
정확도 & 안정성 향상 — 오류·환각 감소

계산 자원은 중가 — 효과적 설계로 효율화

ⓘ 주의: 다중 경로 전략은 항상 정답을 보장하지 않으며, 중요 결론에는 추가 검증(사실 검증, 외부 도구 활용 등)이 필요합니다.

집단 지성 엔진: Self-Consistency 정제 플로우차트

여러 추론 경로를 생성하고 일관성을 기반으로 선뢰도 높은 답변을 도출합니다.



1. Prompting (CoT 기반)

- ✓ 연쇄적 사고를 유도하는 프롬프트 설계
- ✓ 명확한 목표와 제약 조건 제공
- ✓ 에시 기반 가이드(선택)로 품질 향상

2. Sample-and-Marginalize (다양성 샘플링)

- ✓ Temperature로 다양성 조절 (높을수록 다양)
- ✓ N개의 추론 경로를 독립적으로 생성
- ✓ 다양한 관점에서 문제를 탐색하여 편향 감소

3. Majority Vote (일관성 합의)

- ✓ 최종 답변(또는 결론) 기준으로 투표 집계
- ✓ 일관성/신뢰도 점수를 통해 보정
- ✓ 가장 신뢰도 높은 답변을 최종 선택

Self-Consistency 적용 효과

정확도 향상
연산 경로 다양성으로
정답 도달 확률 증가

안정성 강화
편향과 우연한 오류를 줄여
일관된 결과 확보

설명 가능성 제고
여러 추론 경로를 통해
결과 해석 용이

확장성 및 유연성
모델, 도메인, 작업에
폭넓게 적용 가능

작동 예시: 생각의 갈래 분기 및 합의

여러 추론 경로를 탐색하고 일관성 점수를 기반으로 가장 신뢰도 높은 답변을 선택합니다.

1. Prompting (CoT 기반)

- 연쇄적 사고를 유도하는 설계
- 명확한 목표와 제약 조건 제공
- 예시 기반 가이드로 품질 향상

2. Self-Consistency (다중 경로 추론 & 집계)

- Temperature로 다양성 조절
- N개의 추론 경로를 독립적으로 생성
- 다양한 관점에서 문제를 탐색하여 편향 감소

3. Majority Vote (일관성 합의)

- 일관성(신뢰도) 점수로 정량 평가
- 최종 답변 또는 결론 기준으로 투표 집계
- 가장 신뢰도 높은 일관된 답변 선택

기대 효과

정확도 향상
오류 경로 제거로 정답 도달 확률 증가

신뢰도 강화
일관성 점수 기반으로 신뢰도 높은 답변 선택

설명 가능성 제공
여러 추론 경로를 통해 결과 해석 용이

확장성 및 안정성
복잡한 문제, 다양한 도메인에 안정적으로 적용 가능

참고: 개념 구분

- Self-Consistency는 동일한 문제에 대해 무작위성(stochastic sampling)을 활용해 여러 추론 경로를 생성하고, 최종 답을 통계적으로 집계하는 추론 전략입니다.
- 여러 관점으로 질문하기는 다른 프롬프트/관점으로 문제를 재구성하여 답변을 얻고 비교하는 접근(다중 관점 프롬팅, 강건성 점검, 앙상블 프롬팅)에 가깝습니다.

Tip: 두 방법을 함께 사용하면 더 넓은 탐색과 더 안정적인 답변 선택이 가능합니다.

Self-Consistency 실습

목표 : 한번의 답 보다 여러 번의 답을 비교해 더 안정적인 결론을 얻는 감각을 익힌다.

실습 방법 1: 수동 방식

같은 질문을 3번 조금 다르게 묻습니다.

논제

대학교 수업에서 생성형 AI 사용을 적극적으로 허용해야 하는가?

프롬프트 1

대학교 수업에서 생성형 AI 사용을 적극적으로 허용해야 한다는 주장에 대해 가장 설득력 있는 찬성 논거 3개를 제시해줘.

프롬프트 2

대학교 수업에서 생성형 AI 사용을 허용해야 한다는 결론을 미리 전제하지 말고, 가능한 근거들을 검토한 뒤 가장 타당한 찬성 논거 3개를 제시해줘.

프롬프트 3

대학교 수업에서 생성형 AI 활용을 확대해야 한다는 주장에 대해 서로 독립적인 관점에서 찬성 논거 3개를 제시해줘.
그다음, 방금 나온 세 결과를 비교해서 공통으로 등장한 핵심 논거만 추려줘.

실습 방법 2: 하나의 프롬프트 안에 넣기

대학 과제에서 생성형 AI 사용을 어느 범위까지 허용하는 것이 적절한가?

프롬프트

다음 문제를 서로 다른 3가지 관점에서 검토해줘.
각 관점은 가능한 한 서로 겹치지 않도록 해줘.

문제:
대학 과제에서 생성형 AI 사용을 어느 범위까지 허용하는 것이 적절한가?

그 후 세 관점을 비교하여, 공통적으로 동의할 수 있는 결론을 도출해줘.

최종 출력은 다음 형식으로 작성해줘.

관점 1
관점 2
관점 3

출력 형식 구조화: Markdown, 표, 체크리스트, JSON

목표 : 같은 내용도 구조화된 출력 형식으로 요청하면 가독성, 재사용성, 후처리 가능성이 어떻게 달라지는지 이해한다.

과제 C: JSON으로 출력시키기

다음 강의계획을 JSON 형식으로 정리해줘.

필드는 week, topic, objective, activity, assignment 로 해줘.

실습 포인트

그냥 요약 vs 마크다운 표

그냥 정리 vs 체크리스트

그냥 설명 vs JSON 구조

어떤 구조가 재사용하기 쉬운가

추가 실습

회의록 → action items 표

강의계획 → JSON

논문 리뷰 → 섹션별 bullet

발표 질문 예상 → 카테고리별 목록

프로젝트 일정 → 주차별 표

정리된 형식으로 요청하면 AI의 답변이 더 정확하고, 찾기 쉽고, 다시 쓰기 쉬워집니다.

구조화된 출력이 결과 품질과 재사용성에 미치는 영향

Plain Text (비구조화)

생성형 AI는 대규모 데이터를 학습하여 새로운 텍스트나 이미지를 생성하는 인공지능 기술입니다. 창의적인 콘텐츠 제작, 문서 요약, 아이디어 발굴 등 다양한 분야에서 활용할 수 있어 업무 효율을 크게 높여줍니다. 하지만 학습 데이터에 편향이 있거나 사실과 다른 정보를 만들어 내는 환각 현상이 발생할 수 있고, 출처가 불분명한 내용이 포함될 수 있다는 한계도 있습니다. 따라서 중요한 의사결정에는 반드시 사람의 검토가 필요합니다.

Structured Markdown (구조화)

```
# 생성형 AI 소개

## 핵심 개념
- 대규모 데이터를 학습하여 새로운 텍스트, 이미지 등을 생성하는 AI 기술

## 주요 활용 분야
1. 창의적 콘텐츠 제작
2. 문서 요약 및 정리
3. 아이디어 발굴
4. 업무 자동화 및 생산성 향상

## 한계와 주의점
- 학습 데이터 편향 가능성
- 사실과 다른 정보 생성(환각) 위험
- 출처가 불분명한 내용 포함 가능

## 사용 권장 사항
- 중요한 의사결정에는 반드시 사람의 검토 병행
```

MARKDOWN 심볼 설명

 # Heading (제목)
문서의 논리적 계층 구조 생성 및 문단 분할

– List (목록)
다중 지시사항 분리 및 순서/우선순위 부여

**** Emphasis (강조)**
핵심 키워드 가중치 부여 (AI의 주의 집중 유도)

=== Delimiters (구분자)
시스템 지시사항과 입력 데이터의 완벽한 분리

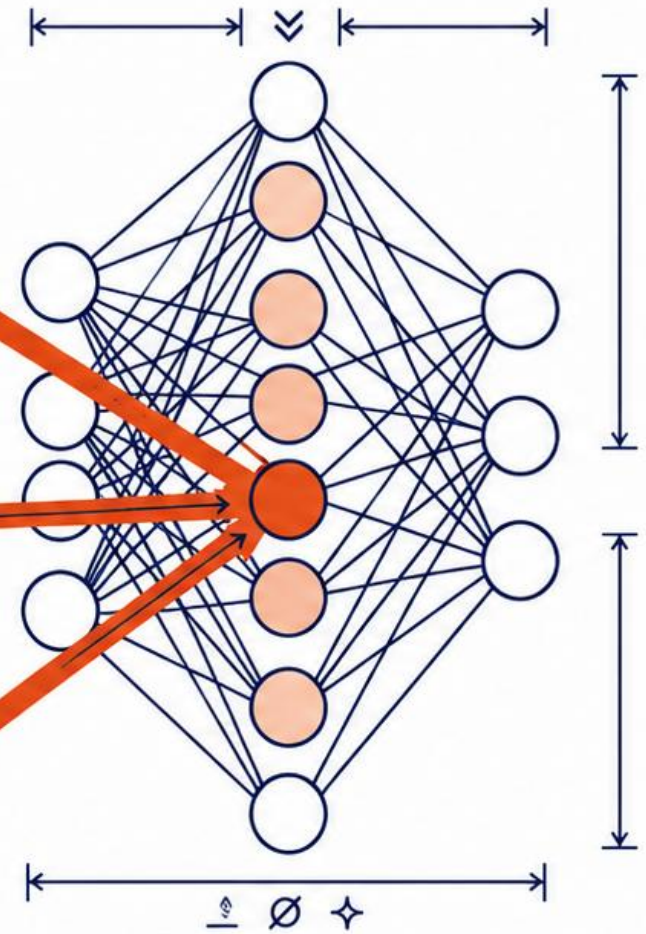
구조화 출력의 장점

- 정보가 명확하고 읽기 쉬움
- 핵심 내용을 빠르게 파악 가능
- 표/JSON 등으로 변환하여 재사용 쉬움
- 일관된 형식으로 반복 사용 가능
- 누락과 중복을 줄여 검토가 용이함

Efficiency Metric 불필요한 자연어 연결사를 줄여 토큰 사용량을 최적화하고, 환각(Hallucination) 현상을 완화하며 결과의 신뢰성과 재사용성을 높여줍니다.

언어의 미세 조정과 심리적 프롬프팅

		모호한 입력 (Ambiguous)	정밀한 입력 (Precise)
1	한국어 최적화	짧게 해줘	요약/축약해줘
2	페르소나 설계	넌 변호사야	전문가에게 조언을 구하는 청자를 위한 역할
3	심리적 트리거	이거 검토해	내 승진이 걸린 중요한 프로젝트입니다 😊



페르소나와 청중 맞춤 실습

목표 : 같은 내용도 누구에게 말하느냐에 따라 표현이 달라져야 함을 익힌다.

주제: 논문 초록을 보고 발표 제목 5개를 만들고, 그중 가장 적절한 제목 1개를 고르기

프롬프트

다음 초록을 바탕으로 발표 제목 후보를 5개 만들어줘.

예시 1

초록 요약: 교육용 챗봇 설계

좋은 제목: 초보 학습자를 위한 대화형 AI 튜터 설계

예시 2

초록 요약: 문서 검색 기반 답변 시스템

좋은 제목: 검색 증강 기반 질의응답 시스템의 설계와 평가

이제 아래 초록에 대해 작업해줘.

절차:

1. 초록의 핵심 키워드 5개를 먼저 뽑아줘.
2. 제목 후보 5개를 만들어줘.
3. 각 제목의 장단점을 한 줄씩 써줘.
4. 가장 적절한 제목 1개를 고르고 이유를 설명해줘.
5. 마지막으로 너무 과장되거나 모호한 제목은 제외했는지 스스로 점검해줘.

복잡성 극복: 패턴의 기준점과 사고의 사슬

프롬프트 초반에 기준점을 설정하고, 점진적으로 사고를 확장하며 검증을 통해 안정적이고 정확한 답변을 도출합니다.

1. Few-Shot Learning

패턴의 기준점(Anchor) 설정

- 2~3개의 정답 예시 제공
- AI가 출력 패턴과 사고 흐름 학습
- 일관된 답변의 기준점 확보

2. Chain-of-Thought (CoT)

단계적 사고 전개

- 'Let's think step by step' 유도
- 중간 추론 단계를 명시적으로 생성
- 논리적 깊이와 정확도 상승

3. Ground-Checking

검증을 통한 신뢰성 확보

- 논리/일관성/사실 검증 수행
- 오답 가능성 조기 감지 및 보정
- 신뢰도 높은 최종 답변 도출

핵심 아이디어 | 도약(Zero-Shot)이 아닌, 단계적 사고와 검증을 통해 복잡한 문제도 일관되고 신뢰성 높은 답변으로 연결합니다.

복잡한 문제를 Few-shot + 단계적 추론 + 근거 검증으로 해결하기

목표 : 어려운 문제는 한 기법만으로 풀지 않는다는 점을 익힌다.

기본 과제

주제: “Transformer”

버전 1

고등학생에게 설명해줘.

버전 2

컴퓨터공학 학부생에게 설명해줘.

버전 3

비전공 교수 면접 청중에게 설명해줘.

버전 4

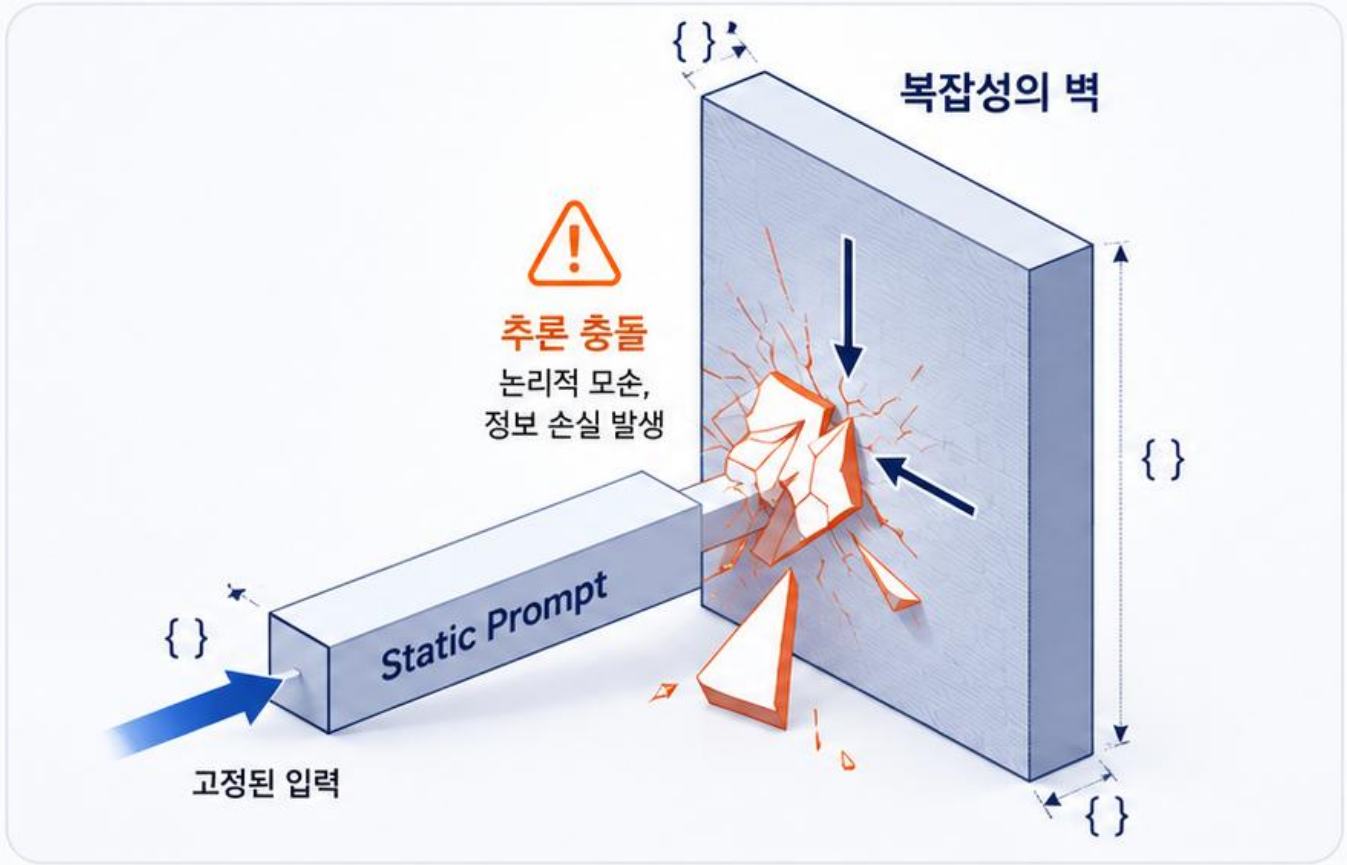
연구 협업 제안서에 넣을 짧은 소개로 써줘.

추가 실습

1. “RAG”를 학생/교수/기업담당자용으로 바꾸기
2. “프롬프트 엔지니어링”을 초보자/실무자/연구자용으로 바꾸기
3. “LLM의 한계”를 일반인/전공자/의사결정자용으로 바꾸기

단방향(Static) 프롬프팅의 구조적 한계

복잡한 추론 문제에서 단방향 프롬프팅은 성능의 병목이 발생합니다.



민감성 (Sensitivity)

작은 단어 변화나 문맥 변화에도
출력이 크게 요동칩니다.

비효율성 (Inefficiency)

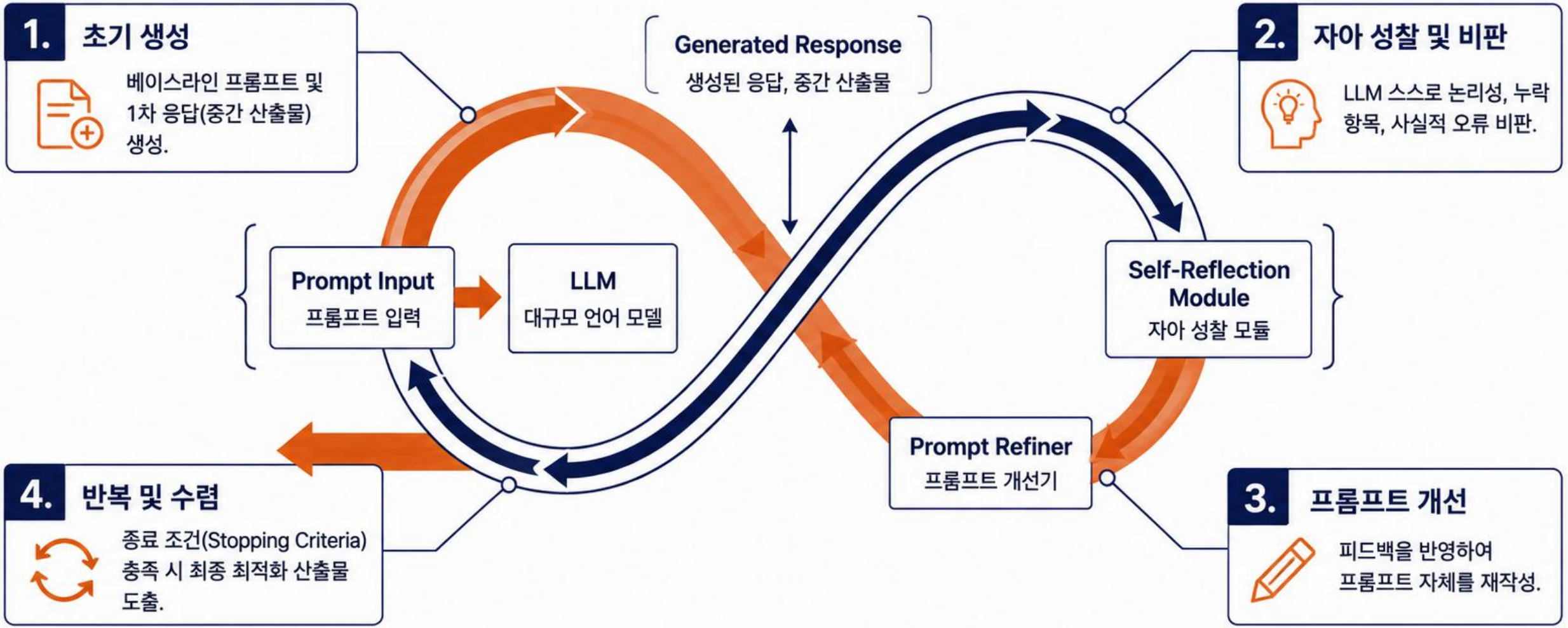
수동적인 시행착오(Trial-and-error)에
전적으로 의존합니다.

취약성 (Vulnerability)

복잡한 추론 문제에서 논리적 모순과
환각(Hallucination) 발생 위험이 높습니다.

Question: LLM이 스스로 자신의 답을 평가하고,
프롬프트 자체를 수정할 수는 없을까?

지능의 자가 진화: 반복적 자아 성찰 프레임워크



자기 성찰 기반 반복 개선

목표 : 한 번에 완벽한 답을 받으려 하지 않고, 개선 루프를 돌린다.

1차 요청

생성형 AI 입문 강의 홍보문을 300자로 작성해줘.

2차 요청: 자기평가 시키기

방금 작성한 홍보문을 다음 기준으로 스스로 평가해줘.

- 초보자 친화성
- 핵심 정보 포함 여부
- 중복 표현
- 신청 유도력

각 항목을 10점 만점으로 채점하고 이유를 써줘.

3차 요청: 개선 시키기

방금 자기평가 결과를 반영해서 홍보문을 다시 작성해줘.

단, 다음 조건을 추가해줘.

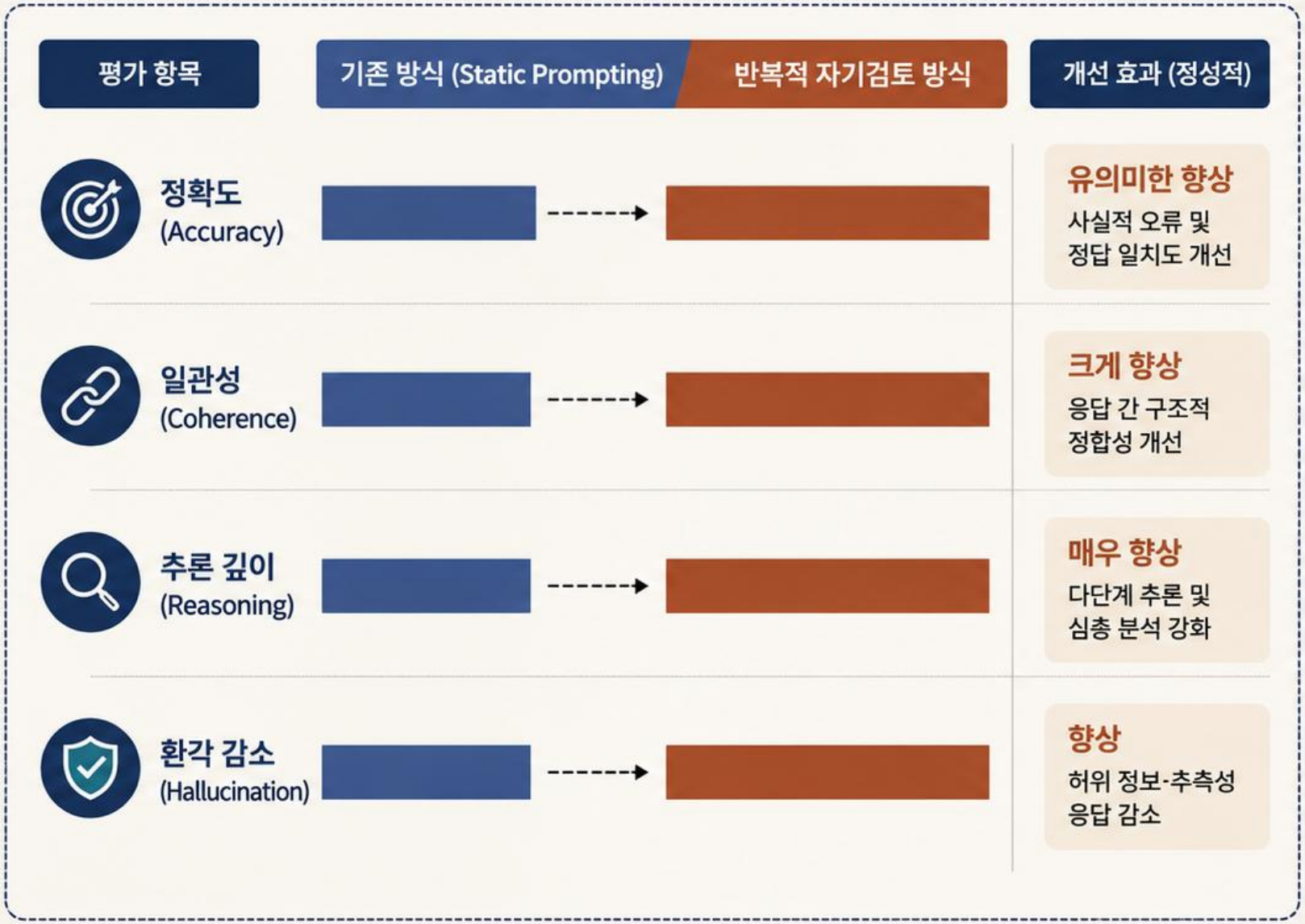
- 지나치게 광고 같지 않게
- 학생이 얻는 실질적 이득이 보이게
- 문장은 4개 이내

4차 요청: 최종 검수

최종본에서 모호한 표현, 중복 표현, 정보 누락이 있는지 다시 점검해줘.

문제가 있으면 수정본만 제시해줘.

성과 입증: 한계를 돌파한 압도적 성능 향상



핵심 데이터 요약 (Key Takeaways)

정확도 (Accuracy):
반복적 검토를 통해 사실적 오류 수정 능력이 크게 향상됩니다.

일관성 (Coherence):
응답의 구조적 일관성과 흐름이 크게 개선됩니다.

추론 깊이 (Reasoning):
더 깊고 체계적인 다단계 추론이 가능해져 통찰력이 강화됩니다.

환각 감소 (Hallucination):
불필요한 추측과 허위 정보 발생이 줄어들어 안전성이 높아집니다.

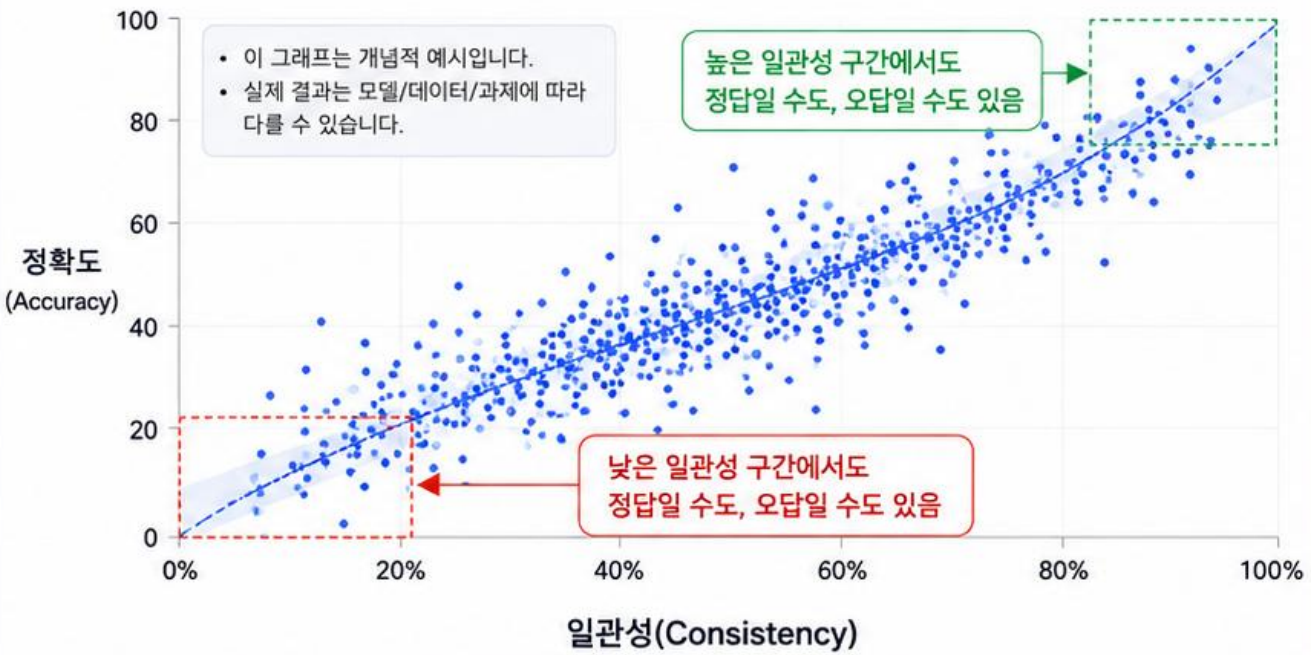
결론 (Conclusion)

반복적 자기검토는 다양한 작업에서 오류 감소, 일관성 및 추론 깊이 향상, 환각 감소에 도움을 줄 수 있습니다. 모델, 작업, 평가 조건에 따라 효과는 달라질 수 있습니다.

응답 일관성과 신뢰도: 일관성만으로는 충분하지 않다

다수의 경로 샘플링과 일관성 기반 검증을 통해 답변의 안정성과 신뢰도를 극대화합니다.

일관성(Consistency) vs 정확도(Accuracy)



일관성(Consistency)은 모델이 동일한 답을 얼마나 자주 생성하는지를 나타내는 지표입니다.
일관성이 높다고 해서 정답일 확률이 보장되지는 않습니다.

결함 복구 능력 (Self-Healing)

예시 프롬프트에 약간의 수식 오류나 모호함이 섞여 있더라도, 다수의 경로 샘플링을 거치면 자체적으로 노이즈를 상쇄하고 정답을 찾아낼 가능성이 높아집니다.

- 노이즈 상쇄
- 정답 수렴

불확실성 측정 지표 (Model Calibration)

일관성(동일한 답이 나온 비율)이 높을수록 정확도가 높아질 가능성은 있지만, 일관성은 정답 확률을 직접 나타내는 지표가 아닙니다. 별도의 신뢰도 조사(확률 추정), 검증 데이터 기반 평가, 온도 스케일링 등으로 보정이 필요할 수 있습니다.

- 일관성 ≠ 신뢰도(정답 확률)
- 신뢰도는 별도 보정 필요

추가 검증의 필요 (Verification is Essential)

동일한 오류가 반복될 수 있으므로, 계산 검증·외부 지식 확인·도구 활용 등 외부 검증 절차를 반드시 병행해야 합니다.

- 외부 검증 병행
- 사실 검증 / 도구 활용

핵심 주의: 일관성은 "모델의 확신"이 아니라 "반복 일치성"을 의미합니다.
반복 생성과 다수결은 응답 안정성을 높일 수 있지만, 동일한 오류가 반복될 수 있으므로 외부 자료 또는 정답 기준을 이용한 검증이 필수적입니다.

반복 생성 (샘플링) | 다수결 (집계) | 외부 검증 (사실 검증/도구)

기억하세요: 일관성은 필요한 지표이지만 충분조건은 아닙니다. 다양한 지표와 검증 절차를 함께 사용해야 신뢰할 수 있는 답변을 얻을 수 있습니다.

비즈니스 스케일링: 산업별 프롬프트 적용 토폴로지

산업별 업무 특성에 맞는 프롬프트 체계를 설계하고, LLM의 잠재력을 비즈니스 성과로 연결합니다.

LLM Applications

콘텐츠 생성 (Content Generation)

Zero/Few-shot 프롬프트 활용으로
고품질 콘텐츠를 빠르게 생성합니다.

주요 적용

- **글쓰기 보조** (Writing Assistance)
  보고서, 제안서, 문서 초안 작성
- **창의적 글쓰기** (Creative Writing)
  아이디어 발상, 스토리, 카피라이팅
- **마케팅 카피** (Marketing Copy)
  광고 문구, 랜딩 페이지, 제품 설명

질의 응답 및 검색 (Question Answering)

Contextual Prompting 기반으로
정확한 정보 검색과 신속한 응답을 지원합니다.

주요 적용

- **기업 내부 검색** (Internal Search)
  사내 문서, 정책, 업무 매뉴얼 검색
- **고객 지원 챗봇** (Customer Support Chatbot)
  FAQ 응답, 문제 해결, 고객 상담 자동화
- **데이터 분석 보조** (Data Analysis Assistant)
  데이터 요약, 인사이트 도출, 리포트 지원

코드 생성 (Code Generation)

CoT(Chain-of-Thought) 프롬프트로
정확하고 최적화된 코드를 생성합니다.

주요 적용

- **자동 완성** (Auto-completion)
  코드 스니펫, 함수, 로직 자동 완성
- **코드 최적화** (Code Optimization)
  성능 개선, 리팩토링, 알고리즘 제안
- **디버깅 지원** (Debugging Support)
  오류 원인 분석, 해결 방안 제시

복잡한 논리적 추론 (Logical Inference)

Self-Consistency 및 다단계 추론으로
복잡한 의사결정을 지원합니다.

주요 적용

- **금융 분석** (Financial Analysis)
  재무 분석, 시장 예측, 리스크 분석
- **전략 도출** (Strategy Formulation)
  시장 전략, 시나리오 분석, 실행 계획
- **리스크 평가** (Risk Assessment)
  위험 식별, 영향도 평가, 대응 방안 제안

핵심 가치

생산성 향상
반복 업무 자동화로
업무 효율 극대화

정확도 및 일관성
표준화된 프롬프트로
신뢰도 높은 결과 재공

확장성 및 재사용성
산업별 템플릿과 프롬프트
재사용으로 빠른 확산

보안 및 거버넌스
접근 제어 및 데이터 보호로
안전한 AI 활용 보장

통합 (Synthesis): 의도를 인지로 번역하는 인터페이스 과학

인간의 복잡한 비즈니스 의도를 프룸프트 프리즘을 통해 구조화하고, 집단 지성 엔진으로 검증·강화하여 정확하고 실행 가능한 결과로 전달합니다.

인간의 복잡한 비즈니스 의도
(Human's complex business intent)

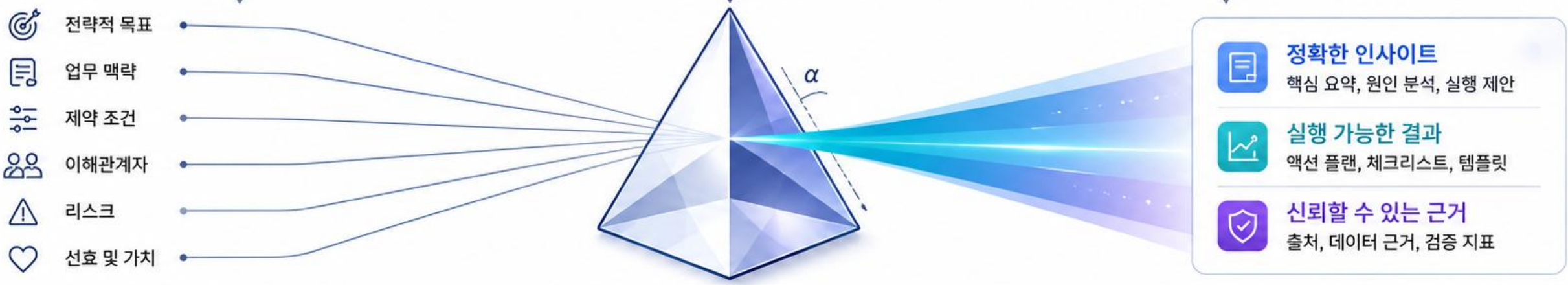
목표, 제약, 맥락, 선호, 리스크 등
다층적 의도 입력

프룸프트 프리즘
(Prompt Prism)

의도를 구조화·분해·정규화하여
모델이 이해할 수 있는 형태로 변환

집단 지성 엔진
(Collective Intelligence Engine)

다중 모델, 지식 그래프, 평가 루프를 통해
결과의 정확성·신뢰성·실행 가능성 강화



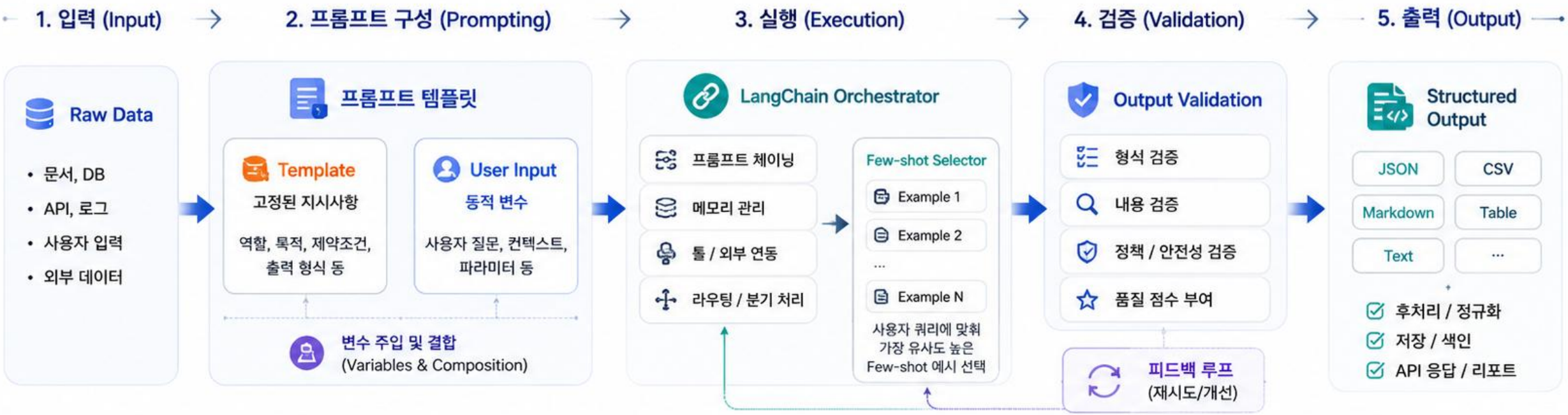
프룸프트 엔지니어링은 AI에게 명령을 내리는 단순한 '타이핑 기술'이 아닙니다.
그것은 확률의 바다에서 논리의 섬을 용기시키고, 환각의 노이즈를 걸러내어,
단일 모델을 '스스로 생각하고 합의를 도출하는 집단 지성 엔진'으로 진화시키는
가장 정교한 인터페이스 과학(Interface Science)입니다.

구글 클라우드 Vertex AI와 같은 인프라 위에서
이 인지 설계를 펼칠 때, 언어 모델의 한계는
비로소 인텔리전스의 새로운 지평으로
전환됩니다.
(Powered by Vertex AI)

의도 입력 다층적 요구사항 수집 › **의도 구조화** 프리즘을 통한 분해·정규화 › **집단 지성 검증** 다중 모델 평가 및 합의 › **신뢰 가능한 출력** 인사이트, 실행안, 근거 제공

AI 파이프라인 구축: LangChain과 프롬프트 템플릿

데이터부터 검증된 출력까지, LangChain 기반의 모듈형 프롬프트 파이프라인으로 안정적이고 확장 가능한 AI 서비스를 구축합니다.



Variables & Templates

고정된 지시사항(Template)과 동적 변수(User Input)를 분리하여 재사용성과 일관성을 확보합니다.

역할 정의 | 제약 조건 | 출력 형식 | 변수 치환

Example Selector

사용자 쿼리의 의미와 유사도가 높은 Few-shot 예시를 자동으로 선택하여 프롬프트 품질과 성능을 향상시킵니다.

임베딩 유사도 | 의미 기반 매칭 | 동적 선택

Output Parser

LLM의 출력을 구조화된 형식으로 변환하고, 검증을 통과한 결과만 안정적으로 활용할 수 있도록 보장합니다.

파싱 | 정규화 | 검증 | 변환

일관된 품질

표준화된 템플릿과 검증으로 안정적 결과 제공

개발 효율성

모듈화된 파이프라인으로 빠른 구축과 확장

신뢰성과 안전성

검증·피드백 루프로 정확도와 안전성 강화

다양한 연동

DB, API, 도구 등 외부 시스템과 유연한 통합

범용 반복 개선 프롬프트 템플릿

실전에서 바로 쓰는 재사용 템플릿

한 번의 응답에 의존하지 않고, **평가 → 개선 → 최종 검수**를 통해 더 높은 품질의 결과를 얻는 반복 프로세스 템플릿

1. 초안 작성

아래 작업의 초안을 작성해줘.

- 주제, 목적, 대상
- 핵심 요구사항
- 포함해야 할 내용

2. 평가 및 검토

다음 평가 기준으로 결과를 검토해줘.

- 평가 기준에 따라 점수(또는 등급)
- 근거와 구체적 의견
- 부족한 항목과 개선점

3. 개선본 작성

부족한 항목을 수정해 개선본을 작성해줘.

- 2단계 피드백을 반영
- 더 나은 결과로 재작성
- 필요한 경우 보완 자료 추가

4. 최종 검수

최종본에서 오류, 중복, 누락을 점검해줘.

- 사실 오류 및 논리 오류
- 중복 내용 확인
- 누락된 필수 항목 확인

5. 최종 출력

문제가 없으면 최종본만 출력해줘.

- 모든 항목이 충족되었는지
- 깔끔하고 완성도 높은 결과
- 최종본만 간결하게 제공

평가 기준 예시 (필요에 따라 조정하세요)

정확성	내용이 사실에 부합하고 논리적으로 일관되는가?
완전성	필수 항목과 요구사항을 빠짐없이 포함했는가?
구조 및 가독성	구성이 체계적이고 읽기 쉽게 정리되었는가?
실용성 및 적합성	목적과 대상에 적합하고 실제 활용이 가능한가?
창의성 및 차별성	기존 방법보다 더 나은 인사이트나 아이디어가 있는가?

이 템플릿을 사용하면 이런 효과가 있습니다

정확도 향상
체계적 검토와 수정을 통해 오류를 줄이고 정확도를 높입니다.

일관성 확보
평가 기준을 유지하여 결과물의 일관성과 품질이 안정됩니다.

효율성 증가
반복 프로세스를 표준화하여 시간과 비용을 절약합니다.

재사용성 극대화
템플릿 형태로 다양한 작업에 빠르게 적용할 수 있습니다.

핵심 메시지 | 좋은 결과는 우연이 아니라 **설계된 프로세스**의 산물입니다. 반복하고 검증하며 개선하는 사람이 더 나은 성과를 만듭니다.

모델별 맞춤형 접근 전략

아래 내용은 일반적인 경향을 설명한 것으로, 버전, 요금제, 연결 도구 및 시점에 따라 달라질 수 있습니다.

ChatGPT

일반적으로 구조화된 작성, 코딩 보조, 아이디어 발상,
다양한 작업에 폭넓게 활용됩니다.

전략

명확한 목적과 형식을 제시하고,
단계별 사고와 검증을 유도하면
정확도와 일관성이 향상됩니다.

- 명확한 지시와 예시 제공
- 단계별 사고(Chain-of-Thought) 활용
- 결과 검토 및 검증 요청

Claude

일반적으로 긴 문서 이해, 미묘한 뉘앙스 파악,
신중하고 안전한 응답이 필요한 작업에 강점이 있습니다.

전략

상세한 배경 맥락과 목표를
제공하고, 제약 조건을 함께
제시하면 응답 적합도가 높아집니다.

- 배경·목표·제약 조건 명확화
- 예시 및 템플릿 제공
- 출력 포맷 지정

Gemini

일반적으로 시각 자료 이해, 실시간 정보 활용,
Google 생태계 연동이 필요한 작업에 유리합니다.

전략

이미지·표·동영상 등 멀티모달 입력과
최신 정보 결합으로 분석의 깊이와
최신성을 높일 수 있습니다.

- 멀티모달 입력 적극 활용
- 실시간 정보/웹 연동 활용
- Google 도구와 연계(검색, 드라이브 등)

Perplexity

일반적으로 출처 기반 검색과 답변이 필요한 작업에
강점을 가지며, 신뢰 가능한 정보를 빠르게 제공합니다.

전략

검색 키워드와 조건을 구체화하고,
출처 범위와 최신성 옵션을 활용하면
신뢰도 높은 답변을 얻을 수 있습니다.

- 구체적인 검색 키워드 사용
- 출처/기간/도메인 범위 설정
- 출처 확인 및 교차 검증

중요 안내

모델 기능과 성능은 버전 및 서비스 업데이트에 따라 변경될 수 있으므로,
실제 작업에서는 최신 기능과 결과를 직접 비교해야 합니다.

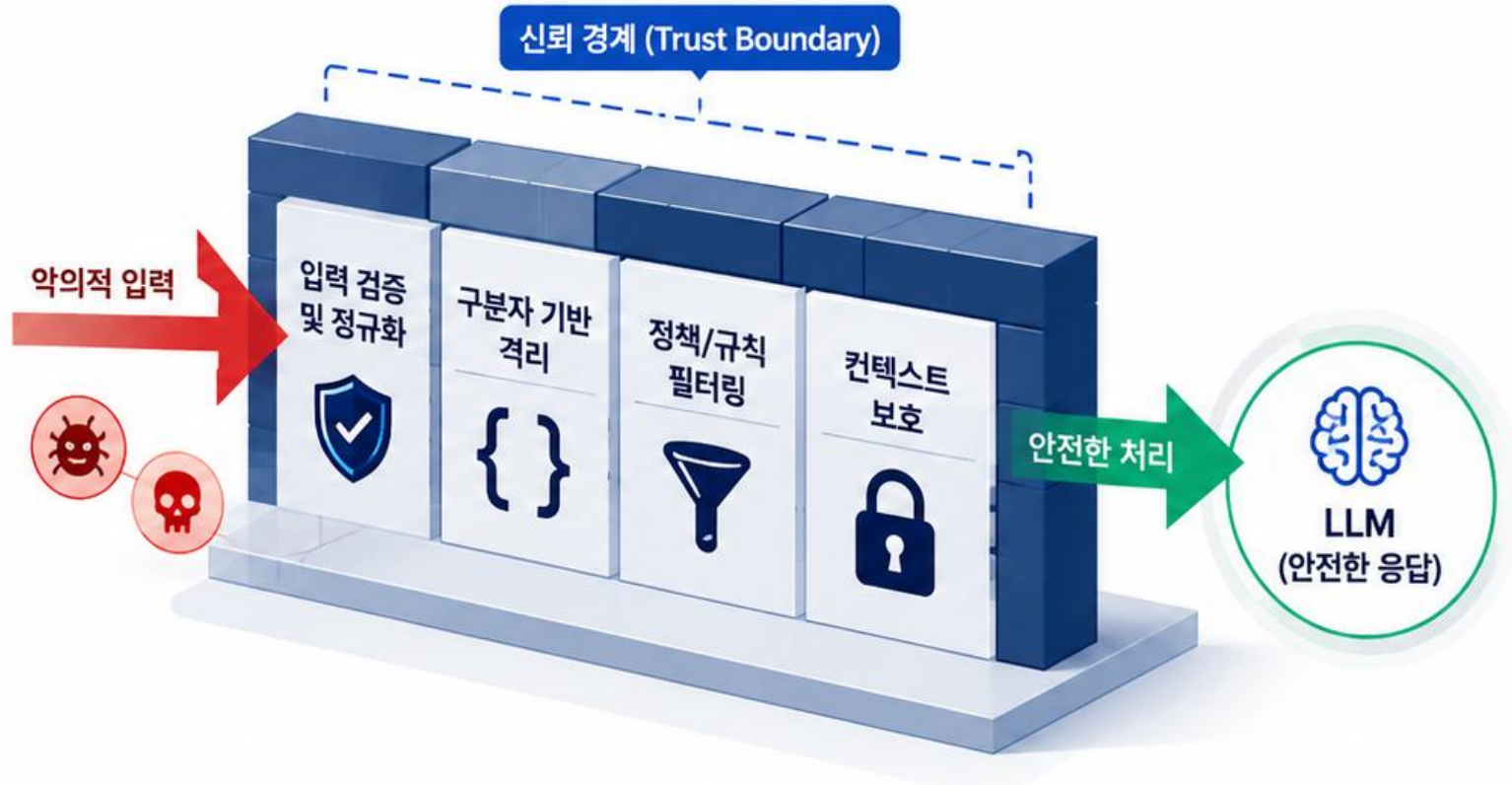
활용 팁

- 여러 모델의 결과를 비교하여 최적의 답변을 선택하세요.
- 중요한 결정이나 사실 확인이 필요한 경우, 교차 검증을 권장합니다.
- 프롬프트를 구체적으로 작성하고, 출력 형식과 평가 기준을 명확히 설정하세요.

보안 설계: 프롬프트 인젝션 방어

사용자의 악의적 입력값이 시스템의 기본 지시사항을 탈취(프롬프트 인젝션)하지 못하도록 방어해야 합니다.

구분자를 통한 논리적 격리가 필수적입니다.



핵심 원칙: 명령과 데이터의 완전한 분리

- 사용자 입력과 시스템 지시를 구조적으로 분리
- 신뢰 경계 설정 및 입력 검증/정규화 수행
- 최소 권한 원칙으로 시스템 컨텍스트 보호

구현 가이드

1. 입력 검증 및 정규화

- 허용 목록 기반 검증
- 길이 제한 및 특수문자 필터링
- 인코딩 정규화

2. 구분자 기반 격리

- 구조화된 템플릿 사용
- 명령과 데이터 명확히 구분
- 강력한 구분자 적용

3. 정책 및 규칙 필터링

- 금지/제한 키워드 탐지
- 의도 분류 및 위험도 평가
- 정책 위반 시 차단/완화

4. 컨텍스트 보호

- 시스템 지시사항 비공개
- 최소 권한으로 정보 노출 제한
- 내부 프롬프트 보호

5. 모니터링 및 검사

- 인젝션 시도 및 이상 탐지
- 로그 수집 및 알림
- 지속적인 모델/정책 업데이트

추가 권장: 정기적인 보안 테스트(레드팀), 프롬프트 보안 평가, 종속 구성 요소 업데이트, 사용자 피드백 기반 개선

프롬프트 엔지니어링을 위한 최종 체크리스트

프롬프트 엔지니어링은 예술이 아니라 구조적 과학입니다. 명확한 규칙, 견고한 구조,
그리고 엄격한 검증 파이프라인을 통해 실험실의 모델을 신뢰할 수 있는 **임상 AI 에이전트**로 전환하십시오.

1 매칭 (Matching)

해결하려는 작업(분류, 추출, 해설)의 본질에 따라 최적의 프롬프트 전략(Heuristic, CoT 등)과 모델을 매칭합니다.

2 구조화 (Structuring)

줄글을 피하고, 마크다운(Markdown), 역할 정의, 출력 형식(스키마) 등을 활용하여
변수와 시스템 규칙을 구조적으로 분리해 모델의 인지적 부담을 최소화합니다.

3 통제 (Controlling)

사전 정의된 규칙(Heuristics)과 명확한 페르소나(Persona) 패턴을 적용하여
LLM의 출력 변동성을 줄이고 일관된 품질을 유지합니다.

4 방어 (Defending)

출력 전 다중 검증 파이프라인(LLM-as-a-Judge, 규칙 기반 검증, 휴먼 리뷰)을 배치하고
구분자를 통한 인젝션 방어벽을 구축하여 보안을 확보합니다.

핵심 원칙 | 명확하게 정의하라 | 구조적으로 설계하라 | 일관되게 통제하라 | 다중 검증으로 신뢰하라 | 안전하게 방어하라

궁극적 진화: Prompter에서 Architect로

프롬프트 엔지니어링의 본질은
단순히 **'단어의 조합'**을 찾는 것이 아닙니다.

그것은 **'맥락의 정제(Context Engineering)'**이며,
인간의 사고 과정을 AI가 이해할 수 있는
'프로세스로 설계'하는 건축 작업입니다.

입력 (Input)

- **목표/질문 정의**
  무엇을 해결하고, 어떤 답을 원하는가?
- **맥락 수집 및 정제**
  필요한 정보와 제약 조건을 선별하고 구조화
- **프로세스 설계**
  사고 흐름을 단계별로 설계하고 연결
- **프롬프트 구조화**
  모델이 일관되게 이해할 수 있도록 구성

출력 (Output)

- **목적에 최적화된 응답**
  정확하고 일관된 결과 도출
- **신뢰성 확보**
  환각 최소화, 근거 기반 답변 제공
- **확장 가능한 설계**
  재사용 및 자동화로 생산성 향상
- **지속적 개선**
  피드백 기반 평가 및 반복 학습

단순히 AI에게 질문하는 **사람(Prompter)**에 머물지 마십시오.
무한한 잠재력을 가진 지능 엔진을 완벽하게 조율하는 **설계자(Architect)**가 되십시오.

핵심 원칙 | 사고를 구조화하라 | 맥락을 정제하라 | 프로세스로 설계하라 | 일관되게 검증하라 | 지속적으로 개선하라

생성형 AI와 프로그래밍의 진화

AI는 코드를 '예측'하는 수준을 넘어, 목표를 이해하고 스스로 계획하고 실행하는 'AI Coding Agent'로 진화했습니다.

1. Traditional Coding

개발자가 설계부터
코드 작성까지 직접 수행

```python
def calculate_average(values):
    return sum(values) / len(values)
```

개발자가 직접 결정해야 하는 것들

- 요구사항 이해
- 알고리즘 설계
- 함수 작성
- 실행
- 오류 수정
- 테스트

2. Code Completion

개발자가 코드를 작성하는 과정에서
AI가 다음 코드를 예측

```python
def calculate_average(
```

AI 예측 결과

```python
def calculate_average(values):
    return sum(values) / len(values)
```

개발자가 주도하고
AI는 일부 코드를 보완한다.

"AI는 다음 코드를 예측하여 개발의 흐름을 돕습니다."

3. AI Coding Assistant

자연어로 요청하면 AI가
코드 작성, 설명, 수정까지 지원

요청 예시

"빈 리스트도 처리할 수 있도록 calculate_average 함수를 수정해줘."

AI 응답 예시

```python
def calculate_average(values):
    if not values:
        return 0
    return sum(values) / len(values)
```

AI가 코드 작성뿐 아니라
설명과 수정까지 도와줍니다.
(작업 단위는 비교적 작음)

4. Vibe Coding

구현 방법보다 원하는 결과를
자연어로 설명

요청 예시

"학생들의 시험 점수를 입력하면
- 평균
- 최고점
- 최저점

을 보여주는 프로그램을 만들어줘."

User Intent → Natural Language → AI → Code

Vibe Coding에서는
'어떻게 코딩할 것인가?'보다
'무엇을 만들 것인가?'의 비중이 커진다.

5. AI Coding Agent

목표를 받아 스스로 계획하고
다양한 작업을 수행

요청 예시 (Codex에 요청)

"학생 성적 관리 웹사이트를 만들어줘.

요구사항
- CSV 업로드
- 평균 점수 계산
- 최고점/최저점 확인
- 학생별 점수 조회
- 성적 분포 그래프
- 예외 처리
- README 작성
- 테스트 코드 작성"

Agentic Workflow (예시)

요구사항 분석 → 프로젝트 탐색 → 계획 수립 → 파일 생성/수정 → 코드 작성 → 명령 실행 → 테스트 실행 → 오류 분석 → 수정 → 재실행 → 결과 검증 → 결과 제공

Codex는 코드베이스를 탐색하고,
변경사항을 만들고, 테스트를 실행하여
리뷰 가능한 결과를 제공합니다.

1교시 실습: 같은 기능을 세 가지 방식으로 요청해보기

① 짧은 요청

"성적 프로그램 만들어줘."

② 구체적인 요청

"학생들의 점수를 입력받아 평균, 최고점, 최저점을 출력하는 Python 프로그램을 만들어줘."

③ Agentic 요청

"학생 성적 관리 프로그램을 만들어줘.
먼저 요구사항을 분석하고
프로젝트 구조를 설계한 뒤 구현해줘.
구현 후
- 예외 처리
- 테스트
- README

까지 작성해줘."

1교시 핵심 메시지

- **Code Completion** = 다음 코드를 예측
- **Coding Assistant** = 개발자를 보조
- **Vibe Coding** = 자연어로 구현
- **Coding Agent** = 목표를 받아 작업 수행

핵심 차이 요약

구분	주도권	작업 범위	AI 역할
Code Completion	개발자	매우 작음 (한 줄, 한 블록)	다음 코드 예측
Coding Assistant	개발자	작음 (함수 단위)	코드 작성/수정 보조
Vibe Coding	사용자(자연어)	중간 (프로그램 단위)	구현 방법 결정 및 코드 생성
AI Coding Agent	AI Agent	큼 (프로젝트 단위)	계획, 구현, 실행, 검증까지 수행

바이브코딩 (Vibe Coding)이란?

자연어로 아이디어와 느낌을 설명하면 AI가 코드를 생성해주는 새로운 개발 방식

> 완벽한 문법을 몰라도 괜찮아요.
> 아이디어와 '바이브'를 전달하면 AI가 구현해줍니다.

어떻게 작동하나요?

1. 아이디어 전달

자연어로 만들고 싶은 기능이나 느낌을 설명합니다.

예) "미니멀한 할 일 관리 웹앱을 만들어줘"

2. AI가 코드 생성

AI가 요구사항을 이해하고 코드를 생성합니다.

3. 결과 확인

생성된 코드를 실행해 결과를 확인합니다.

4. 피드백 & 개선

원하는 부분을 자연어로 피드백하면 AI가 코드를 수정해줍니다.

5. 완성

반복을 통해 아이디어를 빠르게 구현하고 완성합니다.

핵심은 '바이브(Vibe)'

정확한 문법보다 맥락, 의도, 느낌을 AI에게 잘 전달하는 것이 중요합니다.

창의성에 집중하고, 구현은 AI와 함께!
더 빠르게, 더 자유롭게 개발하세요.

바이브코딩의 장점

개발 속도 향상

아이디어를 빠르게 프로토타입으로 구현하고 검증할 수 있습니다.

진입 장벽 감소

프로그래밍 경험이 적어도 개발에 참여할 수 있습니다.

창의성 극대화

구현 부담을 줄여 아이디어와 문제 해결에 더 집중할 수 있습니다.

협업 강화

기획자, 디자이너, 개발자가 자연어로 소통하며 함께 만들 수 있습니다.

기억하세요!

AI가 생성한 코드는 항상 검토하고, 이해하고, 책임감 있게 사용해야 합니다.

AI는 도구일 뿐, 주체는 여러분입니다.

에이전트 AI란?

목표를 이해하고, 스스로 계획·행동·평가하여 과업을 완수하는 자율 지능 시스템

정의

사람이 설정한 목표를 달성하기 위해
스스로 계획을 세우고, 필요한 도구를 사용하며,
결과를 평가하고 다음 행동을 결정하는 AI

핵심 특징

목표 지향
사용자가 원하는 최종 목표를 이해하고 달성을 위해 행동

스스로 계획
복잡한 목표를 작은 단계로 나누고 계획을 수립

도구 활용
검색, 코드 실행, 외부 API 등 다양한 도구를 적절히 사용

기억과 추론
이전 경험과 정보를 기억하고 추론하여 더 나은 의사결정

자율 평가와 개선
결과를 스스로 평가하고 실패 시 원인을 분석해 다시 시도

작동 방식 (Agent Loop)

1. 목표 입력
사용자가 목표를 제시합니다.

→

2. 계획 수립
목표 달성을 위한 계획을 세웁니다.

→

3. 행동 실행
계획에 따라 도구를 사용해 행동합니다.

→

4. 관찰 및 평가
결과를 확인하고 성공 여부를 평가합니다.

→

5. 다음 행동 결정
필요 시 계획을 수정하고 다시 행동합니다.

↩ 목표 달성까지 반복

전통적인 AI와의 차이

	전통적인 AI (단순 모델)	에이전트 AI
입력	단순 질문 또는 명령	목표(Goal)
동작	한 번의 응답 생성	여러 단계를 스스로 수행
도구 사용	제한적	다양한 도구를 선택·사용
기억/맥락	짧은 대화 맥락	장기 기억과 추론 가능
실패 처리	오류 발생 시 종료	원인 분석 후 전략 수정
결과	단일 결과	목표 달성에 집중한 결과

활용 예시

코딩 에이전트
요구사항을 이해하고, 코드 작성 → 실행 → 오류 수정 → 테스트까지 자동 수행

리서치 에이전트
주제 조사 → 정보 수집 → 요약 → 보고서 작성까지 스스로 진행

업무 자동화 에이전트
이메일 정리, 일정 관리, 문서 작성 등 번복 업무를 계획하고 실행

데이터 분석 에이전트
데이터 수집 → 전처리 → 분석 → 시각화 → 인사이트 도출까지 자율적으로 수행

왜 중요한가?

생산성 향상
복잡한 작업을 더 빠르고 효율적으로 완수

24/7 자율 실행
사람의 개입 없이 지속적으로 작업을 수행

확장성과 일관성
여러 작업을 동시에 일관된 품질로 처리

새로운 가치 창출
사람이 더 창의적인 일에 집중할 수 있도록 지원

☆ **핵심 한 줄 요약** 에이전트 AI는 스스로 생각하고 계획하며 행동하여 목표를 끝까지 해내는 '자율 지능 비서'입니다.

코덱스의 진화: 두루마리에서 자율형 AI 에이전트까지

과거: 인류의 지식을 담은 두루마리

지식의 기류과 보존
인류의 지혜를 기록하고
세대를 넘어 보존

정적인 정보
사람이 읽고 해석해야 하는
정적인 형태의 지식

제한된 활용
찾기 어렵고, 업대이트가
느리며 확장성이 낮음

미래: 스스로 생각하고 실행하는 자율형 AI 에이전트

agent.py | goals.yaml | tools.py | memory.db

```python
# Autonomous AI Agent
class AutonomousAgent:
    def __init__(self, knowledge, goals, tools):
        self.knowledge = knowledge
        self.goals = goals
        self.tools = tools
        self.memory = Memory()
        self.status = "idle"

def run(self):
    plan = self.plan()
    for step in plan:
        result = self.execute(step)
        self.learn(result)
    return "Mission Accomplished"
```

AI Agent Capabilities

- **목표 이해 및 계획 수립** — 복잡한 목표를 분석하고 실행 계획을 자동 생성
- **도구 활용 및 실행** — 코드 실행, 웹 검색, API 호출 등 다양한 도구를 활용
- **기억과 학습** — 경험을 기억하고 지속적으로 성능을 개선
- **자율적 의사결정** — 상황을 판단하고 최적의 행동을 스스로 결정
- **결과 평가 및 피드백** — 결과를 분석하고 다음 행동을 개선하여 반복

```
TERMINAL
$ python agent.py
▸ Planning mission... Done
▸ Executing step 1/5: Gather Information ... Done
▸ Executing step 2/5: Analyze Data ... Done
▸ Executing step 3/5: Construct Solution ... Done
▸ Executing step 4/5: Validoze ... Done
▸ Executing step 5/5: Report ... Done
✓ Mission Accomplished
```

- ✓ 실시간성
- ✓ 확장성
- ✓ 스스로 개선
- ✓ 반복적 작업 자동화

전통적 코드 (도구)

- **명령에 따라 실행** — 사람의 지시가 있어야 동작
- **단일 작업 중심** — 정해진 기능만 수행
- **상태 없음** — 기억하지 못하고 매번 초기화
- **유연성 제한** — 예외 상황에 약하고 변화에 취약

VS

코덱스 기반 AI 에이전트 (파트너)

- **목표 기반 자율 실행** — 스스로 계획하고 목표를 달성
- **다단계 복합 작업** — 여러 단계를 연결해 복잡한 문제 해결
- **기억과 학습** — 경험을 기억하고 지속적으로 개선
- **높은 유연성** — 변화에 적응하고 최적의 방법을 찾음

코덱스는 단순한 코드 도구가 아니라, 문제를 해결하는 자율형 AI 파트너입니다.

첫 번째 코덱스: 정보 접근 방식의 혁명

인류는 기록 방식을 혁신하며 지식에 더 빠르고 효율적으로 접근할 수 있게 되었습니다.

파피루스 두루마리 (Papyrus Scroll)

순차적 접근
처음부터 끝까지
순서대로 읽어야 함

시간 소요
특정 정보를 찾기 위해
많은 시간이 필요

제한된 탐색
한 번에 한 부분만
확인 가능

특징 요약
한 번에 한 부분만 읽을 수 있으며,
물리적으로 취약하고 보관이 어려움

시대적 배경

- 기원전 3000년경
  이집트에서 널리 사용
- 길고 비싼 제작 과정으로
  대량 생산이 어려움
- 부피가 크고 보관·이동이
  불편하여 손상 위험 높음

양피지 코덱스 (Parchment Codex)

임의 접근
필요한 페이지로
즉시 이동 가능

양면 활용
Recto/Verso로
효율성 극대화

빠른 탐색
색인·목차를 통한
신속한 정보 탐색

특징 요약
서기 1세기 마르티알리스가 착안한 구조적 혁신.
4세기경 두루마리를 대체하며 프로젝트 전체를
한눈에 파악할 수 있는 기반을 마련

시대적 배경

- 서기 1세기 착안,
  4세기경 널리 확산
- 내구성이 높고 보관·이동이
  용이하여 보급 확대
- 지식 보존과 전파의 효율이
  비약적으로 향상

핵심 인사이트

파피루스 두루마리에서 양피지 코덱스로의 전환은 단순한 기록 매체의 변화가 아니라,
"정보 접근 방식의 혁명"이었습니다. 이는 현대 디지털 시대의 데이터 구조와 UX 혁신으로 이어지고 있습니다.

두 번째 코덱스: 코드베이스 전체를 이해하는 자율성의 탄생

코덱스는 단일 파일 생성에서 벗어나, 코드베이스 전체를 이해하고 스스로 계획·수행하는 자율형 소프트웨어 엔지니어로 진화했습니다.

과거의 AI (두루마리 방식)

ⓘ 한 줄씩 순차적으로 제안하는 기존 AI 자동완성 (Legacy Autocomplete). 프로젝트의 전체 맥락을 알지 못함.

OpenAI Codex (코덱스 방식)

✓ 리포지토리 전체 구조를 읽고, 스스로 분석하여 완전한 기능을 구축하는 자율형 소프트웨어 엔지니어링 에이전트.

전체 맥락 이해
파일·의존성·구조를 파악하여
프로젝트 전체 맥락을 이해

자율 계획 수립
작업을 작은 단위로 분해하고
실행 계획을 스스로 수립

코드 생성·수정
여러 파일을 동시에 생성·수정하여
일관성 있는 코드 구현

실행 및 검증
테스트 실행, 결과 검증으로
품질을 자동으로 확인

지속적 개선
피드백을 반영하여 코드를 개선하고
더 나은 해결책을 제시

OpenAI Codex는 무엇인가?

Codex는 AI가 코드를 이해하고 작성 · 수정 · 검증까지 수행하는 클라우드 기반 자율 코딩 에이전트입니다.

1. 자연어 프롬프트

자연어로 요청하면

Codex가 요구사항을 이해하고
실행 계획을 수립합니다.

예시

"로그인 폼에 이메일 형식 검증을 추가해줘."

2. GPT-5.6-Codex 모델 (최신 코딩 AI)

격리된 클라우드 샌드박스에서
안전하게 자동 실행 및 검증

코딩에 특화된 강력한 추론 엔진

코드 이해, 생성, 수정, 디버깅, 테스트, 문서화는 물론
도구 사용과 복잡한 개발 작업까지 자율적으로 수행합니다.

이해 | 생성 | 수정 | 디버깅 | 테스트 | 문서화

최신 모델: GPT-5.3-Codex NEW

장시간·복잡한 작업 수행, 도구 사용, 인터랙티브 조정, 테스트/검증 지원

3. GitHub Pull Request

완성된 결과를 PR로 제공

변경 사항을 코드, 테스트와 함께
Pull Request로 자동 제출합니다.

예시

기능 구현, 버그 수정, 리팩터링 절과가
PR로 제출됩니다.

Codex의 핵심 특징

클라우드 기반
격리된 실행 환경에서
안전하고 안정적으로 동작

안전한 실행
네트워크 격리, 권한 제한으로
보안 위험 최소화

코드베이스 이해
대규모 저장소의 구조와
의존성을 정확히 파악

자동 검증
빌드, 테스트, 린트 동을
자동 실행하여 검증

반복적 개선
피드백을 반영하여 지속적으로
계획을 보완하고 개선

팀과 협업
개발 워크플로우에 자연스럽게
통합되어 협업 효율 향상

핵심 요약 자연어로 요청하면 Codex가 **이해** → 계획 → **실행** → **검증** → **PR 제출**까지 전 과정을 자율적으로 수행합니다.

기존 AI 코딩 도구가 실패했던 4가지 한계와 Codex의 해결책

Codex는 단순 코드 생성이 아니라, 실제 개발 전 과정을 자율적으로 수행하는 엔지니어링 에이전트입니다.

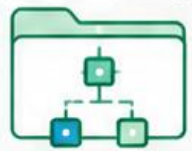
1 컨텍스트 단절 (Context Disconnect)

과거

열려있는 단일 파일
중심의 제한적 이해.

Codex

디렉토리 구조, 의존성 그래프,
코딩 컨벤션을 포함한
리포지토리 전체 분석.



2 단일 파일 한계 (Single-file Limits)

과거

여러 파일에 걸친
기능 구현 불가.

Codex

동시에 여러 모듈을 넘나들며
파일 수정 및 생성.

3 환경 격리 부재 (Lack of Isolation)

과거

로컬 환경에서 직접 코드를
수정하여 예기치 않은
부작용 발생.

Codex

로컬 환경을 전혀 건드리지 않는
클라우드 샌드박스 실행.

4 검증 불가능한 출력 (Unverifiable Output)

과거

개발자가 직접 복사, 붙여넣기,
빌드, 테스트를 수행해야 함.

Codex

생성 후 자체적으로 테스트 스위트
실행 및 결과 검증.

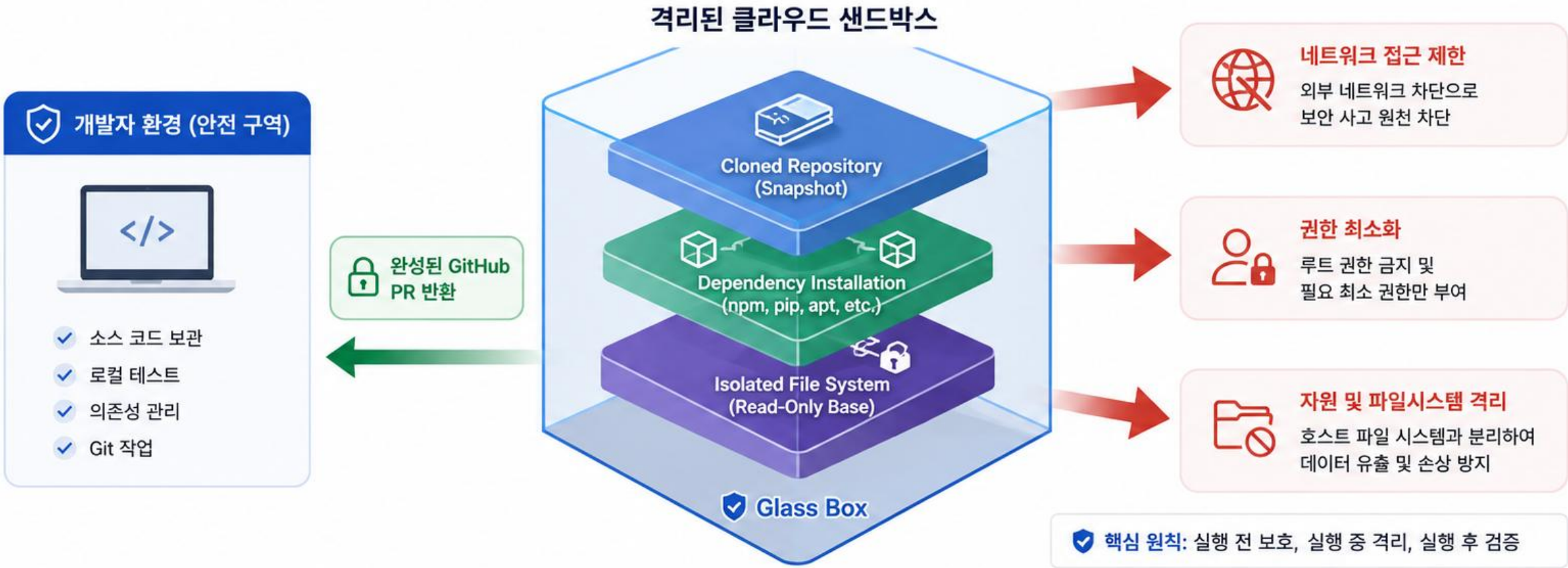
Codex의 핵심 가치

- **전체 맥락 이해** — 프로젝트 전체를 이해하고 일관성 있는 변경 수행
- **완전한 자율성** — 계획 → 구현 → 테스트 → 검증까지 전 과정을 스스로 수행
- **안전한 실행** — 클라우드 기반 격리 환경에서 안전하게 실행 및 검증
- **개발 생산성 향상** — 반복 작업 자동화로 개발자가 더 중요한 일에 집중
- **신뢰할 수 있는 결과** — 검증 가능한 결과물로 높은 품질 보장

Codex는 단순히 코드를 '생성'하는 도구가 아닙니다. **이해하고, 계획하고, 실행하고, 검증하는 완전한 AI 소프트웨어 엔지니어입니다.**

격리 우선(Isolation-First): 안전한 클라우드 샌드박스 엔진

개발 환경을 안전하게 보호하면서, 필요한 작업만 격리된 환경에서 실행합니다.



핵심 원칙: 실행 전 보호, 실행 중 격리, 실행 후 검증

완전한 격리
개발 환경과 완벽 분리된 안전한 실행 공간

보안 강화
네트워크/권한/파일 접근을 철저히 제한

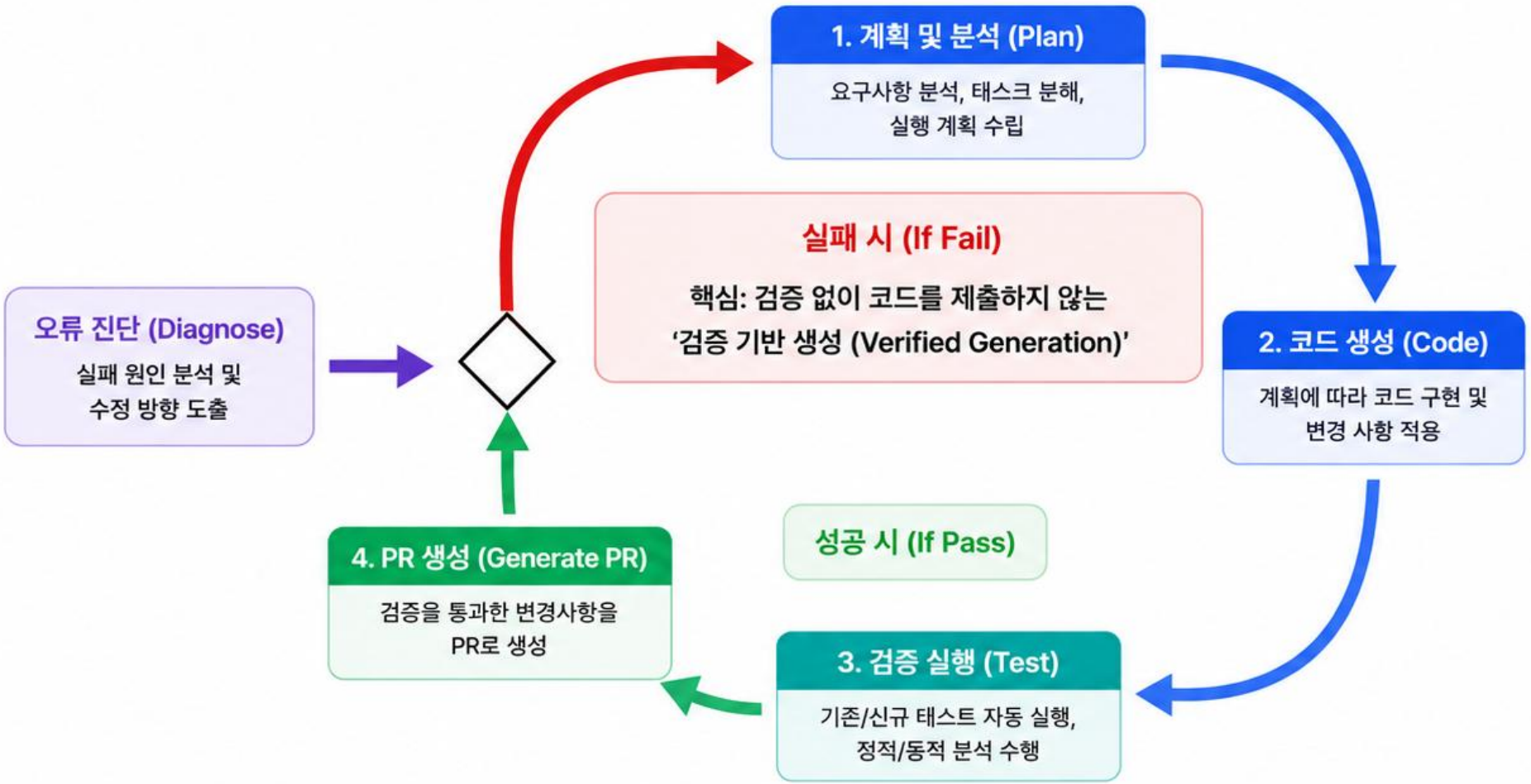
안정성 보장
악성 코드, 의존성 충돌, 시스템 손상 위험으로부터 환경 보호

성능 최적화
캐시 및 증분 빌드로 빠르고 효율적인 실행

신뢰할 수 있는 결과
검증된 결과만 PR로 반환하여 품질과 신뢰도 향상

자율적 진화: 실패를 스스로 바로잡는 자기 수정 루프

계획 → 구현 → 검증 → 피드백을 반복하며 스스로 개선하는 자율형 개발 프로세스



계획 중심
명확한 계획과 태스크 분해로
일관된 실행 보장

자동 구현
계획된 내용을 자율적으로
코드로 구현

검증 우선
테스트, 정적 분석, 빌드 확인으로
품질과 안정성 확보

변경 관리
검증 통과 시에만 PR 생성으로
안전한 변경 관리

지속적 개선
실패 원인 분석 후 계획을 보완하여
반복적으로 성능 향상

핵심 가치

실패를 피드백으로 삼아 스스로 배우고 개선하는 루프를 통해,
더 높은 품질의 코드를 더 빠르고 안전하게 제공하는 자율형 소프트웨어 엔지니어링을 실현합니다.

프로젝트의 뇌(Brain): AGENTS.md 작성 가이드

AGENTS.md는 Codex를 포함한 모든 AI 에이전트가 이 프로젝트에서 일관되고 안전하게 작업하도록 돕는 핵심 지침서입니다.

AGENTS.md

1 기술 스택 및 빌드

- 패키지 매니저는 pnpm을 사용합니다.
- 패키지 설치: `pnpm install`
- 개발 서버 실행: `pnpm dev`
- 빌드: `pnpm build`

2 테스트 규칙

- 테스트 실행: `pnpm test`
- 테스트 관련 파일을 수정했다면, 변경 후 반드시 테스트를 실행합니다.
- 테스트는 빠르고, 독립적이며, 결정적이어야 합니다.

3 UI 컨벤션

- 기존 디자인 시스템과 컴포넌트를 우선 사용합니다.
- 일관된 네이밍, 폴더 구조, 스타일 가이드를 준수합니다.
- 불필요한 코드 스타일 변경은 하지 않습니다.

환경 자동 감지 지시

프로젝트 환경과 도구 버전을
자동으로 감지하고 안전하게 작업하도록 안내합니다.

검증 기준 명시

테스트, 린트, 타입 체크 등
품질 검증 기준을 명확히 합니다.

일관성 유지

코드 스타일, 네이밍, 아키텍처 원칙을
준수하도록 하여 프로젝트 일관성을 유지합니다.

핵심 원칙 | 이 문서는 새로운 주니어 개발자를 위한 온보딩 문서와 같습니다.
명확한 지침은 AI 에이전트의 출력 품질과 프로젝트 생산성을 결정합니다.

완벽한 Codex 프롬프트의 해부학

명확하고 구조화된 프롬프트는 Codex가 정확하게 이해하고 최고의 결과를 내는 핵심입니다.

모호한 요청 (Bad Prompt)

"버그 고쳐줘."

or

"전체 프로젝트를 개선해줘."

결과 (Result):

산만한 결과물, 엉뚱한 파일 수정,
검증 실패로 시간 낭비 발생

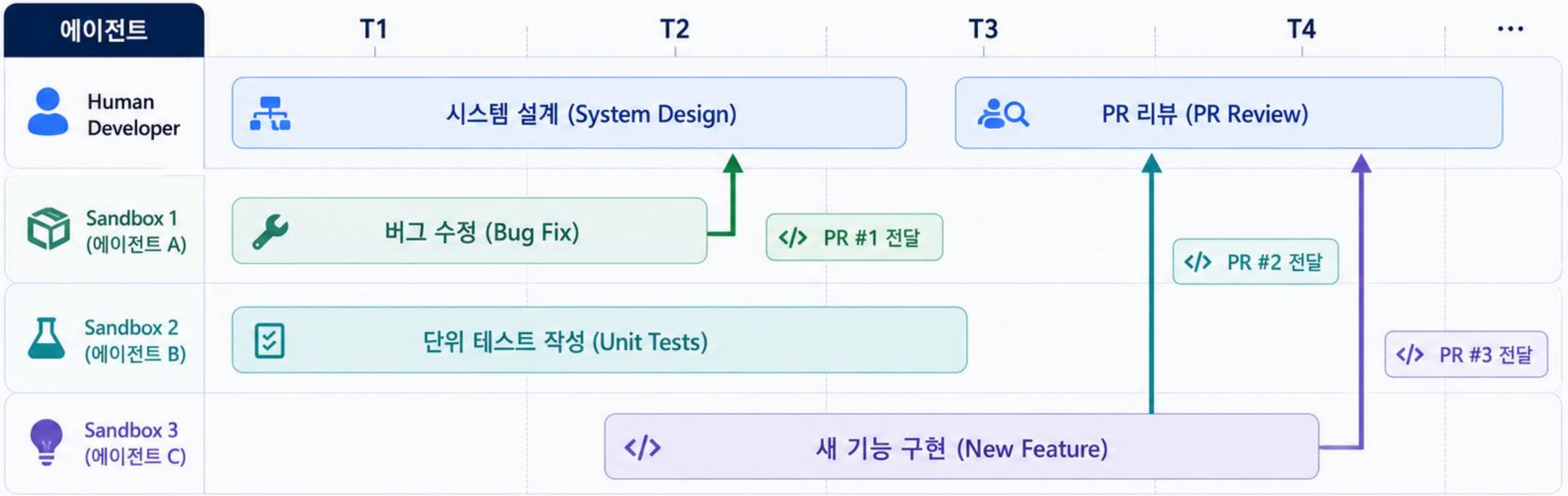
구조화된 요청 (Good Prompt)

목표 (Goal):	로그인 폼에서 JWT 토큰 만료 시 무한 리다이렉트 버그 수정
범위 (Scope):	src/auth/login.ts와 관련 테스트 파일만 수정
기준 (Rules):	만료 감지 시 refresh 시도, 실패 시 로그인 페이지로 1회만 이동
검증 (Verification):	관련 단위 테스트를 실행하고 결과를 요약해줘

핵심: 목표(Goal), 범위(Scope), 기준(Rules), 검증(Verification) — 이 4가지 요소가 프롬프트의 핵심입니다.

병렬 에이전트를 통한 압도적인 멀티태스킹

독립적인 샌드박스 환경에서 여러 에이전트가 동시에 작업하여 개발 병목을 제거하고 생산성을 극대화합니다.



핵심 가치

개발 속도 향상
여러 작업을 병렬로 수행하여 개발 사이클 단축

독립성 보장
각 에이전트는 독립 샌드박스에서 안전하게 작업

품질 향상
테스트, 리뷰 병렬 수행으로 버그 조기 발견

효율성 극대화
병목 제거 및 리소스 최적화로 생산성 극대화

핵심 원칙 | 개발 병목을 제거하고 독립적인 샌드박스에서 여러 에이전트가 동시에 작업하여 더 빠르고 안정적인 결과를 제공합니다.

기능의 무한한 확장: MCP와 플러그인 생태계

MCP(Model Context Protocol)와 플러그인을 통해 Codex 에이전트는 외부 시스템과 연결하고,
개발 전 과정을 자동화하여 생산성을 극대화합니다.

외부 API 연동

Model Context Protocol (MCP) 활용

- REST/GraphQL API 연동
- 타 서비스 및 도구 통합
- 데이터 검색 및 실시간 정보 활용

데이터베이스 스키마

실시간 스키마 검증 및 쿼리 테스트

- 스키마 자동 탐색 및 문서화
- 쿼리 검증 및 최적화 제안
- 변경 사항 감지 및 알림

이메일 및 일정 관리

반복적인 운영 업무 자동화

- 이메일 발송 및 응답 자동화
- 일정 생성 및 조율
- 알림 및 리마인더 관리

웹 배포 자동화

CI/CD 파이프라인 및 배포 도구 연동

- 빌드 및 테스트 자동화
- 배포 파이프라인 연동
- 배포 상태 모니터링

핵심 가치

연결성 확장
외부 시스템과의
손쉬운 통합

생산성 향상
반복 업무 자동화로
개발 집중도 향상

신뢰성 확보
검증 및 모니터링으로
안정적인 운영 지원

유연한 확장
플러그인 생태계로
무한한 기능 확장

AI 코딩 도구 랜드스케이프: 무엇을 선택해야 할까?

각 도구의 특징과 강점을 이해하고, 프로젝트와 팀의 워크플로에 가장 적합한 도구를 선택하세요.

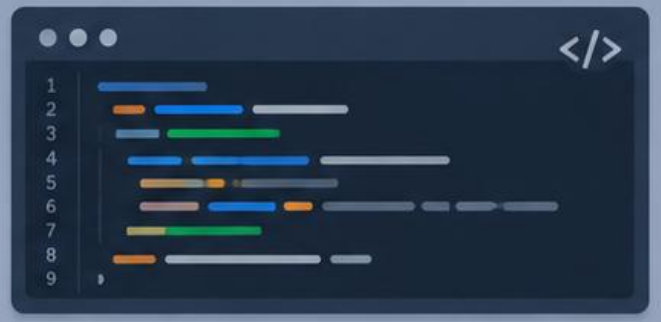
비교 항목	OpenAI Codex	Claude Code	GitHub Copilot / Cursor	Continue
실행 환경	클라우드 샌드박스 (격리된 안전 환경)	로컬 터미널(CLI)	IDE 내장 / 통합 (VS Code, JetBrains 등)	로컬 또는 클라우드 (사용자 환경 기반)
작업 범위	리포지토리 전체 (대규모 작업에 최적)	리포지토리 전체 (로컬 환경 중심)	현재 파일 + 프로젝트 (개발자 작업 보조)	리포지토리 전체 (사용자 설정에 따라)
자동 테스트	자체 샌드박스 실행 (테스트 코드 자동 실행)	로컬 터미널 실행 (개발자 확인 필요)	부분 지원 / 미지원 (수동 실행 중심)	자체/연동 테스트 실행 (설정 및 플러그인 기반)
결과 형태	GitHub PR 반환 (리뷰 및 병합 용이)	로컬 파일 직접 변경 (커밋/푸시 수동)	인라인 제안 및 편집 (즉시 코드 반영)	GitHub PR 반환 (변경사항 추적 가능)

핵심 인사이트 | 실시간 코딩 지원은 **Copilot/Cursor,** 비동기 대규모 태스크 병렬 처리는 **Codex**가 압도적입니다.

결론: 코더(Coder)에서 아키텍트(Architect)로

Codex는 단순 코딩을 넘어 시스템의 설계 · 검증 · 운영까지 책임지는 AI 아키텍트로 진화하고 있습니다.

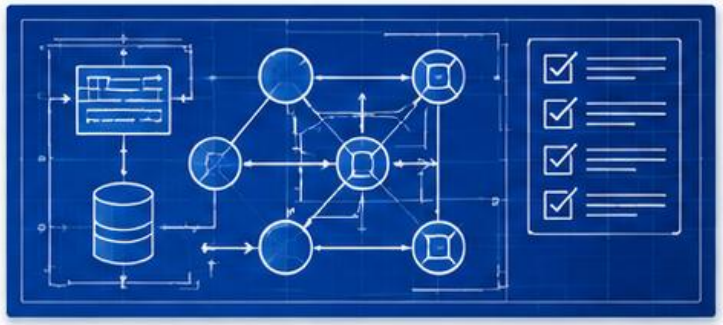
과거: 코더(Coder)



단순 코드 작성
(Line-by-line coding)

정해진 요구사항을 바탕으로
코드를 작성하고 구현하는 역할

미래: 아키텍트(Architect)



시스템 설계 및 품질 보증
(System Design & QA)

시스템 구조 설계, 품질 검증, 테스트 자동화까지
담당하는 역할

핵심 역량의 전환 | 개발자의 핵심 역량은 타이핑 속도가 아닌 시스템 설계와 정확한 커뮤니케이션(프롬프트)으로 이동합니다.

가상의 팀원, Codex | Codex는 개발자를 대체하는 것이 아니라, 설계와 리뷰에 집중할 수 있게 돕는 가장 뛰어난 '가상의 팀원'입니다.

최종 결정권은 인간 | 최종 결정권(Human-in-the-loop)은 항상 아키텍트인 여러분에게 있습니다.

Codex 환경 구축과 기본 사용

Codex는 ChatGPT, IDE, Terminal 등 다양한 환경에서 동일한 AI coding agent로 동작합니다.

Codex 사용 환경

여러 환경에서 동일한 coding-agent 개념으로 활용할 수 있습니다.

수업에서 추천하는 조합 (핵심 3가지)

ChatGPT
아이디어 구상, 대화형 요청, 코드 생성/수정, 설명 요청

+

VS Code
코드 작성/편집, 디버깅, Git 통합 개발 환경

+

Codex CLI
터미널 기반 자동화, 대량 작업, 스크립트 실행

ⓘ 이 조합으로 대부분의 개발 시나리오를 효율적으로 수행할 수 있습니다.

각 환경에서 수행 가능한 주요 작업

환경	주요 특징	수행 가능한 작업 범위	강점
ChatGPT	대화형 AI 인터페이스 자연어 기반 요청	• 코드 생성/수정 • 코드 설명 및 분석 • 오류 원인 분석 • 작업 계획 및 설계	빠르고 직관적 아이디어 구상에 탁월
IDE (VS Code 등)	개발자 중심 IDE 로컬 개발 환경	• 코드 작성/편집 • 리팩토링 및 자동 완성 • 디버깅 및 실행 • Git 통합 작업	실시간 개발 생산성 높은 확장성
Terminal (Codex CLI)	터미널 기반 환경 명령어 중심 작업	• 명령 실행 및 빌드 • 파일/폴더 조작 • 테스트 실행 • 배치/자동화 작업	강력한 자동화 대량 작업에 최적
Codex CLI	로컬 coding agent 명령 기반 AI 작업	• 전체 프로젝트 탐색 • 코드베이스 수정 • 스크립트/테스트 자동화 • 반복 작업 자동 수행	AI agent의 힘을 터미널에서 직접 활용

핵심 정리

- ✓ Codex는 어디서나 동일한 AI coding agent로 동작합니다.
- ✓ 환경을 목적에 맞게 선택하면 개발 효율이 크게 향상됩니다.
- ✓ 이번 수업에서는 ChatGPT + VS Code + Codex CLI 조합을 중심으로 학습합니다.

Codex는 무료로 사용할 수 있을까?

Codex는 무료로도 사용 가능하지만, 사용 환경과 기능, 사용량에 제한이 있습니다.

무료로 사용 가능

무료

- ChatGPT 웹에서 일부 기능을 무료로 체험 가능
- Codex CLI도 무료로 설치 및 기본 사용 가능
- 가벼운 작업, 학습, 간단한 코드 검토/수정에 적합

※ 무료는 "제한된 사용량과 기능"을 의미합니다.

유료 플랜에서 더 강력한 기능 제공

유료

- 더 높은 사용량 한도
- 복잡한 작업 처리, 긴 실행 시간 지원
- 백그라운드 실행, 병렬 작업, 더 빠른 응답 속도
- 우선 접근 및 신규 기능 우선 제공

Tip

단순한 코드 생성이나 학습은 무료로 시작하고,
실무 프로젝트나 대규모 작업은
유료 플랜을 권장합니다.

항목	무료 (ChatGPT Free)	ChatGPT Plus (유료)	Codex CLI (무료)	Codex CLI (Pro / 유료)
사용 가능 여부	일부 기능 체험 가능	완전 사용 가능	설치 후 무료로 사용 가능	Pro 플랜 가입 시 제공
사용 환경	ChatGPT 웹	ChatGPT 웹	터미널 (로컬 환경)	터미널 (로컬 환경)
사용량 / 한도	제한적 (메시지/작업 수 제한)	무료보다 높은 한도	요청 수/토큰/작업 시간 제한	높은 한도 (더 많은 요청/토큰)
작업 복잡도	간단한 코드 생성, 질문, 간단한 수정	중간 수준 작업까지 가능	로컬 파일 기반 간단~중간 작업	복잡한 작업, 대규모 코드베이스 처리
실행 시간	짧은 작업 위주, 긴 작업 제한	더 긴 작업 처리 가능	짧은 작업 위주	긴 실행 시간 지원
백그라운드 실행	불가	일부 기능 지원 (제한적)	불가	지원
병렬 작업	불가	제한적 지원	불가	지원
신규 기능 / 우선 접근	일반 제공	우선 제공	일반 제공	우선 제공
적합한 사용자	학습자, 가벼운 사용, 체험 목적	개인 개발자, 학생, 프리랜서	로컬 개발자, 간단한 자동화 작업	전컬 개발자, 팀 단위, 실무 프로젝트

요금 정보 (2025년 기준)

ChatGPT Plus
$20 / 월
(또는 지역별 요금)

Codex CLI Pro
$20 / 월
(또는 지역별 요금)
API 사용량에 따라
추가 비용이 발생할 수 있음

참고
요금 및 정책은 변경될 수 있으므로
공식 사이트를 확인하세요.

한눈에 정리

무료로 시작 가능 (ChatGPT / Codex CLI) → 더 많은 기능과 한도가 필요하면 (Plus / Pro) → 실무와 대규모 프로젝트는 유료 플랜이 효율적

Codex Prompt Engineering

좋은 프롬프트는 Codex가 더 **정확하게 이해**하고, 더 **안정적인 결과**를 만들어냅니다.

핵심 원칙
목표와 맥락, 제약, 성공 기준을 구체적으로 제공하면
Codex는 더 설계적으로 계획하고 수행할 수 있습니다.

좋은 프롬프트 구조 (G C R C A)

G — Goal
무엇을 만들 것인가?
최종 목표를 명확히 정의합니다.

+

C — Context
현재 상황은 무엇인가?
관련 배경 정보와 데이터를 제공합니다.

+

R — Requirements
필수 기능은 무엇인가?
구현해야 할 기능을 구체적으로 나열합니다.

+

C — Constraints
제약 조건은 무엇인가?
사용 기술, 제한 사항을 명확히 합니다.

+

A — Acceptance Criteria
언제 완료된 것인가?
성공 기준과 검증 방법을 정의합니다.

나쁜 프롬프트 vs 좋은 프롬프트

✕ 나쁜 예

매출 분석 만들어줘.

너무 짧고 모호하여
원하는 결과를 얻기 어렵습니다.

⚠ 개선

sales.csv를 분석해줘.
월별 매출과 제품별 매출을 계산하고
그래프를 만들어줘.

요구는 더 구체적이지만
여전히 제약, 기준, 검증 방법이 부족합니다.

✅ 더 좋은 요청 (GCRCA 활용)

Goal
sales.csv를 이용한 매출 분석 프로그램을 만든다.

Context
컬럼: date, product, region, sales

Requirements
1. 데이터 로드 2. 결측치 확인 3. 월별 매출 계산
4. 상품별 매출 계산 5. 지역별 매출 계산 6. 시각화

Constraints
- pandas 사용 - matplotlib 사용 - 함수 기반으로 구성

Acceptance Criteria
- 오류 없이 실행 - 결측치 처리 - 테스트 작성
- README 작성

먼저 구현 계획만 작성하고 코드는 아직 수정하지 마.

GCRCA 예시: 매출 분석 프로그램

1. Goal
무엇을 만들 것인가?

매출 분석
프로그램을
만들어라.

2. Context
현재 상황은 무엇인가?

sales.csv에는
다음 컬럼이 있다.

- date
- product
- region
- sales

3. Requirements
필수 기능은 무엇인가?

- 월별 매출
- 제품별 매출
- 지역별 매출
- 그래프

4. Constraints
제약 조건은 무엇인가?

- Python
- pandas
- matplotlib
- seaborn
  사용 금지

5. Acceptance Criteria
언제 완료된 것인가?

- 프로그램이 오류 없이 실행된다.
- 결측치를 처리한다.
- 그래프가 정상 출력된다.
- pytest 테스트가 통과한다.
- README가 존재한다.

권장 작업 흐름

1. Plan (계획) → **2. Review (검토)** → **3. Execute (실행)** → **4. Validate (검증)**

1. 먼저 계획을 요청합니다. (설계, 구조, 접근 방법)
2. 계획을 검토하고 수정/보완을 요청합니다.
3. 승인된 계획을 기반으로 구현을 요청합니다.
4. 테스트 및 검증을 통해 완성도를 확인합니다.

★ 핵심 포인트: "먼저 계획만 작성해줘." 라고 요청하면, 성급한 코드 변경을 방지하고 더 나은 결과를 얻을 수 있습니다.

좋은 프롬프트 작성 팁

명확하고 구체적으로
모호한 표현을 피하고,
정확한 목표를 제시하세요.

충분한 맥락 제공
데이터 구조, 현재 상황,
관련 제약 등을 알려주세요.

단계적으로 나열
요구사항을 번호나 목록으로
정리하면 이해도가 높아집니다.

제약과 기준을 명시
기술 스택, 제한, 성공 기준을
명확히 정의하세요.

피드백 루프 유지
결과를 검토하고
반복적으로 개선하세요.

정리: 좋은 프롬프트 = 좋은 설계도
GCRCA 구조를 활용하고, Plan → Review → Execute 흐름을 따르면 Codex가 더 똑똑하게 움직이고, 당신이 원하는 결과를 더 빠르고 정확하게 얻을 수 있습니다.

바이브 코딩 작업 공간과 터미널

효율적인 개발을 위해 작업 공간(IDE)과 터미널을 함께 이해하고 활용합니다.

핵심 포인트

작업 공간(IDE)은 코드를 "작성하고 관리"하는 곳,
터미널은 명령을 "실행하고 결과를 확인"하는 곳입니다.

작업 공간 (Workspace / IDE)

코드를 작성하고, 파일을 관리하고, 프로젝트 구조를 한눈에 파악하는 공간

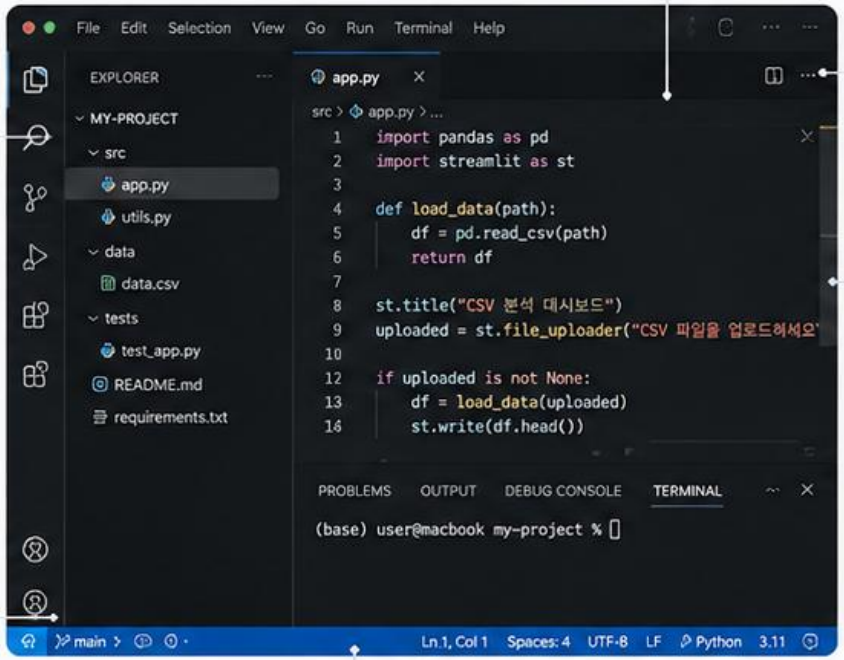
편집기(Editor)
코드를 작성하고 수정하는 영역

탐색기 (Explorer)
프로젝트 파일과 폴더 구조를 확인하고 관리

탭(Tabs)
여러 파일을 탭으로 고 전환

사이드 바(Panel)
검색, Git, 더버그, 확장 기능 등 다양한 기는 제공

상태 표시줄(Status Bar)
브랜치, 오류, 인코딩, 줄/열 정보 등 표시



패널(Panel)
터미널, 실행 결과, 오류 메시지 등을 확인

터미널 (Terminal)

명령을 입력하고 실행 결과를 확인하는 명령줄 인터페이스

```
Terminal
$ pwd
/Users/user/my-project                          # 현재 위치 확인
$ ls
README.md  data  requirements.txt  src  tests   # 파일/폴더 목록 확인
$ python -m venv .venv                          # 가상 환경 생성
$ source .venv/bin/activate                     # 가상 환경 활성화 (Mac/Linux)
(.venv) $ pip install -r requirements.txt       # 의존성 설치
(.venv) $ streamlit run src/app.py              # 앱 실행
You can now view your Streamlit app in your browser.
 Local URL: http://localhost:8501
 Network URL: http://192.168.0.10:8501
```

자주 사용하는 명령어

`pwd`	현재 위치 확인	`pip install -r requirements.txt`	의존성 설치
`ls / dir`	파일 및 폴더 목록 확인	`git status`	Git 상태 확인
`cd <폴더명>`	폴더 이동	`git add .`	변경 사항 스테이징
`python <파일명>.py`	Python 파일 실행	`git commit -m "메시지"`	커밋 생성

바이브 코딩에서의 활용 팁

작업 공간에서
- 코드를 작성하고 구조를 정리
- 파일 탐색과 빠른 이동
- 디버그와 실행 결과 확인

터미널에서
- 패키지 설치 및 환경 설정
- 앱 실행과 서버 관리
- Git 등 개발 도구 사용

함께 사용하기
- 코드는 IDE에서 작성
- 실행/설정은 터미널에서 처리
- 효율적인 개발 흐름 완성

기본 원칙
- 작은 변경 → 터미널에서 실행 → 결과 확인
- 문제가 생기면 터미널 로그를 먼저 확인
- 반복하며 빠르게 개선

핵심 정리

작업 공간(IDE)은 "설계와 작성",
터미널은 "실행과 검증"을 담당합니다.
두 도구를 함께 사용하면
바이브 코딩의 생산성이 극대화됩니다.

Codex 사용 방법 가이드

ChatGPT 계정 방식과 API Key 방식은 별도로 사용됩니다.

공통 사전 조건

- Node.js 18 이상 설치
- Codex CLI 설치: npm install -g @openai/codex
- Codex CLI 버전 확인: codex --version

1. ChatGPT 계정으로 사용 (권장) 간편 사용

별도의 API Key 없이 ChatGPT 계정으로 바로 사용할 수 있습니다.

1 터미널 열기

Terminal, PowerShell, CMD 등을 실행합니다.

2 Codex 실행

아래 명령어를 입력합니다.

명령어

```
codex
```

3 ChatGPT 계정으로 로그인

브라우저가 열리면 ChatGPT 계정으로 로그인/인증합니다.

예시

```
? Sign in to OpenAI? (Y/n) Y
? Opening browser...
✓ Successfully authenticated.
```

4 Codex 사용 시작

로그인 완료 후 자연어로 요청하면 Codex가 작업을 수행합니다.

예시

```
codex > 프로젝트 구조를 분석해줘.
..
codex >
```

장점

- 별도 API Key 불필요
- ChatGPT 유료 계정으로 사용 가능
- 설치와 로그인만으로 바로 시작

이런 분께 추천!

Codex를 빠로게 체험하고 싶거나 AI 코딩을 가볍게 시작하고 싶은 분

2. OpenAI API Key로 사용 (개발자/자동화) 고급 사용

API Key를 통해 더 많은 한도와 자동화 환경에서 사용할 수 있습니다.

1 OpenAI API Key 발급

OpenAI Platform에서 API Key를 발급받습니다.

https://platform.openai.com/api-keys

2 환경 변수 설정

터미널에서 환경 변수에 API Key를 설정합니다.

예시 (macOS / Linux)

```
export OPENAI_API_KEY="sk-..."
```

예시 (Windows PowerShell)

```
$env:OPENAI_API_KEY="sk-..."
```

3 Codex 실행

Codex CLI를 실행합니다.

명령어

```
codex
```

4 Codex 사용

API Key로 인증되어 작업을 수행합니다.

예시

```
codex > 복잡한 리팩터링을 도와줘.
..
codex >
```

장점

- 더 높은 사용 한도
- 자동화 스크립트/CI 환경에 적합
- 세밀한 사용 제어 가능

이런 분께 추천!

대량 작업, 자동화, 팀/서버 환경에서 Codex를 사용하려는 분

공통 주의사항

- ChatGPT 계정 방식과 API Key 방식은 서로 다른 인증 방식입니다.
- API Key는 외부에 노출되지 않도록 안전하게 관리하세요.
- 두 방식 모두 Codex CLI의 모든 기능을 사용할 수 있습니다.

Windows에서 WSL2 설치하기 최신 가이드)

WSL(Windows Subsystem for Linux)은 Windows에서 Linux 환경을 실행할 수 있게 해주는 기능입니다.

핵심 요약

- ☑ Windows 11은 'wsl --install' 한 줄로 설치 가능
- ☑ Windows 10은 버전에 따라 방법 선택
- ☑ WSL2가 기본 권장 (성능 및 호환성 향상)

Q. wsl --install 한 줄로 끝낼 수 있을까?

가능합니다! (권장)

대부분의 최신 Windows 10(2004 이상) / Windows 11은 아래 명령 한 줄로 설치됩니다.

```
PowerShell (관리자 권한)
wsl --install
```

설치되는 항목: WSL + Virtual Machine Platform + 기본 Linux 배포판(예: Ubuntu)
설치 후 재부팅이 필요할 수 있습니다.

언제 수동 설치가 필요할까?

- Windows 10 2004 미만(빌드 19041 미만)
- 기업/학교 정책으로 기능 설치가 제한된 경우
- 특정 기능만 개별적으로 제어하고 싶은 경우
- 설치 중 오류 발생 시 수동으로 의존성 해결 필요

필수 요구 사항

- ✓ Windows 10 2004 빌드 19041 이상 또는 Windows 11
- ✓ 64비트 시스템
- ✓ CPU 가상화 지원 (Intel VT-x / AMD-V)
  BIOS/UEFI에서 활성화 필요
- ✓ BIOS/UEFI에서 가상화 기능 활성화

방법 1. 간편 설치 (권장) – wsl --install

1 관리자 PowerShell 실행
시작 메뉴에서 'PowerShell'을 검색 → '관리자 권한으로 실행'

2 설치 실행
다음 명령 실행

```
wsl --install
```

3 재부팅
설치 완료 후 PC를 재부팅합니다.

4 설치 확인
아래 명령으로 확인

```
wsl --list --verbose
```

ⓘ 최신 Windows에서는 대부분 이 방법으로 자동 설치됩니다.

방법 2. 수동 설치 (필요 시)

1 WSL 기능 활성화
관리자 PowerShell에서 실행

```
C:\Windows\System32\dism.exe
/online /enable-feature
/featurename:Microsoft-Windows-
Subsystem-Linux /all /norestart
```

2 Virtual Machine Platform 활성화
관리자 PowerShell에서 실행

```
C:\Windows\System32\dism.exe
/online /enable-feature
/featurename:VirtualMachinePlatform
/all /norestart
```

3 Hyper-V (선택) 활성화
일부 환경에서 필요 (권장)

```
C:\Windows\System32\dism.exe
/online /enable-feature
/featurename:Hyper-V
/all /norestart
```

4 PC 재부팅
활성화 후 PC를 재부팅합니다.

5 Linux 커널 업데이트 패키지 설치
아래 링크에서 최신 패키지 다운로드 후 설치
https://aka.ms/wsl2kernel

6 기본 배포판 설치
다음 명령 중 선택

```
wsl --install -d Ubuntu
또는
Microsoft Store에서 설치
```

7 설치 확인
아래 명령으로 확인

```
wsl --list --verbose
```

상치 확인 및 기본 사용

```
PowerShell
wsl --list --verbose
```

	NAME	STATE	VERSION
*	Ubuntu	Running	2
	docker-desktop	Running	2

- ☑ STATE가 Running이고
- ☑ VERSION이 2이면 WSL2 정상 설치 완료!
- ☑ 처음 실행 시 사용자 계정/비밀번호 설정이 필요할 수 있습니다.

주요 명령어

`wsl --list --verbose`	설치된 배포판 확인
`wsl -d <배포판>`	특정 배포판 실행
`wsl --set-default-version 2`	기본 버전 설정(WSL2)
`wsl --update`	WSL 업데이트
`wsl --shutdown`	WSL 중료

문제 해결 팁

- 명령을 사용할 수 없음 → PC 재부팅 후 다시 시도
- VERSION이 1로 표시됨 → `wsl --set-version <배포판> 2`
- 가상화가 사용 안 됨 → BIOS/UEFI에서 가상화(VT-x/AMD-V) 활성화
- WSL 관련 오류 → Windows 업데이트 최신 상대로 유지

보안 참고
기업/학교 환경에서는 관리자 권한이 필요할 수 있으며, 정책에 따라 설치가 제한될 수 있습니다.

자세한 공식 문서
https://learn.microsoft.com/windows/wsl/install

TIP
도커(Docker), VS Code Dev Containers 등과 함께 사용하면 더 강력한 개발 환경을 구축할 수 있습니다!

Node.js란?

Node.js는 Chrome V8 JavaScript 엔진으로 빌드된
JavaScript 런타임 환경입니다. 브라우저 밖에서 JavaScript를 실행할 수 있게 해주며,
이벤트 기반, 논블로킹 I/O 모델로 빠르고 확장성 있는 네트워크 애플리케이션을 개발할 수 있습니다.

JavaScript 사용
프론트엔드와 백엔드 언어 통일

빠른 성능
V8 엔진 기반의 고성능 실행

확장성
이벤트 기반 아키텍처로 높은 동시성 처리

풍부한 생태계
NPM을 통한 다양한 패키지 제공

주요 특징

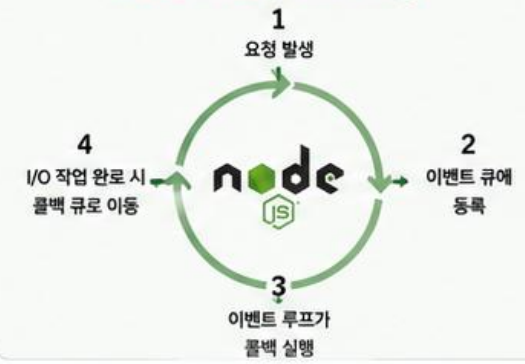
이벤트 기반 (Event-Driven)
이벤트 루프가 요청을 처리하여 자원을 효율적으로 사용

논블로킹 I/O (Non-blocking I/O)
I/O 작업이 블로킹되지 않아 뛰어난 성능과 확장성 제공

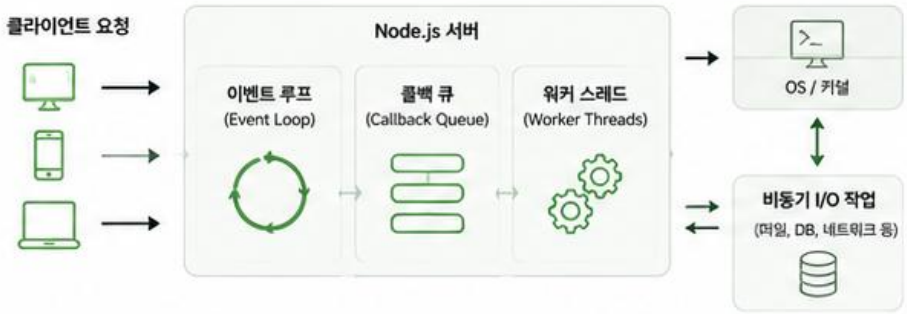
단일 스레드 모델
단일 스레드로 동작하지만 비동기 방식으로 여러 작업 처리

크로스 플랫폼
Windows, macOS, Linux 등 다양한 플랫폼에서 실행

이벤트 루프 (Event Loop)



Node.js 아키텍처



간단한 Node.js 예제

```js
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, Node.js!');
});

server.listen(3000, () => {
  console.log('서버가 http://localhost:3000 에서 실행 중입니다.');
});
```

실행: `node app.js`

주요 활용 분야

웹 서버 / API 서버
Express.js, NestJS 등을 이용한 RESTful API 개발

실시간 채팅 / 스트리밍
Socket.io 등을 활용한 실시간 애플리케이션

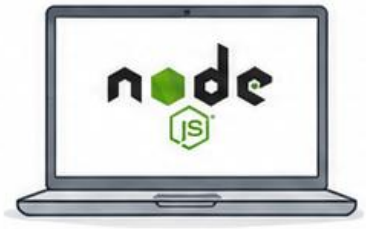
마이크로서비스
경량의 서비스들을 빠르게 개발하고 배포

빌드 도구 / CLI 도구
Webpack, ESLint, npm 등 다양한 개발 도구가 Node.js 기반

설치 방법

1. **Node.js 다운로드**
   https://nodejs.org/ 에서 LTS 버전 다운로드
2. **설치 및 확인**
   설치 후 터미널(또는 명령 프롬프트)에서 확인
3. **버전 확인**

```
node -v   # Node.js 버전
npm -v    # npm 버전
```



팁: nvm(Node Version Manager)를 사용하면 Node.js 버전을 쉽게 관리할 수 있습니다.

핵심 내장 모듈

http	fs	path	events	crypto
웹 서버 생성	파일 시스템 접근	경로 처리	이벤트 처리	암호화 기능

이 외에도 os, util, stream, buffer 등 다양한 내장 모듈 제공

생태계: npm (Node Package Manager)

- ✓ 전 세계 최대 JavaScript 패키지 저장소
- ✓ 수백만 개의 오픈소스 패키지 사용 가능
- ✓ 패키지 설치: `npm install <패키지명>`
- ✓ 패키지 관리, 버전 관리, 의존성 관리가 용이

대표적인 패키지 예시

`express` `lodash` `axios` `mongoose` `socket.io`

node 설치 시 오류 발생?

PowerShell 실행 정책(Execution Policy) 때문에 npm 명령 실행이 차단되는 경우입니다.

핵심 요약

- [x] Node.js는 정상 설치됨 (node -v는 동작)
- [x] npm 실행 시 PowerShell 실행 정책 문제 발생
- [x] 실행 정책을 CurrentUser 범위에서 RemoteSigned로 변경하면 해결

오류 화면

PowerShell에서 npm -v 실행 시 다음과 같은 오류가 발생합니다.

```
PS C:\Windows\system32> npm -v
npm : 이 시스템에서 스크립트를 실행할 수 없으므로
C:\Program Files\nodejs\npm.ps1 파일을 로드할 수 없습니다.
자세한 내용은 about_Execution_Policies
(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시오.
위치 줄:1 문자:1
+ npm -v
+ ~~~
    + CategoryInfo          : 보안 오류: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

원인

PowerShell의 기본 실행 정책이 Restriced로 설정되어 있어,
npm.ps1(PowerShell 스크립트) 실행이 차단되기 때문입니다.

정상 동작 확인

Node.js는 정상 설치된 상태입니다.

```
PS C:\Windows\system32> node -v
v20.11.1
PS C:\Windows\system32> npm.cmd -v   (또는 CMD에서 npm -v)
10.2.4
```

참고

- node -v → 정상
- npm.cmd -v → 정상
- npm -v → 실행 정책 문제

해결 방법 (권장)

현재 사용자 계정에 대해서만 실행 정책을 변경하면 안전하게 해결할 수 있습니다.

1 현재 실행 정책 확인 (선택)

```
Get-ExecutionPolicy -List
```

현재 설정 상태를 확인합니다.

2 현재 사용자 범위로 실행 정책 변경

```
Set-ExecutionPolicy -Lopoo
-Scope CurrentUser -ExecutionPolicy RemoteSigned
```

확인 메시지가 나오면
'Y'를 입력합니다.

3 PowerShell 닫고 다시 실행

변경 사항을 적용하기 위해
PowerShell을 다시 엽니다.

4 정상 동작 확인

```
node -v
npm -v
```

버전 번호가 출력되면
정상적으로 해결된 것입니다.

실행 정책 옵션 비교

RemoteSigned (권장)	• 로컬 스크립트는 실행 가능, 인터넷에서 받은 스크립트는 서명 필요
Unrestricted	• 모든 스크립트 실행 가능 (보안 위원 증가)
Bypass	• 모든 제한 무시 (권장하지 않음)
Restricted (기본값)	• 스크립트 실행 불가 (npm.ps1 실행 차단)

다른 방법 (우회 방법)

실행 정책을 변경하고 싶지 않다면 아래 방법을 사용할 수 있습니다.

- CMD에서 실행: npm -v
- PowerShell에서 npm.cmd 사용: npm.cmd -v

문제 해결 흐름 요약

Node.js 설치 완료 (node -v 정상) → npm 실행 시 오류 (PSSecurityException) → 실행 정책 변경 (RemoteSigned) → npm 정상 실행 (해결 완료)

기업/학교 환경에서는 CurrentUser 범위로 변경하는 것이 가장 안전합니다. 시스템 전체 변경(-Scope LocalMachine)은 관리자 권한이 필요하며 신중히 사용하세요.

챗 GPT 계정으로 Codex CLI 실행하기

Codex CLI는 ChatGPT 계정으로 간편하게 로그인하여 바로 사용할 수 있습니다.

핵심 포인트

- OpenAI API 키 없이도 사용 가능
- ChatGPT 계정만 있으면 OK
- 로그인 후 바로 Codex CLI 사용

1 Codex CLI 설치

npm을 이용하여 Codex CLI를 전역으로 설치합니다.

설치 명령어

```
npm install -g @openai/codex
```

TIP

Node.js 18 이상과 npm이 설치되어 있어야 합니다.

설치 확인

```
codex --version
1.0.0
```

2 터미널 실행

PowerShell, CMD, Terminal 중 편한 터미널을 실행합니다.

예시 (PowerShell)

또는

- Windows Terminal
- CMD
- Mac / Linux Terminal

3 Codex CLI 실행

아래 명령어를 입력하여 Codex CLI를 실행합니다.

명령어

```
codex
```

실행 예시

```
PS C:\Users\user> codex
Welcome to Codex CLI!
Let's get you signed in.
```

4 로그인 방법 선택

로그인 방식을 선택하라는 메시지가 표시됩니다.

화면 예시

```
How would you like to sign in?
1. ChatGPT account (recommended)
2. API key
Select an option:
```

TIP

일반 사용자는 1번(ChatGPT 계정)을 선택하세요.

5 브라우저 로그인

1번을 선택하면 브라우저가 열리고 OpenAI 로그인 페이지가 표시됩니다.

브라우저 예시

ChatGPT 계정으로 로그인합니다. (이메일/비밀번호 또는 다른 방법)

6 인증 완료

로그인 후 브라우저에서 인증이 완료되었다는 메시지가 표시됩니다.

브라우저 예시

브라우저를 닫아도 됩니다. 터미널로 돌아가세요.

7 Codex CLI 사용 시작

터미널에 로그인 성공 메시지가 표시되며 Codex CLI를 바로 사용할 수 있습니다.

터미널 예시

```
✓ Signed in successfully!
Welcome back, user!

codex >
```

이제 Codex CLI를 자유롭게 사용하세요!

자주 하는 질문 (FAQ)

Q. API 키가 꼭 필요한가요?

A. 아니요. ChatGPT 계정으로 로그인하면 API 키 없이도 Codex CLI를 사용할 수 있습니다.

Q. 무료로 사용할 수 있나요?

A. 네. Codex CLI는 ChatGPT Free를 포함한 ChatGPT 플랜에서 사용할 수 있습니다.

Q. 여러 계정으로 로그인할 수 있나요?

A. 네. `codex logout` 명령어로 로그아웃 후 다른 계정으로 다시 로그인할 수 있습니다.

유용한 명령어

`codex`	Codex CLI 실행
`codex logout`	로그아웃
`codex status`	로그인 상태 확인
`codex --help`	도움말 보기

보안 안내

공용 PC에서는 사용 후 반드시 로그아웃해 주세요.

```
codex logout
```

Codex를 처음 실행해 보자!

간단한 프롬프트로 Codex CLI 사용을 시작해 봅시다.

핵심 포인트

- Codex CLI를 실행하면 인터랙티브 모드로 시작됩니다.
- 자연어로 원하는 작업을 요청하면 Codex가 코드를 작성하고 실행까지 도와줍니다.
- 결과를 확인하고 추가 요청을 통해 더 발전시킬 수 있습니다.

1 터미널 열기

PowerShell, CMD, Terminal 등을 실행합니다.

- PowerShell
- CMD
- Terminal (Mac / Linux)

2 Codex CLI 실행

아래 명령어를 입력합니다.

명령어

```
codex
```

실행 예시

```
PS C:\Users\user> codex
Welcome to Codex CLI!
Type your request and
press Enter.
(help 를 입력하면 도움말 확인 )
codex >
```

3 자연어 프롬프트 입력

원하는 작업을 자연어로 입력합니다.
(예: 간단한 파일 생성 요청)

예시 프롬프트

```
codex > 간단한 할 일 목록을
관리하는 todo.py 파일을 만들어줘
```

4 Codex가 작업 수행

Codex가 코드를 작성하고
필요한 경우 실행까지 도와줍니다.

실행 예시

```
• todo.py 파일을 생성했습니다.
• 필요한 코드를 작성했습니다.
• 실행하여 정상 작동을 확인했습니다.
```

생성된 파일 예시

todo.py

5 결과 확인 및 추가 요청

결과를 확인하고, 추가로 개선이나
다른 기능을 요청할 수 있습니다.

실행 예시

```
codex > python todo.py

[1] 할 일 추가
[2] 할 일 목록 보기
[3] 할 일 완료
[4] 종료
선택하세요:
```

추가 요청 예시

→ 필터 기능을 추가해줘
→ 데이터를 파일에 저장하도록 바꿔줘

프롬프트 작성 팁

- 구체적으로 요청할수록 원하는 결과를 얻기 쉽습니다.
- 기능, 입력/출력 예시, 제약 조건을 함께 적어주세요.
- 필요하면 단계별로 요청하세요.

좋은 프롬프트 예시

Python으로 간단한 할 일 목록 결리 프로그램을 만들어줘.

기능:

- 할 일 추가, 목록 보기, 완료 표시, 삭제
- 데이터는 파일(todo.json)에 저장
- 메뉴 방식으로 동작

깔끔한 코드로 작성하고 주석도 추가해줘.

실습 과제 (직접 해보기)

1. **기본 파일 생성 요청**
   README.md 파일에 프로젝트 소개를 작성해줘.
2. **간단한 계산기 만들기**
   덧셈, 뺄셈, 곱셈, 나눗셈을 하는 calc.py를 만들어줘.
3. **웹 서버 만들기 (선택)**
   Flask를 사용해서 "Hello Codex!"를 출력하는 웹 서버를 만들어줘.

TIP

- 도움말: `/help` 입력
- 대화 기록: ↑/↓ 키로 이전 명령 확인
- 종료: `/exit` 또는 Ctrl+C
- 환경 정보: `/status`
- 문제 발생 시: `/reset`

실행 결과에 만족하지 않으면
추가 설명을 덧붙여 다시 요청해 보세요!

VS Code에서 Codex 사용하기

Codex CLI를 VS Code의 터미널에서 실행하여 AI와 함께 코드를 작성하고 개선하세요.

핵심 포인트

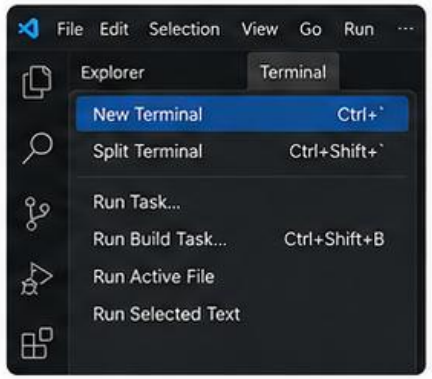
- VS Code 내장 터미널에서 Codex CLI 실행
- 자연어로 요청하면 Codex가 코드 작성/수정/설명
- 파일 변경 사항을 직접 확인하고 반영 가능

1 터미널 열기

VS Code에서 터미널을 엽니다.

`Ctrl` + `` ` ``

메뉴: Terminal > New Terminal



2 Codex 실행

터미널에서 codex 명령어을 입력하여 실행합니다.

명령어

```
codex
```

실행 예시

3 자연어로 요청 입력

원하는 작업을 자연어로 입력합니다.
(예: 함수 생성, 버그 수정, 설명 요청 등)

예시 요청

```
파이썬으로 피보나치 수열을
구하는 함수를 만들어줘
```

TIP

구체적일수록 원하는 결과를 정확하게 얻을 수 있어요!

4 Codex가 작업 수행

Codex가 코드를 작성하거나 수정하고 결과를 터미널에 표시합니다.

실행 예시

```
codex > 파이썬으로 피보나치 수열을
구하는 함수를 만들어줘
✓ 작업을 시작합니다...
생성된 코드 (fibonacci.py)
def fibonacci(n):
    if n <= 0:
        return []
    if n == 1:
        return [0]
    seq = [0, 1]
    for i in range(2, n):
        seq.append(seq[-1] + seq[-2])
    return seq
```

5 파일 변경 확인

Codex가 생성/수정한 파일을 VS Code에서 확인합니다.

예시

6 추가 요청 및 개선

추가 기능, 수정, 설명 등을 계속 요정할 수 있습니다.

예시 요청

```
입력값 검증을 추가해줘
함수 사용 예시도 보여줘
```

실행 예시

```
codex > 입력값 검증을 추가하고
사용 예시도 보여줘
✓ fibonacci.py 업데이트 완료
✓ 사용 예시 추가 완료
```

추천 사용 흐름

계획	자연어로 요구사항을 설명
생성/수정	Codex가 코드 작성 및 수정
확인	파일 변경 사항 확인 및 테스트
반복 개선	추가 요청으로 점진적 개선

AI와 함께 더 빠르고 스마트하게 개발하세요!

자주 사용하는 요청 예시

- **특정 기능 구현**
  "사용자 로그인 기능을 만들어줘"
- **버그 수정**
  "이 코드의 버그를 찾아서 고쳐줘"
- **코드 설명**
  "이 함수가 어떻게 동작하는지 설명해줘"
- **리팩터링**
  "이 코드를 더 깔끔하게 리팩터링해줘"
- **테스트 코드**
  "이 함수에 대한 테스트 코드를 작성해줘"

{ }

유용한 명령어

명령어	설명
/help	도움말 보기
/status	현재 작업 상태 확인
/context	현재 컨텍스트(파일/폴더) 확인
/history	대화 히스트리 보기
/clear	대화 내용 지우기
/exit	Codex 종료

주의사항

- 민감한 정보(API Key 등)는 입력하지 마세요.
- 생성된 코드는 반드시 테스트 후 사용하세요.
- 대규모 변경 사에는 Git으로 버전 관리하세요.

TIP

복잡한 프로젝트에서는 .cursorrules 파일을 활용하면 일관된 코드 스타일과 규칙을 유지할 수 있습니다.

자세히: cursor.com/docs/cursorrules

VS Code Extensions에서 Codex 설치하고 사용하기

OpenAI의 공식 확장 프로그램(Codex)을 설치하면 VS Code 안에서 AI와 대화하며 코드를 작성·수정할 수 있습니다.

핵심 포인트

- VS Code 안에서 바로 AI와 대화하며 개발
- 코드 생성, 수정, 설명, 리팩터링을 자연어로 요청
- 변경 사항을 미리 확인하고 적용(또는 취소) 가능

1 Extensions 열기

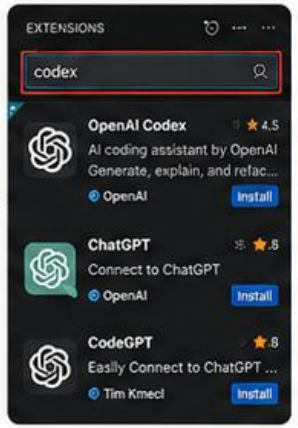
VS Code 좌측 Activity Bar에서 Extensions 아이콘(⊞)을 클릭합니다.

단축키

`Ctrl + Shift + X`

2 Codex 검색

검색창에 'Codex' 또는 'OpenAI Codex'를 입력합니다.

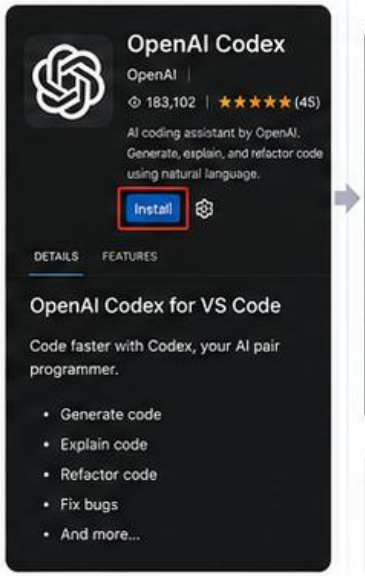


Tip

공식 확장 프로그램 제작자: OpenAI (로고 확인!)

3 설치

OpenAI Codex 확장 프로그램의 "Install" 버튼을 클릭합니다.



4 로그인

설치가 완료되면 Codex 패널이 열립니다. "Sign in"을 클릭하고 OpenAI/ChatGPT 계정으로 로그인합니다.

계정 요구 사항

ChatGPT 계정이면 바로 사용 가능 (별도 API Key 불필요)

5 프로젝트 폴더 열기

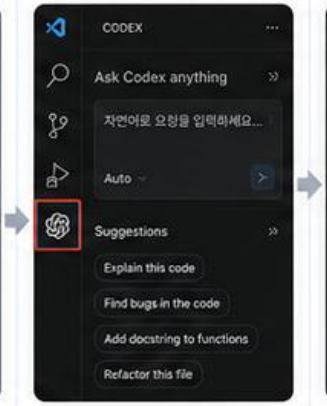
AI가 프로젝트를 잘 이해할 수 있도록 작업할 폴더를 엽니다.

권장 사항

프로젝트 루트 폴더를 열어야 파일 분석과 수정이 정확합니다.

6 Codex 패널 열기

좌측 Activity Bar의 Codex 아이콘(또는 View > Codex)을 클릭하여 Codex 패널을 엽니다.



단축키

View > Codex 또는
`Ctrl + Alt + I`

7 자연어로 요청

원하는 작업을 자연어로 입력하고 Enter를 누릅니다.

요청 예시

- 오류가 나는 부분을 찾아줘.
- 이 코드를 더 효율적으로 바꿔줘
- README.md를 작성해줘
- API 엔드포인트를 추가해줘

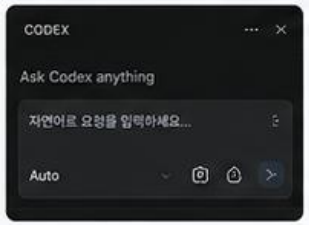
8 결과 확인 및 적용

Codex가 제안한 변경 내용을 확인하고 필요 시 "Apply"를 눌러 파일에 반영합니다.

안전하게 사용

적용 전 변경 내용을 항상 확인하고 필요 시 되돌리거나 수정하세요.

Codex 패널 주요 기능



새 대화 시작	새로운 대화를 시작합니다.
컨텍스트 첨부	파일/폴더/선택한 코드 등을 첨부합니다.
모드 선택 (Auto / Ask / Edit)	자동으로 최적 모드를 선택하거나 수동 지정합니다.
실행 요청 전송	요청을 전송하고 결과를 기다립니다.

실행 모드 안내

Auto (자동)	Codex가 요청 내용을 분석하여 가장 적합한 모드로 자동 실행
Ask (질문)	질문/설명/분석 등 정보 제공에 최적화
Edit (편집)	파일 생성/수정/삭제 등 변경 작업에 최적화

자주 사용하는 요청 예시

- 이 함수의 동작을 설명해줘.
- 이 코드의 시간 복잡도를 분석해줘.
- 에러 메시지의 원인을 찾아줘.
- 이 기능을 구현해줘.
- 변수명을 더 명확하게 바꿔줘.
- 주석과 타입 힌트를 추가해줘.

주의사항

- 민감한 정보나 비공개 코드를 입력하지 마세요.
- 생성된 코드는 반드시 테스트 후 사용하세요.
- 대규모 변경 전에는 Git으로 버전 관리하세요.
- 인터넷 연결이 필요합니다.

Codex CLI 기본 명령어 정리

Codex CLI에서 자주 사용하는 기본 명령어를 한눈에 확인하세요.

사용 방법

1. 터미널에서 codex 명령어를 입력하여 시작
2. 자연어로 요청하기나 옵션을 함께 사용
3. Codex가 코드를 작성/수정하고 결과를 보여줌

기본 명령어

명령어	설명	예시	설명
`codex`	Codex CLI 시작 (대화형 모드 실행)	`codex`	대화형 모드로 Codex 시작
`codex "요청 내용"`	한 번의 요청을 실행하고 결과 출력	`codex "README.md 파일을 생성해줘"`	비대화형 모드로 실행 후 종료
`codex --cd <경로>`	지정한 경로에서 Codex 실행	`codex --cd ./my-project`	특정 폴더에서 작업할 때 사용
`codex --model <모델>`	모델을 지정하여 실행	`codee --model o4-mini` `codex --model gpt-4.1`	사용 가능한 모델로 실행
`codex --file <파일>`	특정 파일을 참고하여 요청 수행	`codex --file app.py "이 코드 리뷰해줘"`	지정 파일 내용을 기반으로 응답
`codex --ask`	비대화형 모드에서 질문 입력 (STDIN)	`echo "프로젝트 구조를 분석해줘" \` `| codex --ask`	파이프와 함께 사용 가능
`codex --output <파일>`	결과를 파일로 저장	`codex "요약 정리해줘" \` `--output summary.md`	결과를 파일로 내보낼 때 사용
`codex --history`	이전 대화 히스토리 확인	`codex --history`	이전 요청과 응답을 확인
`codex --version`	버전 확인	`codex --version`	설치된 Codex CLI 버전 출력
`codex --help`	도움말 보기	`codex --help`	사용 가능한 옵션 확인

자주 사용하는 옵션

`--cd <경로>`	지정한 폴더에서 실행
`--model <모델>`	사용할 모델 지정 (예: o4-mini, gpt-4.1)
`--file <파일>`	참고할 파일 지정
`--output <파일>`	결과를 파일로 저장
`--ask`	비대화형 모드로 질문 입력
`--verbose`	상세 로그 출력
`--dangerously-bypass-permissions`	권한 확인 경고 생략 (주의해서 사용)

효과적인 요청 팁

- 구체적이고 명확하게 요청하세요.
- 원하는 결과 형식이나 조건을 함께 알려주세요.
- 관련 파일이나 코드를 함께 제공하면 더 정확한 결과를 얻을 수 있습니다.

요청 예시

```
# README.md 파일 생성
codex "프로젝트에 대한 README.md 파일을 작성해줘"
```

```
# 특정 파일 코드 지뷰
codex --file app.py
"이 코드의 문제점을 찾아줘"
```

```
# 포로젝트 구조 분석
codex "현재 포로젝트 구조를 분석하고 설망해줘"
```

```
# 테스트 코드 생성
codex --file utils.py
이 파일에 대한 테스트 코드를 작성해줘"
```

주의사항

- 민감한 정보(API Key 동)는 입력하지 마세요.
- 자동으로 실행된 코드/명령은 반드시 검토 후 사용하세요.
- 대규모 변경 전 에는 Git으로 버전 관리를 권장합니다.

컨텍스트 윈도우(Context Window)

AI가 한 번에 '기억할 수 있는 정보의 최대 길이(토큰 수)'를 의미합니다.

핵심 요약

- 컨텍스트 윈도우 = 모델이 동시에 처리할 수 있는 최대 토큰 수
- 매모리 버퍼 (작업 기억 공간)에 대화, 파일, 코드, 검색 결과 등이 담김
- 창을 초과하면 잘려나가거나 요약되어 정확도가 떨어질 수 있음

1. 컨텍스트 윈도우란?

컨텍스트 윈도우는 AI 모델의 **'작업 메모리 크기'**입니다.
사용자의 입력, 이전 대화, 시스템 지시, 파일/도구 결과 등
모든 정보가 이 안에 담겨 모델이 이해하고 응답합니다.

대화 내역
이전에 주고받은 메시지

시스템 프롬프트
모델의 역할, 규칙, 지시사항

사용자 입력 / 파일 내용
질문, 문서, 코드, 표 등

도구 결과 / 검색 결과
웹 검색, 코드 실행 결과 등

2. 메모리 버퍼(컨텍스트 버퍼)의 구조

AI는 입력된 정보를 토큰 단위로 변환하여 버퍼에 순서대로 담습니다.

컨텍스트 버퍼 (Context Window)

시스템 프롬프트	대화 내역 (이전 메시지들)	사용자 입력 (현재 질문)	도구 결과 / 파일 내용 (검색, 코드 실행 등)	버어있는 공간

처음(오래된 정보) → 최근(최신 정보)

모델(LLM)이 버퍼의 정보를 기반으로 응답 생성

버퍼 초과 시 어떻게 될까?

새로운 입력이 들어와 버퍼 용량을 초과하면,
모델은 오래된 정보를 다음 방식으로 처리합니다.

- 자동으로 압축(요약)된 내용을 잘라냄 (most common)
- 요약하여 압축 후 버퍼에 유지
- 중요도 기반으로 일부만 보존 (모델/제품에 따라 다름)

3. 토큰(Token)이란?

토큰은 모델이 이해하는 최소 단위입니다.
대략 단어의 일부 또는 1~4글자에 해당하며,
언어와 내용에 따라 길이가 달라집니다.

대략적인 기준 (영문 기준)

- 1 토큰 ≈ 0.75 단어
- 100 토큰 ≈ 75 단어
- 1,000 토큰 ≈ 750 단어
- 4,000 토큰 ≈ 3,000 단어

예시

"안녕하세요, 오늘 날씨가 어떤가요?"

↓

약 10 ~ 15 토큰

코드, 표, 특수문자는 더 많은 토큰을 사용합니다.

4. 컨텍스트 윈도우가 중요한 이유

1. 더 긴 대화 유지
더 많은 정보를 기억해 일관성 있는 대화 가능

2. 긴 문서/코드 처리
논문, 문서, 긴 코드 파일을 한 번에 이해 가능

3. 정확도 향상
충분한 맥락을 제공하면 정확하고 풍부한 답변 생성

4. 재작업 감소
중간 요약/재설명 없이 한 번에 작업 진행

TIP

중요한 정보는 최근에 두세요!

AI는 '최근 정보'에 더 높은 가중치를 둡니다.

Best Practice

- 긴 작업은 단계별로 나누기
- 핵심 정보는 주기적으로 요약하여 유지
- 불필요한 긴 출력/코드를 줄이기

5. 주요 모델별 컨텍스트 윈도우 (2026.08 기준)

모델	컨텍스트 윈도우 (토큰)	특징 / 비고
GPT-5	1M (1,048K)	최신 플래그십, 긴 문서/코드 작업에 최적화
GPT-5 mini	1M (1,048K)	경량 · 고속 모델
GPT-5 nano	256K	온디바이스/저지연 최적화
o3	200K	추론 특화(Reasoning) 모델
o4-mini	200K	추론 경량 모델
Gemini 2.5 Pro	2M (2,097K)	최대 2M 컨텍스트 지원
Gemini 2.5 Flash	1M (1,048K)	빠른 응답 & 긴 컨텍스트
Claude 4 Opus	200K	복잡한 작업, 긴 컨텍스트에 강점
Claude 4 Sonnet	200K	균형 잡힌 성능과 효율성
Claude 4 Haiku	200K	빠르고 효율적인 모델
Llama 4 Maverick	1M (1,048K)	균형형 오픈소스 모델
Llama 4 Scout	10M (10,485K)	초장문 컨텍스트 특화 모델
Mistral Large 3	128K	고성능 오픈 가중치 모델
DeepSeek-V3.1	128K	오픈소스 범용 모델
DeepSeek-R1	256K	추론 특화 오픈소스 모델

※ 제공사 및 버전에 따라 변경될 수 있습니다.

기억하세요!

- 컨텍스트 윈도우 = AI의 작업 메모리 크기
- 토큰 수가 많을수록 더 많은 정보를 한 번에 처리
- 초과 시 정보가 잘려 정확도 저하 가능
- 모델마다 최대 길이가 다르므로 작업에 맞는 모델 선택이 중요
- 효율적인 프롬프트와 요약 습관이 AI 성능을 크게 끌어올립니다!

GPT-6 모델 가격 정책 비교 (2026년 10월 기준)

GPT-6 Astra / GPT-6.1 Sol / GPT-6 Luna 모델의 특징과 가격을 비교합니다. (OpenAI 공식 API 요금 기준, 2026년 10월 22일 확인)

한눈에 보는 핵심 포인트

- 작업 특적과 난이도에 따라 적합한 모델 선택
- Standard 모드 · 짧은 컨텍스트 기준 가격 비교
- 캐싱, 도구 호출, 장문 입력 등 조건에 따라 실제 비용 상이

항목		GPT-6 Astra 최고 성능 · 최상위 모델	GPT-6.1 Sol 균형 잡힌 고성능 모델	GPT-6 Luna 비용 효율 · 경량 모델
모델 포지션		최고 난도의 추론, 연구, 복잡한 코딩을 위한 최상위 모델	폭넓은 전문 작업에 적합한 고성능·균형 모델	비용 효율적인 모델로 빠른 응답과 대량 처리에 최적화
주요 특징		복잡한 문제 해결, 깊이 있는 분석, 고난도 연구 및 개발 작업에 강점	우수한 성능과 비용의 균형, 다양한 전문 작업에 적합	빠른 응답 속도와 낮은 비용, 일상적인 작업 및 대량 처리에 적합
권장 사용 사례		심층 연구, 복잡한 코딩, 고난도 문제 해결, 전문 분석	문서 작성, 데이터 분석, 비즈니스 업무, 일반 개발 작업	일상 대화, 요약, 분류, 번역 등 비용 우선의 대량 처리 작업
최대 컨텍스트 길이		**1,050,000 토큰** (최대 출력 128,000 토큰)	**미공개** (공식 문서 참고)	**미공개** (공식 문서 참고)
API 가격 (100만 토큰당, Standard · 짧은 컨텍스트)	입력 (프롬프트)	**$10.00**	**$2.00**	**$0.10**
	캐시 입력	**$1.00**	**$0.10**	**$0.01**
	출력 (응답)	**$50.00**	**$10.00**	**$0.50**
과금 예시 (입력 10만 + 출력 20만 토큰, 캐시 적용 없음)		**$11.00** (입력 $1.00 + 출력 $10.00)	**$2.20** (입력 $0.20 + 출력 $2.00)	**$0.11** (입력 $0.01 + 출력 $0.10)

가격 이해 가이드

- **100만 토큰이란?**
  - 약 75만 단어 분량 (영문 기준)
  - 일반 문서 약 1,500~2,000페이지 분량
- **계산 방법 (Standard · 짧은 컨텍스트 기준)**
  - 입력 토큰 수 × 입력 가격 + 출력 토큰 수 × 출력 가격
  - 캐시 적용 시 캐시 입력 가격이 적용됩니다.
- **과금 예시 (입력 10만 + 출력 20만 토큰)**
  - Astra: $11.00 (입력 $1.00 + 출력 $10.00)
  - Sol: $2.20 (입력 $0.20 + 출력 $2.00)
  - Luna: $0.11 (입력 $0.01 + 출력 $0.10)

모델 선택 가이드

GPT-6 Astra가 적합한 경우
- 최고 난도의 추론과 복잡한 문제 해결이 필요할 때
- 심층 연구, 전문 분석, 고난도 코딩 작업

GPT-6.1 Sol이 적합한 경우
- 우수한 성능과 비용의 균형이 필요할 때
- 문서 작성, 데이터 분석, 비즈니스 업무 등

GPT-6 Luna가 적합한 경우
- 비용을 우선 고려해야 할 때
- 빠른 응답과 대량 처리, 일상적인 작업에 적합

참고 사항

- 본 내용은 OpenAI 공식 API 요금 기준 (2026년 10월 22일 확인)입니다.
- API 가격이며 ChatGPT 구독료와 별개입니다.
- Standard 모드 · 짧은 컨텍스트 기준 가격으로, 캐싱, 도구 호출, 장문 입력 등 추가 조건에 따라 실제 비용이 다를 수 있습니다.
- 공식 출처: developers.openai.com/api/docs/pricing

ChatGPT 명령어 한눈에 보기 (2026년 10월 기준)

데스크톱 앱에서 사용할 수 있는 슬래시 명령어로, 더 빠르고 효율적으로 작업해 보세요.

핵심 포인트

- 입력창에서 / 입력하면 명령어 메뉴가 나타납니다.
- 명령어를 선택하면 바로 적용됩니다.
- 작업 목적에 맞게 명령어를 조합하여 사용해 보세요.

1 채팅 관리

대화 시작, 정리, 맥락 관리를 위한 명령어

- **/new** — 새 대화를 시작합니다.
- **/rename** — 현재 대화의 이름을 변경합니다.
- **/archive** — 현재 대화를 보관합니다.
- **/compact** — 긴 대화의 맥락을 압축합니다.

2 모델·설정

모델과 추론 수준, 세션 상태 등을 설정

- **/model** — 사용할 모델을 선택합니다.
- **/reasoning** — 추론 수준을 선택합니다.
- **/status** — 현재 세션의 상태를 확인합니다.
- **/fast** — 빠른 모드로 전환합니다. (지원 시)

3 작업·개발

계획 수립, 목표 설정, 개발 작업을 지원

- **/plan** — 다단계 계획 모드로 전환합니다.
- **/goal** — 지속형 목표를 설정합니다.
- **/review** — 코드 리뷰를 요청합니다.
- **/init** — AGENTS.md 초안을 생성합니다.

4 환경·도구

프로젝트 연동, 클라우드, 서버 연결 등

- **/project** — 프로젝트를 선택합니다.
- **/cloud** — 클라우드 실행 환경으로 작업합니다. (지원 시)
- **/mcp** — 연결된 서버의 상태를 확인합니다.
- **/feedback** — 제품 피드백을 보냅니다.

사용 방법 (데스크톱 앱 기준)

1. **입력창에서 / 입력** — 채팅 입력창에 슬래시(/)를 입력합니다.
2. **명령어 메뉴에서 선택** — 나타나는 명령어 목록에서 원하는 명령어를 선택합니다.
3. **바로 적용** — 선택한 명령어가 적용되어 작업을 시작할 수 있습니다.

참고 사항 | 명령어 제공 여부는 앱·계정·작업 환경에 따라 다르며, 일부는 데스크톱·Codex 작업 환경 전용입니다. 일반 ChatGPT 웹의 메뉴와 다를 수 있습니다.

출처: ChatGPT Learn — Slash commands

바이브코딩 첫걸음: AI와 함께 간단한 계산기 만들기

자연어로 설명하고, AI가 코드를 만들어주는 가장 기초적인 경험

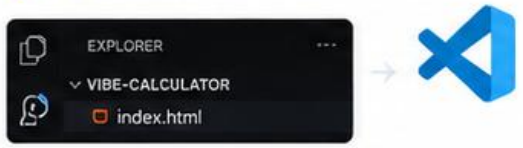
이번 예제에서 만들 것

- ✅ 브라우저에서 실행되는 간단한 계산기
- ✅ 덧셈, 뺄셈, 곱셈, 나눗셈 기능
- ✅ 사용자 친화적인 UI

1 준비하기

VS Code를 열고 새 폴더를 만듭니다.

1. 새 폴더 생성 (예: vibe-calculator)
2. 폴더 열기
3. 새 파일: index.html



TIP
VS Code 확장: GitHub Copilot 또는 Codeium, Continue 등을 설치하면 더 좋아요!

2 AI에게 요청하기

챗봇(예: ChatGPT, Copilot Chat)에 다음과 같이 요청합니다.

> 브라우저에서 실행되는 간단한 계산기를 만들어줘.
> 덧셈, 뺄셈, 곱셈, 나눗셈 기능이 필요하고,
> 결과를 화면에 보여줘. HTML, CSS, JavaScript로
> 한 파일(index.html)에 작성해줘.

좋아요! 아래는 간단한 계산기 코드입니다.

index.html

```
<!DOCTYPE html>
<html lang="ko">
<head> ...
```

AI가 코드를 생성해줍니다.

3 코드 복사 & 저장

AI가 생성한 코드를 복사하여 index.html에 붙여넣고 저장합니다.

index.html × | index.html › html › body › div.calculator

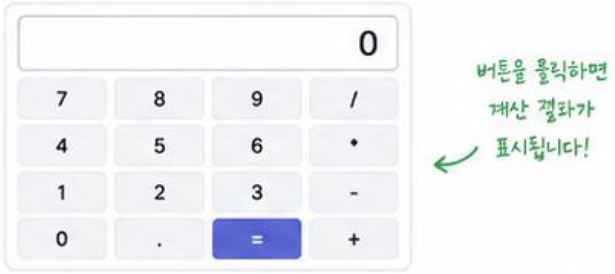
```
1  <!DOCTYPE html>
2  <html lang="ko">
3  <head>
4    <meta charset="UTF-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6    <title>간단한 계산기</title>
7    <style>
8      * { box-sizing: border-box; }
9      body {
10       display: flex;
11       justify-content: center;
12       align-items: center;
```

저장 단축키: Ctrl + S (Windows) / Cmd + S (Mac)

4 브라우저에서 실행

index.html 파일을 브라우저에서 열어 실행합니다.

방법 ① 파일을 더블 클릭 → 브라우저로 열림
방법 ② VS Code에서 'Live Server' 확장 설치 후 우클릭 → Open with Live Server



버튼을 올리하면 계산 결과가 표시됩니다!

5 기능 확장해보기 (프롬프트 예시)

원하는 기능을 자연어로 추가 요청해보세요!

- "키보드 입력도 지원하게 해줘."
- "계산 기록 기능을 추가해줘."
- "다크모드/리이트모드를 추가해줘."
- "소수점 계산 정확도를 높여줘."

계산 기록 예시

```
12 + 5 = 17
7 × 3 = 21
100 ÷ 4 = 25
9 - 2 = 7
```

TIP
작게 나누어 요청할수록 원하는 결과를 얻기 쉬워요!
(한 번에 하나씩 추가 기능 요청)

6 정리 및 다음 단계

원성된 코드를 정리하고, 배포까지 도전해보세요!

- **코드 정리**
  불필요한 주석/공백 제거, 파일 구조 정리
- **버전 관리 (선택)**
  Git으로 버전 관리 시작
- **배포 (선택)**
  GitHub Pages, Vercel, Netlify 등에 배포
- **더 큰 프로젝트로 확장!**
  투두리스트, 날씨 앱, 메모 앱 등 도전해보세요.

핵심 포인트

자연어로 설명하기 → AI가 코드 생성 → 실행 & 확인 → 기능 확장 → 배포까지!

✅ 바이브코딩의 핵심은 '대화하듯이 개발'하는 경험입니다!

Thank you.

스마트제조_2일차
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.